

```
const fs = require("fs");  
const path = require("path");  
const app = express();  
app.use(express.json());  
async function 불러오기() {  
  return JSON.parse(글자);  
} catch (에러) {
```

2 권 · 저장과 화면

백엔드

요청을 받아 응답을 돌려주는 것에서 시작해,
파일에 저장하고 화면과 잇는 데까지 갑니다.

5

단원

20

개념

134

직접 해보기

68

문제

차례

07	데이터 저장하기	9
01	메모리에서 파일로	10
02	저장소로 묶기	20
03	계층 나누기 (routes · controllers · services · repositories)	29
04	동시에 쓰면 데이터가 사라진다	38
05	저장소 완성하기, 그리고 왜 데이터베이스인가	48
08	정적파일과 CORS	89
01	정적 파일 서비스하기 (express.static)	90
02	CORS 란 무엇인가	98
03	CORS 직접 열어 보기	107
04	cors 패키지 쓰기	117
05	프론트와 붙이는 세 가지 방법	128
09	파일 업로드	153
01	파일 업로드의 원리	154
02	multer 시작하기 (그리고 한글 함정 세 가지)	162
03	제한과 검증	173
04	올린 파일 보여 주고 내려받게 하기	178
05	파일과 기록을 함께 관리하기	189
10	프론트엔드 연동	205
01	fetch 로 API 부르기	206
02	로딩·에러·빈 결과 보여 주기	215
03	폼과 검증 결과 보여 주기	223
04	목록 화면 만들기 (페이징 · 검색 · 정렬)	231
05	업로드 화면 만들기	240
11	종합 실습	261

부록 가	연습문제·종합연습 정답	301
------	---------------------	-----

부록 나	심화문제 정답	367
------	----------------	-----

여는 글

시작하기 전에

다 배우고 나면 이런 것을 직접 만듭니다. 지금은 무슨 코드인지 하나도 몰라도 됩니다.

이 책의 표시

블록마다 왼쪽 가장자리 표시가 다릅니다. 표시만 보고 “지금 내가 무엇을 해야 하는지” 알 수 있습니다.

```
console.log("안녕하세요");
```

— 직접 쳐 볼 코드입니다. 파일을 열고 그대로 따라 치세요.

출력 **안녕하세요**

— 실행하면 컴퓨터가 내놓는 값입니다. 내 화면과 같은지 맞춰 보세요.

▢ 직접 해보기 **1**

자기 이름을 출력해 보세요.

— **여러분 차례입니다.** 이 표시가 보이면 손을 움직이세요. 정답은 그 개념 맨 끝에 있습니다.

여기에 코드를 쓰세요

— 연필로 직접 적는 자리입니다.

이런 실수를 합니다

따옴표를 빠뜨림

```
console.log(안녕하세요);
```

따옴표가 없으면 컴퓨터는 ‘안녕하세요’라는 이름의 변수를 찾습니다.

— 여기서 많이 넘어집니다. 미리 보고 피해 가세요.

RUN node 개념01_console_log로_출력하기.js

— 개념마다 맨 위에 있습니다. 이대로 실행하면 됩니다.

시작하기

```
cd 백엔드자료
npm install          # 한 번만
node 01_Node_시작하기/개념01_Node란_무엇인가.js
```

패키지는 **맨 위 폴더에 한 번만** 설치합니다. Node 가 `node_modules` 를 위쪽으로 거슬러 올라가며 찾기 때문에, 단위 폴더 안에서 실행해도 잘 찾습니다.

도구	왜
Node 18 이상	<code>fetch</code> 가 내장돼 있어야 합니다 (24 권장)
VS Code	아무 에디터나 됩니다
Postman	03단원부터 필수입니다. 브라우저로는 POST 를 못 보냅니다
크롬	08·10단원에서 F12 를 계속 씁니다

파일 이름 규칙

개념01_....js	읽고 실행하며 배우는 것
연습문제.js	서버 없이 푸는 문제 (TODO 를 채웁니다)
연습문제_정답.js	
연습문제_서버.js	서버를 직접 만드는 문제
연습문제_서버_정답.js	

정답을 먼저 보지 마세요. 설명은 정답 파일에 더 많지만, 막힌 뒤에 읽어야 남습니다.

이 자료의 약속

적힌 출력은 전부 진짜입니다

```
console.log(1 + 2);
// 출력: 3
```

이 주석은 손으로 적은 것이 아니라 **기계로 대조한 것**입니다. 서버 자료도 마찬가지입니다.

```
// 확인: GET /health
// 응답: 200 {"status":"ok"}
```

이건 진짜로 서버를 띄워 요청을 보내고 응답을 맞춰 본 것입니다. 화면도 진짜 크롬으로 버튼을 눌러 확인했습니다.

결과가 다르면 여러분이 틀린 것입니다. 자료를 의심하지 말고 코드를 보세요. 그래도 다르면 알려 주세요. 자료가 틀렸을 수 있습니다.

틀린 예를 일부러 넣어 뒀습니다

자주 하는 실수 절에 있는 것들은 대부분 **자료를 만들면서 실제로 겪은** 것입니다.  직접 해보기 에 "일부러 망가뜨려 보세요" 가 자주 나옵니다. 꼭 해 보세요. **확인한 뒤에는 반드시 되돌리세요.**

한글은 값에만 씁니다

이 자료가 계속 강조하는 규칙입니다.

곳	한글	안 쓰면 무슨 일이
라우트 주소	×	에러 없이 조용히 404
응답 헤더 값	×	500 으로 터짐
요청 헤더 값	×	요청이 아예 안 나감
업로드 필드 이름	×	Unexpected field
업로드 파일 이름	△	깨져서 도착 (되돌려야 함)
쿼리·경로·본문·응답	○	—

안 되는 것은 전부 **헤더에 실려 가는** 것입니다. HTTP 헤더는 아주 오래된 약속이라 영어와 숫자만 담깁니다.

데이터 저장하기

OPEN 07_데이터_저장하기

```
const fs = require("fs");
const path = require("path");
const app = express();
app.use(express.json());
```

- 01 메모리에서 파일로
- 02 저장소로 묶기
- 03 계층 나누기 (routes · controllers · services · repositories)
- 04 동시에 쓰면 데이터가 사라진다
- 05 저장소 완성하기, 그리고 왜 데이터베이스인가
- 연습문제
- 서버 연습문제

메모리에서 파일로

RUN node 개념01_메모리에서_파일로.js

그러면 Ctrl + C

03단원부터 계속 이 말을 했습니다.

“서버를 꺾다 켜면 사라집니다.”

이제 고칩니다. 02단원에서 배운 파일 다루기를 서버에 붙입니다.

★ 파일 저장이 최종 답은 아닙니다. 진짜 답은 데이터베이스입니다. PART 4 에서 배웁니다. 그래도 파일을 한 번 거쳐야 하는 이유가 있습니다.

- ① "저장한다" 는 게 무슨 일인지 손으로 겪어 봐야 합니다
- ② 데이터베이스가 무엇을 해결해 주는지 알게 됩니다
- ③ 작은 프로젝트는 파일로도 충분할 때가 있습니다

```
const express = require("express");
const fs = require("fs");
const path = require("path");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
```

저장할 파일 위치 정하기

섹션 1

★ __dirname 을 꼭 쓰세요. (02단원 개념01) “data/설비.json” 이라고만 쓰면 ‘터미널이 열려 있는 위치’ 기준이 됩니다. 다른 폴더에서 node 로 실행하면 엉뚱한 데를 찾거나 만듭니다.

```
const 데이터폴더 = path.join(__dirname, "data");
const 설비파일 = path.join(데이터폴더, "설비.json");
```

폴더가 없으면 만듭니다. 첫 실행 때 반드시 필요합니다.

```
fs.mkdirSync(데이터폴더, { recursive: true });
```

읽기 — 파일이 없을 수도 있습니다

섹션 2

```

async function 불러오기() {
  try {
    const 글자 = await fs.promises.readFile(설비파일, "utf-8");
    return JSON.parse(글자);
  } catch (에러) {
    if (에러.code === "ENOENT") {
      return []; // 아직 파일이 없으면 빈 목록으로 시작합니다
    }
    throw 에러; // 그 밖의 에러는 숨기지 않습니다
  }
}

```

★★ ENOENT 만 골라서 처리하는 것이 중요합니다.

catch 로 전부 잡아서 [] 를 돌려주면 어떻게 될까요? JSON 이 깨져 있어도, 권한이 없어도, 조용히 “데이터가 없습니다” 가 됩니다. 데이터가 멀쩡히 파일에 있는데 화면에는 아무것도 안 나옵니다. 그런데 에러도 안 나니 원인을 찾을 수가 없습니다.

“없는 것” 과 “못 읽는 것” 은 완전히 다릅니다. 없는 것만 [] 로 처리하고, 나머지는 그대로 터뜨리세요.

쓰기

섹션 3

```

async function 저장하기(목록) {
  await fs.promises.writeFile(설비파일, JSON.stringify(목록, null, 2), "utf-8");
}

```

★ `JSON.stringify(목록, null, 2)` 의 2 줄바꿈과 들여쓰기를 넣어 줍니다. 사람이 파일을 열어 볼 수 있습니다. 없으면 한 줄로 쭉 이어져서 읽을 수가 없습니다.

대신 파일이 조금 커집니다. 데이터가 아주 많으면 빠기도 합니다. 지금은 눈으로 확인하는 게 더 중요하니 넣습니다.

★ “utf-8” 을 빼먹으면 쓸 때는 괜찮은데 읽을 때 Buffer 가 나옵니다. (02단원 개념02)

컬 때 초기값 넣기

섹션 4

이 파일은 수업용이라, 컬 때마다 같은 상태에서 시작하게 합니다. 진짜 서버라면 이러면 안 됩니다. 기존 데이터가 날아갑니다.

```
const 처음데이터 = [
  { id: 1, name: "컨베이어 1호", line: "A", status: "가동" },
  { id: 2, name: "프레스 1호", line: "B", status: "정지" },
];

fs.writeFileSync(설비파일, JSON.stringify(처음데이터, null, 2), "utf-8");
```

★ 여기서만 동기(writeFileSync)를 씁니다. 서버가 켜지기 전, 요청을 받기 전이라 아무도 안 기다립니다. '시작할 때 한 번'은 동기여도 괜찮습니다. 요청을 처리하는 중에는 절대 안 됩니다. 섹션 7 에서 봅니다.

읽고 → 고치고 → 쓰기

섹션 5

```
app.get("/equipments", async (req, res) => {
  const 목록 = await 불러오기();
  res.json({ data: 목록 });
});
```

GET /equipments

```
200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"},{"id":2,"name":"프레스 1호","line":"B","status":"정지"}]}
```

```
app.post("/equipments", async (req, res) => {
  const { name, line } = req.body || {};

  if (!name || !line) {
    return res.status(400).json({ error: "name 과 line 이 필요합니다" });
  }
});
```

① 읽고

```
const 목록 = await 불러오기();
```

② 고치고

```
const 다음번호 = 목록.length === 0 ? 1 : Math.max(...목록.map((설비) => 설비.id)) + 1;
const 새설비 = { id: 다음번호, name, line, status: "정지" };
목록.push(새설비);
```

③ 쓰기

```
await 저장하기(목록);
```

```
res.status(201).json({ data: 새설비 });
});
```

```
POST /equipments
```

```
{ "name": "용접로봇 1호", "line": "C" }
```

```
201 { "data": { "id": 3, "name": "용접로봇 1호", "line": "C", "status": "정지" } }
```

```
GET /equipments
```

```
200 { "data": [ { "id": 1, "name": "컨베이어 1호", "line": "A", "status": "가동" }, { "id": 2, "name": "프레스 1호", "line": "B", "status": "정지" }, { "id": 3, "name": "용접로봇 1호", "line": "C", "status": "정지" } ] }
```

★★ '읽고 → 고치고 → 쓰기' 가 파일 저장의 기본 모양입니다. 메모리에 있을 때는 목록.push 한 줄이면 끝이었습니다. 파일은 통째로 읽고 통째로 다시 써야 합니다.

설비가 10만 개면 하나 추가하려고 10만 개를 읽고 10만 개를 씁니다. 이게 파일 저장의 한계입니다. 개념05에서 정리합니다.

★ 다음 번호를 `Math.max` 로 구하는 이유 목록.length + 1 로 하면 안 됩니다. 3개 중 2번을 지우면 [1, 3] 이 되는데, length + 1 은 3 입니다. 겹칩니다. 지금 있는 번호 중 제일 큰 것 + 1 이 안전합니다.

...목록.map(...) 은 배열을 인자들로 펼치는 것입니다. (JS자료 09단원) `Math.max([1,2,3])` 은 NaN 입니다. `Math.max(1,2,3)` 이어야 합니다.

목록이 비어 있으면 `Math.max()` 가 -Infinity 라서 따로 처리했습니다.

```
app.delete("/equipments/:id", async (req, res) => {
  const 번호 = Number(req.params.id);
  const 목록 = await 불러오기();

  const 있나 = 목록.some((설비) => 설비.id === 번호);
  if (!있나) {
    return res.status(404).json({ error: "설비를 찾을 수 없습니다" });
  }

  await 저장하기(목록.filter((설비) => 설비.id !== 번호));

  res.sendStatus(204);
});
```

DELETE /equipments/3

204

GET /equipments

200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"}, {"id":2,"name":"프레스 1호","line":"B","status":"정지"}]}

파일을 직접 열어 보세요

섹션 6

```
app.get("/raw", async (req, res) => {
  const 글자 = await fs.promises.readFile(설비파일, "utf-8");
  res.type("text/plain").send(글자);
});
```

data/설비.json 을 에디터로 열어 보세요. 우리가 저장한 내용이 사람이 읽을 수 있는 글자로 들어 있습니다.

★ 이게 파일 저장의 장점이자 단점입니다. 장점: 열어 보면 바로 압니다. 문제가 생겨도 눈으로 확인됩니다.
단점: 서버에 접근할 수 있는 사람은 누구나 다 봅니다.

비밀번호 같은 건 이렇게 저장하면 안 됩니다.

★★ 서버에서 동기 파일 읽기를 쓰면 안 되는 이유

섹션 7

02단원에서 readFileSync 와 fs.promises.readFile 을 둘 다 배웠습니다. 서버에서는 반드시 비동기 (fs.promises)를 써야 합니다. 왜일까요?

말로만 하면 안 와닿으니 직접 재 봅니다.

방법: 타이머를 0초 뒤에 실행하라고 걸어 두고, 파일을 300번 읽습니다.

읽는 동안 타이머가 실행됐는지 봅니다.

```
app.get("/sync-test", (req, res) => {
  let 타이머실행됨 = false;
  setTimeout(() => {
    타이머실행됨 = true;
  }, 0);

  const 시작 = Date.now();
  for (let i = 0; i < 300; i++) {
    fs.readFileSync(설비파일, "utf-8"); // 동기
```

```

}
const 걸린시간 = Date.now() - 시작;

console.log(` 동기 300회: ${걸린시간}ms`);

res.json({ 방식: "동기", 도중에_타이머가_실행됐나: 타이머실행됨 });
});

```

GET /sync-test

200 {"방식":"동기","도중에_타이머가_실행됐나":false}

```

app.get("/async-test", async (req, res) => {
  let 타이머실행됨 = false;
  setTimeout(() => {
    타이머실행됨 = true;
  }, 0);

  const 시작 = Date.now();
  for (let i = 0; i < 300; i++) {
    await fs.promises.readFile(설비파일, "utf-8"); // 비동기
  }
  const 걸린시간 = Date.now() - 시작;

  console.log(` 비동기 300회: ${걸린시간}ms`);

  res.json({ 방식: "비동기", 도중에_타이머가_실행됐나: 타이머실행됨 });
});

```

GET /async-test

200 {"방식":"비동기","도중에_타이머가_실행됐나":true}

★★★ 결과를 보세요.

동기 → 도중에 타이머가 한 번도 실행되지 않았습니다 (**false**) 비동기 → 도중에 타이머가 실행됐습니다 (**true**)

동기로 파일을 읽는 동안 Node 는 '아무것도' 못 합니다. 0초 뒤에 실행하라고 한 타이머조차 못 돌립니다. 그 사이에 들어온 다른 사람의 요청도 전부 줄을 서서 기다립니다.

02단원 개념03 의 '일꾼 한 명' 이야기가 여기서 진짜로 보입니다. Node 는 일꾼이 한 명입니다. 그 한 명이 파일 읽기에 붙들려 있으면 식당 전체가 멈춥니다.

★ 그런데 터미널의 시간을 보세요. 뜻밖의 결과가 나옵니다.

동기 300회: 11ms
비동기 300회: 35ms

동기가 3배 빠릅니다. 비동기는 매번 넘겨주고 받아 오는 비용이 있어서입니다.

그래도 서버에서는 비동기를 씁니다. “이 요청 하나가 빠른가” 가 아니라 “다른 요청이 기다리지 않는가” 가 중요하기 때문입니다.

혼자 쓰는 도구라면 동기가 낫습니다. 실제로 이 파일도 섹션 4의 ‘결 때 한 번’ 은 동기를 씁니다. 여러 사람이 동시에 쓰는 서버에서만 비동기가 필요합니다.

```
app.use((req, res) => {
  res.status(404).json({ error: "그런 주소가 없습니다" });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  res.status(err.status || 500).json({ error: "서버에서 문제가 생겼습니다" });
});

app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}/equipments`);
  console.log(`데이터 파일: ${설비파일}`);
});
```

메모리 저장 vs 파일 저장

메모리(지금까지)	파일(지금부터)
-----------	----------

꺾다 켜면 사라짐 남아 있음

읽기	배열 그대로	읽고 <code>JSON.parse</code>
쓰기	<code>push</code> 한 줄	전부 읽고 전부 다시 쓰기
속도	아주 빠름	느림

눈으로 확인 못 함 파일을 열면 보임 여러 서버가 공유 불가능 같은 파일이면 가능 (문제 많음)

★ 파일 저장으로 해결된 것: 꺾다 켜도 남습니다. 이거 하나뿐입니다. 나머지는 전부 나빠졌습니다. 그런데도 쓰는 이유는 ‘남는 것’ 이 그만큼 중요하기 때문입니다.

직접 해 볼 것

▫ 직접 해보기 1

설비를 하나 등록하고, Ctrl+C 로 서버를 꺾다 켜세요. 목록에 남아 있나요? (섹션 4 때문에 사라집니다. 그 줄을 주석 처리하고 다시 해 보세요)

▫ 직접 해보기 2

data/설비.json 을 에디터로 열어 직접 고쳐 보세요. 이름을 바꾸고 저장한 뒤 GET /equipments 를 부르면 바뀐 이름이 나오나요? 서버를 다시 켜야 하나요?

▫ 직접 해보기 3

data/설비.json 의 중괄호 하나를 지워서 JSON 을 깨뜨려 보세요. GET /equipments 가 어떻게 되나요? 불러오기의 catch 가 ENOENT 만 잡는 게 왜 중요한지 보이나요?

▫ 직접 해보기 4

data/설비.json 을 통째로 지우고 GET /equipments 를 부르세요. (섹션 4 를 주석 처리한 뒤에 하세요) 빈 배열이 나오나요? 에러가 나나요?

▫ 직접 해보기 5

PATCH /equipments/:id 를 만들어 status 를 바꿔 보세요. '읽고 → 찾고 → 고치고 → 쓰기' 순서입니다.

▫ 직접 해보기 6

/sync-test 를 브라우저 탭 두 개로 거의 동시에 열어 보세요. 반복 횟수를 300 에서 30000 으로 늘리면 더 잘 보입니다. 두 번째 탭은 언제 응답을 받나요? /async-test 로도 같은 실험을 해 보세요.

▫ 직접 해보기 7

다음번호 를 목록.length + 1 로 바꿔 보세요. 설비를 3개 만들고, 2번을 지우고, 하나 더 만들면 어떻게 되나요?

자주 하는 실수

이런 실수를 합니다

경로에 __dirname 을 안 씀

터미널 위치에 따라 파일을 못 찾거나 엉뚱한 곳에 만듭니다.

이런 실수를 합니다

catch 로 모든 에러를 [] 로 처리

JSON 이 깨져도 조용히 "데이터 없음" 이 됩니다. 원인을 못 찾습니다. ENOENT 만 골라내세요.

이런 실수를 합니다

폴더를 안 만들고 파일을 씀

→ ENOENT. writeFile 은 파일은 만들어도 폴더는 안 만듭니다. mkdirSync(폴더, { recursive: true }) 를 먼저 부르세요.

이런 실수를 합니다

await 를 빠뜨림

저장하기(목록) 만 쓰면 저장이 끝나기 전에 응답이 나갑니다. 바로 다음 요청에서 옛날 데이터가 읽힐 수 있습니다.

이런 실수를 합니다

다음 번호를 length + 1 로 구함

지운 뒤에 번호가 겹칩니다. Math.max 를 쓰세요.

이런 실수를 합니다

서버 요청 처리 중에 readFileSync 를 씀

그동안 다른 모든 요청이 멈춥니다. 섹션 7 에서 재 본 그대로입니다.

이런 실수를 합니다

JSON.stringify 를 빼고 저장

파일에 "[object Object]" 만 들어갑니다. 데이터가 통째로 사라집니다.

정리

- 1 메모리에 담아 둔 데이터는 서버를 끄면 사라진다. 파일로 옮기면 남는다.
- 2 경로는 반드시 __dirname 을 기준으로 만든다. 안 쓰면 '어디서 node 를 쳤느냐' 에 따라 파일 위치가 달라진다.
- 3 파일은 처음에 없을 수 있다. 없을 때만 빈 목록으로 시작한다.
- 4 catch 로 모든 에러를 [] 로 넘기면 안 된다. 진짜 고장을 못 보게 된다. '없어서 나는 에러' 인지 코드로 가려 낸다.
- 5 고치는 일은 읽고 → 고치고 → 쓰기 세 단계다.
- 6 await 를 빠뜨리면 아직 안 끝난 것을 쓰게 된다. 에러도 안 나서 더 나쁘다.
- 7 다음 번호를 length + 1 로 구하지 않는다. 중간을 지우면 번호가 겹친다.
- 8 객체는 JSON.stringify 로 글자로 바꿔서 저장한다.

- 9 서버가 요청을 처리하는 중에는 `readFileSync` 를 쓰지 않는다. 일꾼이 한 명인데 붙잡히면 그동안 아무도 응답을 못 받는다.

저장소로 묶기

RUN node 개념02_저장소로_묶기.js

그러면 Ctrl + C

이 파일은 혼자가 아닙니다. 옆의 repositories 폴더를 함께 보세요.

07_데이터_저장하기/

개념02_저장소로_묶기.js	← 지금 이 파일
repositories/	
설비저장소.js	← 파일을 읽고 쓰는 코드가 전부 여기
data/	
설비.json	← 실제 데이터

개념01에서는 불러오기·저장하기를 라우트 파일 안에 두었습니다. 라우트가 열 개가 되면 어떻게 될까요?

· 모든 라우트가 '읽고 → 고치고 → 쓰기'를 각자 씁니다 · id를 붙이는 방법이 라우트마다 조금씩 다를 수 있습니다 · 나중에 데이터베이스로 바꾸면 열 곳을 전부 고쳐야 합니다

그래서 '데이터를 다루는 코드'를 한 파일에 모읍니다. 이것을 저장소라고 부릅니다.

```
const express = require("express");
const 설비저장소 = require("./repositories/설비저장소");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
```

컬 때 초기값 (수업용)

섹션 1

```
const 처음데이터 = [
  { id: 1, name: "컨베이어 1호", line: "A", status: "가동" },
  { id: 2, name: "프레스 1호", line: "B", status: "정지" },
];
```

★ 초기화는 '비동기'입니다. 파일에 다 쓰이기 전에 요청이 들어오면 안 됩니다. 그래서 이 파일은 맨 아래에서 await로 기다린 뒤에 listen을 부릅니다. 여기서는 부르지 않습니다. 아래 시작하기()를 보세요.

라우트가 짧아졌습니다

섹션 2

```
app.get("/equipments", async (req, res) => {
  res.json({ data: await 설비저장소.전부() });
});
```

GET /equipments

```
200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"},{"id":2,"name":"프레스 1호","line":"B","status":"정지"}]}
```

```
app.get("/equipments/:id", async (req, res) => {
  const 설비 = await 설비저장소.하나(Number(req.params.id));
```

★ 여기서 404 를 냅니다. 저장소는 null 만 줄 뿐입니다.

```
if (설비 === null) {
  return res.status(404).json({ error: { code: "NOT_FOUND", message: "설비를 찾을 수 없습니다" } });
}

res.json({ data: 설비 });
});
```

GET /equipments/1

```
200 {"data":{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"}}
```

GET /equipments/99

```
404 {"error":{"code":"NOT_FOUND","message":"설비를 찾을 수 없습니다"}}
```

★★ 역할이 나뉜 것을 보세요.

저장소	"찾아봤는데 없습니다" → null
라우트	"그러면 404 를 보내자"

저장소는 HTTP 를 모릅니다. 404 라는 숫자를 본 적도 없습니다. 이 경계를 지키는 것이 이번 단원의 핵심입니다.

```
app.post("/equipments", async (req, res) => {
  const { name, line } = req.body || {};

  if (!name || !line) {
    return res.status(400).json({ error: { code: "VALIDATION_FAILED", message: "name
과 line 이 필요합니다" } });
  }
}
```

★ status 는 서버가 정합니다. id 는 저장소가 붙입니다.

```
const 새설비 = await 설비저장소.추가({ name, line, status: "정지" });

res.status(201).location(`/equipments/${새설비.id}`).json({ data: 새설비 });
});
```

```
POST /equipments
{"name":"용접로봇 1호","line":"C"}

201 {"data":{"id":3,"name":"용접로봇 1호","line":"C","status":"정지"}}
```

```
POST /equipments
{"name":"용접로봇 2호"}

400 {"error":{"code":"VALIDATION_FAILED","message":"name 과 line 이 필요합니다"}}
```

```
app.patch("/equipments/:id", async (req, res) => {
  const { status } = req.body || {};

  if (!["가동", "정지", "점검중"].includes(status)) {
    return res.status(400).json({ error: { code: "VALIDATION_FAILED", message:
"status 가 올바르지 않습니다" } });
  }

  const 고쳐진것 = await 설비저장소.수정(Number(req.params.id), { status });

  if (고쳐진것 === null) {
    return res.status(404).json({ error: { code: "NOT_FOUND", message: "설비를 찾을
수 없습니다" } });
  }

  res.json({ data: 고쳐진것 });
});
```

PATCH /equipments/3

{ "status": "가동" }

200 { "data": { "id": 3, "name": "용접로봇 1호", "line": "C", "status": "가동" } }

PATCH /equipments/99

{ "status": "가동" }

404 { "error": { "code": "NOT_FOUND", "message": "설비를 찾을 수 없습니다" } }

PATCH /equipments/3

{ "status": "이상" }

400 { "error": { "code": "VALIDATION_FAILED", "message": "status 가 올바르지 않습니다" } }

```
app.delete("/equipments/:id", async (req, res) => {
  const 지워졌나 = await 설비저장소.삭제(Number(req.params.id));

  if (!지워졌나) {
    return res.status(404).json({ error: { code: "NOT_FOUND", message: "설비를 찾을 수 없습니다" } });
  }

  res.sendStatus(204);
});
```

DELETE /equipments/3

204

DELETE /equipments/3

404 { "error": { "code": "NOT_FOUND", "message": "설비를 찾을 수 없습니다" } }

id 는 못 바꿉니다

섹션 3

```
app.patch("/equipments/:id/try-id-change", async (req, res) => {
```

일부러 id 를 바꿔 달라고 넘겨 봅니다.

```
const 결과 = await 설비저장소.수정(Number(req.params.id), { id: 999, name: "이름만 바뀜" });

res.json({ data: 결과 });
});
```

PATCH /equipments/1/try-id-change

200 {"data":{"id":1,"name":"이름만 바뀜","line":"A","status":"가동"}}

★ name 은 바뀌었는데 id 는 1 그대로입니다. 저장소의 수정 함수가 마지막에 id 를 다시 넣기 때문입니다.

```
목록[자리] = { ...목록[자리], ...바꿀것, id };
                └─
                맨 뒤라서 이깁니다
```

번호가 바뀌면 그 설비를 가리키던 모든 것이 어긋납니다. 점검기록의 equipmentId, 프론트가 저장해 둔 주소, 남이 북마크한 링크까지요. "번호는 한 번 정해지면 안 바뀐다" 는 규칙을 저장소가 지켜 줍니다.

저장소는 서버 없이도 시험할 수 있습니다

섹션 4

```
app.get("/self-test", async (req, res) => {
```

저장소만 따로 불러 쓴 것입니다. Express 와 아무 상관이 없습니다.

```
const 전부 = await 설비저장소.전부();
const 없는것 = await 설비저장소.하나(99999);
const 지우기실패 = await 설비저장소.삭제(99999);

res.json({
  전부개수: 전부.length,
  없는것: 없는것,
  지우기실패: 지우기실패,
});
});
```

GET /self-test

200 {"전부개수":2,"없는것":null,"지우기실패":false}

★ 이 세 줄은 서버 없이 그냥 node 로도 실행됩니다.


```
const 저장소 = require("../repositories/설비저장소");
저장소.하나(99999).then(console.log); → null
```

저장소가 Express 를 안 쓰기 때문에 가능합니다. 테스트를 짤 때 이 차이가 아주 큼니다.

```
app.use((req, res) => {
  res.status(404).json({ error: { code: "ROUTE_NOT_FOUND", message: "그런 주소가 없습니다" } });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  res.status(500).json({ error: { code: "INTERNAL_ERROR", message: "서버에서 문제가 생겼습니다" } });
});
```

준비가 끝난 뒤에 요청을 받습니다

섹션 5

```
async function 시작하기() {
```

① 먼저 데이터를 준비합니다 (수업용 초기화)

```
await 설비저장소.초기화(처음데이터);
```

② 준비가 끝난 뒤에야 문을 엽니다

```
app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}/equipments`);
});
}

시작하기();
```

★★ 왜 listen 을 async 함수 안에 넣었나

초기화(처음데이터) 는 파일에 쓰는 일이라 시간이 걸립니다. await 없이 부르고 바로 listen 하면 이런 일이 생깁니다.

```
서버가 켜짐 → 요청이 들어옴 → 아직 파일이 안 쓰였음 → 빈 목록
```

대부분은 파일 쓰기가 더 빨라서 문제가 안 보입니다. 그래서 더 위험합니다. 열 번에 한 번씩 이상하게 동작합니다.

“준비가 끝난 뒤에 문을 연다” 는 서버의 기본 순서입니다. PART 4 에서 데이터베이스에 연결할 때도 똑같이 합니다.

```
await 데이터베이스연결();
app.listen(PORT, ...);
```

★ 시작하기() 안에서 에러가 나면 준비에 실패했는데 서버가 켜지면 안 됩니다. 실무에서는 이렇게 씁니다.

```
시작하기().catch((에러) => {
  console.error("서버를 켜지 못했습니다:", 에러);
  process.exit(1);
});
```

저장소가 지켜야 할 규칙

① HTTP 를 모른다

req, res 를 쓰지 않습니다. 상태코드를 정하지 않습니다.

② 못 찾으면 null 이나 false 를 돌려준다

에러를 던져도 되지만, 그러면 "없는 것" 이 예외가 됩니다.
없는 것은 흔한 일이지 예외가 아닙니다.

③ 어떻게 저장하는지를 밖에 안 알린다

파일인지 데이터베이스인지 부르는 쪽이 몰라야 합니다.
readFile 을 내보내면 안 되는 이유입니다.

④ 규칙을 스스로 지킨다

id 를 붙이는 것, id 를 못 바꾸게 하는 것은 저장소의 몫입니다.

★ 반대로 저장소가 하면 안 되는 것 “이 사람이 볼 권한이 있나” 같은 판단은 저장소가 하지 않습니다. 그건 다음 층의 일입니다. 개념03 에서 봅니다.

직접 해 볼 것

▫ 직접 해보기

1

새 파일 시험.js 를 만들고 아래를 실행해 보세요.

```
const 저장소 = require("./repositories/설비저장소");
(async () => {
  console.log(await 저장소.전부());
  console.log(await 저장소.하나(1));
  console.log(await 저장소.하나(99));
})();
```

서버를 안 켜고도 되는 것을 확인하세요.

▫ 직접 해보기 2

저장소에 라인별로 찾는 함수 라인별(line) 을 추가하고 GET /equipments?line=A 를 만들어 보세요.

▫ 직접 해보기 3

저장소의 수정 함수에서 맨 뒤의 id 를 지워 보세요. PATCH /equipments/1/try-id-change 가 어떻게 되나요? 그 뒤에 GET /equipments/1 을 부르면요? (확인 후 반드시 되돌리세요)

▫ 직접 해보기 4

저장소의 findIndex 확인을 if (!자리) 로 바꿔 보세요. PATCH /equipments/1 이 어떻게 되나요? 왜 그럴까요? (확인 후 되돌리세요)

▫ 직접 해보기 5

module.exports 에 읽기 와 쓰기 를 추가해 보세요. 이제 라우트에서 저장소.쓰기([]) 를 부를 수 있습니다. 이게 왜 위험한지 생각해 보세요. (확인 후 되돌리세요)

▫ 직접 해보기 6

저장소의 삭제 함수가 그냥 true 를 돌려주게 바꿔 보세요. DELETE /equipments/99 가 어떻게 되나요? 프론트는 무엇을 잘못 알게 될까요? (확인 후 되돌리세요)

자주 하는 실수

이런 실수를 합니다

저장소에서 res 를 씀

저장소가 HTTP 를 알게 됩니다. 다른 곳에서 못 씁니다. res 가 필요해 보이면, 그건 저장소가 아니라 라우트가 할 일입니다.

이런 실수를 합니다

findIndex 결과를 if (!자리) 로 확인

0번 자리를 '못 찾음' 으로 봅니다. === -1 로 확인하세요.

이런 실수를 합니다

저장소 함수에 `await` 를 안 붙임

Promise 가 그대로 응답에 담깁니다. `{}` 만 나옵니다.

이런 실수를 합니다

저장소가 `id` 를 밖에서 받음

부르는 곳마다 다르게 정하면 번호가 겹칩니다.

이런 실수를 합니다

읽기·쓰기를 밖으로 내보냄

저장소에 가둔 의미가 사라집니다. 아무나 파일을 통째로 덮어쓸 수 있습니다.

이런 실수를 합니다

못 찾았을 때 빈 객체 `{}` 를 돌려줌

부르는 쪽이 “찾았다” 고 착각합니다. `null` 로 분명히 알려세요.

정리

- 1 읽고 쓰는 일을 '저장소' 로 묶으면 라우트가 짧아진다. 라우트는 요청을 받고 응답을 주는 일만 한다.
- 2 저장소는 `req` 도 `res` 도 모른다. 알면 서버 없이 시험할 수 없다.
- 3 그래서 저장소는 서버를 안 켜도 그냥 불러서 시험할 수 있다. 그게 목적이다.
- 4 `id` 는 저장소가 붙인다. 밖에서 받으면 겹치거나 건너뛴다.
- 5 `id` 는 나중에 못 바꾼다. 바꾸면 그것을 가리키던 것들이 전부 어긋난다.
- 6 `findIndex` 결과는 `if (!자리)` 로 확인하면 안 된다. 0 번째가 없는 것이 되어 버린다. `=== -1` 로 정확히 본다.
- 7 저장소 함수는 파일을 다루므로 부를 때 `await` 를 붙인다.
- 8 못 찾았으면 빈 객체가 아니라 `null` 을 돌려준다. `{}` 를 주면 부르는 쪽이 '찾았다' 고 오해한다.
- 9 읽기·쓰기 같은 속살은 밖으로 내보내지 않는다. 내보내면 아무나 파일을 건드린다.

계층 나누기 (routes · controllers · services · repositories)

RUN node 개념03_계층_나누기.js

그러면 Ctrl + C

이 파일에는 로직이 거의 없습니다. 조립만 합니다. 진짜 내용은 옆 폴더들에 있습니다. 순서대로 열어 보세요.

07_데이터_저장하기/

개념03_계층_나누기.js	← 조립 (지금 이 파일)
routes/설비라우트.js	주소 ↔ 함수 연결
controllers/설비컨트롤러.js	요청에서 꺼내고 응답 만들기
services/설비서비스.js	업무 규칙
repositories/설비저장소.js	파일 읽고 쓰기
utils/AppError.js	에러 만들기

★ 왜 이렇게까지 나누나

개념02 의 라우트 하나를 다시 봅시다.

```
app.post("/equipments", async (req, res) => {
  const { name, line } = req.body || {};          ← HTTP
  if (!name || !line) return res.status(400)... ← 규칙 + HTTP
  const 새설비 = await 설비저장소.추가(...);      ← 저장
  res.status(201).json({ data: 새설비 });         ← HTTP
});
```

짧아서 괜찮아 보입니다. 그런데 규칙이 늘어나면 이렇게 됩니다.

- 이름이 겹치는지 확인 (저장된 것을 전부 읽어야 함)
- 라인이 A/B/C 인지 확인
- 상태는 무조건 '정지' 로
- 등록한 사람을 기록
- 등록 알림을 보냄

라우트 하나가 60줄이 됩니다. 그리고 같은 규칙이 다른 라우트에도 필요해집니다. 복사해서 붙이면 한쪽만 고쳐지는 날이 옵니다.

```
const express = require("express");
const 설비라우트 = require("./routes/설비라우트");
const 설비저장소 = require("./repositories/설비저장소");
const { AppError, 에러 } = require("./utils/AppError");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
```

라우터 붙이기

섹션 1

```
app.use("/api/v1/equipments", 설비라우트);
```

잘 도는지 확인

섹션 2

GET /api/v1/equipments

```
200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"},{"id":2,"name":"프레스 1호","line":"B","status":"정지"},{"id":3,"name":"검사기 1호","line":"A","status":"점검중"}]}
```

GET /api/v1/equipments/2

```
200 {"data":{"id":2,"name":"프레스 1호","line":"B","status":"정지"}}
```

GET /api/v1/equipments/99

```
404 {"error":{"code":"NOT_FOUND","message":"설비를 찾을 수 없습니다"}}
```

★ “설비를”입니다. utils/한국어.js의 조사 함수가 붙여 준 것입니다. 06단원 연습문제에서 만든 것을 그대로 가져왔습니다.

업무 규칙이 정말 지켜지는지

섹션 3

규칙 ①: 이름 2글자, 라인 A/B/C

POST /api/v1/equipments

```
{ "name": "용", "line": "Z" }
```

400 { "error": { "code": "VALIDATION_FAILED", "message": "입력값이 올바르지 않습니다", "details": [{ "키": "name", "이유": "2글자 이상이어야 합니다" }, { "키": "line", "이유": "A, B, C 중 하나여야 합니다" }] } }

규칙 ②: 같은 이름은 못 만든다

POST /api/v1/equipments

```
{ "name": "프레스 1호", "line": "C" }
```

409 { "error": { "code": "DUPLICATED", "message": "이미 있는 설비 이름입니다" } }

규칙 ③: 새 설비는 무조건 '정지'

POST /api/v1/equipments

```
{ "name": "용접로봇 1호", "line": "C", "status": "가동" }
```

201 { "data": { "id": 4, "name": "용접로봇 1호", "line": "C", "status": "정지" } }

status: "가동" 을 보냈는데 "정지" 로 만들어졌습니다. 서비스가 보낸 값을 안 쓰고 무조건 "정지" 로 넣기 때문입니다.

규칙 ④: 점검중 → 가동 은 바로 못 간다

PATCH /api/v1/equipments/3

```
{ "status": "가동" }
```

409 { "error": { "code": "CONFLICT", "message": "점검중인 설비는 정지를 거쳐야 가동할 수 있습니다" } }

PATCH /api/v1/equipments/3

```
{ "status": "정지" }
```

200 { "data": { "id": 3, "name": "검사기 1호", "line": "A", "status": "정지" } }

PATCH /api/v1/equipments/3

```
{ "status": "가동" }
```

200 { "data": { "id": 3, "name": "검사기 1호", "line": "A", "status": "가동" } }

정지를 거치니 가동이 됩니다. 순서를 강제하는 규칙이 동작합니다.

규칙 ⑤: 가동 중인 설비는 못 지운다

DELETE /api/v1/equipments/3

409 {"error":{"code":"CONFLICT","message":"가동 중인 설비는 삭제할 수 없습니다. 먼저 정지시키세요"}}}

PATCH /api/v1/equipments/3

{"status":"정지"}

200 {"data":{"id":3,"name":"검사기 1호","line":"A","status":"정지"}}

DELETE /api/v1/equipments/3

204

★★ 여기가 이 단원의 핵심입니다.

위 다섯 가지 규칙은 전부 services/설비서비스.js 한 파일에 적혀 있습니다. 컨트롤러에도, 라우트에도, 저장소에도 없습니다.

“이 시스템의 규칙이 뭐예요?” 라는 질문에 “services 폴더를 보세요” 라고 답할 수 있습니다.

규칙이 여기저기 흩어져 있으면 그 질문에 아무도 답을 못 합니다.

404 와 에러 처리기

섹션 4

```
app.use((req, res, next) => {  
  next(에러.주소없음());  
});
```

GET /없는주소

404 {"error":{"code":"ROUTE_NOT_FOUND","message":"그런 주소가 없습니다"}}

```
app.use((err, req, res, next) => {  
  console.error(`[에러] ${req.method} ${req.path} - ${err.name}: ${err.message}`);  
  
  if (err.type === "entity.parse.failed") {  
    return res.status(400).json({ error: { code: "INVALID_JSON", message: "JSON  
형식이 아닙니다" } });  
  }  
  
  if (err instanceof AppError) {  
    const 몸통 = { code: err.code, message: err.message };  
    if (err.details !== undefined) 몸통.details = err.details;  
    return res.status(err.status).json({ error: 몸통 });  
  }  
});
```



```
res.status(500).json({ error: { code: "INTERNAL_ERROR", message: "서버에서 문제가 생겼습니다" } });
});
```

★ 에러 처리기는 조립하는 이 파일에 둡니다. “에러를 어떤 모양으로 내보낼지” 는 서비스가 아니라 서버 전체의 결정입니다.

준비하고 문 열기

섹션 5

```
const 처음데이터 = [
  { id: 1, name: "컨베이어 1호", line: "A", status: "가동" },
  { id: 2, name: "프레스 1호", line: "B", status: "정지" },
  { id: 3, name: "검사기 1호", line: "A", status: "점검중" },
];

async function 시작하기() {
  await 설비저장소.초기화(처음데이터);

  app.listen(PORT, () => {
    console.log(`서버가 켜졌습니다. http://localhost:${PORT}/api/v1/equipments`);
  });
}

시작하기().catch((에러) => {
  console.error("서버를 켜지 못했습니다:", 에러);
  process.exit(1);
});
```

각 층이 아는 것과 모르는 것

HTTP	업무규칙	저장방법			
routes		○	×	×	주소와 함수를 잇는다
controllers		○	×	×	꺼내고 부르고 응답한다
services		×	○	×	되는지 안 되는지 판단한다
repositories		×	×	○	읽고 쓴다

화살표는 한쪽으로만 갑니다.

routes → controllers → services → repositories

거꾸로 가면 안 됩니다. 저장소가 서비스를 부르면 서로 물고 물리는 관계가 됩니다. 서비스가 res 를 쓰면 웹 밖에서 못 씁니다.

★ 이 규칙 하나만 지키면 됩니다. “아래층은 위층을 모른다.”

무엇을 어디에 둘지 헛갈릴 때

“주소가 /equipments 인가 /api/v1/equipments 인가”

→ routes. 주소 이야기입니다.

“본문이 안 왔을 때 터지지 않게”

→ controllers. HTTP 사정입니다.

“req.params.id 를 숫자로”

→ controllers. 경계에서 타입을 맞춥니다.

“이름이 2글자 이상이어야 한다”

→ services. 업무 규칙입니다.

“가동 중이면 못 지운다”

→ services. 업무 규칙입니다.

“id 는 지금 있는 것 중 최댓값 + 1”

→ repositories. 저장 방법의 일부입니다.
데이터베이스로 바꾸면 데이터베이스가 붙여 줍니다.

“404 를 돌려준다”

→ 에러 처리기. 서비스는 '없다' 는 사실만 알립니다.

★ 판단이 애매한 것도 많습니다. 중요한 건 팀이 하나로 정하는 것입니다. “우리는 검증을 서비스에서 한다”
를 정해 두면, 찾을 때 헤매지 않습니다.

나눠서 좋은 점을 언제 느끼나

지금은 오히려 파일만 많아진 것 같습니다. 맞습니다. 설비 하나뿐인 지금은 나눈 이득이 잘 안 보입니다.

이럴 때 확 느낍니다.

① 자원이 늘 때

작업지시·불량기록·사용자가 생기면, 같은 구조를 복사해 만들면 됩니다.
매번 "어디에 뭘 쓰지?" 를 고민하지 않습니다.

② 저장 방법을 바꿀 때

파일 → 데이터베이스는 repositories 만 고칩니다. (PART 4)
services·controllers·routes 는 한 줄도 안 고칩니다.

③ 화면이 늘 때

웹 말고 관리자 도구를 따로 만들어도 services 를 그대로 씁니다.
규칙이 두 곳에서 어긋날 일이 없습니다.

④ 시험할 때

"가동 중이면 못 지운다" 를 확인하려고 서버를 켜고 Postman 을 열 필요가 없습니다.
서비스 함수만 불러서 확인하면 됩니다.

★ 반대로 — 언제 안 나누나 혼자 쓰는 작은 도구, 한 번 쓰고 버릴 스크립트는 나누지 마세요. 파일 다섯 개를 오가는 게 더 번거롭습니다. "누가 이 코드를 몇 년 동안 고칠 것인가" 가 기준입니다.

직접 해 볼 것

▫ 직접 해보기 1

파일 다섯 개를 순서대로 열어 보세요. routes → controllers → services → repositories POST 요청 하나가 어디를 거쳐 가는지 손으로 따라가 보세요.

▫ 직접 해보기 2

규칙을 하나 추가해 보세요. "한 라인에 설비가 5개를 넘으면 등록할 수 없다" 어느 파일을 고쳐야 할까요?

▫ 직접 해보기 3

컨트롤러에 검증 코드를 넣어 보세요. if (!name) return res.status(400)... 동작은 합니다. 그런데 서비스의 검증과 중복됩니다. 어느 쪽을 지워야 할까요? 왜죠? (확인 후 되돌리세요)

▫ 직접 해보기 4

서비스에서 res.status(404) 를 써 보세요. → TypeError 가 납니다. res 가 없으니까요. 이게 왜 '좋은 일' 인지 생각해 보세요.

▫ 직접 해보기 5

서버 없이 서비스를 시험해 보세요.

```
const 서비스 = require("./services/설비서비스");
(async () => {
  try { await 서비스.삭제(1); }
  catch (e) { console.log(e.status, e.code, e.message); }
})();
```

1번은 가동 중이라 409 가 나와야 합니다.
Express 를 안 켜고 업무 규칙을 확인했습니다.

▹ 직접 해보기 6

routes/설비라우트.js 에서 컨트롤러에 괄호를 붙여 보세요. 설비컨트롤러.목록() 서버를 켜면 어떻게 되나요? (확인 후 되돌리세요)

▹ 직접 해보기 7

05단원의 인증·역할확인을 가져와 POST 와 DELETE 에만 붙여 보세요. 어느 파일을 고치나요?

자주 하는 실수

이런 실수를 합니다

서비스에서 req 나 res 를 씀

그 순간 서비스가 웹 전용이 됩니다. 다른 곳에서 못 씁니다.

이런 실수를 합니다

컨트롤러에 업무 규칙을 씀

같은 규칙이 필요한 다른 입구가 생기면 복사하게 됩니다. 그리고 한쪽만 고쳐집니다.

이런 실수를 합니다

저장소가 서비스를 부름

서로 물고 물리는 관계가 됩니다. 화살표는 한쪽으로만.

이런 실수를 합니다

라우트에 컨트롤러를 괄호와 함께 넘김

설비컨트롤러.목록() 은 지금 실행합니다. 서버 켜자마자 에러가 납니다.

이런 실수를 합니다

층을 나눠 놓고 컨트롤러가 저장소를 직접 부름

서비스를 건너뛰면 업무 규칙이 안 지켜집니다. “가동 중이면 못 지운다” 를 통과해 버립니다.

이런 실수를 합니다

작은 프로젝트에 억지로 다 나눔

파일 20개에 코드가 5줄씩 들어 있으면 오히려 못 읽습니다. 나누는 건 커진 뒤에 해도 늦지 않습니다.

정리

- 1 층을 셋으로 나눈다. 컨트롤러(요청·응답) · 서비스(업무 규칙) · 저장소(읽고 쓰기).
- 2 부르는 방향은 한쪽이다. 컨트롤러 → 서비스 → 저장소. 저장소가 서비스를 부르면 고리가 생겨서 풀 수 없게 된다.
- 3 서비스에서 req 나 res 를 쓰지 않는다. 쓰는 순간 서버 없이 시험할 수 없다.
- 4 업무 규칙은 서비스에 둔다. 컨트롤러에 쓰면 같은 규칙을 여러 곳에 또 쓰게 된다.
- 5 층을 나눠 놓고 컨트롤러가 저장소를 직접 부르면 나눈 뜻이 없어진다.
- 6 라우트에는 컨트롤러 함수를 괄호 없이 넘긴다. 붙이면 그 자리에서 실행돼 버린다.
- 7 작은 프로젝트에 억지로 다 나누지 않는다. 파일만 늘고 읽기 어려워진다. 나눌 이유가 생겼을 때 나눈다.
- 8 404 처리기와 에러 처리기는 여기서도 맨 아래다.

동시에 쓰면 데이터가 사라진다

RUN node 개념04_동시에_쓰면_사라진다.js

그러면 Ctrl + C

★ 이 파일은 이번 단원에서 가장 중요합니다.

파일로 저장하면 안 되는 진짜 이유가 여기 있습니다.

02단원 개념05 마지막에 이렇게 적었습니다.

“동시에 두 사람이 저장하면 한쪽이 덮어써집니다.”

그때는 말로만 했습니다. 이번에는 직접 재 봅니다.

```
const express = require("express");
const fs = require("fs");
const path = require("path");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());

const 파일경로 = path.join(__dirname, "data", "동시성.json");
fs.mkdirSync(path.dirname(파일경로), { recursive: true });
```

지금까지 쓰던 방식

섹션 1

```
async function 읽기() {
  try {
    return JSON.parse(await fs.promises.readFile(파일경로, "utf-8"));
  } catch (에러) {
    if (에러.code === "ENOENT") return [];
    throw 에러;
  }
}

async function 쓰기(목록) {
  await fs.promises.writeFile(파일경로, JSON.stringify(목록, null, 2), "utf-8");
}
```

★ 읽다가 깨진 횟수를 세어 둡니다. 왜 필요한지는 섹션 2-2 에서 봅니다.

```
let 읽다깨진횟수 = 0;

async function 그냥추가(값) {
  let 목록;

  try {
    목록 = await 읽기(); // ①
  } catch (에러) {
```

다른 요청이 파일을 쓰는 도중이라 반쯤 쓰인 것을 읽었습니다.

```
    읽다깨진횟수 += 1;
    목록 = [];
  }

  const id = 목록.length === 0 ? 1 : Math.max(...목록.map((x) => x.id)) + 1;
  목록.push({ id, ...값 });
  await 쓰기(목록); // ②
}
```

지금까지 만든 것과 똑같습니다. 아무 문제 없어 보입니다. Postman 으로 하나씩 눌러 보면 정말 잘 됩니다.

★ try/catch 는 원래 없던 것입니다. 이 파일을 만들면서 /race 가 가끔 500 을 내길래 붙였습니다. 무슨 일이었는지는 섹션 2-2 에서 설명합니다.

★ 20명이 동시에 등록하면

섹션 2

```
app.post("/race", async (req, res) => {
  await 쓰기([]); // 빈 상태에서 시작
  읽다깨진횟수 = 0;
```

20개를 '동시에' 추가합니다. 20명이 같은 순간에 등록 버튼을 누른 것입니다.

```
await Promise.all(
  Array.from({ length: 20 }, (_, i) => 그냥추가({ name: `설비${i}` })))
);

const 결과 = await 읽기();

console.log(` /race - 읽다가 깨진 횟수: ${읽다깨진횟수}`);

res.json({
  시도: 20,
```

```
실제저장됨: 결과.length,  
  사라진것: 20 - 결과.length,  
});  
});
```

POST /race

200 {"시도":20,"실제저장됨":1,"사라진것":19}

★★★ 20건을 넣었는데 1건만 남았습니다. 19건이 사라졌습니다.

에러가 하나도 안 났습니다. 서버는 20번 모두 "저장했습니다" 라고 답했습니다. 그런데 19명의 데이터가 없습니다.

열 번을 반복해서 재 봐도 언제나 1건이었습니다. 우연이 아닙니다.

2: 사고가 하나 더 있습니다

섹션 2

터미널을 보세요. 이런 줄이 나올 때가 있습니다.

/race — 읽다가 깨진 횟수: 1

대부분은 0 입니다. 그런데 가끔 1 이나 2 가 나옵니다.

무슨 일일까요?

writeFile 은 파일을 '먼저 비우고' 새 내용을 씁니다. 비운 직후, 아직 다 안 쓴 그 짧은 순간에 다른 요청이 읽으면 반쯤 쓰인 글자를 읽게 됩니다.

```
읽은 것: [{"id":1,"name":"설비3  
JSON.parse → SyntaxError
```

이걸 '찢긴 읽기' 라고 합니다.

★ 이 파일을 만들 때 실제로 겪었습니다. /race 를 열 번 넘게 부르는 동안 한 번 500 이 났습니다. 재현이 잘 안 돼서 처음에는 원인을 몰랐습니다.

이런 버그가 제일 무섭습니다. "가끔 이상해요" 라는 문의는 있는데, 개발자가 눌러 보면 잘 됩니다. 동시에 일어나야만 생기는 문제라 혼자서는 재현이 안 됩니다.

★ 지금은 try/catch 로 덮어 뒀습니다. 진짜 해결이 아닙니다. 에러는 안 나지만 데이터를 못 읽은 건 마찬가지입니다. 제대로 고치는 방법은 섹션 5 에 있습니다.

왜 사라지나

섹션 3

```
app.get("/why", (req, res) => {
  res.json({
    설명: [
      "그냥추가 는 '읽기 → 고치기 → 쓰기' 세 단계입니다.",
      "await 읽기() 에서 Node 는 다른 요청에게 순서를 넘깁니다.",
      "그 사이에 다른 요청도 읽기를 시작합니다. 아직 빈 파일입니다.",
      "20개 요청이 전부 '빈 목록' 을 읽어 갑니다.",
      "각자 자기 것 하나만 담은 목록을 만들어 통째로 씁니다.",
      "나중에 쓴 것이 앞엿것을 덮어씁니다. 결국 1건만 남습니다.",
    ],
    이름: "읽어버린 갱신 (lost update)",
  });
});
```

GET /why

```
200 {"설명":["그냥추가 는 '읽기 → 고치기 → 쓰기' 세 단계입니다.", "await 읽기() 에서
Node 는 다른 요청에게 순서를 넘깁니다.", "그 사이에 다른 요청도 읽기를 시작합니
다. 아직 빈 파일입니다.", "20개 요청이 전부 '빈 목록' 을 읽어 갑니다.", "각자 자기
것 하나만 담은 목록을 만들어 통째로 씁니다.", "나중에 쓴 것이 앞엿것을 덮어씁니
다. 결국 1건만 남습니다."], "이름":"읽어버린 갱신 (lost update)"}
```

그림으로 보면 이렇습니다.

요청A	읽기[]	—————	쓰기[A]	
요청B	읽기[]	—————	쓰기[B]	← A 를 덮어씀
요청C	읽기[]	—————	쓰기[C]	← B 를 덮어씀

출력 [C] 하나

★ 02단원 개념03의 '일꾼 한 명'이 여기서 반대로 문제가 됩니다. 일꾼이 한 명이라 동시에 두 줄이 실행되지는 않습니다. 그런데 await 에서 일을 놓고 다른 손님에게 갑니다. 놓았다가 다시 잡는 사이에 세상이 바뀌어 있는 것입니다.

★ "Node 는 싱글 스레드라 안전하다" 는 말은 절반만 맞습니다. '한 줄이 실행되는 도중'에는 안전합니다. await 를 만나는 순간 안전하지 않습니다.

해결 ① 줄 세우기

섹션 4

동시에 하지 못하게 막습니다. 한 번에 한 명씩 차례로 하게 합니다.

```
let 대기줄 = Promise.resolve();
```

```
function 줄서서(작업) {
```

앞사람이 끝난 뒤에 시작하도록 이어 붙입니다.

```
const 다음 = 대기줄.then(작업, 작업);
```

앞사람이 실패해도 줄이 끊기면 안 되니 에러를 삼킨 것을 대기줄로 삼습니다.

```
대기줄 = 다음.catch(() => {});
```

```
return 다음;
```

```
}
```

★ 이 여섯 줄이 하는 일 대기줄은 “지금까지 걸린 일들이 끝나는 시점”을 가리키는 약속입니다. 새 작업이 오면 그 뒤에 붙입니다. 자연스럽게 한 줄로 세워집니다.

★ `.then(작업, 작업)`에 작업을 두 번 쓴 이유 첫 번째는 성공했을 때, 두 번째는 실패했을 때입니다.

앞사람이 실패했다고 내 차례가 안 오면 안 되니 둘 다 넣었습니다.

```
app.post("/race-queue", async (req, res) => {
  await 쓰기([]);

  await Promise.all(
    Array.from({ length: 20 }, (_, i) => 줄서서(() => 그냥추가({ name: `설비${i}` })))
  );

  const 결과 = await 읽기();

  res.json({
    시도: 20,
    실제저장됨: 결과.length,
    사라진것: 20 - 결과.length,
  });
});
```

```
POST /race-queue
```

```
200 {"시도":20,"실제저장됨":20,"사라진것":0}
```

★ 20건이 전부 저장됐습니다.

달라진 것은 딱 하나입니다. 동시에 안 하고 차례로 했습니다.

★ 대가가 있습니다 20명이 줄을 서니 마지막 사람은 19명을 기다립니다. 설비가 10만 개라면 한 명당 파일 전체를 읽고 쓰는 시간을 기다립니다. 사람이 늘수록 점점 느려집니다.

그래도 데이터가 사라지는 것보다는 낫습니다. “느린 것” 과 “틀린 것” 중에는 언제나 느린 쪽을 고릅니다.

해결 ② 쓰다가 죽어도 안 깨지게

섹션 5

섹션 2-2 의 ‘찢긴 읽기’ 를 제대로 고칩니다. 그리고 더 큰 문제 하나도 함께 해결됩니다.

writeFile 은 파일을 먼저 비우고 새 내용을 씁니다. 비운 직후에 서버가 죽거나 전원이 나가면? 파일이 반쯤 쓰인 채로 남습니다. JSON 이 깨져서 아예 못 읽습니다. 데이터 전체가 날아갑니다. 다시 켜도 복구가 안 됩니다.

찢긴 읽기는 ‘가끔 한 요청이 실패’ 하는 것이지만, 이걸 ‘데이터 전체가 사라지는’ 사고입니다.

그래서 이렇게 씁니다.

① 임시 파일에 전부 쓴다 ② 다 쓰이면 원래 이름으로 바꾼다 (rename)

이름 바꾸기는 ‘한순간에’ 일어납니다. 반쯤 바뀌는 일이 없습니다. 그래서 파일은 언제나 ‘옛날 것 전체’ 이거나 ‘새것 전체’ 입니다.

```
async function 안전하게쓰기(목록) {
  const 임시경로 = 파일경로 + ".tmp";

  await fs.promises.writeFile(임시경로, JSON.stringify(목록, null, 2), "utf-8");
  await fs.promises.rename(임시경로, 파일경로);
}

async function 안전추가(값) {
  const 목록 = await 읽기();
  const id = 목록.length === 0 ? 1 : Math.max(...목록.map((x) => x.id)) + 1;
  목록.push({ id, ...값 });
  await 안전하게쓰기(목록);
}

app.post("/safe", async (req, res) => {
  await 안전하게쓰기([]);

  await Promise.all(
    Array.from({ length: 20 }, (_, i) => 줄서서(() => 안전추가({ name: `설비${i}` })))
  );

  const 결과 = await 읽기();
  const 임시파일남았나 = fs.existsSync(파일경로 + ".tmp");
```

```
res.json({ 저장됨: 결과.length, 임시파일남았나 });
});
```

POST /safe

```
200 {"저장됨":20,"임시파일남았나":false}
```

★ 임시 파일이 안 남았습니다. rename 이 옮겨 갔기 때문입니다.

★ 이 방법을 원자적 쓰기(atomic write)라고 합니다. '원자' 는 더 쪼갤 수 없다는 뜻입니다. 반쯤 되는 일이 없습니다.

★ 주의 — 임시 파일은 같은 디스크에 두세요 이름 바꾸기는 같은 디스크(파일시스템) 안에서만 됩니다. 다른 디스크로 rename 하면 EXDEV 에러가 나며 실패합니다. (mv 명령이 다른 디스크로도 되는 것은, 그때 복사 + 삭제로 대신해 주기 때문입니다. rename 자체가 그러는 게 아닙니다) 그래서 파일경로 + ".tmp" 처럼 바로 옆에 만듭니다.

그래도 남는 문제

섹션 6

```
app.get("/limits", (req, res) => {
  res.json({
    남는문제: [
      "서버를 두 대로 늘리면 줄 세우기가 소용없습니다. 각 서버가 자기 줄만 압니다.",
      "PM2 로 프로세스를 4개 띄워도 마찬가지입니다. (PART 4)",
      "10만 건이면 하나 고치려고 10만 건을 읽고 씁니다.",
      "'A라인 가동 중인 것만' 을 찾으려면 전부 읽어서 훑어야 합니다.",
      "중간에 실패하면 되돌릴 방법이 없습니다.",
      "누가 언제 무엇을 바꿨는지 알 수 없습니다.",
    ],
    답: "데이터베이스",
  });
});
```

GET /limits

```
200 {"남는문제":["서버를 두 대로 늘리면 줄 세우기가 소용없습니다. 각 서버가 자기 줄만 압니다.", "PM2 로 프로세스를 4개 띄워도 마찬가지입니다. (PART 4)", "10만 건이면 하나 고치려고 10만 건을 읽고 씁니다.", "'A라인 가동 중인 것만' 을 찾으려면 전부 읽어서 훑어야 합니다.", "중간에 실패하면 되돌릴 방법이 없습니다.", "누가 언제 무엇을 바꿨는지 알 수 없습니다."], "답":"데이터베이스"}
```

★★ 첫 줄이 결정적입니다.

줄 세우기는 '한 프로세스 안에서만' 통합니다. 서버를 두 대 띄우거나, PM2 로 프로세스를 4개 돌리면 각자 자기 대기줄만 알고 있어서 다시 덮어쓰기가 일어납니다.

그런데 서버를 여러 대 띄우는 건 트래픽이 늘면 반드시 하게 됩니다. 즉, 파일 저장은 '서비스가 잘되면 반드시 깨집니다'.

★ 데이터베이스는 이걸 어떻게 푸나 여러 서버가 붙어도 데이터베이스는 한 곳입니다. 그리고 '트랜잭션' 과 '락' 이라는 장치가 안에 들어 있습니다. 우리가 만든 줄 세우기를, 훨씬 똑똑하게 (필요한 줄만 잠그며) 해 줍니다. PART 4 에서 배웁니다.

```
app.use((req, res) => {
  res.status(404).json({ error: "그런 주소가 없습니다" });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  res.status(500).json({ error: "서버에서 문제가 생겼습니다" });
});

app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. POST http://localhost:${PORT}/race 로 시험해 보세요`);
});
```

정리

문제 해결 남는 한계

동시에 쓰면 덮어쓰 줄 세우기 프로세스가 여러 개면 소용없음 쓰다가 죽으면 깨짐 임시 파일 + rename -
전부 읽고 전부 씀 - 데이터가 늘면 답이 없음 조건 검색이 느림 - 전부 훑어야 함 되돌릴 수 없음 -
트랜잭션이 필요함

★ 파일 저장을 써도 되는 경우 · 쓰는 사람이 사실상 한 명 (관리자 도구, 개인 프로젝트) · 데이터가 적고 잘 안 바뀜 (설정 파일, 목록 데이터) · 읽기가 대부분이고 쓰기는 아주 가끔

★ 절대 쓰면 안 되는 경우 · 여러 사람이 동시에 쓴다 · 돈이나 재고처럼 틀리면 안 되는 값 · 서버를 여러 대 띄울 계획이 있다

직접 해 볼 것

▫ 직접 해보기

1

POST /race 를 여러 번 눌러 보세요. 매번 1건인가요? 20 을 100 으로 늘리면 어떻게 되나요?

▫ 직접 해보기

2

섹션 2의 Promise.all 을 for 문 + await 로 바꿔 보세요.

```
for (let i = 0; i < 20; i++) await 그냥추가({ name: `설비${i}` });
```

몇 건이 저장되나요? 왜 그럴까요?

(힌트: `for` + `await` 는 하나씩 차례로 합니다. 줄 세우기와 같습니다)

▹ 직접 해보기 3

그냥추가 에서 읽기와 쓰기 사이에 일부러 시간을 넣어 보세요.

```
await new Promise((r) => setTimeout(r, 50));
```

/race-queue 는 얼마나 느려지나요?

줄 세우기의 대가를 눈으로 확인하세요.

▹ 직접 해보기 4

data/동시성.json 을 열어 놓고 POST /race 를 눌러 보세요. 내용이 어떻게 바뀌는지 보세요.

▹ 직접 해보기 5

줄서서 의 .then(작업, 작업) 에서 두 번째 작업을 지워 보세요. 그리고 그냥추가 가 가끔 실패하도록 만들어 보세요.

```
if (Math.random() < 0.2) throw new Error("일부러 실패");
```

한 번 실패한 뒤로 줄이 어떻게 되나요?

(확인 후 되돌리세요)

▹ 직접 해보기 6

안전하게쓰기 에서 rename 줄을 지우고 임시 파일만 쓰게 해 보세요. /safe 를 부르면 저장됨 이 몇 건인가요? 왜 그럴까요?

▹ 직접 해보기 7

07단원의 repositories/설비저장소.js 에 줄 세우기를 넣어 보세요. 그게 이 단원의 마무리입니다.

자주 하는 실수

이런 실수를 합니다

“Node 는 싱글 스레드라 안전하다” 고 믿음

await 를 만나면 다른 요청이 끼어듭니다. 안전하지 않습니다.

이런 실수를 합니다**Postman 으로 하나씩 눌러 보고 “잘 된다” 고 판단**

혼자 천천히 누르면 절대 안 겹칩니다. 동시에 눌러 봐야 합니다.

이런 실수를 합니다**에러가 없으니 잘 됐다고 생각**

잃어버린 갱신은 에러를 내지 않습니다. 조용히 사라집니다. 그래서 며칠 뒤에 “데이터가 이상해요” 라는 문의로 알게 됩니다.

이런 실수를 합니다**writeFile 하나면 안전하다고 생각**

쓰는 도중에 죽으면 파일이 깨집니다. 임시 파일 + rename 을 쓰세요.

이런 실수를 합니다**줄 세우기를 해 놓고 서버를 여러 대 띄움**

각 서버가 자기 줄만 압니다. 다시 덮어쓰기가 일어납니다.

이런 실수를 합니다**대기줄에 에러 처리를 안 함**

앞사람이 실패하면 뒤로 줄이 통째로 끊깁니다. 그 뒤의 모든 요청이 영원히 멈춥니다.

정리

- 1 읽고 → 고치고 → 쓰기 사이에 다른 요청이 끼어들면 먼저 쓴 것이 사라진다. 에러가 하나도 안 나기 때문에 눈으로는 절대 못 잡는다.
- 2 “Node 는 싱글 스레드라 안전하다” 는 틀렸다. await 에서 다른 요청으로 넘어가므로 그 틈이 그대로 사고 자리다.
- 3 Postman 으로 하나씩 눌러 보면 언제나 잘 된다. 동시에 보내 봐야 드러난다.
- 4 해결 ① 줄 세우기 — 읽고 쓰는 일을 한 번에 하나씩만 하도록 대기줄을 만든다.
- 5 해결 ② 임시 파일에 다 쓴 뒤 이름을 바꾼다. 쓰다가 죽어도 원본이 반쯤 망가진 채로 남지 않는다.
- 6 writeFile 하나만으로는 안전하지 않다. 문제는 쓰는 순간이 아니라 읽고 쓰기 사이이다.
- 7 대기줄에도 에러 처리를 해 둔다. 안 하면 하나가 터질 때 뒤가 전부 멈춘다.
- 8 줄 세우기는 서버가 한 대일 때만 듣는다. 여러 대로 늘리면 다시 사라진다. 거기서부터는 데이터베이스가 할 일이다.

저장소 완성하기, 그리고 왜 데이터베이스인가

RUN node 개념05_저장소_완성하고_다음으로.js

그러면 Ctrl + C

개념04 에서 배운 두 가지를 저장소에 넣었습니다. repositories/안전한설비저장소.js 를 열어 보세요.

① 쓰는 작업을 줄 세운다 ② 임시 파일에 쓰고 이름을 바꾼다

그리고 부르는 쪽은 한 글자도 안 고쳤습니다. 그게 이 단원의 성과입니다.

```
const express = require("express");
const 저장소 = require("./repositories/안전한설비저장소");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
```

부르는 쪽은 그대로입니다

섹션 1

```
app.get("/equipments", async (req, res) => {
  res.json({ data: await 저장소.전부() });
});

app.post("/equipments", async (req, res) => {
  const { name, line } = req.body || {};

  if (!name || !line) {
    return res.status(400).json({ error: "name 과 line 이 필요합니다" });
  }

  const 새설비 = await 저장소.추가({ name, line, status: "정지" });

  res.status(201).json({ data: 새설비 });
});
```

POST /equipments

```
{ "name": "용접로봇 1호", "line": "C" }
```

201 { "data": { "id": 3, "name": "용접로봇 1호", "line": "C", "status": "정지" } }

개념02 의 라우트와 완전히 같습니다. require 하는 파일 이름만 바뀌었습니다.

이제 동시에 써도 안 사라집니다

섹션 2

```
app.post("/race", async (req, res) => {
  await 저장소.초기화([]);

  await Promise.all(
    Array.from({ length: 20 }, (_, i) => 저장소.추가({ name: `설비${i}`, line: "A"
  }))
);

  const 결과 = await 저장소.전부();
```

번호가 겹치지 않았는지도 확인합니다.

```
const 번호들 = 결과.map((설비) => 설비.id);
const 겹친번호 = 번호들.length !== new Set(번호들).size;

res.json({ 시도: 20, 저장됨: 결과.length, 번호가겹쳤나: 겹친번호 });
});
```

```
POST /race
200 {"시도":20,"저장됨":20,"번호가겹쳤나":false}
```

★ 개념04 에서는 20건 중 1건만 남았습니다. 이제 20건 전부 남습니다.

★ 번호도 안 겹쳤습니다. 줄을 안 세우면 20개 요청이 전부 '최댓값 1' 을 읽어서 전부 id 2 를 붙이려 합니다. 번호가 겹치면 더 큰 사고가 납니다.

★ new Set(배열).size 는 중복을 뺀 개수입니다. (JS자료 06단원) 개수가 다르면 어딘가 겹쳤다는 뜻입니다.

```
app.get("/still-broken", (req, res) => {
  res.json({
    프로세스가여러개면: "줄 세우기는 이 프로세스 안에서만 통합니다. PM2 로 4개 띄우면
    다시 깨집니다.",
    데이터가 많으면: "하나 고치려고 10만 건을 전부 읽고 전부 다시 씁니다.",
    조건검색: "'A라인 가동 중' 을 찾으려면 전부 읽어서 하나씩 훑어야 합니다.",
    중간실패: "설비를 만들고 점검기록도 만들어야 하는데, 두 번째에서 실패하면 되돌릴
    수 없습니다.",
    변경이력: "누가 언제 무엇을 바꿨는지 알 방법이 없습니다.",
    동시에읽고쓰기: "한 명이 쓰는 동안 다른 사람은 기다립니다. 읽기만 하는
    사람도요.",
  });
});
```

GET /still-broken

```
200 {"프로세스가여러개면":"줄 세우기는 이 프로세스 안에서만 통합니다. PM2 로 4개 띄
우면 다시 깨집니다.", "데이터가 많으면":"하나 고치려고 10만 건을 전부 읽고 전부 다
시 씁니다.", "조건검색":"'A라인 가동 중' 을 찾으려면 전부 읽어서 하나씩 훑어야 합
니다.", "중간실패":"설비를 만들고 점검기록도 만들어야 하는데, 두 번째에서 실패하
면 되돌릴 수 없습니다.", "변경이력":"누가 언제 무엇을 바꿨는지 알 방법이 없습니
다.", "동시에읽고쓰기":"한 명이 쓰는 동안 다른 사람은 기다립니다. 읽기만 하는 사
람도요."}
```

★ '중간실패' 를 좀 더 봅시다.

설비를 등록하면서 첫 점검기록도 함께 만들어야 한다고 합니다.

```
await 설비저장소.추가(설비);      ← 성공
await 기록저장소.추가(첫기록);   ← 여기서 디스크가 꽉 참
```

설비는 만들어졌는데 기록이 없습니다. 데이터가 어긋난 상태로 남습니다. 되돌리려면 방금 만든 설비를 지워야 하는데, 그 코드를 우리가 직접 써야 합니다. 그러다 지우는 도중에 또 실패하면? 끝이 없습니다.

데이터베이스에는 '트랜잭션' 이 있습니다. "둘 다 되거나 둘 다 안 되거나" 를 데이터베이스가 보장해 줍니다.

데이터베이스가 해 주는 일

섹션 4

```
app.get("/why-database", (req, res) => {
  res.json({
    비교: [
      { 항목: "여러 서버가 동시에", 파일: "깨짐", 데이터베이스: "정상" },
      { 항목: "조건으로 찾기", 파일: "전부 흠", 데이터베이스: "색인으로 바로 찾음" },
    ],
    { 항목: "하나만 고치기", 파일: "전부 다시 씀", 데이터베이스: "그 줄만 고침" },
    { 항목: "여러 작업 묶기", 파일: "불가능", 데이터베이스: "트랜잭션" },
    { 항목: "동시에 읽기", 파일: "줄 서야 함", 데이터베이스: "동시에 됨" },
    { 항목: "10만 건", 파일: "느려서 못 씀", 데이터베이스: "문제없음" },
  ],
});
});
```

GET /why-database

```
200 {"비교":[{"항목":"여러 서버가 동시에","파일":"깨짐","데이터베이스":"정상"},{"항목":"조건으로 찾기","파일":"전부 흠","데이터베이스":"색인으로 바로 찾음"},{"항목":"하나만 고치기","파일":"전부 다시 씀","데이터베이스":"그 줄만 고침"},{"항목":"여러 작업 묶기","파일":"불가능","데이터베이스":"트랜잭션"},{"항목":"동시에 읽기","파일":"줄 서야 함","데이터베이스":"동시에 됨"},{"항목":"10만 건","파일":"느려서 못 씀","데이터베이스":"문제없음"}]}
```

07

바꿀 때 무엇을 고치나

섹션 5

```
app.get("/migration-plan", (req, res) => {
  res.json({
    고칠파일: ["repositories/설비저장소.js"],
    안고칠파일: [
      "routes/설비라우트.js",
      "controllers/설비컨트롤러.js",
      "services/설비서비스.js",
      "utils/AppError.js",
    ],
    설명: "저장 방법이 바뀌어도 주소·업무 규칙·응답 형식은 그대로입니다.",
  });
});
```

GET /migration-plan

```
200 {"고칠파일":["repositories/설비저장소.js"],"안고칠파일":["routes/설비라우트.js","controllers/설비컨트롤러.js","services/설비서비스.js","utils/AppError.js"],"설명":"저장 방법이 바뀌어도 주소·업무 규칙·응답 형식은 그대로입니다."}
```

★★ 이게 개념03 에서 층을 나눈 진짜 이유입니다.

전부() 안이 이렇게 바뀔 뿐입니다.

```
지금      return JSON.parse(await fs.promises.readFile(파일경로, "utf-8"));
PART 4 return db.query("SELECT * FROM equipments");
```

서비스는 여전히 "전부() 를 부르면 배열이 온다" 만 알면 됩니다. 컨트롤러는 서비스만 압니다. 라우트는 컨트롤러만 압니다.

한 층만 갈아 끼우고 나머지는 그대로 — 그게 목표였습니다.

준비하고 문 열기

섹션 6

```
const 처음데이터 = [
  { id: 1, name: "컨베이어 1호", line: "A", status: "가동" },
  { id: 2, name: "프레스 1호", line: "B", status: "정지" },
];

async function 시작하기() {
  await 저장소.초기화(처음데이터);

  app.listen(PORT, () => {
    console.log(`서버가 켜졌습니다. http://localhost:${PORT}/equipments`);
  });
}

시작하기().catch((에러) => {
  console.error("서버를 켜지 못했습니다:", 에러);
  process.exit(1);
});
```

07단원에서 한 일

개념01 메모리 → 파일. 꺾다 쳐도 남습니다

그리고 서버에서 동기 파일 읽기를 쓰면 안 되는 이유를 재 봤습니다

개념02 파일 다루는 코드를 저장소 한 곳에 모았습니다

저장소는 HTTP 를 모릅니다. 못 찾으면 null 을 줄 뿐입니다

개념03 네 층으로 나눴습니다

routes → controllers → services → repositories
업무 규칙 다섯 개가 services 한 곳에 모였습니다

개념04 동시에 쓰면 20건 중 19건이 사라지는 것을 재 봤습니다

줄 세우기와 원자적 쓰기로 막았습니다

개념05 저장소에 그 둘을 넣었습니다. 부르는 쪽은 안 고쳤습니다

★ 파일 저장은 임시 답입니다. 그런데 여기서 배운 것 중 버릴 게 하나도 없습니다.

- 층을 나누는 법
- 저장소가 HTTP 를 모르게 하는 법
- 동시에 쓰면 왜 깨지는가
- 왜 트랜잭션이 필요한가

데이터베이스를 배울 때 이 경험이 전부 쓰입니다. “왜 이런 게 있지?” 가 아니라 “아, 그거 때문이구나” 가 됩니다.

직접 해 볼 것

▫ 직접 해보기 1

POST /race 를 여러 번 눌러 보세요. 항상 20건인가요? 개념04 의 POST /race 와 비교해 보세요.

▫ 직접 해보기 2

안전한설비저장소.js 의 추가 함수에서 줄서서() 를 벗겨 보세요.

```
return (async () => { ... })();
```

POST /race 가 몇 건이 되나요? (확인 후 되돌리세요)

▫ 직접 해보기 3

줄서서() 안에서 읽기만 감싸 보세요.

```
const 목록 = await 줄서서(() => 읽기());
```

... 나머지는 밖에서

이래도 될까요? 왜 안 될까요?

▫ 직접 해보기 4

전부() 에도 줄서서() 를 붙여 보세요. 동작은 합니다. 무엇이 나빠질까요? (힌트: 읽기만 하는 100명이 쓰기 한 명을 기다리게 됩니다)

▹ 직접 해보기 5

개념03의 개념03_계층_나누기.js에서 require를 안전한설비저장소로 바꿔 보세요. 다른 파일을 고쳐야 하나요?

(설비서비스.js의 require 한 줄만 고치면 됩니다.
컨트롤러·라우트는 손댈 게 없습니다)

▹ 직접 해보기 6

07단원 전체를 다시 훑으면서, 각 파일이 '무엇을 알고 무엇을 모르는지'를 한 줄로 적어 보세요.

자주 하는 실수

이런 실수를 합니다

읽기와 쓰기를 따로 줄 세움

그 사이에 다른 요청이 끼어듭니다. 아무 소용이 없습니다. '읽고-고치고-쓰기'를 한 덩어리로 감싸세요.

이런 실수를 합니다

읽기만 하는 함수까지 줄 세움

틀리지는 않는데 괜히 느려집니다. 읽기는 겹쳐도 됩니다.

이런 실수를 합니다

저장소를 바꾸면서 부르는 쪽도 함께 고침

함수 이름과 돌려주는 모양이 같으면 안 고쳐도 됩니다. 고쳐야 한다면 저장소의 경계가 잘못 그려진 것입니다.

이런 실수를 합니다

파일 저장으로 실제 서비스를 시작함

사람이 늘면 반드시 깨집니다. 그때는 이미 데이터가 망가진 뒤입니다.

이런 실수를 합니다

"우리는 사용자가 적으니 괜찮다"고 생각

두 명이 같은 순간에 저장 버튼을 누르면 그걸로 충분합니다. 확률의 문제일 뿐 언젠가는 일어납니다.

정리

- 1 저장소 안을 통째로 바꿔도 부르는 쪽은 한 줄도 안 고친다. 층을 나눠 둔 값어치가 여기서 나온다.
- 2 읽기와 쓰기는 같은 대기줄에 세운다. 따로 세우면 그 사이로 다시 끼어든다.
- 3 읽기만 하는 함수까지 줄 세우면 느려지지만 한다. 고치는 일만 세운다.
- 4 이제 동시에 써도 안 사라진다. 다만 파일 저장으로 못 하는 것이 아직 많다. 찾기·정렬이 느리고, 서버를 여러 대로 못 늘리고, 부분 실패를 못 되돌린다.
- 5 그 일들을 데이터베이스가 해 준다. PART 4 에서 SQL 로 옮긴다.
- 6 옮길 때 고치는 것은 저장소 파일뿐이다. 컨트롤러와 서비스는 그대로다.
- 7 파일 저장으로 실제 서비스를 시작하지 않는다. “우리는 사용자가 적으니 괜찮다” 가 가장 흔한 착각이다. 사용자가 적어도 요청은 겹친다.

저장과 계층

RUN node 연습문제.js

★ 이 파일은 Express 를 한 줄도 안 씁니다. 그런데도 저장소와 서비스를 전부 시험할 수 있습니다. 왜 가능한지 생각하면서 푸세요.

★ 문제 1·2·3·4·5·7·9·10·11 은 출력이 있습니다. 전부 async 함수 `main()` 안에서 이 순서로 실행하세요. 순서가 바뀌면 출력 순서도 바뀝니다.

```
const 저장소 = require("./repositories/설비저장소");
const 서비스 = require("./services/설비서비스");
```

문제 1 저장소를 서버 없이 시험하기

저장소를 아래 두 건으로 초기화한 뒤, 순서대로 출력하세요.

```
[{ id: 1, name: "컨베이어 1호", line: "A", status: "가동" }, { id: 2, name: "프레스 1호", line: "B", status: "정지" }]
```

출력할 것 (이 순서로) ① 전부() 의 개수 ② 하나(1) 의 name ③ 하나(99) ④ 삭제(99) ⑤ 삭제(2) ⑥ 전부() 의 id 배열 ⑦ 추가({ name: "용접로봇 1호", line: "C", status: "정지" }) 의 id

```
기대 출력    2
              컨베이어 1호
              null
              false
              true
```

— 1

2

★★ ⑦이 2 입니다. 방금 지운 번호가 다시 나왔습니다. 왜 그럴까요? 이게 왜 위험한지 설명할 수 있어야 합니다. (힌트: 프레스 1호의 점검기록에 `equipmentId: 2` 가 남아 있다면?)

여기에 코드를 쓰세요

문제 2 서비스의 업무 규칙 시험하기

저장소를 아래 세 건으로 다시 초기화한 뒤, 규칙이 지켜지는지 확인하세요.

1 컨베이어 1호 / A / 가동 2 프레스 1호 / B / 정지 3 검사기 1호 / A / 점검중

확인할 것 (이 순서로) ① 서비스.삭제(1) → 가동 중이라 안 됩니다. status 와 code 출력 ② 서비스.삭제(2) 뒤 전부()의 id 배열 ③ 서비스.상태바꾸기(3, "가동") → 점검중이라 안 됩니다. status 와 code 출력 ④ 정지를 거친 뒤 가동으로 바꾸고 status 출력 ⑤ 서비스.하나(99) → status, code, message 출력 ⑥ 서비스.등록({ name: "컨베이어 1호", line: "B" }) → 이름 중복 ⑦ 서비스.등록({ name: "포장기 1호", line: "C", status: "가동" }) 의 status

힌트: 에러가 나는 것은 try/catch 로 감싸고 catch 에서 출력하세요.

```
console.log(`${에러.status} ${에러.code}`)
```

기대 출력 409 CONFLICT

— 1, 3

409 CONFLICT 가동 404 NOT_FOUND 설비를 찾을 수 없습니다 409 DUPLICATED 정지

★★ ⑦을 보세요. status: "가동" 을 보냈는데 "정지" 가 나옵니다.

★★ 그리고 이 문제 전체를 보세요. Express 를 한 줄도 안 켜고 업무 규칙 여섯 가지를 확인했습니다. 서비스에 res.status(409) 를 한 줄만 넣었어도 이게 불가능해집니다. 왜일까요?

여기에 코드를 쓰세요

문제 3 다음 번호 붙이기

두 가지 방법을 만들어 비교하세요.

다음번호_틀린방법(목록) → 목록.length + 1

다음번호_맞는방법(목록) → 지금 있는 id 중 최댓값 + 1 (빈 배열이면 1)

힌트: `Math.max(...목록.map((x) => x.id))` 입니다.

`Math.max([1,2,3])` 은 `NaN` 입니다. 배열을 펼쳐서 넘겨야 합니다.
`Math.max()` 는 인자가 없으면 `-Infinity` 입니다. 빈 배열을 따로 처리하세요.

시험할 것 (이 순서로, 두 방법을 한 줄에 나란히)

— `{id:1},{id:2},{id:3}`

`{id:1},{id:3}` · ← 2번을 지운 뒤

[]

```
기대 출력    4 4
              3 4
              1 1
```

(두 번째 줄이 이 문제의 전부입니다. 무엇이 문제인가요?)

여기에 코드를 쓰세요

문제 4 무엇을 어디에 두나

아래 열 가지 일은 어느 층이 해야 할까요? `routes` · `controllers` · `services` · `repositories` · 에러 처리기
중에서 고르세요.

```
기대 출력    주소를 /api/v1/equipments 로 정한다 → routes
              req.params.id 를 숫자로 바꾼다 → controllers
              본문이 안 왔을 때 터지지 않게 || {} 를 붙인다 → controllers
              이름은 2글자 이상이어야 한다 → services
              가동 중인 설비는 못 지운다 → services
              같은 이름의 설비는 못 만든다 → services
              id 는 지금 최댓값 + 1 로 붙인다 → repositories
              파일이 없으면 빈 배열을 돌려준다 → repositories
              쓰기를 줄 세운다 → repositories
              404 를 어떤 JSON 모양으로 내보낼지 정한다 → 에러 처리기
```

★ 헷갈리는 두 개를 미리 짚습니다. “|| {} 는 검증 아닌가?” → “본문이 안 왔다” 는 HTTP 사정입니다 “id 붙이기는 업무 규칙 아닌가?” → 데이터베이스로 바꾸면 DB 가 붙여 줍니다

★ 이 블록을 `main()` 밖에 두면 출력이 맨 앞으로 갑니다. 왜일까요?

여기에 코드를 쓰세요

문제 5 줄 세우기가 순서를 어떻게 바꾸나

아래 함수를 만들고, 두 방식으로 실행해 순서를 비교하세요.

```
async function 일하기(이름, 걸리는시간) {
```

```
  순서.push(`${이름} 시작`);
  await (걸리는시간 만큼 기다리기);
  순서.push(`${이름} 끝`);
```

```
}
```

① 그냥 동시에 Promise.all([일하기("A", 30), 일하기("B", 10)]) ② 줄 세워서 Promise.all([줄서서(() ⇒ 일하기("A", 30)), 줄서서(() ⇒ 일하기("B", 10))])

줄서서 함수도 직접 만드세요. (개념04 섹션 4)

```
let 대기줄 = Promise.resolve();
function 줄서서(작업) { ... }
```

각각 순서 배열을 " / " 로 이어서 출력하세요. 두 번째 실험 전에 순서.length = 0 으로 비우세요.

```
기대 출력  A 시작 / B 시작 / B 끝 / A 끝
           A 시작 / A 끝 / B 시작 / B 끝
```

★ 위쪽에서 A 와 B 가 어떻게 겹쳤는지 보세요. '읽고-고치고-쓰기' 가 이렇게 겹치면 데이터가 사라집니다.

★ 아래쪽이 더 느립니다. 얼마나 느린지, 왜 그런지 설명해 보세요.

여기에 코드를 쓰세요

문제 6 원자적 쓰기가 필요한 이유

writeFile 만 쓰면 무슨 일이 생기나 (두 가지): ① _____ ② _____

②는 왜 특히 찾기 어려운가: _____

어떻게 고치나 (두 단계): ① _____ ② _____

왜 이게 안전한가: _____

임시 파일을 어디에 만들어야 하나, 왜 그런가: _____

문제 7 findIndex 함정

아래 배열에서 id 10 과 id 99 의 자리를 찾고, 두 가지 방법으로 “찾았나” 를 확인해 출력하세요.

```
const 설비들 = [{ id: 10 }, { id: 20 }];
```

잘못된 확인: !자리 로 판단 올바른 확인: 자리 === -1 로 판단

```
기대 출력    0 -1
              잘못된 확인 → 못 찾음
              올바른 확인 → 찾음
```

★★ 어떤 버그가 되는지 한 줄로 설명해 보세요. (힌트: 목록에서 ‘몇 번째’ 항목만 수정이 안 될까요?)

★ find 는 어떤가요? find 도 같은 문제가 있나요? 왜 그럴까요?

여기에 코드를 쓰세요

문제 8 파일 저장의 한계

줄 세우기로 해결되는 것: _____

줄 세우기로도 해결이 안 되는 것: _____

그게 왜 치명적인가: _____ (힌트: 서버를 여러 대 띄우는 것은 언제 하게 되나요?)

그 밖에 파일로 못 하는 것 네 가지: ① _____ ② _____
 _____ ③ _____ ④ _____

데이터베이스가 해 주는 것: _____

그래도 07단원에서 배운 것 중 버릴 게 없는 이유: _____

문제 9 저장 경로 만들기

파일 이름을 받아 data 폴더 안의 경로를 돌려주는 저장경로(파일이름) 를 만드세요. 기준은 이 파일이 있는 폴더입니다.

출력할 것 (이 순서로) ① 저장경로("설비.json") 의 뒤 두 칸만 (사람마다 앞부분이 다르니까요) ② 그 경로가 절대 경로인지

힌트: path.join(__dirname, "data", 파일이름)

뒤 두 칸은 .split(path.sep).slice(-2).join("/") 로 잘라서 찍으세요.
 절대 경로인지는 path.isAbsolute 로 봅니다.

기대 출력 data/설비.json
 true

★ "./data/설비.json" 처럼 적으면 무엇이 달라질까요? 에러가 나나요, 조용히 이상해지나요?

main() 안에 쓰세요

문제 10 없는 파일과 진짜 고장 가려내기

파일을 읽다가 난 에러를 받아 무엇인지 알려 주는 읽기결과(에러) 를 만드세요.

에러가 null 이면 → "정상" 에러.code 가 "ENOENT" 면 → "처음 실행 — 빈 목록으로 시작" 그 밖이면 → "진짜 고장 — 코드"

시험할 것 (이 순서로)

null { code: "ENOENT" } { code: "EACCES" } { code: "EISDIR" }

기대 출력	정상
	처음 실행 - 빈 목록으로 시작
	진짜 고장 - EACCES
	진짜 고장 - EISDIR

★ catch 로 전부 [] 를 돌려주면 EACCES(권한 없음)까지 '데이터 없음' 이 됩니다. 그다음에 저장하면 무슨 일이 벌어질지 생각해 보세요.

main() 안에 쓰세요

문제 11 읽고 → 고치고 → 쓰기 한 바퀴

파일 내용(글자)과 고치는 함수를 받아, 저장할 글자를 돌려주는 한바퀴(원본, 고치기) 를 만드세요. 파일은 안 씁니다. 글자만 다룹니다.

① `JSON.parse` 로 읽고 ② 고치기 를 통과시키고 ③ `JSON.stringify` 로 되돌립니다

아래를 넣고, 1번 설비의 status 를 "정지" 로 바꾸세요. [{ id: 1, name: "컨베이어 1호", status: "가동" }]

출력할 것 (이 순서로) ① 바뀐 결과를 다시 parse 해서 [0].status ② 바뀐 결과의 `typeof`

기대 출력	정지
	string

★ ②가 왜 중요한지 생각해 보세요. `JSON.stringify` 를 빼고 객체를 그대로 `writeFile` 에 넘기면 어떻게 되나요?

main() 안에 쓰세요

문제 12 서버에서 Sync 를 쓰면

개념01 섹션7 에서 재 본 것을 정리해 보세요.

`readFileSync` 가 도는 동안 다른 요청은 어떻게 되나: _____

요청이 100건 몰리면 마지막 사람은 얼마나 기다리나: _____

그런데 서버가 처음 켜질 때 설정 파일을 Sync 로 읽는 것은 왜 괜찮나:

"우리 파일은 작아서 금방 읽힌다" 는 왜 위험한 생각인가: _____

주소와 함수를 짝지어 주는 곳

이 파일에는 로직이 한 줄도 없습니다. 연결만 합니다.

```
const express = require("express");
const 설비컨트롤러 = require("../controllers/설비컨트롤러");

const router = express.Router();

router.get("/", 설비컨트롤러.목록);
router.get("/:id", 설비컨트롤러.하나);
router.post("/", 설비컨트롤러.등록);
router.patch("/:id", 설비컨트롤러.상태바꾸기);
router.delete("/:id", 설비컨트롤러.삭제);

module.exports = router;
```

이 파일을 보면 알 수 있는 것

다섯 줄만 보면 이 자원에 무엇이 있는지 한눈에 들어옵니다. 처음 보는 프로젝트에 들어가면 routes 폴더부터 여는 이유가 이것입니다.

★ 미들웨어도 여기서 붙입니다 05단원의 인증·역할확인을 붙인다면 이렇게 됩니다.

```
router.post("/", 인증, 역할확인("admin"), 설비컨트롤러.등록);
router.delete("/:id", 인증, 역할확인("admin"), 설비컨트롤러.삭제);
```

“누가 할 수 있는가”가 이 파일에 한눈에 보입니다. 컨트롤러를 열어 보지 않아도 알 수 있습니다.

★ 괄호를 붙이면 안 됩니다

```
router.get("/", 설비컨트롤러.목록())   x 지금 실행해서 결과를 넘김
router.get("/", 설비컨트롤러.목록)     o 함수 자체를 넘김
```

Express가 요청이 올 때 부를 것이니, 우리는 함수만 건네주면 됩니다. 괄호를 붙이면 서버 켤 때 한 번 실행되고 끝납니다. 그러면 req가 undefined라서 켜자마자 에러가 납니다.

업무 규칙이 있는 곳

“무엇을 해도 되고 무엇을 하면 안 되는가” 를 정하는 층입니다.

★★ 이 파일의 규칙

HTTP 를 모릅니다 req, res 를 쓰지 않습니다 파일을 모릅니다 readFile 을 부르지 않습니다. 저장소에게 시킵니다 업무 규칙만 압니다 “가동 중인 설비는 못 지운다” 같은 것

이 층이 왜 필요한가 저장소는 “시키면 저장한다” 만 합니다. 되는지 안 되는지는 판단하지 않습니다.

컨트롤러는 “요청을 받아 응답한다” 만 합니다. 그럼 “가동 중이면 못 지운다” 는 규칙은 누가 알고 있어야 할까요?

컨트롤러에 두면, 나중에 배치 작업에서 설비를 지울 때 그 규칙이 안 지켜집니다. 저장소에 두면, 저장소가 업무를 알게 되어 다른 곳에서 못 씁니다. 그래서 가운데 층을 하나 둡니다.

```
const 설비저장소 = require("../repositories/설비저장소");
const { 에러 } = require("../utils/AppError");

const 라인값들 = ["A", "B", "C"];
const 상태값들 = ["가동", "정지", "점검중"];

async function 목록() {
  return 설비저장소.전부();
}

async function 하나(id) {
  const 설비 = await 설비저장소.하나(id);
```

★ 저장소는 null 을 줍니다. 여기서 에러로 바꿉니다. 이렇게 해 두면 이 함수를 부르는 쪽은 null 확인을 안 해도 됩니다. “없으면 에러가 날 것” 이라고 믿고 쓰면 됩니다.

```
if (설비 === null) {
  throw 에러.없음("설비");
}

return 설비;
}

async function 등록(값) {
```

규칙 1: 모양이 맞아야 한다

```
const 항목 = [];

if (!값.name) 항목.push({ 키: "name", 이유: "필수입니다" });
else if (값.name.length < 2) 항목.push({ 키: "name", 이유: "2글자 이상이어야 합니다" });

if (!값.line) 항목.push({ 키: "line", 이유: "필수입니다" });
else if (!라인값들.includes(값.line))
  항목.push({ 키: "line", 이유: `${라인값들.join(", ")} 중 하나여야 합니다` });

if (항목.length > 0) {
  throw 에러.검증실패(항목);
}
```

규칙 2: 이름이 겹치면 안 된다

★ 이건 저장소가 못 하는 판단입니다. 저장소는 “이름이 겹치면 안 된다” 는 규칙을 모릅니다. 시키면 저장할 뿐입니다. 그리고 데이터를 봐야 알 수 있으니 미들웨어에서도 못 합니다. 딱 이 층이 할 일입니다.

```
const 전부 = await 설비저장소.전부();

if (전부.some((설비) => 설비.name === 값.name)) {
  throw 에러.중복("설비 이름");
}
```

규칙 3: 새 설비는 언제나 '정지' 로 시작한다

```
return 설비저장소.추가({
  name: 값.name,
  line: 값.line,
  status: "정지", // 보낸 값을 안 씁니다
});

async function 상태바꾸기(id, 새상태) {
  if (!상태값들.includes(새상태)) {
    throw 에러.검증실패([
      { 키: "status", 이유: `${상태값들.join(", ")} 중 하나여야 합니다` },
    ]);
  }
}
```

설비 데이터를 읽고 쓰는 곳

“데이터를 어디에 어떻게 저장하는가” 를 이 파일 하나에 가둡니다.

★★ 이 파일의 규칙 — 여기에는 HTTP 가 없습니다.

req, res 를 쓰지 않습니다 상태코드를 모릅니다 (404 를 내지 않습니다) 못 찾으면 `null` 을 돌려줄 뿐입니다

왜 그럴까요? 이 파일은 “설비 데이터를 다루는 법” 만 압니다. 그걸 웹으로 서비스할지, 명령줄 도구로 만들지, 배치 작업으로 돌릴지는 이 파일이 알 바가 아닙니다.

나중에 “매일 밤 설비 목록을 엑셀로 뽑는 프로그램” 을 만든다면 이 파일을 그대로 가져다 쓸 수 있습니다. Express 없이도요.

```
const fs = require("fs");
const path = require("path");
```

.. 로 한 칸 올라갔다가 data 로 내려갑니다.

```
const 파일경로 = path.join(__dirname, "..", "data", "설비.json");
```

폴더가 없으면 만듭니다.

```
fs.mkdirSync(path.dirname(파일경로), { recursive: true });
```

안에서만 쓰는 함수 (내보내지 않습니다) —————

```
async function 읽기() {
  try {
    const 글자 = await fs.promises.readFile(파일경로, "utf-8");
    return JSON.parse(글자);
  } catch (에러) {
    if (에러.code === "ENOENT") return [];
    throw 에러;
  }
}

async function 쓰기(목록) {
  await fs.promises.writeFile(파일경로, JSON.stringify(목록, null, 2), "utf-8");
}
```

★ 읽기 와 쓰기 는 module.exports 에 없습니다. 밖에서 마음대로 파일을 만지면 이 파일에 가둔 의미가 없어집니다. “안에서만 쓰는 것” 과 “밖에 내주는 것” 을 나누는 것이 중요합니다. (01단원 계산기.js 에서 SECRET_KEY 를 안 내보낸 것과 같은 이야기입니다)

밖에 내주는 것

```
async function 전부() {
  return 읽기();
}

async function 하나(id) {
  const 목록 = await 읽기();
  return 목록.find((설비) => 설비.id === id) ?? null;
}
```

★ 못 찾으면 null 입니다. undefined 가 아닙니다. find 는 undefined 를 주는데, 일부러 null 로 바꿨습니다.

```
undefined "그런 걸 물어본 적 없다" 는 느낌
null "찾아봤는데 없더라" 는 느낌
```

여기서는 분명히 찾아봤습니다. 그래서 null 이 맞습니다. 중요한 건 ‘하나로 정하고 지키는 것’ 입니다.

```
async function 추가(값) {
  const 목록 = await 읽기();

  const 다음번호 = 목록.length === 0 ? 1 : Math.max(...목록.map((설비) => 설비.id)) + 1;
  const 새설비 = { id: 다음번호, ...값 };

  목록.push(새설비);
  await 쓰기(목록);

  return 새설비;
}
```

★ id 를 저장소가 정합니다. 밖에서 id 를 정해서 넘기면 겹칠 수 있습니다. “번호는 저장하는 쪽이 붙인다” 가 규칙입니다.

★ { id: 다음번호, ...값 } 의 순서 id 를 앞에 두면 JSON 에서도 id 가 먼저 나옵니다. 보기 좋습니다. ...값 을 앞에 두면 값 안에 id 가 있을 때 그게 이겨 버립니다. id 를 앞에 두면 뒤엣것이 이기니까... 반대입니다. 주의하세요.

```
{ id: 1, ...{ id: 99 } } → { id: 99 } ← 뒤엣것이 이깁니다
```

확실히 하려면 값에서 id 를 빼고 넘기거나, 아래처럼 씁니다.

```
{ ...값, id: 다음번호 }
```

```
async function 수정(id, 바꿀것) {
  const 목록 = await 읽기();
  const 자리 = 목록.findIndex((설비) => 설비.id === id);

  if (자리 === -1) return null; // 없으면 null
}
```

온 것만 덮어씁니다. id 는 못 바꾸게 마지막에 다시 넣습니다.

```
목록[자리] = { ...목록[자리], ...바꿀것, id };

await 쓰기(목록);
return 목록[자리];
}
```

★ `findIndex` 는 못 찾으면 `-1` 입니다. `undefined` 가 아닙니다. `if (!자리)` 로 쓰면 안 됩니다. `0`번 자리가 `falsy` 라서 걸립니다. 반드시 `=== -1` 로 확인하세요. 아주 흔한 실수입니다.

★ 맨 뒤의 id 가 하는 일 바꿀것 에 id 가 들어 있어도 원래 id 로 되돌립니다. 누가 { id: 999 } 를 보내도 번호가 안 바뀝니다.

```
async function 삭제(id) {
  const 목록 = await 읽기();
  const 남길것 = 목록.filter((설비) => 설비.id !== id);

  if (남길것.length === 목록.length) return false; // 지운 게 없음 await 쓰기(남길것);
  return true;
}
```

★ 개수를 비교해서 “정말 지웠는지” 를 알려 줍니다. 그냥 `true` 를 돌려주면, 없는 걸 지우라고 해도 성공처럼 보입니다. 부르는 쪽이 404 를 낼지 204 를 낼지 정할 수 있어야 합니다.

수업용입니다. 진짜 서버에는 이런 게 있으면 안 됩니다.

```
async function 초기화(목록) {
  await 쓰기(목록);
}

module.exports = { 전부, 하나, 추가, 수정, 삭제, 초기화 };
```

이 파일이 있어서 좋은 점

① 파일 다루는 코드가 여기에만 있습니다 라우트 열 개가 각자 `readFile` 을 부르지 않습니다.

② 나중에 데이터베이스로 바꿀 때 이 파일만 고칩니다 전부() 안이 SELECT * FROM equipments 로 바뀔 뿐, 부르는 쪽은 한 줄도 안 고칩니다. (PART 4)

③ 서버 없이 시험할 수 있습니다

```
const 저장소 = require("./repositories/설비저장소");
저장소.전부().then(console.log);
```

④ 규칙이 한 곳에 모입니다 id 를 어떻게 붙이는지, 없으면 무엇을 돌려주는지가 여기에만 적혀 있습니다.

HTTP 와 업무를 잇는 곳

요청에서 값을 꺼내고, 서비스를 부르고, 응답을 만듭니다.

★★ 이 파일의 규칙

HTTP 를 압니다 req 에서 꺼내고 res 로 보냅니다 업무 규칙은 모릅니다 “가동 중이면 못 지운다” 를 여기 쓰면 안 됩니다 저장을 모릅니다 파일인지 데이터베이스인지 모릅니다

컨트롤러가 하는 일은 딱 세 가지입니다.

① 요청에서 값을 꺼낸다 req.params, req.query, req.body ② 서비스를 부른다 await 서비스.등록(값)

③ 응답을 만든다 상태코드 + { data }

이 세 줄이 넘어가면 뭔가를 잘못 넣은 것입니다.

```
const 설비서비스 = require("../services/설비서비스");
```

목록 보기

```
async function 목록(req, res) {
  const 설비들 = await 설비서비스.목록();

  res.json({ data: 설비들 });
}
```

하나 보기

```
async function 하나(req, res) {
  const 설비 = await 설비서비스.하나(Number(req.params.id));

  res.json({ data: 설비 });
}
```

★ 여기에 if (설비 === null) 이 없습니다. 서비스가 없으면 에러를 던지기 때문입니다. 던져진 에러는 Express 5 가 잡아서 에러 처리기로 보냅니다. (04단원 개념04)

컨트롤러가 “잘된 경우” 만 적으면 되니 아주 짧아집니다.

등록

```

async function 등록(req, res) {
  const { name, line } = req.body || {};

  const 새설비 = await 설비서비스.등록({ name, line });

  res.status(201).location(`/api/v1/equipments/${새설비.id}`).json({ data: 새설비 });
}

```

★ 검증을 여기서 안 합니다. “name 이 2글자 이상” 은 업무 규칙입니다. 서비스가 압니다. 컨트롤러는 값을 꺼내서 넘기기만 합니다.

★ 그런데 || {} 는 여기 있습니다. “본문이 아예 안 왔다” 는 HTTP 사정이지 업무 규칙이 아니기 때문입니다. 이런 게 층을 나눌 때 판단이 필요한 부분입니다.

상태 바꾸기

```

async function 상태바꾸기(req, res) {
  const { status } = req.body || {};

  const 고쳐진것 = await 설비서비스.상태바꾸기(Number(req.params.id), status);

  res.json({ data: 고쳐진것 });
}

```

지우기

```

async function 삭제(req, res) {
  await 설비서비스.삭제(Number(req.params.id));

  res.sendStatus(204);
}

module.exports = { 목록, 하나, 등록, 상태바꾸기, 삭제 };

```

컨트롤러가 짧아진 이유

개념02 의 라우트와 비교해 보세요.

개념02 (섞여 있을 때) 이 파일 (나뉘었을 때)

검증 코드 4줄 없음 (서비스가 함) null 확인 + 404 응답 3줄 없음 (서비스가 던짐) 저장소 부르기 서비스 부르기 응답 만들기 응답 만들기

10~15줄 → 2~3줄

★ 짧아진 게 목적이 아닙니다. “이 파일을 열면 HTTP 이야기만 보인다” 가 목적입니다. 업무 규칙이 궁금하면 서비스를, 저장 방법이 궁금하면 저장소를 엽니다. 찾을 곳이 분명해지는 것이 진짜 이득입니다.

★ `Number(req.params.id)` 는 왜 여기 있나 “주소에서 온 값은 글자다” 는 HTTP 의 사정입니다. 서비스는 숫자를 받는 게 당연하다고 생각합니다. 경계에서 타입을 맞춰 주는 것도 컨트롤러의 일입니다.

줄 세우기 + 원자적 쓰기를 넣은 저장소

개념02 의 설비저장소.js 에 개념04 에서 배운 두 가지를 더했습니다.

- ① 쓰는 작업을 줄 세운다 → 동시에 써도 안 사라짐
- ② 임시 파일에 쓰고 이름을 바꾼다 → 쓰다가 죽어도 안 깨짐

```
const fs = require("fs");
const path = require("path");

const 파일경로 = path.join(__dirname, "..", "data", "설비_안전.json");
fs.mkdirSync(path.dirname(파일경로), { recursive: true });
```

줄 세우기

```
let 대기줄 = Promise.resolve();

function 줄서서(작업) {
  const 다음 = 대기줄.then(작업, 작업);
  대기줄 = 다음.catch(() => {}); // 앞사람이 실패해도 줄은 이어집니다 return 다음;
}
```

★ 읽기만 하는 작업은 줄을 안 세워도 됩니다. 읽기는 아무것도 안 바꾸니 겹쳐도 문제가 없습니다. 줄을 세우면 괜히 느려지기만 합니다.

‘읽고-고치고-쓰기’ 한 덩어리만 줄을 세웁니다.

안에서만 쓰는 함수

```
async function 읽기() {
  try {
    return JSON.parse(await fs.promises.readFile(파일경로, "utf-8"));
  } catch (에러) {
    if (에러.code === "ENOENT") return [];
    throw 에러;
  }
}

async function 쓰기(목록) {
  const 임시경로 = `${파일경로}.tmp`;
```

점검기록을 같은 구조로 추가하기

RUN npx nodemon 연습문제_서버.js

★ 이 문제의 목적 “구조를 나눠 두면 두 번째 자원부터는 쉽다” 를 손으로 겪는 것입니다.

네 개의 파일을 새로 만듭니다. 그중 셋은 거의 복사입니다. 진짜 생각이 필요한 것은 services 하나뿐입니다.

만들 파일

repositories/점검기록저장소.js	← 설비저장소.js 복사 + 이름 바꾸기
services/점검기록서비스.js	← ★ 여기만 새로 생각
controllers/점검기록컨트롤러.js	← 설비컨트롤러.js 복사
routes/점검기록라우트.js	← 설비라우트.js 복사

만들 주소

GET	/api/v1/logs	전체 목록
GET	/api/v1/logs?equipmentId=1	그 설비의 기록만
GET	/api/v1/logs/:id	하나
POST	/api/v1/logs	등록

DELETE /api/v1/logs/:id 삭제

PATCH 는 안 만듭니다. 점검기록은 ‘일어난 사실’ 이라 고치는 게 이상합니다. 잘못 적었으면 지우고 다시 남깁니다.

```
const express = require("express");

const 설비저장소 = require("../repositories/설비저장소");
const 설비라우트 = require("../routes/설비라우트");
const { AppError, 에러 } = require("../utils/AppError");
```

점검기록저장소와 점검기록라우트를 여기서 require 하세요

```
const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());

app.use("/api/v1/equipments", 설비라우트);
```

점검기록라우트를 /api/v1/logs 에 붙이세요

문제 1 repositories/점검기록저장소.js

설비저장소.js 를 복사해서 만드세요. 고칠 것은 세 가지뿐입니다.

① 파일경로를 data/점검기록.json 으로 ② 변수 이름을 기록으로 (설비 → 기록) ③ 함수 하나를 더 만들기: 설비별(equipmentId)

그 설비의 기록만 걸러서 돌려줍니다

내보낼 것: 전부, 설비별, 하나, 추가, 삭제, 초기화

★ 개념05 의 안전한설비저장소.js 처럼 줄 세우기와 원자적 쓰기를 넣으세요. 처음부터 안전하게 만드는 게 낫습니다.

★ 수정 함수는 안 만들어도 됩니다. PATCH 를 안 만들 거니까요.

문제 2 services/점검기록서비스.js ★ 여기가 진짜 문제입니다

아래 다섯 가지 규칙을 이 파일에 담으세요.

① 없는 설비의 기록을 물으면 404 (0건이 아닙니다)

GET /api/v1/logs?equipmentId=99 → 404 "설비를 찾을 수 없습니다"

② 없는 설비에는 기록을 남길 수 없다

POST { equipmentId: 99, ... } → 404

③ result 는 "정상" 또는 "이상", 담당자는 필수

```
POST { result: "이상함" } → 400, details 에 result
POST { 담당자 없이 } → 400, details 에 담당자
```

④ ★★ result 가 "이상" 이면 그 설비의 status 를 "점검중" 으로 바꾼다

⑤ 기록을 지워도 설비 상태는 되돌리지 않는다

★ ①과 ②는 같은 확인을 씁니다. 함수 하나로 빼세요.

```
async function 설비확인(equipmentId) { ... 없으면 throw 에러.없음("설비") }
```

★★ ④를 어떻게 할까요? 점검기록서비스가 설비저장소를 함께 require 하면 됩니다.

```
const 설비저장소 = require("../repositories/설비저장소");
```

서비스가 저장소를 여러 개 봐도 됩니다. 금지된 것은 '아래층이 위층을 보는 것' 뿐입니다. 저장소가 서비스를 require 하면 안 됩니다.

★ ④를 컨트롤러에 쓰면 안 되는 이유를 생각해 보세요. 나중에 관리자 도구나 배치 작업에서 기록을 남기면 어떻게 될까요?

내보낼 것: 목록, 하나, 등록, 삭제

문제 3 controllers/점검기록컨트롤러.js

설비컨트롤러.js 를 복사해서 서비스 이름만 바꾸세요.

★ 규칙을 여기 쓰면 안 됩니다. 한 줄도요. 꺼내고 → 서비스 부르고 → 응답 만들기. 세 줄이면 끝납니다.

★ 목록에서 주의할 것?equipmentId 가 없으면 undefined 를 그대로 넘기세요. Number(undefined) 는 NaN 이라 "전부 달라" 가 "NaN번 설비 달라" 가 됩니다.

```
equipmentId === undefined ? undefined : Number(equipmentId)
```

문제 4 routes/점검기록라우트.js

설비라우트.js 를 복사해서 컨트롤러 이름만 바꾸세요. PATCH 는 빼세요.

문제 5 404 와 에러 처리기

개념03 과 똑같습니다. 자원이 늘어도 에러 처리기는 하나뿐입니다.

404 처리기와 에러 처리기를 여기에 쓰세요

문제 6 준비하고 문 열기

두 저장소를 모두 초기화한 뒤에 listen 하세요.

설비: 1 컨베이어 1호 / A / 가동

2 프레스 1호 / B / 정지

점검기록: 1 equipmentId 1 / 정상 / 김민준

2 equipmentId 2 / 이상 / 이서연

`async function` 시작하기() {

두 저장소를 초기화하세요

```
app.listen(PORT, () => {  
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}/api/v1/logs`);  
});  
}
```

```
시작하기().catch((에러) => {  
  console.error("서버를 켜지 못했습니다:", 에러);  
  process.exit(1);  
});
```

다 만들었으면 이 순서대로 눌러 보세요

```

1 GET    /api/v1/logs                2건
2 GET    /api/v1/logs?equipmentId=1 1건
3 GET    /api/v1/logs?equipmentId=99 404 * 0건이 아닙니다
4 GET    /api/v1/logs/1 1번 기록
5 GET    /api/v1/logs/99 404 "점검기록을" 6 POST /api/v1/logs { equipmentId: 99,
... } 404 7 POST /api/v1/logs { result: "이상함" } 400 8 POST /api/v1/logs {
담당자 없이 } 400 9 POST /api/v1/logs 정상 등록 201, id 3
10 GET   /api/v1/equipments/1          status 가 "가동"
11 POST  /api/v1/logs { result: "이상" } 201, id 4
12 GET   /api/v1/equipments/1          status 가 "점검중" **
13 DELETE /api/v1/logs/4 204
14 GET   /api/v1/equipments/1          여전히 "점검중"

```

★ 3번과 12번이 이 문제의 핵심입니다. 3번: 없는 설비와 기록 없는 설비를 구별했나 12번: 기록을 남겼는데 다른 자원의 상태가 바뀌었나

다 만든 뒤 답해 보세요

① 네 파일 중 '복사해서 이름만 바꾼' 것은 몇 개인가요? 진짜로 새로 생각한 파일은 어느 것인가요?

② 3번(없는 설비의 기록)을 구별하려면 무엇이 필요했나요? 저장소만으로는 왜 안 되나요?

③ 12번 규칙을 컨트롤러에 쓰면 무엇이 문제가 되나요? 프론트엔드에 쓰면요?

④ 기록을 저장한 뒤 설비 수정에서 실패하면 어떻게 되나요? 어떻게 해야 "둘 다 되거나 둘 다 안 되거나"를 보장할 수 있나요? _____

⑤ 저장소 파일 두 개가 거의 같은 코드입니다. 이걸 하나로 묶는 게 좋을까요? 언제 묶는 게 좋을까요?

⑥ 이제 '불량기록' 자원을 하나 더 추가한다면 몇 시간이 걸릴까요? 개념02 처럼 한 파일에 다 넣었다면 어땠을까요? _____

잘 안 될 때 보는 곳

?equipmentId 없이 부르면 빈 배열 → Number(undefined) 가 NaN 이 되어 "NaN번 설비" 를 찾고 있습니다.

?equipmentId=99 가 404 가 아니라 빈 배열 → 서비스에서 설비확인 을 안 했습니다. 저장소는 이걸 못 막습니다.

이상 기록을 남겼는데 설비 상태가 그대로 → 규칙 ④를 안 넣었거나, 설비저장소.수정 에 await 를 빠뜨렸습니다.

Cannot find module './repositories/점검기록저장소' → services 폴더에서는 ../ 로 한 칸 올라가야 합니다.

순환 참조 에러가 남 → 저장소가 서비스를 require 했습니다. 아래층은 위층을 모릅니다.

기록은 남는데 설비 상태가 가끔 안 바뀜 → 두 저장소가 각자 대기줄을 갖고 있어서 순서가 어긋난 것입니다.

이런 게 트랜잭션이 필요한 이유입니다. (개념05 섹션 3)

점검기록라우트

설비라우트.js 를 복사해서 컨트롤러 이름만 바꿨습니다.

```
const express = require("express");
const 점검기록컨트롤러 = require("../controllers/점검기록컨트롤러");

const router = express.Router();

router.get("/", 점검기록컨트롤러.목록);
router.get("/:id", 점검기록컨트롤러.하나);
router.post("/", 점검기록컨트롤러.등록);
router.delete("/:id", 점검기록컨트롤러.삭제);

module.exports = router;
```

★ 설비와 달리 PATCH 가 없습니다. 점검기록은 '일어난 사실' 이라 나중에 고치는 게 이상합니다. 잘못 적었으면 지우고 다시 남깁니다.

자원마다 필요한 메서드가 다릅니다. 다섯 개를 전부 만들 필요는 없습니다.

점검기록의 업무 규칙

네 파일 중 여기만 '새로 생각' 이 필요했습니다.

나머지는 설비 것을 복사해서 이름만 바꿨습니다.

```
const 점검기록저장소 = require("../repositories/점검기록저장소");
const 설비저장소 = require("../repositories/설비저장소");
const { 에러 } = require("../utils/AppError");

const 결과값들 = ["정상", "이상"];
```

★ 서비스가 저장소를 두 개 봅니다. 점검기록을 다루려면 “그 설비가 있는가” 를 알아야 하기 때문입니다. 같은 층끼리(서비스 ↔ 서비스)도, 아래층 여러 개도 볼 수 있습니다. 금지된 것은 아래층이 위층을 보는 것뿐입니다.

```
async function 설비확인(equipmentId) {
  const 설비 = await 설비저장소.하나(equipmentId);

  if (설비 === null) {
    throw 에러.없음("설비");
  }

  return 설비;
}

async function 목록(equipmentId) {
```

equipmentId 가 없으면 전부, 있으면 그 설비 것만

```
if (equipmentId === undefined) {
  return 점검기록저장소.전부();
}
```

규칙 ①: 없는 설비의 기록을 물으면 404

이게 없으면 “그런 설비 없는데요” 대신 “기록이 0건입니다” 가 나옵니다. 요청한 쪽은 설비 번호를 잘못 쓴 줄 모르고 계속 헤맬니다.

```
await 설비확인(equipmentId);
```

07단원 · repositories/점검기록저장소.js

점검기록을 읽고 쓰는 곳

설비저장소.js 를 복사해서 파일 이름과 변수 이름만 바꿨습니다.

구조를 정해 두면 두 번째 자원부터는 이렇게 빨라집니다.

```
const fs = require("fs");
const path = require("path");

const 파일경로 = path.join(__dirname, "..", "data", "점검기록.json");
fs.mkdirSync(path.dirname(파일경로), { recursive: true });

let 대기줄 = Promise.resolve();

function 줄서서(작업) {
  const 다음 = 대기줄.then(작업, 작업);
  대기줄 = 다음.catch(() => {});
  return 다음;
}

async function 읽기() {
  try {
    return JSON.parse(await fs.promises.readFile(파일경로, "utf-8"));
  } catch (에러) {
    if (에러.code === "ENOENT") return [];
    throw 에러;
  }
}

async function 쓰기(목록) {
  const 임시경로 = `${파일경로}.tmp`;
  await fs.promises.writeFile(임시경로, JSON.stringify(목록, null, 2), "utf-8");
  await fs.promises.rename(임시경로, 파일경로);
}

async function 전부() {
  return 읽기();
}

async function 설비별(equipmentId) {
  const 목록 = await 읽기();
  return 목록.filter((기록) => 기록.equipmentId === equipmentId);
}
```

```

}

async function 하나(id) {
  const 목록 = await 읽기();
  return 목록.find((기록) => 기록.id === id) ?? null;
}

async function추가(값) {
  return 줄서서(async () => {
    const 목록 = await 읽기();
    const 다음번호 = 목록.length === 0 ? 1 : Math.max(...목록.map((기록) => 기록.id))
    + 1;
    const 새기록 = { id: 다음번호, ...값 };

    목록.push(새기록);
    await 쓰기(목록);

    return 새기록;
  });
}

async function삭제(id) {
  return 줄서서(async () => {
    const 목록 = await 읽기();
    const 남길것 = 목록.filter((기록) => 기록.id !== id);

    if (남길것.length === 목록.length) return false;

    await 쓰기(남길것);
    return true;
  });
}

async function초기화(목록) {
  return 줄서서(() => 쓰기(목록));
}

module.exports = { 전부, 설비별, 하나, 추가, 삭제, 초기화 };

```

★ 설비저장소에 없던 함수가 하나 있습니다 — 설비별(equipmentId) 자원마다 필요한 조회가 조금씩 다릅니다. 그 차이만 더하고 나머지는 그대로 두면 됩니다.

07단원 · controllers/점검기록컨트롤러.js

점검기록컨트롤러

설비컨트롤러.js 를 복사해서 서비스 이름만 바꿨습니다.

```
const 점검기록서비스 = require("../services/점검기록서비스");

async function 목록(req, res) {
```

?equipmentId=1 이 오면 숫자로 바뀌어서 넘깁니다. 없으면 `undefined` 를 그대로 넘깁니다. "전부 달라" 는 뜻입니다.

```
const { equipmentId } = req.query;

const 기록들 = await 점검기록서비스.목록(
  equipmentId === undefined ? undefined : Number(equipmentId)
);

res.json({ data: 기록들 });
}

async function 하나(req, res) {
  const 기록 = await 점검기록서비스.하나(Number(req.params.id));
  res.json({ data: 기록 });
}

async function 등록(req, res) {
  const { equipmentId, result, 담당자 } = req.body || {};

  const 새기록 = await 점검기록서비스.등록({
    equipmentId: Number(equipmentId),
    result,
    담당자,
  });

  res.status(201).location(`/api/v1/logs/${새기록.id}`).json({ data: 새기록 });
}

async function 삭제(req, res) {
  await 점검기록서비스.삭제(Number(req.params.id));
  res.sendStatus(204);
}
```

```
module.exports = { 목록, 하나, 등록, 삭제 };
```

★ 여기서도 규칙은 하나도 없습니다. "없는 설비면 404" 도, "이상이면 점검중" 도 여기 없습니다. 꺼내고, 서비스를 부르고, 응답을 만드는 것뿐입니다.

★ Number(equipmentId) 가 여기 있는 이유 쿼리에서 온 값은 글자입니다. 그건 HTTP 사정입니다. 서비스는 숫자를 받는 게 당연하다고 생각합니다. 경계에서 타입을 맞춰 주는 것이 컨트롤러의 일입니다.

07단원 · utils/한국어.js

조사 붙이기

06단원 연습문제에서 만든 것을 파일로 뺐습니다.

```
function 받침있나(글자) {
  const 코드 = 글자.charCodeAt(0);
```

완성형 한글은 가(44032) ~ 힉(55203)

```
if (코드 < 44032 || 코드 > 55203) return null;
```

한글 = 44032 + (초성 × 21 + 중성) × 28 + 종성 앞부분이 전부 28의 배수라, 나머지가 곧 종성(받침)입니다.

```
return (코드 - 44032) % 28 !== 0;
}

function 조사(단어, 받침있을때, 받침없을때) {
  const 있나 = 받침있나(단어[단어.length - 1]);

  if (있나 === null) return `${단어}(${받침있을때}/${받침없을때})`;

  return 단어 + (있나 ? 받침있을때 : 받침없을때);
}

module.exports = { 받침있나, 조사 };
```

상태코드와 코드가 붙은 에러

06단원 개념04 에서 만든 것을 파일로 뺐습니다.

```
class AppError extends Error {
  constructor(status, code, message, details) {
    super(message);

    this.name = "AppError";
    this.status = status;
    this.code = code;
    this.details = details;
  }
}

const { 조사 } = require("./한국어");

const 에러 = {
```

★ 06단원에서 만든 조사 함수를 씁니다. "설비을(를)" 이 아니라 "설비를" 로 나옵니다.

```
없음: (무엇) => new AppError(404, "NOT_FOUND", `${조사(무엇, "을", "를")}` 찾을 수 없습니다`),

검증실패: (항목들) => new AppError(400, "VALIDATION_FAILED", "입력값이 올바르지 않습니다", 항목들),

중복: (무엇) => new AppError(409, "DUPLICATED", `이미 있는 ${무엇}입니다`),

상태충돌: (메시지) => new AppError(409, "CONFLICT", 메시지),

주소없음: () => new AppError(404, "ROUTE_NOT_FOUND", "그런 주소가 없습니다"),
};

module.exports = { AppError, 에러 };
```


정적파일과 CORS

OPEN 08_정적파일과_CORS

```
const path = require("path");  
const app = express();  
app.use(express.json());  
res.json({
```

01 정적 파일 서비스하기 (express.static)

02 CORS 란 무엇인가

03 CORS 직접 열어 보기

04 cors 패키지 쓰기

05 프론트와 붙이는 세 가지 방법

- 연습문제

- 서버 연습문제

정적 파일 서비스하기 (express.static)

RUN node 개념01_정적_파일_서비스하기.js

그다음 브라우저에서 `http://localhost:3000` 을 여세요.

그러면 `Ctrl + C`

지금까지 만든 것은 전부 JSON 이었습니다. 그런데 사람이 보는 것은 화면입니다. HTML·CSS·이미지가 필요합니다.

이런 파일들을 '정적 파일' 이라고 부릅니다.

정적(static) 누가 요청하든 똑같은 것을 돌려줌. HTML·CSS·JS·이미지 동적(dynamic) 요청에 따라 달라짐. 우리가 만든 API 응답

정적 파일은 만들 것이 없습니다. 있는 파일을 그대로 보내면 됩니다. 그래서 Express 에 이미 들어 있습니다.

```
const express = require("express");
const path = require("path");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
```

한 줄이면 됩니다

섹션 1

```
app.use(express.static(path.join(__dirname, "public")));
```

이 한 줄이 하는 일

요청이 오면 public 폴더에서 같은 이름의 파일을 찾습니다. 있으면 그 파일을 보내고 끝냅니다. (`next()` 를 안 부릅니다) 없으면 `next()` 를 불러 아래 라우트로 넘깁니다.

GET / → public/index.html GET /css/공통.css → public/css/공통.css GET /js/화면.js → public/js/화면.js GET /api/v1/... → public 에 없으니 아래 라우트로

★ 미들웨어입니다. 05단원에서 배운 그것입니다. "요청이 오면 먼저 여기를 거친다" 는 것이죠. 찾으면 응답하고 끝, 못 찾으면 `next()`. 딱 그 모양입니다.

★ `__dirname` 을 꼭 쓰세요 `express.static("public")` 이라고만 쓰면 터미널 위치 기준이 됩니다. 다른 폴더에서 `node` 로 실행하면 파일을 못 찾습니다. (02단원 개념01)

```
GET /
```

```
200
```

```
GET /css/공통.css
```

```
200
```

```
GET /js/화면.js
```

```
200
```

index.html 은 자동입니다

섹션 2

GET / 로 요청하면 `public/index.html` 을 찾아 줍니다. 아무 폴더나 마찬가지로입니다. `/docs/` 면 `public/docs/index.html` 을 찾습니다.

★ 이걸 아주 오래된 약속입니다. 웹 서버는 폴더를 요청받으면 `index.html` 을 찾습니다. 그래서 홈 화면 파일 이름을 `index.html` 로 짓는 것입니다.

없는 파일은 아래로 넘어갑니다

섹션 3

```
app.get("/api/v1/equipments", (req, res) => {
  res.json({
    data: [
      { id: 1, name: "컨베이어 1호", line: "A", status: "가동" },
      { id: 2, name: "프레스 1호", line: "B", status: "정지" },
    ],
  });
});
```

```
GET /api/v1/equipments
```

```
200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"},{"id":2,"name":"프레스 1호","line":"B","status":"정지"}]}
```

`public` 폴더에 `api` 라는 파일이 없으니 `static` 이 `next()` 를 불렀고, 이 라우트가 걸렸습니다.

★★ 그래서 순서가 중요합니다. `static` 을 라우트보다 아래에 두면? `/css/공통.css` 요청이 라우트를 먼저 훑고 내려옵니다. 결과는 같습니다.

그런데 이런 경우가 문제입니다. `public` 에 `api` 라는 폴더를 만들면, `static` 이 그걸 먼저 찾아서 우리 API 라우트가 영영 실행되지 않습니다.

그래서 API 주소에 /api 접두어를 붙이고, public 안에는 api 라는 이름을 안 쓰는 것이 안전합니다. (06단원 개념01)

없는 파일을 요청하면

섹션 4

```
GET /없는파일.png
404
```

static 이 못 찾고 next() 를 불렀고, 아래 404 처리기까지 내려온 것입니다.

★ 여기서 판단이 필요합니다. /없는파일.png 는 404 JSON 이 맞을까요, 404 HTML 페이지가 맞을까요?

API 서버라면 JSON 입니다. 화면까지 주는 서버라면, 브라우저 주소창으로 들어온 것에는 HTML 이 낫습니다. 이 자료는 API 가 중심이라 JSON 으로 통일합니다.

★ 상위 폴더로 못 나갑니다

섹션 5

이런 요청을 보내면 어떻게 될까요?

GET ../../package.json GET ../../%2F..%2F.env

public 밖으로 나가서 서버 파일을 가져가려는 시도입니다. 실제로 아주 흔한 공격입니다. '경로 탈출(path traversal)' 이라고 합니다.

```
GET ../../package.json
404
```

```
GET ../../%2F..%2Fpackage.json
404
```

★ express.static 이 막아 줍니다. 요청 경로를 정리한 뒤, public 폴더 밖으로 나가는지 확인합니다. 나가려 하면 그냥 안 줍니다.

★★ 그런데 직접 만들면 이게 안 막힙니다. "정적 파일 정도는 직접 만들지" 하고 이렇게 쓰면 큰일 납니다.

```
app.get("/files/:name", (req, res) => {
  res.sendFile(path.join(__dirname, "public", req.params.name));
});
```

:name 에 ../../.env 를 넣으면 서버 비밀 파일이 그대로 나갑니다. 비밀번호와 API 키가 들어 있는 파일입니다.

express.static 을 쓰세요. 남이 이미 고민해서 막아 둔 것을 쓰는 게 낫습니다.

옵션 몇 가지

섹션 6

자주 쓰는 것만 봅니다.

```
express.static(폴더, {
  maxAge: "1d",           브라우저에게 "하루 동안 다시 안 물어봐도 된다"
  index: false,           index.html 자동 찾기를 끄
  dotfiles: "ignore",     .env 같은 점으로 시작하는 파일은 무시 (기본값)
  extensions: ["html"],   /about 으로 about.html 을 찾아 줌
})
```

★ maxAge 를 주면 빨라집니다 브라우저가 파일을 저장해 두고 다시 안 받아 갑니다. 그런데 파일을 고쳐도 사용자 화면이 안 바뀝니다. 하루 동안요.

그래서 실무에서는 파일 이름에 번호를 붙입니다.

```
화면.a3f21c.js
```

내용이 바뀌면 이름이 바뀌니 브라우저가 새로 받아 갑니다. React 를 빌드하면 이런 이름이 나오는 이유입니다.

★ 지금은 maxAge 를 안 주는 게 낫습니다 개발 중에는 고칠 때마다 바로 보여야 하니까요. "왜 안 바뀌지?" 의 원인이 캐시인 경우가 정말 많습니다. 그럴 때는 Ctrl + Shift + R 로 강제 새로고침해 보세요.

```
app.use(
  "/docs",
  express.static(path.join(__dirname, "public"), { index: false })
);
```

```
GET /docs/css/공통.css
```

200

```
GET /docs/
```

404

같은 폴더를 /docs 아래에도 붙였습니다. index: false 라 /docs/ 는 404 입니다.

★ static 은 여러 번 붙일 수 있습니다.

```
app.use(express.static("public"))
app.use(express.static("uploads"))
```

위에서부터 찾다가 처음 있는 것을 줍니다. (09단원에서는 접두어를 붙인 변형 `app.use("/uploads", express.static(...))` 을 씁니다)

404 와 에러 처리기

섹션 7

★ 07단원에서는 없는 주소를 `ROUTE_NOT_FOUND` 로 구분했지만, 이 단원의 서버는 '없는 파일' 의 404 까지 겸하므로 `NOT_FOUND` 하나로 씁니다.

```
app.use((req, res) => {
  res.status(404).json({ error: { code: "NOT_FOUND", message: "찾을 수 없습니다" } });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  res.status(500).json({ error: { code: "INTERNAL_ERROR", message: "서버에서 문제가 생겼습니다" } });
});

app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}`);
  console.log("브라우저로 열어서 버튼을 눌러 보세요.");
});
```

브라우저로 확인하기

`http://localhost:3000` 을 여세요.

① 화면이 나옵니다 (`public/index.html`) ② 글자에 스타일이 붙어 있습니다 (`public/css/공통.css`) ③ 버튼을 누르면 목록이 나옵니다 (`public/js/화면.js` 가 fetch)

F12 → Network 탭을 켜고 새로고침해 보세요. 네 개의 요청이 보입니다.

localhost	document	index.html
공통.css	stylesheet	
화면.js	script	
equipments	fetch	← 버튼을 눌렀을 때

★★ 여기가 이 단원의 출발점입니다.

네 개가 전부 `localhost:3000` 입니다. 같은 곳에서 왔습니다. 이걸 '같은 출처(same origin)' 라고 합니다.

`화면.js` 의 `fetch` 를 보세요.

```
fetch("/api/v1/equipments")
```

주소를 반쪽만 썼습니다. 앞부분(`http://localhost:3000`)이 없습니다. 브라우저가 “지금 페이지와 같은 곳”으로 알아서 붙여 줍니다.

같은 출처라서 아무 문제가 없습니다. 출처가 다르면? 그때부터 CORS 이야기가 시작됩니다. 개념02 에서 봅니다.

정적 파일을 어디서 줄 것인가

① 우리 Express 서버가 준다 (지금 방식)

장점: 서버 하나면 끝. CORS 문제가 없음
 단점: 화면을 고칠 때마다 서버를 다시 올려야 함
 언제: 작은 프로젝트, 관리자 화면

② 프론트를 따로 띄운다 (React 개발 서버 등)

장점: 프론트와 백엔드를 따로 개발·배포
 단점: ★ 출처가 달라서 CORS 설정이 필요함
 언제: 프론트가 React·Vue 인 경우. PART 3 이후

③ S3 같은 곳에 올린다

장점: 서버가 안 죽고, 전 세계에서 빠름
 단점: 역시 CORS 설정 필요
 언제: 실제 서비스 배포. PART 4

★ ②와 ③은 반드시 CORS 를 만납니다. 여러분이 만들 프로젝트도 대부분 ②나 ③입니다. 그래서 이 단원이 필요합니다.

직접 해 볼 것

◦ 직접 해보기 1

`public/index.html` 의 제목을 바꾸고 새로고침해 보세요. 서버를 다시 켜야 하나요? 왜 그럴까요? (힌트: HTML 은 서버가 켜질 때 읽는 게 아닙니다)

◦ 직접 해보기 2

`public` 안에 `사진.png` 를 아무거나 넣고 `http://localhost:3000/사진.png` 로 열어 보세요. 한글 파일 이름도 되나요?

◦ 직접 해보기 3

`express.static` 줄을 API 라우트 ‘아래’ 로 옮겨 보세요. 화면이 여전히 나오나요? 무엇이 달라지나요?

▹ 직접 해보기 4

public 안에 api 라는 폴더를 만들고 그 안에 v1/equipments 라는 파일을 만들어 보세요. 이제 GET /api/v1/equipments 는 무엇을 돌려주나요? (확인 후 지우세요)

▹ 직접 해보기 5

섹션 5의 위험한 코드를 실제로 만들어 보세요.

```
app.get("/files/:name", (req, res) => {  
  res.sendFile(path.join(__dirname, "public", req.params.name));  
});
```

그리고 /files/..%2F..%2Fpackage.json 을 열어 보세요.
무엇이 나오나요? (확인 후 반드시 지우세요)

▹ 직접 해보기 6

F12 → Network 에서 공통.css 를 눌러 보세요. Response Headers 에 Content-Type 이 무엇으로 되어 있나요? Express 가 어떻게 알았을까요? (힌트: 파일 확장자를 봅니다)

자주 하는 실수

이런 실수를 합니다

express.static 에 __dirname 을 안 씀

터미널 위치에 따라 파일을 못 찾습니다.

이런 실수를 합니다

폴더 이름을 주소에 포함시킴

public/index.html 은 /index.html 로 접근합니다. /public/index.html 이 아닙니다. static 에 준 폴더가 '뿌리' 가 됩니다.

이런 실수를 합니다

정적 파일을 직접 sendFile 로 만들

경로 탈출을 막아야 하는데 대부분 빠뜨립니다. express.static 을 쓰세요.

이런 실수를 합니다

public 안에 비밀 파일을 둬

.env, 설정 파일, 백업 파일이 public 에 있으면 누구나 받아 갑니다. public 은 '전 세계에 공개하는 폴더' 라고 생각하세요.

이런 실수를 합니다**고쳤는데 안 바뀐다고 서버를 다시 켜**

정적 파일은 요청할 때마다 읽습니다. 서버를 다시 켤 필요가 없습니다. 안 바뀌면 브라우저 캐시입니다. Ctrl + Shift + R 을 눌러 보세요.

이런 실수를 합니다**API 주소와 파일 이름이 겹침**

public 에 api 폴더가 있으면 API 라우트가 안 걸립니다. 에러도 안 나서 원인을 찾기 어렵습니다.

정리 —————

- 1 app.use(express.static(폴더)) 한 줄이면 그 폴더의 파일이 그대로 서비스된다.
- 2 경로는 여기서도 __dirname 기준으로 만든다.
- 3 폴더 이름은 주소에 안 들어간다. public/a.html 은 /a.html 로 열린다.
- 4 index.html 은 자동이다. / 로 열면 나온다.
- 5 없는 파일이면 아래로 넘어간다. 그래서 맨 아래의 404 처리기가 받는다.
- 6 상위 폴더로는 못 나간다. 주소에 ../ 를 넣어도 Express 가 막는다.
- 7 public 안에 비밀 파일을 두지 않는다. 주소만 알면 누구나 받아 간다.
- 8 고쳤는데 안 바뀌면 서버가 아니라 브라우저 캐시를 먼저 의심한다.
- 9 API 주소와 파일 이름이 겹치지 않게 한다. 먼저 걸리는 쪽이 이긴다.

CORS란 무엇인가

RUN node 개념02_CORS란_무엇인가.js

그러면 Ctrl + C

★ 이 파일은 '일부러 CORS 를 안 열어 둔' 서버입니다.

front/다른출처.html 을 다른 포트로 열어서 막히는 것을 직접 보세요.

개념01 마지막에서 이렇게 끝냈습니다.

"같은 출처라서 아무 문제가 없었습니다. 출처가 다르면?"

이제 그 이야기입니다.

```
const express = require("express");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
```

출처(origin)란

섹션 1

출처는 세 가지가 합쳐진 것입니다.

http://localhost:3000

프로토콜 호스트 포트

이 셋이 '전부' 같아야 같은 출처입니다. 하나라도 다르면 다른 출처입니다.

```
app.get("/origin-quiz", (req, res) => {
  const 기준 = "http://localhost:3000";

  res.json({
    기준,
    비교: [
      { 주소: "http://localhost:3000/api/x", 같은출처: true, 이유: "경로는 상관없습니다" },
```

```
{ 주소: "http://localhost:3000/?q=1", 같은출처: true, 이유: "쿼리도 상관없습니다" },
  { 주소: "http://localhost:5500", 같은출처: false, 이유: "포트가 다릅니다" },
  { 주소: "https://localhost:3000", 같은출처: false, 이유: "프로토콜이 다릅니다"
},
  { 주소: "http://127.0.0.1:3000", 같은출처: false, 이유: "글자가 다르면 다른
호스트입니다" },
  { 주소: "http://api.mysite.com", 같은출처: false, 이유: "호스트가 다릅니다" },
],
});
});
```

GET /origin-quiz

```
200 {"기준":"http://localhost:3000","비교":[{"주소":"http://localhost:3000/api/
x","같은출처":true,"이유":"경로는 상관없습니다"},{"주소":"http://localhost:300
0/?q=1","같은출처":true,"이유":"쿼리도 상관없습니다"},{"주소":"http://localhost:
5500","같은출처":false,"이유":"포트가 다릅니다"},{"주소":"https://localhost:300
0","같은출처":false,"이유":"프로토콜이 다릅니다"},{"주소":"http://127.0.0.1:300
0","같은출처":false,"이유":"글자가 다르면 다른 호스트입니다"},{"주소":"http://ap
i.mysite.com","같은출처":false,"이유":"호스트가 다릅니다"}]}
```

★★ localhost 와 127.0.0.1 은 다른 출처입니다. 같은 컴퓨터를 가리키는데도 그렇습니다. 브라우저는 글자로만 비교합니다. "분명 같은 곳인데 왜 막히지?" 의 흔한 원인입니다.

★ 포트만 달라도 다른 출처입니다. React 개발 서버는 5173, 우리 서버는 3000. 이미 다른 출처입니다. 그래서 프론트를 따로 만들면 반드시 CORS 를 만납니다.

왜 이런 규칙이 있나

섹션 2

```
app.get("/why-sop", (req, res) => {
  res.json({
    시나리오: [
      "여러분이 은행 사이트에 로그인해 둡니다.",
      "그 상태로 나쁜 사이트에 들어갑니다.",
      "그 사이트의 스크립트가 fetch('https://은행.com/내계좌') 를 부릅니다.",
      "브라우저는 은행 쿠키를 자동으로 붙여 보냅니다. 은행은 로그인한 사람으로
봅니다.",
      "막지 않으면 나쁜 사이트가 여러분의 계좌 정보를 읽습니다.",
    ],
    그래서: "브라우저는 '다른 출처의 응답을 스크립트가 읽는 것' 을 기본으로
막습니다.",
    이름: "같은 출처 정책 (Same-Origin Policy)",
    CORS는: "그 막힌 것을 서버가 '애는 괜찮다' 고 허락해 주는 방법입니다.",
  });
});
```

GET /why-sop

```
200 {"시나리오":["여러분이 은행 사이트에 로그인해 둡니다.", "그 상태로 나쁜 사이트에 들어갑니다.", "그 사이트의 스크립트가 fetch('https://은행.com/내계좌') 를 부릅니다.", "브라우저는 은행 쿠키를 자동으로 붙여 보냅니다. 은행은 로그인한 사람으로 봅니다.", "막지 않으면 나쁜 사이트가 여러분의 계좌 정보를 읽습니다."], "그래서": "브라우저는 '다른 출처의 응답을 스크립트가 읽는 것' 을 기본으로 막습니다.", "이름": "같은 출처 정책 (Same-Origin Policy)", "CORS는": "그 막힌 것을 서버가 '얘는 괜찮다' 고 허락해 주는 방법입니다."}
```

★ CORS 는 '보안 장치' 가 아니라 '허락하는 방법' 입니다. 이름 그대로입니다. Cross-Origin Resource Sharing — 출처를 넘어 자원 공유하기. 막는 쪽은 브라우저이고, CORS 는 그걸 푸는 쪽입니다.

그래서 "CORS 를 열면 위험한가?" 는 상황에 따라 다릅니다. 누구에게 여는지가 중요합니다. 개념03 에서 봅니다.

★★ 브라우저만 막습니다

섹션 3

```
app.get("/api/v1/equipments", (req, res) => {
```

★ CORS 헤더를 하나도 안 붙입니다. 일부러입니다.

```
  res.set("X-Total-Count", "2");
  res.json({
    data: [
      { id: 1, name: "컨베이어 1호", line: "A", status: "가동" },
      { id: 2, name: "프레스 1호", line: "B", status: "정지" },
    ],
  });
});
```

GET /api/v1/equipments

Origin: http://localhost:5500

```
200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"}, {"id":2,"name":"프레스 1호","line":"B","status":"정지"}]}
access-control-allow-origin=(없음)
```

★★★ 여기를 보세요.

다른 출처(localhost:5500)에서 온 것처럼 Origin 헤더를 붙여 보냈습니다. 그런데 서버는 200 과 데이터를 그대로 돌려줬습니다. Access-Control-Allow-Origin 헤더는 없습니다.

즉 서버는 아무것도 안 막았습니다.

★ 이 검증은 Node로 요청을 보낸 것입니다. Node는 CORS를 안 따집니다. Postman도, curl도 마찬가지입니다. 전부 잘 됩니다.

막는 것은 오직 브라우저입니다. 정확히는 "브라우저에서 도는 스크립트가 응답을 읽는 것"만 막습니다.

★★ 그래서 이런 일이 벌어집니다.

프론트 개발자: "API가 안 돼요. CORS 에러가 나요." 백엔드 개발자: "Postman으로는 잘 되는데요?"

둘 다 맞습니다. Postman은 브라우저가 아니니까요. 이 대화를 안 하려면 백엔드가 CORS를 알아야 합니다.

브라우저에서 실제로 막히는 것 보기

섹션 4

```
app.get("/how-to-test", (req, res) => {
  res.json({
    준비: [
      "이 서버를 켜 둡니다 (포트 3000).",
      "front/다른출처.html 을 '다른 포트' 로 엽니다.",
      "VS Code 라면 파일을 우클릭 → Open with Live Server (보통 5500 포트).",
      "또는 터미널을 하나 더 열어 npx serve front 를 실행합니다.",
    ],
    주의: [
      "파일을 더블클릭해서 열면 주소가 file:// 이 됩니다.",
      "그것도 다른 출처라 막히긴 하는데, 에러 메시지가 다르게 나옵니다.",
      "http:// 로 여는 것이 실제 상황과 같습니다.",
    ],
    반드시: "F12 → Console 탭을 함께 여세요. 진짜 원인은 콘솔에만 나옵니다.",
  });
});
```

GET /how-to-test

```
200 {"준비":["이 서버를 켜 둡니다 (포트 3000).","front/다른출처.html 을 '다른 포트'
로 엽니다.","VS Code 라면 파일을 우클릭 → Open with Live Server (보통 5500 포
트)."],"또는 터미널을 하나 더 열어 npx serve front 를 실행합니다."},"주의":["파
일을 더블클릭해서 열면 주소가 file:// 이 됩니다.","그것도 다른 출처라 막히긴 하
는데, 에러 메시지가 다르게 나옵니다.","http:// 로 여는 것이 실제 상황과 같습니
다."],"반드시":"F12 → Console 탭을 함께 여세요. 진짜 원인은 콘솔에만 나옵니다."}
```

★ 에러 메시지 읽는 법

섹션 5

진짜 크롬으로 다섯 가지 경우를 실제로 재 봤습니다. 아래는 지어낸 것이 아니라 크롬이 찍은 그대로입니다.

```

app.get("/error-messages", (req, res) => {
  res.json({
    자바스크립트가받는것: "TypeError: Failed to fetch",
    설명: "이 메시지만 봐서는 원인을 알 수 없습니다. 네트워크가 끊겨도 같은 메시지가
    나옵니다.",
    콘솔에나오는진짜이유: [
      {
        경우: "서버가 CORS 헤더를 아예 안 줌",
        메시지: "No 'Access-Control-Allow-Origin' header is present on the requested
        resource.",
        고치는법: "서버에서 Access-Control-Allow-Origin 을 붙여 주세요.",
      },
      {
        경우: "POST/PATCH/DELETE 인데 CORS 헤더가 없음 (프리플라이트가 막힌 경우)",
        메시지: "Response to preflight request doesn't pass access control check: No
        'Access-Control-Allow-Origin' header is present on the requested resource.",
        고치는법: "OPTIONS 요청에도 CORS 헤더를 주세요. 앞부분에 preflight 이 붙어
        있으면 OPTIONS 문제입니다.",
      },
      {
        경우: "허용한 출처가 내 출처와 다름",
        메시지: "The 'Access-Control-Allow-Origin' header has a value
        'http://example.com' that is not equal to the supplied origin.",
        고치는법: "허용 목록에 내 출처를 추가하세요. 포트까지 정확히 맞춰야 합니다.",
      },
      {
        경우: "Authorization 같은 헤더를 보냈는데 허용 안 함",
        메시지: "Request header field authorization is not allowed by Access-Control-
        Allow-Headers in preflight response.",
        고치는법: "Access-Control-Allow-Headers 에 그 헤더 이름을 넣으세요.",
      },
      {
        경우: "DELETE 를 보냈는데 허용 안 함",
        메시지: "Method DELETE is not allowed by Access-Control-Allow-Methods in
        preflight response.",
        고치는법: "Access-Control-Allow-Methods 에 그 메서드를 넣으세요.",
      },
      {
        경우: "요청은 됐는데 응답 헤더를 못 읽음",
        메시지: "(에러가 안 납니다. 200 이 오는데 헤더만 null 입니다)",
        고치는법: "Access-Control-Expose-Headers 에 그 헤더 이름을 넣으세요.",
      },
    ],
  });
});

```

GET /error-messages

200

★★ 다섯 번째가 제일 무섭습니다.

요청은 성공하고 200 이 옵니다. 에러도 안 납니다. 그런데 `response.headers.get("X-Total-Count")` 가 `null` 입니다.

06단원에서 "전체 개수를 헤더로 주는 방법도 있다" 고 했는데, 그때 "CORS 를 쓰면 헤더를 따로 열어 줘야 보인다" 고 적은 게 이것입니다.

화면에 "전체 0건" 이 나오는데 목록은 잘 나옵니다. 원인을 CORS 라고는 아무도 생각 안 합니다.

★ 읽는 순서 ① Console 탭의 빨간 글씨를 끝까지 읽습니다 ② "blocked by CORS policy:" 뒤가 진짜 이유입니다 ③ 그 이유가 위 표의 어느 줄인지 찾습니다 ④ Network 탭에서 그 요청을 눌러 Response Headers 를 확인합니다

Network 탭에서 무엇을 보나

섹션 6

```
app.get("/network-tab", (req, res) => {
  res.json({
    볼것: [
      "요청이 두 번 보이면 앞엿것이 OPTIONS 입니다. 이게 프리플라이트입니다 (개념03).",
      "Request Headers 에 Origin 이 있는지 - 없으면 같은 출처라 CORS 대상이 아닙니다.",
      "Response Headers 에 Access-Control-Allow-Origin 이 있는지.",
      "그 값이 내 Origin 과 글자 단위로 같은지 - 끝의 / 하나도 다르면 안 됩니다.",
      "상태가 200 인데도 빨간 줄이면, 응답은 왔지만 브라우저가 읽기를 막은 것입니다.",
    ],
    함정: "Network 탭에는 200 으로 보이는데 코드에서는 실패합니다. 정상입니다. 응답은 도착했고, 읽는 것만 막힌 것입니다.",
  });
});
```

GET /network-tab

200

★ 마지막 줄이 정말 헷갈리는 부분입니다. "서버는 200 을 줬는데 왜 실패라고 나오지?" 응답이 도착한 것과 스크립트가 읽을 수 있는 것은 다릅니다. 브라우저가 중간에서 가로채 버립니다.

```
app.use((req, res) => {
  res.status(404).json({ error: { code: "NOT_FOUND", message: "찾을 수 없습니다" } })
});
```

```
});
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  res.status(500).json({ error: { code: "INTERNAL_ERROR", message: "서버에서 문제가 생겼습니다" } });
});

app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}/api/v1/equipments`);
  console.log("* 이 서버는 일부러 CORS 를 안 열어 두었습니다.");
  console.log(" front/다른출처.html 을 다른 포트로 열어서 막히는 것을 확인하세요.");
});
```

실험 순서

- 1 이 서버를 켵니다.
- 2 브라우저에서 직접 `http://localhost:3000/api/v1/equipments` 를 엽니다.

→ 잘 나옵니다. 주소창으로 연 것은 같은 출처니까요.

3. Postman 으로 같은 주소를 부릅니다.

→ 잘 됩니다. Postman 은 브라우저가 아닙니다.

4. front/다른출처.html 을 Live Server(5500 포트)로 엽니다.

버튼을 누릅니다.
→ 실패합니다. 화면에는 "TypeError: Failed to fetch" 만 나옵니다.

5. F12 → Console 을 봅니다.

→ No 'Access-Control-Allow-Origin' header is present...

6. F12 → Network 에서 그 요청을 누릅니다.

→ Status 는 200 입니다. 응답은 왔습니다. 읽기만 막힌 것입니다.

2·3번은 되고 4번만 안 되는 것 — 이게 CORS 의 전부입니다.

직접 해 볼 것

▫ 직접 해보기 1

위 실험 순서를 그대로 해 보세요. 여섯 단계 전부요.

▫ 직접 해보기 2

front/다른출처.html 의 API 를 "http://127.0.0.1:3000" 으로 바꿔 보세요. 같은 컴퓨터인데도 막히나요? 왜 그럴까요?

▫ 직접 해보기 3

다른출처.html 을 public 폴더로 옮기고 http://localhost:3000/다른출처.html 로 열어 보세요. 이제 되나요? 무엇이 달라졌나요?

▫ 직접 해보기 4

다른출처.html 을 파일 탐색기에서 더블클릭해 열어 보세요. 주소가 file:// 로 시작합니다. 콘솔 메시지가 어떻게 다른가요?

▫ 직접 해보기 5

Network 탭에서 ⊙ POST 버튼을 눌러 보세요. 요청이 몇 개 보이나요? 앞엿것의 Method 는 무엇인가요? (이게 개념03 의 주제입니다)

▫ 직접 해보기 6

아래 다섯 상황이 CORS 대상인지 판단해 보세요.

- ① 브라우저 주소창에 API 주소를 직접 치
- ② ``
- ③ `fetch("http://다른서버/api")`
- ④ Postman 으로 요청
- ⑤ 서버에서 다른 서버로 fetch (Node 에서)

(힌트: "브라우저의 스크립트가 응답을 읽는가?" 를 물어보세요)

자주 하는 실수

이런 실수를 합니다

"Failed to fetch" 만 보고 서버가 죽었다고 생각

콘솔의 빨간 글씨를 끝까지 읽으세요. 진짜 이유는 거기 있습니다.

이런 실수를 합니다

Postman 으로 되니까 서버는 문제없다고 결론

Postman 은 CORS 를 안 따집니다. 브라우저로 확인해야 합니다.

이런 실수를 합니다

localhost 와 127.0.0.1 을 같다고 생각

브라우저는 글자로 비교합니다. 다른 출처입니다.

이런 실수를 합니다

프론트 코드를 고치려고 함

CORS 는 서버가 허락해 주는 것입니다. 프론트에서 할 수 있는 게 없습니다. (브라우저 확장으로 끄는 방법이 있지만, 내 컴퓨터에서만 되는 임시방편입니다)

이런 실수를 합니다

Network 탭이 200 이라 CORS 가 아니라고 생각

응답이 온 것과 읽을 수 있는 것은 다릅니다.

이런 실수를 합니다

응답 헤더가 안 읽히는데 CORS 를 의심 안 함

에러가 안 나서 알아채기 어렵습니다. Expose-Headers 를 떠올리세요.

정리

- 1 출처(origin)는 프로토콜·호스트·포트 셋을 합친 것이다. 하나라도 다르면 다른 출처다. localhost 와 127.0.0.1 도 다른 출처다. 같은 컴퓨터인 것은 상관없다.
- 2 CORS 는 브라우저만 막는다. 서버는 응답을 이미 정상으로 보냈다.
- 3 그래서 Postman 으로는 언제나 잘 된다. Postman 이 된다고 문제가 없는 것이 아니다.
- 4 Network 탭에도 200 으로 보인다. 응답은 왔고 브라우저가 안 넘겨준 것뿐이다. 200 이니까 CORS 가 아니라고 판단하면 안 된다.
- 5 "Failed to fetch" 만 보고 서버가 죽었다고 하지 않는다. Console 에 있는 CORS 문구를 끝까지 읽는다.
- 6 고칠 곳은 서버다. 프론트 코드를 아무리 고쳐도 안 된다.
- 7 응답 헤더가 안 읽히는 것도 CORS 다. 기본으로는 몇 개만 보여 준다.

CORS 직접 열어 보기

RUN node 개념03_CORS_직접_열어보기.js

그러면 Ctrl + C

개념02 에서 막히는 것을 봤습니다. 이제 엽니다.

★ 다음 파일(개념04)에서 cors 패키지를 씁니다. 한 줄이면 끝납니다. 그런데 한 번은 직접 붙여 봐야 합니다. 패키지가 무슨 헤더를 붙이는지 알아야, 안 될 때 어디를 볼지 압니다.

```
const express = require("express");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
```

아무나 볼 수 있는 자료라면

섹션 1

로그인이 필요 없고, 누가 봐도 상관없는 자료입니다. 공지사항, 공개 통계, 오픈 데이터 같은 것들이죠.

```
app.get("/public-api/notice", (req, res) => {
  res.set("Access-Control-Allow-Origin", "*");
  res.json({ data: { 제목: "8월 정기 점검 안내" } });
});
```

```
GET /public-api/notice
Origin: http://example.com

200 {"data":{"제목":"8월 정기 점검 안내"}}
access-control-allow-origin=*
```

```
GET /public-api/notice
Origin: http://another-site.com

200 {"data":{"제목":"8월 정기 점검 안내"}}
access-control-allow-origin=*
```

★ * 는 “어느 출처든 읽어도 된다” 는 뜻입니다. 어느 출처에서 오든 같은 * 가 나옵니다.

★ * 를 써도 되는 경우 · 로그인 필요 없다 · 누가 봐도 문제없는 자료다 · 쿠키를 안 쓴다 ← 이게 중요합니다. 섹션 6 에서 봅니다

★ * 를 쓰면 안 되는 경우 · 로그인한 사람만 보는 자료 · 사내에서만 쓰는 API * 를 열어 두면 아무 사이트나 우리 API 를 자기 화면에 붙일 수 있습니다.

정해진 곳만 허락하기

섹션 2

우리 프론트엔드만 허락하고 싶습니다. 출처 목록을 만들어 두고, 그 안에 있을 때만 허락합니다.

```
const 허용출처들 = [
  "http://localhost:5500", // VS Code Live Server "http://localhost:5173", // Vite
  개발 서버 (React) "https://설비관리.example.com", // 실제 서비스 주소
];

app.use((req, res, next) => {
  const 출처 = req.get("Origin");
```

★ Origin 헤더는 '다른 출처에서 왔을 때만' 붙습니다. 주소창으로 직접 열거나 Postman 으로 부르면 없습니다.

```
if (출처 && 허용출처들.includes(출처)) {
```

★★ * 가 아니라 '요청한 출처 그대로' 를 돌려줍니다. Allow-Origin 에는 여러 개를 쉼표로 못 씁니다. 딱 하나만 됩니다. 그래서 매번 "이번 요청의 출처" 를 확인해서 그것만 적어 줍니다.

```
res.set("Access-Control-Allow-Origin", 출처);
```

★ Vary: Origin 은 왜 필요한가 중간에 캐시 서버가 있으면, 한 출처에게 준 응답을 다른 출처에게도 줍니다. 그러면 A 사이트에게 준 Allow-Origin 이 B 사이트에게 전달되어 막힙니다. "출처에 따라 응답이 달라진다" 를 알려 주는 것이 Vary 입니다.

```
res.vary("Origin");
}

next();
});

app.get("/api/v1/equipments", (req, res) => {
  res.set("X-Total-Count", "2");
  res.json({
    data: [
      { id: 1, name: "컨베이어 1호", line: "A", status: "가동" },
      { id: 2, name: "프레스 1호", line: "B", status: "정지" },
    ],
  });
});
```

```
});
});
```

GET /api/v1/equipments

Origin: http://localhost:5500

```
200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"}, {"id":2,"name":"프레스 1호","line":"B","status":"정지"}]}
```

access-control-allow-origin=http://localhost:5500 | vary=Origin

GET /api/v1/equipments

Origin: http://evil.example.com

```
200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"}, {"id":2,"name":"프레스 1호","line":"B","status":"정지"}]}
```

access-control-allow-origin=(없음)

GET /api/v1/equipments

```
200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"}, {"id":2,"name":"프레스 1호","line":"B","status":"정지"}]}
```

access-control-allow-origin=(없음)

★★★ 두 번째를 잘 보세요.

허락하지 않은 출처인데도 200 과 데이터가 그대로 나갔습니다. 서버는 아무것도 안 막았습니다. Allow-Origin 헤더만 안 붙였을 뿐입니다.

막는 것은 브라우저입니다. 브라우저가 그 헤더를 보고 판단합니다. 헤더가 없으니 “읽지 마” 하고 스크립트에게 안 넘겨줍니다.

★★ 그래서 CORS 는 보안이 아닙니다.

Postman·curl·다른 서버는 헤더를 무시하고 데이터를 그대로 가져갑니다. “CORS 로 막아 뒀으니 안전하다” 는 완전히 틀린 생각입니다.

진짜 보안은 인증(05단원)입니다. 로그인 안 한 사람에게 데이터를 아예 안 주는 것이죠. CORS 는 “브라우저에서 남의 사이트가 내 로그인 상태를 이용하는 것” 만 막습니다.

★ 세 번째 — Origin 이 아예 없으면 헤더도 안 붙습니다. 같은 출처이거나 브라우저가 아닌 경우입니다. 붙일 필요가 없습니다.

프리플라이트 — 미리 물어보기

섹션 3

개념02 의  5 에서 “POST 를 누르면 요청이 두 개 보인다” 고 했습니다. 앞엿것이 프리플라이트 (preflight)입니다.

브라우저가 진짜 요청을 보내기 전에 먼저 물어봅니다.

브라우저: "OPTIONS 입니다. localhost:5500 인데, POST 로 Content-Type 붙여

보내도 될까요?"

서버: "204. 됩니다. GET·POST·PATCH·DELETE 되고,

Content-Type 과 Authorization 헤더 받습니다."

브라우저: "좋습니다." → 그제야 진짜 POST 를 보냅니다.

★ 왜 미리 물어보나 DELETE 요청을 그냥 보냈다가 서버가 진짜로 지워 버리면 늦습니다. "읽지 마" 라고 해도 이미 지워진 뒤입니다. 그래서 '바꾸는 요청' 은 먼저 허락을 받고 보냅니다.

```
app.get("/preflight-rules", (req, res) => {
  res.json({
    프리플라이트가없는요청: {
      조건: [
        "메서드가 GET · HEAD · POST 중 하나",
        "Content-Type 이 text/plain · multipart/form-data · application/x-www-form-
        urlencoded 중 하나",
        "직접 붙인 헤더가 없음 (Authorization 같은 것)",
      ],
      설명: "이 조건을 전부 만족하면 '단순 요청' 이라 바로 보냅니다. HTML 폼으로도 할
      수 있는 일이라서요.",
    },
    프리플라이트가생기는요청: [
      "PATCH · DELETE · PUT",
      "Content-Type: application/json ← * 우리 API 가 거의 다 여기 걸립니다",
      "Authorization 헤더를 붙임",
      "직접 만든 헤더를 붙임 (X-Request-Id 등)",
    ],
  });
});
```

GET /preflight-rules

200

★★ 두 번째 목록의 두 번째 줄이 핵심입니다.

우리가 만든 API 는 전부 JSON 을 주고받습니다. POST 에 Content-Type: application/json 을 붙이는 순간 프리플라이트가 생깁니다.

즉 "GET 은 되는데 POST 만 안 돼요" 라는 말이 나오면 십중팔구 OPTIONS 를 처리 안 한 것입니다.

OPTIONS 처리하기

섹션 4

```
app.use((req, res, next) => {
  if (req.method !== "OPTIONS") {
    return next();
  }

  const 출처 = req.get("Origin");
```

허락한 출처가 아니면 CORS 헤더 없이 그냥 끝냅니다.

```
if (!출처 || !허용출처들.includes(출처)) {
  return res.sendStatus(204);
}
```

Allow-Origin 은 위쪽 미들웨어가 이미 붙였습니다.

```
res.set("Access-Control-Allow-Methods", "GET,POST,PATCH,DELETE");
res.set("Access-Control-Allow-Headers", "Content-Type, Authorization");
```

★ 이 답을 10분 동안 기억해 두라는 뜻입니다. 없으면 요청마다 OPTIONS 가 한 번씩 더 갑니다. 요청 수가 두 배가 됩니다.

```
res.set("Access-Control-Max-Age", "600");
```

★ 204 는 '성공했고 돌려줄 내용 없음' 입니다. 프리플라이트에 딱 맞습니다. 200 을 줘도 됩니다. 다만 본문이 없으니 204 가 정확합니다.

```
res.sendStatus(204);
});
```

OPTIONS /api/v1/equipments

Origin: http://localhost:5500; Access-Control-Request-Method: POST; Access-Control-Request-Headers: content-type

204

access-control-allow-origin=http://localhost:5500 | access-control-allow-methods=GET,POST,PATCH,DELETE | access-control-allow-headers=Content-Type, Authorization | access-control-max-age=600

OPTIONS /api/v1/equipments

Origin: http://evil.example.com; Access-Control-Request-Method: POST

204

access-control-allow-origin=(없음) | access-control-allow-methods=(없음)

★ 허락 안 한 출처에는 헤더를 안 줍니다. 브라우저는 "허락을 못 받았다" 고 보고 진짜 요청을 아예 안 보냅니다.

```
app.post("/api/v1/equipments", (req, res) => {
  const { name, line } = req.body || {};
  res.status(201).json({ data: { id: 3, name, line, status: "정지" } });
});
```

```
POST /api/v1/equipments
Origin: http://localhost:5500
{"name":"용접로봇 1호","line":"C"}

201 {"data":{"id":3,"name":"용접로봇 1호","line":"C","status":"정지"}}
access-control-allow-origin=http://localhost:5500
```

★ 프리플라이트가 통과한 뒤에 오는 진짜 요청입니다. 여기에도 Allow-Origin 이 있어야 합니다. OPTIONS 에만 붙이면 안 됩니다. 프리플라이트와 진짜 요청은 별개입니다. 둘 다 허락을 받아야 합니다.

```
app.delete("/api/v1/equipments/:id", (req, res) => {
  console.log(` * DELETE 가 서버까지 도착했습니다. id=${req.params.id}`);
  res.sendStatus(204);
});
```

```
DELETE /api/v1/equipments/1
Origin: http://localhost:5500

204
access-control-allow-origin=http://localhost:5500
```

★ 터미널의 ★ 표시를 보세요. 프리플라이트가 막히면 이 줄이 안 찍힙니다. 진짜 요청이 아예 안 오니까요. Allow-Methods 에서 DELETE 를 빼고 다시 해 보면 확인할 수 있습니다. (✎ 4)

응답 헤더를 읽게 해 주기

섹션 5

개념02 에서 본 '조용한 사고' 입니다. 200 이 오는데 X-Total-Count 만 null 인 경우요.

```
app.get("/api/v1/equipments-with-count", (req, res) => {
  res.set("X-Total-Count", "12");
```

★ 이 줄이 없으면 브라우저가 X-Total-Count 를 못 읽게 합니다. 에러도 안 나고 그냥 null 이 나옵니다.

```
res.set("Access-Control-Expose-Headers", "X-Total-Count");

res.json({ data: [{ id: 1 }, { id: 2 }] });
});
```


GET /api/v1/equipments-with-count

Origin: http://localhost:5500

200 {"data":[{"id":1},{"id":2}]}access-control-allow-origin=http://localhost:5500 | access-control-expose-headers=X-Total-Count |
x-total-count=12

★ 브라우저가 기본으로 읽게 해 주는 응답 헤더는 몇 개뿐입니다.

**Content-Type, Content-Length, Content-Language,
Cache-Control, Expires, Last-Modified, Pragma**

그 밖의 헤더는 전부 **Expose-Headers**에 적어야 보입니다. 우리가 만든 **X-Request-Id**, **X-Total-Count** 같은 것들이죠.

★ 실제로 재 봤습니다 (진짜 크롬으로)

Expose-Headers 없음 → headers.get("X-Total-Count")는 null
Expose-Headers 있음 → "12"

요청은 둘 다 200으로 성공합니다. 차이는 헤더가 보이느냐뿐입니다.

쿠키를 함께 보내려면

섹션 6

08

```
app.get("/credentials-rules", (req, res) => {
  res.json({
    프론트에서: 'fetch(주소, { credentials: "include" })',
    서버에서: 'Access-Control-Allow-Credentials: true'를 붙인다',
    금지: "이때는 Allow-Origin에 *를 쓸 수 없습니다. 정확한 출처를 적어야 합니다.",
    이유: "*는 '아무나'라는 뜻인데, 쿠키까지 아무나 쓰게 하면 개념02의 은행  
시나리오가 그대로 일어납니다.",
    에러메시지:
      "The value of the 'Access-Control-Allow-Origin' header must not be the wildcard  
'*' when the request's credentials mode is 'include'."
  });
});
```

GET /credentials-rules**200**

★ 우리 자료는 쿠키 대신 **Authorization** 헤더를 씁니다 (05단원). 헤더 방식은 **credentials**가 필요 없습니다. 프론트가 직접 붙여 보내니까요. 브라우저가 자동으로 붙이는 게 아닙니다.

대신 **Allow-Headers**에 **Authorization**을 넣어야 합니다. 섹션 4에 있습니다.

```

app.use((req, res) => {
  res.status(404).json({ error: { code: "NOT_FOUND", message: "찾을 수 없습니다" } });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  res.status(500).json({ error: { code: "INTERNAL_ERROR", message: "서버에서 문제가 생겼습니다" } });
});

app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}/api/v1/equipments`);
  console.log(`허용 출처: ${허용출처들.join(", ")}`);
});

```

CORS 헤더 정리

Access-Control-Allow-Origin	누가 읽어도 되나	(필수)
Access-Control-Allow-Methods	어떤 메서드가 되나	(프리플라이트 응답)
Access-Control-Allow-Headers	어떤 헤더를 받나	(프리플라이트 응답)
Access-Control-Max-Age	프리플라이트를 몇 초 기억하나	
Access-Control-Expose-Headers	어떤 응답 헤더를 읽게 하나	
Access-Control-Allow-Credentials	쿠키를 함께 받나	

Vary: Origin 캐시가 출처별로 나누게

요청 쪽 헤더 (브라우저가 자동으로 붙임 — 우리가 안 씁니다)

Origin	지금 페이지의 출처
Access-Control-Request-Method	프리플라이트에서 "이 메서드로 갈게요"
Access-Control-Request-Headers	프리플라이트에서 "이 헤더 붙일게요"

직접 해 볼 것

▫ 직접 해보기 1

이 서버를 켜고 front/다른출처.html 을 5500 포트로 여세요. 다섯 버튼이 전부 되나요? 개념02 서버일 때와 비교해 보세요.

▫ 직접 해보기 2

허용출처들 에서 http://localhost:5500 을 빼고 다시 켜세요. 콘솔 메시지가 개념02 때와 어떻게 다른가요? ('헤더가 없다' 와 '값이 다르다' 는 다른 메시지입니다)

▫ 직접 해보기 3

섹션 4의 Allow-Headers 에서 Authorization 을 빼고 ㉠ 증표 버튼을 눌러 보세요. 콘솔에 뭐라고 나오나요?

▫ 직접 해보기 4

Allow-Methods 에서 DELETE 를 빼고 ㉠ 삭제 버튼을 누르세요. Network 탭에 요청이 몇 개 보이나요? 진짜 DELETE 가 서버까지 갔나요? (터미널을 보세요)

▫ 직접 해보기 5

섹션 5의 Expose-Headers 줄을 지우고 ㉠ 헤더읽기 버튼을 눌러 보세요. 에러가 나나요? X-Total-Count 는 어떻게 나오나요?

▫ 직접 해보기 6

Max-Age 를 지우고 Network 탭을 보면서 ㉠ POST 를 여러 번 눌러 보세요. OPTIONS 가 매번 가나요? 다시 넣으면 어떻게 되나요?

▫ 직접 해보기 7

허용출처들 을 함수로 바꿔 보세요. "localhost 로 시작하면 전부 허용" 처럼요. 개발 중에는 편한데, 실제 서비스에서는 왜 위험할까요?

자주 하는 실수

이런 실수를 합니다

Allow-Origin 에 여러 출처를 쉼표로 씀

"http://a.com, http://b.com" 은 통째로 하나의 값으로 봅니다. 안 됩니다. 요청의 Origin 을 확인해서 하나만 적어 주세요.

이런 실수를 합니다

OPTIONS 를 처리 안 함

"GET 은 되는데 POST 만 안 돼요" 의 1번 원인입니다.

이런 실수를 합니다

OPTIONS 에만 헤더를 붙이고 진짜 요청에는 안 붙임

프리플라이트는 통과하는데 본 요청에서 막힙니다.

이런 실수를 합니다

주소 끝에 / 를 붙임

"http://localhost:5500/" 과 "http://localhost:5500" 은 다릅니다. Origin 헤더에는 / 가 안 붙습니다. 목록에도 붙이지 마세요.

이런 실수를 합니다

Vary: Origin 을 빼먹음

개발 중에는 티가 안 납니다. 중간에 캐시가 있는 실제 서비스에서 터집니다.

이런 실수를 합니다

CORS 를 보안이라고 생각

Postman 으로는 그냥 뚫립니다. 진짜 보안은 인증입니다.

이런 실수를 합니다

credentials 를 쓰면서 Allow-Origin 에 * 를 씀

브라우저가 거부합니다. 정확한 출처를 적어야 합니다.

정리

- 1 허락은 응답 헤더로 한다 — Access-Control-Allow-Origin.
- 2 여러 출처를 심표로 못 쓴다. 하나만 적거나, 요청한 출처를 보고 그것만 돌려준다.
- 3 출처를 보고 정할 때는 Vary: Origin 을 함께 붙인다. 안 붙이면 캐시가 다른 출처에 엉뚱한 답을 준다.
- 4 주소 끝에 / 를 붙이면 안 맞는다. 출처는 글자 그대로 비교된다.
- 5 단순하지 않은 요청은 브라우저가 먼저 OPTIONS 로 물어본다. 이것이 프리플라이트다. 이걸 처리 안 하면 진짜 요청은 오지도 않는다.
- 6 OPTIONS 에만 헤더를 붙이면 안 된다. 진짜 요청의 응답에도 붙어야 한다.
- 7 응답 헤더를 프론트가 읽게 하려면 Expose-Headers 로 따로 열어 준다.
- 8 쿠키를 함께 보내려면 credentials 를 켜는데, 그때는 Allow-Origin 에 * 를 못 쓴다.
- 9 CORS 는 보안 장치가 아니다. 브라우저 안에서만 도는 규칙이다. 진짜 보호는 인증과 권한이 한다.

cors 패키지 쓰기

```
RUN node 개념04_cors_패키지.js
```

그러면 Ctrl + C

개념03 에서 직접 붙였습니다. 40줄쯤 됐습니다. 같은 일을 해 주는 패키지가 있습니다.

```
npm install cors
```

★ 직접 만들어 본 뒤에 패키지를 쓰는 이유 이제 이 패키지가 무슨 헤더를 붙이는지 압니다. 안 될 때 Network 탭에서 무엇을 봐야 할지도 압니다. 처음부터 패키지만 썼다면 “왜 안 되지?” 에서 막혔을 것입니다.

```
const express = require("express");
const cors = require("cors");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
```

아무 옵션 없이

섹션 1

`cors()` 하나면 개념03 섹션 1의 * 방식이 됩니다. 여기서는 공개 자료에만 붙여 봅니다.

```
app.get("/public-api/notice", cors(), (req, res) => {
  res.json({ data: { 제목: "8월 정기 점검 안내" } });
});
```

```
GET /public-api/notice
Origin: http://example.com

200 {"data":{"제목":"8월 정기 점검 안내"}}
access-control-allow-origin=*
```

★ 라우트마다 붙일 수도 있고, `app.use(cors())` 로 전부에 붙일 수도 있습니다. 미들웨어니까요. (05단원)

★★ `app.use(cors())` 한 줄로 끝내지 마세요. 많은 블로그가 이렇게만 알려 줍니다. 개발 중에는 편합니다. 그런데 그 순간 우리 API 를 아무 사이트나 쓸 수 있게 됩니다. 섹션 2 처럼 목록을 정하세요.

```
const 허용출처들 = [
  "http://localhost:5500",
  "http://localhost:5173",
  "https://설비관리.example.com",
];

const cors설정 = {
```

origin 에 배열을 주면 그 안에 있을 때만 허락합니다.

```
  origin: 허용출처들,
```

프리플라이트 응답에 들어갈 값들

```
  methods: ["GET", "POST", "PATCH", "DELETE"],
  allowedHeaders: ["Content-Type", "Authorization"],
```

브라우저가 읽게 해 줄 응답 헤더

```
  exposedHeaders: ["X-Total-Count", "X-Request-Id"],
```

프리플라이트를 몇 초 기억할지

```
  maxAge: 600,
};

app.use("/api", cors(cors설정));

app.get("/api/v1/equipments", (req, res) => {
  res.set("X-Total-Count", "2");
  res.json({
    data: [
      { id: 1, name: "컨베이어 1호", line: "A", status: "가동" },
      { id: 2, name: "프레스 1호", line: "B", status: "정지" },
    ],
  });
});
```

GET /api/v1/equipments

Origin: http://localhost:5500

```
200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"},{"id":2,"name":"프레스 1호","line":"B","status":"정지"}]}
```

```
access-control-allow-origin=http://localhost:5500 | access-control-expose-headers=X-Total-Count,X-Request-Id | vary=Origin
```

★ Vary: Origin 을 자동으로 붙여 줍니다. 개념03 에서 직접 써야 했던 것입니다. 이런 걸 빠뜨리지 않는 게 패키지의 장점입니다.

GET /api/v1/equipments

Origin: http://evil.example.com

```
200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"},{"id":2,"name":"프레스 1호","line":"B","status":"정지"}]}
```

```
access-control-allow-origin=(없음)
```

★ 개념03 과 똑같습니다. 서버는 안 막습니다. 헤더만 안 붙입니다. 막는 것은 브라우저입니다.

front/다른출처.html 의 ☉ 버튼이 부르는 주소입니다. 08단원의 모든 서버를 같은 페이지로 확인하려고 이름을 맞춰 두었습니다.

```
app.get("/api/v1/equipments-with-count", (req, res) => {
  res.set("X-Total-Count", "2");
  res.json({ data: [{ id: 1 }, { id: 2 }] });
});
```

GET /api/v1/equipments-with-count

Origin: http://localhost:5500

```
200 {"data":[{"id":1},{"id":2}]}
```

```
access-control-allow-origin=http://localhost:5500 | x-total-count=2 | access-control-expose-headers=X-Total-Count,X-Request-Id
```

```
app.post("/api/v1/equipments", (req, res) => {
  const { name, line } = req.body || {};
  res.status(201).json({ data: { id: 3, name, line, status: "정지" } });
});

app.delete("/api/v1/equipments/:id", (req, res) => {
  console.log(`  * DELETE 가 서버까지 도착했습니다. id=${req.params.id}`);
  res.sendStatus(204);
});
```

```
DELETE /api/v1/equipments/1
Origin: http://localhost:5500

204
access-control-allow-origin=http://localhost:5500
```

★ 터미널의 ★ 표시를 보세요. 프리플라이트가 막히면 이 줄이 안 찍힙니다. 진짜 요청이 아예 안 오니까요. methods 에서 DELETE 를 빼고 다시 해 보면 확인할 수 있습니다. (🖋 5)

```
OPTIONS /api/v1/equipments
Origin: http://localhost:5500; Access-Control-Request-Method: POST; Access-Control-Request-Headers:
content-type

204
access-control-allow-origin=http://localhost:5500 | access-control-allow-methods=GET,POST,PATCH,DEL
ETE | access-control-allow-headers=Content-Type,Authorization | access-control-max-age=600
```

```
POST /api/v1/equipments
Origin: http://localhost:5500
{"name":"용접로봇 1호","line":"C"}

201 {"data":{"id":3,"name":"용접로봇 1호","line":"C","status":"정지"}}
access-control-allow-origin=http://localhost:5500
```

★ OPTIONS 라우트를 하나도 안 만들었는데 204 가 나왔습니다. cors 미들웨어가 “이건 프리플라이트구나” 하고 알아서 답하고 끝납니다. 개념03 에서 직접 쓴 열 줄이 사라졌습니다.

```
OPTIONS /api/v1/equipments
Origin: http://evil.example.com; Access-Control-Request-Method: POST

204
access-control-allow-origin=(없음)
```


조건을 함수로 정하기

섹션 4

배열로 안 되는 경우가 있습니다. · 우리 회사 도메인의 하위 주소는 전부 허용 · 개발 중에는 localhost 아무 포트나 허용

이럴 때는 함수를 줍니다.

```
function 출처판단(출처, 알려주기) {
```

★ 출처가 `undefined` 인 경우 Postman·curl·같은 출처 요청은 Origin 헤더가 없습니다. 이때 거절하면 Postman 으로 시험할 수가 없습니다. 보통 허용합니다.

```
if (!출처) {
  return 알려주기(null, true);
}

const 허용 =
  허용출처들.includes(출처) ||
  출처.endsWith(".example.com") || /^http:\/\/localhost:\d+$/ .test(출처);
```

★ 첫 번째 인자는 '에러', 두 번째는 '허용할지' 입니다. 거절할 때 에러를 던지면 500 이 됩니다. 그냥 `false` 를 주세요.

```
알려주기(null, 허용);
}

app.use("/open-api", cors({ origin: 출처판단 }));

app.get("/open-api/stats", (req, res) => {
  res.json({ data: { 가동중: 8, 정지: 3, 점검중: 1 } });
});
```

```
GET /open-api/stats
Origin: http://localhost:9999

200 {"data":{"가동중":8,"정지":3,"점검중":1}}
access-control-allow-origin=http://localhost:9999
```

```
GET /open-api/stats
Origin: https://sub.example.com

200 {"data":{"가동중":8,"정지":3,"점검중":1}}
access-control-allow-origin=https://sub.example.com
```

GET /open-api/stats

Origin: http://evil.example.net

200 {"data":{"가동중":8,"정지":3,"점검중":1}}

access-control-allow-origin=(없음)

GET /open-api/stats

200 {"data":{"가동중":8,"정지":3,"점검중":1}}

access-control-allow-origin=(없음)

OPTIONS /open-api/stats

Origin: http://evil.example.net; Access-Control-Request-Method: POST

404

OPTIONS /api/v1/equipments

Origin: http://evil.example.net; Access-Control-Request-Method: POST

204

★★ 위 두 줄을 비교해 보세요. 같은 '거절' 인데 상태코드가 다릅니다.

/open-api 는 origin 을 '함수' 로 찼고, /api 는 '배열' 로 찼습니다. 직접 재 본 결과입니다.

origin 설정	허용 출처	거절 출처
-----------	-------	-------

배열	204 + Allow-Origin	204, 헤더 없음
함수(false)	204 + Allow-Origin	404, 헤더 없음

함수가 false 를 돌려주면 cors 는 응답을 안 만들고 next() 를 부릅니다. 그래서 요청이 아래로 흘러가 404 처리하기까지 내려갑니다.

브라우저 입장에서는 둘 다 똑같이 막힙니다. Allow-Origin 이 없으면 상태코드가 무엇이든 거절이니까요.

다만 서버 기록에서 헛갈릴 수 있습니다. "OPTIONS 가 404 로 잔뜩 찍히는데 공격인가요?" 라는 질문의 답이 이것입니다. 허락 안 한 출처가 두드리고 있는 것뿐입니다.

★★ 세 번째를 보세요. evil.example.net 은 막혔습니다. endsWith(".example.com") 이라 .example.net 은 안 걸립니다.

그런데 이런 걸 조심해야 합니다.

출처.includes("example.com") ← x 위험합니다

"http://example.com.evil.net" 도 통과합니다. 공격자가 이런 도메인을 삽니다.

endsWith 를 쓰되 앞에 점을 붙이세요. ".example.com" 더 확실하게 하려면 정규식으로 전체를 맞추세요.

```
/^https:\/\/([a-z0-9-]+\.)?example\.com$/
```

개발과 운영을 나누기

섹션 5

```
app.get("/env-strategy", (req, res) => {
  res.json({
    개발: "localhost 아무 포트나 허용. 팀원마다 포트가 다르니까요.",
    운영: "실제 프론트 주소만. 목록을 코드에 박지 말고 환경변수로 받으세요.",
    예시: 'const 허용 = process.env.CORS_ORIGINS ?
process.env.CORS_ORIGINS.split(",") : ["http://localhost:5173"];',
    이유: "주소가 바뀔 때 코드를 고치고 다시 배포할 필요가 없습니다. (PART 4)",
    주의: "운영에서 * 를 쓰지 마세요. 아무 사이트나 우리 API 를 자기 화면에 붙일 수
있습니다.",
  });
});
```

GET /env-strategy

200

★ 환경변수는 PART 4 에서 자세히 배웁니다. 지금은 "코드에 주소를 박지 않는다" 만 기억하세요.

08

직접 만든 것과 비교

섹션 6

```
app.get("/compare", (req, res) => {
  res.json({
    직접만들기: {
      줄수: "약 40줄",
      장점: "무슨 일이 일어나는지 전부 보입니다",
      빠뜨리기쉬운것: [
        "Vary: Origin",
        "OPTIONS 를 라우트보다 위에 두기",
        "프리플라이트와 진짜 요청 둘 다에 헤더 붙이기",
        "Origin 이 없을 때의 처리",
      ],
    },
    cors패키지: {
      줄수: "약 8줄 (설정 포함)",
      장점: "빠뜨릴 게 없습니다. 위 네 가지를 전부 알아서 합니다",
      단점: "안에서 무슨 일이 일어나는지 안 보입니다",
    }
  });
});
```

```

},
  결론: "직접 한 번 만들어 보고, 실무에서는 패키지를 쓰세요.",
});
});

```

GET /compare

200

```

app.use((req, res) => {
  res.status(404).json({ error: { code: "NOT_FOUND", message: "찾을 수 없습니다" } });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  res.status(500).json({ error: { code: "INTERNAL_ERROR", message: "서버에서 문제가 생겼습니다" } });
});

app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}/api/v1/equipments`);
  console.log(`허용 출처: ${허용출처들.join(", ")}`);
});

```

cors 옵션 정리

origin 누구를 허락하나

```

true      아무나 (*)
"http://..." 그 하나만

```

배열 · 그중 하나면

```

함수      직접 판단

```

methods 프리플라이트에서 허락할 메서드

```

기본: GET, HEAD, PUT, PATCH, POST, DELETE

```

allowedHeaders 받을 요청 헤더

```

안 주면 브라우저가 물어본 것을 그대로 허락합니다

```

exposedHeaders 브라우저가 읽게 해 줄 응답 헤더

X-Total-Count 같은 것. 안 주면 못 읽습니다

credentials 쿠키를 받을지 (true 면 origin 에 * 못 씀)

maxAge 프리플라이트를 몇 초 기억할지

optionsSuccessStatus 프리플라이트 응답 코드 (기본 204)

★ allowedHeaders 를 안 주면 어떻게 되나 브라우저가 "Content-Type 불일게요" 하면 "Content-Type 됩니다" 라고 답합니다. 즉 아무 헤더나 허락하는 셈입니다. 편하지만, 무엇을 받는지 코드에 안 남습니다. 적어 두는 편이 낫습니다.

직접 해 볼 것

▫ 직접 해보기 1

이 서버를 켜고 front/다른출처.html 을 5500 으로 여세요. 다섯 버튼이 전부 되나요? 개념03 과 결과가 같나요?

▫ 직접 해보기 2

㉟ 헤더읽기 버튼을 눌러 X-Total-Count 가 보이는지 확인하세요. exposedHeaders 에서 X-Total-Count 를 빼고 다시 해 보세요.

▫ 직접 해보기 3

허용출처들 에서 5500 을 빼고 다시 켜세요. 출처판단 함수 때문에 /open-api 는 여전히 될 것입니다. /api 와 /open-api 의 차이를 확인하세요.

▫ 직접 해보기 4

출처판단 의 endsWith 를 includes 로 바꾸고, Origin 을 http://example.com.evil.net 으로 보내 보세요. (Postman 의 Headers 에 Origin 을 직접 넣으면 됩니다) 통과하나요? 왜 위험한가요? (확인 후 되돌리세요)

▫ 직접 해보기 5

cors 설정에서 methods 를 ["GET"] 으로 바꾸고 ㉠ POST 버튼을 눌러 보세요. 콘솔에 뭐라고 나오나요?

▫ 직접 해보기 6

app.use(cors()) 를 맨 위에 한 줄만 두는 방식으로 바꿔 보세요. 전부 잘 됩니다. 그런데 무엇이 위험해졌나요? (힌트: 아무 사이트나 우리 API 를 부를 수 있습니다)

07단원의 계층 구조 서버에 cors 를 붙여 보세요. 어느 파일에 붙이는 게 맞을까요?

자주 하는 실수

이런 실수를 합니다

app.use(cors()) 한 줄로 끝냄

아무 사이트나 우리 API 를 쓸 수 있게 됩니다. 목록을 정하세요.

이런 실수를 합니다

origin 함수에서 거절할 때 에러를 던짐

콜백(`null, false`) 를 쓰세요. 에러를 던지면 500 이 됩니다. CORS 로 막히는 것과 서버가 터지는 것은 다릅니다.

이런 실수를 합니다

출처를 includes 로 확인

example.com.evil.net 이 통과합니다. `endsWith(".example.com")` 을 쓰세요.

이런 실수를 합니다

exposedHeaders 를 빠뜨림

에러가 안 나서 못 알아챱니다. 헤더만 조용히 `null` 이 됩니다.

이런 실수를 합니다

cors 를 라우트 아래에 붙임

미들웨어입니다. 라우트보다 위에 있어야 합니다. (05단원)

이런 실수를 합니다

credentials: true 와 origin: true 를 함께 씀

`origin: true` 는 * 를 뜻합니다. 브라우저가 거부합니다.

정리

- 1 직접 만들던 것을 cors 패키지가 대신해 준다. 프리플라이트도 알아서 처리한다.
- 2 `app.use(cors())` 한 줄로 끝내지 않는다. 그건 모두에게 여는 것이다.

- 3 허용 목록을 정해서 준다. 확인은 `includes` 가 아니라 정확히 같은지로 한다. `includes` 로 하면 `evil-mysite.com` 같은 것이 통과한다.
- 4 조건이 복잡하면 `origin` 자리에 함수를 준다.
- 5 거절할 때 에러를 던지지 않는다. 던지면 500 이 된다. '허락 안 함' 으로 넘긴다.
- 6 프론트가 읽어야 할 응답 헤더가 있으면 `exposedHeaders` 에 적는다.
- 7 `cors` 는 라우트보다 위에 붙인다. 아래에 두면 안 거친다.
- 8 `credentials: true` 와 `origin: true` 를 함께 쓰지 않는다. 요청한 곳을 전부 허락하면서 쿠키까지 보내는 셈이 된다.
- 9 개발과 운영의 허용 목록을 나눠 둔다.

프론트와 붙이는 세 가지 방법

RUN node 개념05_프론트와_붙이는_세_가지_방법.js

그러면 Ctrl + C

★ 이 파일은 서버를 '두 개' 띄웁니다.

API 서버 하나, 프론트 서버 하나. 실제 상황을 흉내 내기 위해서입니다.

프론트엔드와 백엔드를 붙이는 방법은 셋뿐입니다.

① 같은 서버가 화면도 준다 → CORS 문제 없음 (개념01) ② 출처가 다르고 CORS 를 연다 → 개념03·04

③ 프론트 서버가 대신 전달해 준다 → 프록시. 이번 파일의 주제

React·Vue 를 쓰면 개발 중에는 ③을 씁니다. 무슨 일이 일어나는지 모르고 쓰는 사람이 많습니다. 직접 만들어 봅시다.

```
const express = require("express");
```

★★ Number() 를 꼭 씁주세요. 안 그러면 서버가 안 켜집니다.

process.env.PORT 는 '글자' 입니다. 환경변수는 언제나 글자입니다. Number 없이 PORT + 100 을 하면 더하기가 아니라 '이어 붙이기' 가 됩니다.

```
"3000" + 100 → "3000100"    ← 이런 포트는 없습니다
3000 + 100   → 3100         ← 이게 맞습니다
```

이 파일을 만들 때 실제로 이 실수를 했습니다. 에러 메시지가 포트 이야기를 안 해서 원인을 찾는 데 한참 걸렸습니다.

쿼리(04단원), 경로 파라미터(04단원), 환경변수(여기) — 밖에서 들어오는 값은 전부 글자입니다. 숫자로 쓸 거면 Number 를 거치세요.

```
const PORT = Number(process.env.PORT) || 3000;
const API포트 = PORT + 100; // API 서버는 다른 포트에서 둥니다
```

1부 — API 서버 (다른 포트에서 도는 백엔드)

```
const api = express();
api.use(express.json());
```


★ CORS 를 하나도 안 열었습니다. 일부러입니다. 프록시를 쓰면 CORS 가 필요 없다는 것을 보여 주려고요.

```
const 설비들 = [
  { id: 1, name: "컨베이어 1호", line: "A", status: "가동" },
  { id: 2, name: "프레스 1호", line: "B", status: "정지" },
];

api.get("/api/v1/equipments", (req, res) => {
  res.set("X-Total-Count", String(설비들.length));
  res.json({ data: 설비들 });
});

api.post("/api/v1/equipments", (req, res) => {
  const { name, line } = req.body || {};

  if (!name || !line) {
    return res.status(400).json({ error: { code: "VALIDATION_FAILED", message: "name
과 line 이 필요합니다" } });
  }

  res.status(201).json({ data: { id: 3, name, line, status: "정지" } });
});

api.use((req, res) => {
  res.status(404).json({ error: { code: "NOT_FOUND", message: "찾을 수 없습니다" }
});
});
```

2부 — 프론트 서버 (화면을 주고, API 요청을 대신 전달)

```
const app = express();
app.use(express.json());
```

프록시란 무엇인가

섹션 1

프록시(proxy)는 '대신 해 주는 것' 입니다.

프록시 없이

브라우저 → API 서버 (다른 출처. 막힘)

프록시로

브라우저 → 프론트 서버 → API 서버
(같은 출처) (서버끼리라 CORS 없음)

브라우저가 보기에는 같은 출처와만 이야기합니다. 그래서 CORS 가 안 걸립니다. 프론트 서버가 뒤에서 몰래 API 서버에 다녀오는 것이죠.

```
app.use("/api", async (req, res, next) => {
```

브라우저가 /api/v1/equipments 를 부르면 우리가 대신 http://localhost:API포트/api/v1/equipments 를 부릅니다.

```
const 진짜주소 = `http://localhost:${API포트}${req.originalUrl}`;

try {
  const 보낼것 = {
    method: req.method,
    headers: {},
  };
```

본문이 있는 요청만 본문을 실어 보냅니다.

```
if (req.method !== "GET" && req.method !== "HEAD") {
  보낼것.headers["Content-Type"] = "application/json";
  보낼것.body = JSON.stringify(req.body ?? {});
}
```

증표는 그대로 전달합니다. 없으면 안 붙입니다.

```
const 증표 = req.get("Authorization");
if (증표) 보낼것.headers["Authorization"] = 증표;

const 응답 = await fetch(진짜주소, 보낼것);
const 글자 = await 응답.text();
```

상태코드와 몇몇 헤더를 그대로 옮깁니다.

```

res.status(응답.status);

const 형식 = 응답.headers.get("Content-Type");
if (형식) res.set("Content-Type", 형식);

const 개수 = 응답.headers.get("X-Total-Count");
if (개수) res.set("X-Total-Count", 개수);

res.send(글자);
} catch (에러) {

```

API 서버가 꺼져 있으면 여기로 옵니다.

```

next(에러);
}
});

```

GET /api/v1/equipments

```

200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"},{"id":2,"name":"프레스 1호","line":"B","status":"정지"}]}
x-total-count=2 | access-control-allow-origin=(없음)

```

POST /api/v1/equipments

```

{"name":"용접로봇 1호","line":"C"}

```

```

201 {"data":{"id":3,"name":"용접로봇 1호","line":"C","status":"정지"}}

```

POST /api/v1/equipments

```

{"name":"용접로봇 1호"}

```

```

400 {"error":{"code":"VALIDATION_FAILED","message":"name 과 line 이 필요합니다"}}

```

GET /api/v1/없는것

```

404 {"error":{"code":"NOT_FOUND","message":"찾을 수 없습니다"}}

```

★★★ Allow-Origin 헤더가 없습니다. 그런데도 브라우저에서 잘 됩니다.

왜냐하면 브라우저는 프론트 서버(같은 출처)와만 이야기했기 때문입니다. CORS 를 따질 일이 아예 없습니다.

API 서버는 CORS 를 한 줄도 안 열었습니다. 그래도 됩니다. 요청을 보낸 것이 브라우저가 아니라 프론트 서버(Node)니까요.

★ 상태코드와 에러도 그대로 전달됩니다. 400 이든 404 든 API 서버가 준 그대로 브라우저에 갑니다.
프록시는 '전달' 만 합니다. 내용을 바꾸지 않습니다.

화면

섹션 2

```
app.get("/", (req, res) => {
  res.type("html").send(`<!DOCTYPE html>
<html lang="ko"><head><meta charset="UTF-8"><title>프록시 확인</title>
<style>body{font-family:"맑은 고딕",sans-serif;max-width:640px;margin:40px
auto;padding:0 20px;line-height:1.7}
pre{background:#f5f5f5;border:1px solid #ddd;padding:12px;white-space:pre-wrap}
button{font-size:15px;padding:8px 16px;cursor:pointer}</style></head>
<body>
<h1>프록시로 부르기</h1>
<p>이 화면은 <b>${PORT}</b> 번 포트가 주었습니다.<br>
API 서버는 <b>${API포트}</b> 번 포트에서 돌고 있고, CORS 를 하나도 안 열었습니다.</p>
<button id="누르기">설비 목록 불러오기</button>
<pre id="결과">버튼을 누르세요.</pre>
<script>
document.querySelector("#누르기").addEventListener("click", async () => {
  const 결과 = document.querySelector("#결과");
  결과.textContent = "불러오는 중...";
  try {
```

★ 주소를 반쪽만 씁니다. 지금 페이지와 같은 곳으로 갑니다.

```
const 응답 = await fetch("/api/v1/equipments");
const 답 = await 응답.json();
결과.textContent =
  "성공 " + 응답.status + "\n" +
  "X-Total-Count: " + 응답.headers.get("X-Total-Count") + "\n\n" +
  JSON.stringify(답, null, 2);
} catch (에러) {
  결과.textContent = "실패: " + 에러.message;
}
});
</script></body></html>`);
});
```

GET /

200

실제로는 직접 안 만듭니다

섹션 3

```
app.get("/real-world", (req, res) => {
  res.json({
    설명: "React·Vue 개발 서버에 프록시 기능이 이미 있습니다. 설정 몇 줄이면 됩니다.",
    Vite: {
      파일: "vite.config.js",
      설정: "server: { proxy: { '/api': 'http://localhost:3000' } }",
      뜻: "/api 로 시작하는 요청은 3000 번으로 대신 보내 줘",
    },
    CRA: {
      파일: "package.json",
      설정: '"proxy": "http://localhost:3000"',
      뜻: "내가 못 찾는 요청은 3000 번으로 보내 줘",
    },
    Next: {
      파일: "next.config.js",
      설정: "rewrites() { return [{ source: '/api/:path*', destination: 'http://localhost:3000/api/:path*' }] }",
    },
    Express에서: "http-proxy-middleware 패키지를 씁니다. 지금 만든 것을 훨씬 잘 해 줍니다.",
  });
});
```

GET /real-world

200

★ 우리가 만든 프록시는 아주 단순합니다. 실제 프록시는 이런 것들까지 처리합니다.

- 파일 업로드처럼 큰 본문을 통째로 안 읽고 흘려보내기 (09단원)
- 쿠키와 헤더를 정확히 옮기기
- 웹소켓 연결 전달하기
- 연결이 끊겼을 때 처리

그래서 실무에서는 직접 안 만듭니다. 다만 무슨 일이 일어나는지는 알아야 합니다.

세 가지 방법 비교

섹션 4

```
app.get("/compare", (req, res) => {
  res.json({
    같은서버가화면도움: {
      CORS: "필요 없음",
    },
  });
});
```

```

장점: "설정이 없습니다. 배포도 서버 하나면 끝",
단점: "화면을 고칠 때마다 서버에 다시 올려야 합니다",
언제: "관리자 화면, 작은 프로젝트, 사내 도구",
},
CORS를 연다: {
  CORS: "필요함",
  장점: "프론트와 백엔드를 완전히 따로 배포할 수 있습니다",
  단점: "허용 목록을 관리해야 합니다. 주소가 바뀌면 서버도 고쳐야 합니다",
  언제: "프론트를 S3·Vercel 같은 곳에 따로 올릴 때",
},
프록시: {
  CORS: "필요 없음",
  장점: "프론트가 주소를 '/api/...' 로만 씁니다. 개발이 편합니다",
  단점: "프록시가 한 단계 더 거칩니다. 배포할 때 따로 설정이 필요합니다",
  언제: "React·Vue 개발 중. 그리고 배포에서도 Nginx 로 같은 구조를 만듭니다",
},
});
});

```

GET /compare

200

배포하면 무엇이 바뀌나

섹션 5

```

app.get("/deploy", (req, res) => {
  res.json({
    개발중: "프론트 5173, 백엔드 3000. 프록시로 붙임",
    배포후: [
      "프론트를 빌드하면 HTML·CSS·JS 파일 몇 개가 나옵니다.",
      "그 파일들을 Nginx 나 S3 에 올립니다.",
      "Nginx 가 /api 로 오는 요청만 백엔드로 넘깁니다. 개발할 때의 프록시와 같은 구조입니다.",
      "브라우저가 보기에겐 전부 한 주소입니다. 그래서 CORS 가 필요 없습니다.",
    ],
    그런데: "프론트를 Vercel 에, 백엔드를 다른 곳에 올리면 출처가 달라집니다. 그때는 CORS 를 열어야 합니다.",
    결론: "개발 중에 어떻게 했든, 배포 구조에 따라 다시 정해야 합니다. PART 4 에서 합니다.",
  });
});

```

GET /deploy

200

★★ 가장 흔한 사고 개발 중에는 프록시라 잘 되다가, 배포하고 나서 CORS 에러가 납니다. 개발 환경과 배포 환경의 구조가 다르기 때문입니다.

막으려면 배포 구조를 미리 정하고, 개발에서도 비슷하게 맞춰 두는 게 좋습니다.

```
app.use((req, res) => {
  res.status(404).json({ error: { code: "NOT_FOUND", message: "찾을 수 없습니다" } });
});

app.use((err, req, res, next) => {
  console.error(`[프록시 에러] ${req.method} ${req.path} - ${err.message}`);
  res.status(502).json({
    error: { code: "BAD_GATEWAY", message: "API 서버에 연결하지 못했습니다" },
  });
});
```

★ 502 Bad Gateway “내가 대신 물어보러 갔는데 그쪽이 답을 안 준다” 는 뜻입니다. 프록시가 있을 때만 나오는 상태코드입니다. 500 과 구별해서 쓰면 원인을 훨씬 빨리 찾습니다.

두 서버를 함께 켵니다

```
api.listen(API포트, () => {
  console.log(`API 서버: http://localhost:${API포트} (CORS 안 열었음)`);

  app.listen(PORT, () => {
    console.log(`프론트 서버: http://localhost:${PORT} (프록시 있음)`);
    console.log("");
    console.log(`브라우저로 http://localhost:${PORT} 를 열고 버튼을 눌러 보세요.`);
    console.log(`직접 http://localhost:${API포트}/api/v1/equipments 를 열어 봐도 됩니다.`);
  });
});
```

무엇을 고를까

화면이 몇 개 안 되고 혼자 만든다

→ ① 같은 서버. public 폴더에 넣으면 끝입니다.

React·Vue 로 만들고, 프론트 담당이 따로 있다

→ 개발 중에는 ③ 프록시, 배포는 구조에 따라 ①이나 ②

프론트를 Vercel·S3 에 올리고 백엔드는 따로 둔다

→ ② CORS. 허용 목록을 환경변수로 관리하세요.

모바일 앱도 이 API 를 쓴다

→ 앱은 브라우저가 아니라 CORS 와 무관합니다.
대신 인증(05단원)이 더 중요해집니다.

★ 어느 쪽이든 인증은 따로 해야 합니다. CORS 는 보안이 아닙니다. 개념03 섹션 2 에서 본 그대로입니다.

직접 해 볼 것

▫ 직접 해보기 1

브라우저로 이 서버(3000)를 열고 버튼을 눌러 보세요. F12 → Network 에서 요청 주소가 무엇인가요?
3100 번이 보이나요?

▫ 직접 해보기 2

브라우저 주소창에 `http://localhost:3100/api/v1/equipments` 를 직접 쳐 보세요. 나오나요? 같은
주소를 `front/다른출처.html` 에서 부르면요?

▫ 직접 해보기 3

프록시 안의 `fetch` 주소를 없는 포트로 바꿔 보세요. 버튼을 누르면 어떤 상태코드가 나오나요? 502 가 왜
맞는 답인지 설명해 보세요.

▫ 직접 해보기 4

프록시에서 `Authorization` 전달 부분을 지워 보세요. 05단원의 인증이 붙은 API 라면 무슨 일이
생길까요?

▫ 직접 해보기 5

프록시가 `X-Total-Count` 를 안 옮기게 바꿔 보세요. 화면에 어떻게 나오나요? (개념03 의 `Expose-
Headers` 문제와 증상이 같습니다)

▫ 직접 해보기 6

API 서버에 `cors` 를 붙이고, 프록시를 지워 보세요. 화면의 `fetch` 주소를 `http://localhost:3100/...` 으로
바꾸면 ②번 방법이 됩니다. 둘 다 되는지 확인하세요.

자주 하는 실수

이런 실수를 합니다

프록시를 쓰면서 프론트에서 전체 주소를 씀

`fetch("http://localhost:3000/api/...")` 라고 쓰면 프록시를 건너뛰니다. 그러면 다시 CORS 문제가 됩니다. `"/api/..."` 로만 쓰세요.

이런 실수를 합니다

프록시가 헤더를 안 옮김

Authorization 을 안 옮기면 전부 401 이 됩니다. Content-Type 을 안 옮기면 본문이 안 읽힙니다.

이런 실수를 합니다

개발에서만 확인하고 배포함

프록시는 개발 서버 기능입니다. 배포하면 사라집니다. Nginx 설정이나 CORS 를 따로 준비해야 합니다.

이런 실수를 합니다

프록시 에러를 500 으로 냄

내 코드가 터진 게 아니라 상대가 응답을 안 준 것입니다. 502 가 정확합니다.

이런 실수를 합니다

CORS 와 프록시를 둘 다 씀

틀리지는 않지만, 어느 쪽이 동작하는지 헷갈립니다. 하나만 고르세요.

이런 실수를 합니다

"프록시를 쓰면 인증이 필요 없다" 고 생각

프록시는 출처 문제만 없앱니다. 누가 부르는지는 여전히 확인해야 합니다.

정리

- 1 프론트와 붙이는 방법은 셋이다 — CORS 열기 · 프록시 · 같은 서버에서 함께 주기.
- 2 프록시는 프론트 쪽 서버가 요청을 대신 받아 백엔드로 넘겨 주는 것이다. 브라우저가 보기에는 같은 출처라 CORS 가 아예 안 생긴다.
- 3 프록시를 쓰면 프론트에서 전체 주소를 쓰지 않는다. `/api/...` 처럼 상대 주소로 쓴다. 전체 주소를 쓰면 프록시를 지나치지 않고 곧장 가서 다시 막힌다.
- 4 프록시가 헤더를 안 옮기면 인증이 통째로 빠진다.
- 5 프록시를 쓴다고 인증이 필요 없어지는 것이 아니다. 출처만 같아 보이게 만든 것이다.
- 6 CORS 와 프록시를 둘 다 켜지 않는다. 문제가 생겼을 때 어디가 원인인지 못 가린다.
- 7 개발에서만 확인하고 배포하지 않는다. 배포하면 출처가 바뀐다.
- 8 실제로는 프록시를 직접 만들 일이 거의 없다. Vite 같은 도구가 해 준다. 다만 무엇을 해 주는지는 알고 써야 문제가 생겼을 때 찾을 수 있다.

정적 파일과 CORS

RUN node 연습문제.js

서버 없이 푸는 문제들입니다. 여기서 만든 함수를 연습문제_서버.js 에서 그대로 씁니다.

문제 1 같은 출처인가

두 주소가 같은 출처인지 판단하는 함수 같은출처인가(a, b) 를 만드세요.

출처 = 프로토콜 + 호스트 + 포트. 경로와 쿼리는 상관없습니다.

힌트: new URL(주소) 로 분석하고 `\${분석.protocol}/\${분석.host}` 를 비교합니다.

protocol 에는 이미 콜론이 붙어 있습니다. ("http:")
host 에는 포트가 포함되고, hostname 에는 안 됩니다.
주소 모양이 아니면 URL 이 TypeError 를 던집니다. try/catch 로 false 를 주세요.

시험할 것 (이 순서로) ("http://localhost:3000/a", "http://localhost:3000/b?q=1")
("http://localhost:3000", "http://localhost:5500") ("http://localhost:3000",
"https://localhost:3000") ("http://localhost:3000", "http://127.0.0.1:3000") ("https://a.com",
"https://a.com:443") ("http://a.com", "http://a.com:80") ("http://a.com", "http://a.com:8080")

기대 출력 true
 false
 false
 false
 true
 true
 false

★ 다섯 번째와 여섯 번째가 true 인 이유를 설명할 수 있어야 합니다. ★ 네 번째가 false 인 것도요. 같은 컴퓨터인데 왜 다를까요?

여기에 코드를 쓰세요

문제 2 프리플라이트가 필요한가

요청을 받아 프리플라이트(OPTIONS)가 생기는지 판단하는 함수를 만드세요.

프리플라이트가 '없는' 조건 (전부 만족해야 함) · 메서드가 GET · HEAD · POST 중 하나 · 직접 붙인 헤더가 없음

(그냥 붙는 헤더: accept, accept-language, content-language, content-type)

· Content-Type 이 text/plain · multipart/form-data ·

application/x-www-form-urlencoded 중 하나

힌트: Content-Type 에 charset 이 붙어 올 수 있습니다.

split(";")[0].trim() 으로 앞부분만 비교하세요.

시험할 것 (이 순서로)

```
{ method: "GET" }
{ method: "POST", headers: { "Content-Type": "application/x-www-form-urlencoded" } }
{ method: "POST", headers: { "Content-Type": "application/json" } }
{ method: "DELETE" }
{ method: "GET", headers: { Authorization: "Bearer key-user-1" } }
{ method: "POST", headers: { "Content-Type": "text/plain; charset=utf-8" } }
```

기대 출력	false
	false
	true
	true
	true
	false

★ 세 번째가 우리 API 의 거의 전부입니다. "GET 은 되는데 POST 만 안 돼요" 가 왜 생기는지 설명해 보세요.

★ 두 번째와 여섯 번째는 왜 '단순 요청' 일까요? (힌트: HTML 폼으로도 보낼 수 있는 요청인가요?)

여기에 코드를 쓰세요

문제 3 허용할 출처인가

출처를 받아 허락할지 판단하는 함수 허용인가(출처) 를 만드세요.

Origin 이 없으면(undefined) 허용 (Postman·curl·같은 출처) 허용목록에 있으면 허용 호스트가 example.com 이거나 .example.com 으로 끝나면 허용 그 밖 거절

허용목록: ["http://localhost:5500", "http://localhost:5173"]

★★ 글자로 비교하지 마세요. 출처.includes("example.com") 으로 쓰면

"https://example.com.evil.net" 이 통과합니다. 공격자가 이런 도메인을 삽니다.

new URL(출처).hostname 을 꺼내서 비교하세요.

시험할 것 (이 순서로) undefined "http://localhost:5500" "https://example.com"

"https://sub.example.com" "https://example.com.evil.net" "https://evilexample.com" "not-a-url"

기대 출력	true
	true
	true
	true
	false
	false
	false

★ 여섯 번째 — endsWith("example.com") 이라고 썼다면 통과합니다. 앞에 점을 붙여야 하는 이유입니다.

여기에 코드를 쓰세요

문제 4 에러 메시지 → 고칠 헤더

아래 일곱 가지 상황에 어떤 헤더를 고쳐야 할까요? 짝지어 출력하세요.

기대 출력 No 'Access-Control-Allow-Origin' header is present → Access-Control-Allow-Origin
 Response to preflight request doesn't pass access control check → OPTIONS 응답에도 Access-Control-Allow-Origin
 has a value 'http://다른곳' that is not equal to the supplied origin → 허용 목록에 내 출처 추가
 Request header field authorization is not allowed → Access-Control-Allow-Headers
 Method DELETE is not allowed → Access-Control-Allow-Methods
 200 인데 headers.get 이 null → Access-Control-Expose-Headers
 must not be the wildcard '*' when the request's credentials mode is 'include' → Allow-Origin 을 정확한 출처로

★ 여섯 번째만 에러가 안 납니다. 왜 그게 더 위험한지 생각해 보세요. ★ 두 번째에 preflight 이라는 단어가 있으면 어디를 봐야 하나요?

여기에 코드를 쓰세요

문제 5 상황별로 어떤 헤더가 필요한가

일곱 가지 상황에 필요한 헤더를 짚지어 출력하세요.

기대 출력 공개 자료를 아무나 읽게 → Access-Control-Allow-Origin: *
 우리 프론트만 읽게 → Access-Control-Allow-Origin: (그 출처) + Vary: Origin
 JSON 을 POST 로 받게 → OPTIONS 에 Allow-Methods, Allow-Headers
 Authorization 헤더를 받게 → Access-Control-Allow-Headers: Authorization
 X-Total-Count 를 읽게 → Access-Control-Expose-Headers: X-Total-Count
 OPTIONS 를 10분간 기억하게 → Access-Control-Max-Age: 600
 쿠키를 함께 받게 → Access-Control-Allow-Credentials: true (* 금지)

여기에 코드를 쓰세요

문제 6 CORS 는 왜 보안이 아닌가

막는 것은 누구인가: _____

서버는 무엇을 하나 (데이터를 안 주나?): _____

그래서 무엇이 뚫리나: _____

그럼 CORS 는 무엇을 막나: _____

진짜 보안은 무엇인가: _____

한 줄로 정리하면: _____

문제 7 프록시가 CORS 를 없애는 이유

프록시가 없을 때 브라우저는 누구와 이야기하나: _____

프록시가 있을 때는: _____

API 서버에 요청을 보내는 것은 누구인가: _____

그래서 API 서버는 CORS 를 열어야 하나: _____

프론트에서 주소를 어떻게 써야 하나, 왜: _____

배포하면 무엇이 달라지나: _____

문제 8 Expose-Headers 함정

무슨 일이 일어나나 (상태코드와 에러 여부를 포함해서): _____

왜 그런가: _____

왜 특히 위험한가: _____

화면에는 어떤 증상으로 나타나나: _____

어떻게 고치나: _____

06단원에서 이 이야기가 이미 나왔습니다. 어디였나요: _____

문제 9 주소를 파일 경로로 바꾸기

express.static 이 속으로 하는 일을 손으로 해 봅시다. 주소를 받아 public 안의 파일 경로를 돌려주는 파일경로(주소) 를 만드세요.

· 기준 폴더는 `path.join(__dirname, "public")` 입니다. · 주소가 "/" 이거나 "/" 로 끝나면 `index.html` 을 붙입니다. · 결과는 기준 폴더에서 본 '상대 경로' 로 출력하세요.

절대 경로는 사람마다 다르니 `path.relative` 로 찍습니다.

힌트: `path.join(공개폴더, 주소)` · `path.relative(공개폴더, 만든경로)`

윈도우는 폴더 구분자가 다르니 `.split(path.sep).join("/")` 로 맞춰서 찍으세요.

시험할 것 (이 순서로) `"/" "/index.html" "/css/공통.css" "/img/"`

```
기대 출력  index.html
           index.html
           css/공통.css
           img/index.html
```

여기에 코드를 쓰세요

문제 10 상위 폴더로 못 나가게 막기

문제 9로 만든 경로가 정말 `public` 안인지 보는 안전한가(주소) 를 만드세요.

★ `startsWith` 로 볼 때 폴더 구분자까지 붙여서 봐야 합니다(개념04 실수 6). 그냥 `startsWith(공개폴더)` 로 보면 `public2` 같은 이름이 통과합니다.

힌트: `path.join` 은 `../` 를 알아서 정리해 줍니다.

그래서 `"/css/../index.html"` 은 안쪽으로 정리되어 통과해야 맞습니다.

시험할 것 (이 순서로) `"/index.html" "/css/../index.html" "../비밀.txt" "../../etc/passwd"`

```
기대 출력  true
           true
           false
           false
```


여기에 코드를 쓰세요

문제 11 먼저 걸리는 쪽이 이긴다

Express 는 등록한 순서대로 찾습니다. 그래서 정적 파일과 API 주소가 겹치면 먼저 등록된 쪽이 이깁니다. 그걸 흉내 내 보세요.

등록 하나는 { 이름, 접두 } 모양이고, 주소가 접두로 시작하면 걸립니다. 먼저걸리는것(등록들, 주소) 를 만들어 걸린 것의 이름을 돌려주세요.

```
const 등록들 = [
```

```
  { 이름: "정적파일", 접두: "/" },
  { 이름: "API", 접두: "/api" },
```

```
];
```

시험할 것 (이 순서로) 먼저걸리는것(등록들, "/api/v1/equipments") 먼저걸리는것
(등록들.slice().reverse(), "/api/v1/equipments")

기대 출력	정적파일 API

★ 첫 줄이 왜 사고인지 생각해 보세요. public 안에 실수로 api 라는 폴더가 생기면 무슨 일이 일어날까요?
에러가 나나요, 조용히 이상해지나요?

여기에 코드를 쓰세요

문제 12 public 에 두면 안 되는 것

아래 중 public 폴더에 두면 안 되는 것을 전부 고르고 이유를 쓰세요.

① 로고 이미지 ② .env ③ 관리자 화면 html ④ 회원 명단 엑셀 ⑤ 공통.css ⑥ 아직 안 쓰는 백업.sql

고른 것: _____

왜 안 되나 (한 문장으로): _____

③ 관리자 화면은 왜 애매한가: _____

"주소를 아무도 모르니 괜찮다" 는 왜 안 통하나: _____

화면도 주고 CORS 도 여는 서버

RUN npx nodemon 연습문제_서버.js

★ 이 문제의 목적 한 서버가 세 가지를 동시에 하게 만드는 것입니다.

- ① 화면(HTML·CSS·JS)을 준다 → 같은 출처라 CORS 불필요
- ② 공개 API 를 아무나 쓰게 연다 → *
- ③ 내부 API 를 정해진 곳만 쓰게 연다 → 허용 목록

그리고 확인은 '세 가지 방법' 으로 합니다.

같은 출처(브라우저) · 다른 출처(브라우저) · 브라우저가 아닌 곳(Postman)

★ 세 번째 확인이 이 단원의 결론입니다.

```
const express = require("express");
const cors = require("cors");
const path = require("path");

const app = express();
const PORT = Number(process.env.PORT) || 3000;
```

★ `Number()` 를 씌운 것에 주의하세요. 환경변수는 언제나 '글자' 입니다. 숫자로 쓸 거면 반드시 바꿔야 합니다. (개념05 에서 이 실수로 서버가 안 켜졌습니다)

```
app.use(express.json());
```

문제 1 출처 판단 함수

연습문제.js 문제 3의 허용인가(출처) 를 여기로 가져오세요. 그리고 cors 가 쓸 수 있는 모양으로 감싸세요.

```
function 출처판단(출처, 알려주기) {

  알려주기(null, 허용인가(출처));

}
```

★ 거절할 때 에러를 던지지 마세요. 500 이 됩니다. 알려주기(`null`, `false`) 로 "허락 안 함" 을 알립니다.

허용목록: ["http://localhost:5500", "http://localhost:5173"] 그리고 example.com 과 그 하위 도메인

여기에 코드를 쓰세요

문제 2 공개 API 는 아무나

/public-api 아래를 아무 출처나 읽을 수 있게 하세요. 그리고 GET /public-api/notice 를 만드세요.

기대 출력

GET /public-api/notice (Origin 이 무엇이든)
→ 200 {"data":{"제목":"8월 정기 점검 안내","작성일":"2026-08-01"}}
헤더에 Access-Control-Allow-Origin: *

★ * 를 써도 되는 경우인지 판단해 보세요. 로그인 이 필요 없고, 누가 봐도 상관없고, 쿠키를 안 쓰는
자료인가요?

여기에 코드를 쓰세요

문제 3 내부 API 는 허용 목록만

/api 아래에 cors 를 붙이세요. 옵션은 이렇게 주세요.

origin	문제 1의 출처판단 함수
methods	GET, POST, PATCH, DELETE
allowedHeaders	Content-Type, Authorization
exposedHeaders	X-Total-Count
maxAge	600

그리고 두 주소를 만드세요. 둘 다 X-Total-Count 헤더를 붙입니다.

GET /api/v1/equipments GET /api/v1/equipments-with-count

← front/다른출처.html 의 ㉟ 버튼이 이 주소를 부릅니다.
08단원의 모든 서버를 같은 페이지로 확인하려고 이름을 맞춰 둔 것입니다.

```

기대 출력  Origin: http://localhost:5500      → 200, Allow-Origin 있음, X-Total-Count 읽힘
            Origin: https://sub.example.com   → 200, Allow-Origin 있음
            Origin: https://example.com.evil.net → 200, Allow-Origin 없음 ★

```

★★ 세 번째를 보세요. 데이터는 그대로 나옵니다. 서버가 막는 게 아니라 헤더를 안 붙일 뿐입니다. 이게 “CORS 는 보안이 아니다” 의 증거입니다.

여기에 코드를 쓰세요

문제 4 프리플라이트

POST /api/v1/equipments 와 DELETE /api/v1/equipments/:id 를 만드세요.

POST name 과 line 이 없으면 400, 있으면 201 DELETE 204. 그리고 터미널에 “★ DELETE 가 서버까지 도착했습니다” 를 찍으세요

★ OPTIONS 라우트는 만들지 마세요. cors 가 알아서 합니다.

```

기대 출력  OPTIONS /api/v1/equipments (허용 출처)
            → 204, Allow-Methods 와 Allow-Headers 와 Max-Age 가 있음
            POST (허용 출처, 정상 본문) → 201, Allow-Origin 있음
            DELETE (허용 출처)          → 204, 터미널에 ★ 찍힘
            OPTIONS (거절 출처)          → ?

```

★★ 마지막 상태코드를 직접 확인해 보세요. 204 일까요 404 일까요? origin 을 '함수' 로 줬는지 '배열' 로 줬는지에 따라 다릅니다. 왜 그런지, 브라우저 입장에서는 차이가 있는지 설명해 보세요. (개념04 섹션 4 아래에 재 본 결과가 있습니다)

여기에 코드를 쓰세요

문제 5 정적 파일

public 폴더를 정적으로 서비스하세요.

★★ 어디에 두어야 할까요? API 라우트보다 위일까요 아래일까요? public 안에 실수로 api 라는 폴더가 생기면 어떻게 될지 생각해 보세요. 에러가 나나요, 조용히 이상해지나요?

```
기대 출력  GET /                → 200 (public/index.html)
            GET /css/공통.css   → 200
```

여기에 코드를 쓰세요

문제 6 404 와 에러 처리기

개념03:04 와 같습니다.

```
기대 출력  GET /없는주소 → 404 {"error":{"code":"NOT_FOUND","message":"찾을 수 없습니다"}}
```

여기에 코드를 쓰세요

```
app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}`);
});
```

세 가지 방법으로 확인하기

① 같은 출처

브라우저로 `http://localhost:3000` 을 엽니다. 버튼을 누르면 목록이 나옵니다. F12 → Network 에서 Origin 헤더가 있나요? 없을 것입니다. 같은 출처니까요.

② 다른 출처 (CORS)

front/다른출처.html 을 다른 포트로 엽니다.

- VS Code 라면 우클릭 → Open with Live Server (보통 5500)
- 또는 터미널을 하나 더 열어 `npx serve front`

다섯 버튼을 전부 눌러 보세요.

- ① 단순 GET → 성공
- ② POST + JSON → 성공 (프리플라이트가 통과했다는 뜻)
- ③ Authorization → 성공 (Allow-Headers 가 있다는 뜻)
- ④ DELETE → 성공 (Allow-Methods 가 있다는 뜻)
- ⑤ 응답 헤더 읽기 → X-Total-Count 가 숫자로 보여야 합니다

⑤가 "못 읽음 (null)" 이면 exposedHeaders 를 빠뜨린 것입니다.

③ 브라우저가 아닌 곳 _____

Postman 으로 GET `http://localhost:3000/api/v1/equipments` 를 부릅니다. Headers 에 Origin: `https://example.com.evil.net` 을 직접 넣어 보세요.

데이터가 그대로 나옵니다. Allow-Origin 헤더만 없습니다.

★★★ ③이 이 단원의 결론입니다. CORS 로는 아무것도 못 막습니다. 진짜로 막으려면 05단원의 인증을 붙여야 합니다.

다 만든 뒤 답해 보세요

① ②번 확인에서 ②번 버튼(POST)을 누를 때 Network 탭에 요청이 몇 개 보이나요? 앞엿것의 Method 는 무엇인가요? _____

② exposedHeaders 를 빼면 ⑤번 버튼이 어떻게 되나요? 에러가 나나요? 콘솔에 뭐라도 나오나요?

③ allowedHeaders 에서 Authorization 을 빼면 ⑤번 버튼이 어떻게 되나요? 콘솔 메시지를 그대로 적어 보세요. _____

④ methods 에서 DELETE 를 빼고 ④번을 누르면, 터미널에 "★ DELETE 가 서버까지 도착했습니다" 가 찍히나요? 그게 무슨 뜻인가요? _____

⑤ ③번 확인(Postman)에서 데이터가 그대로 나온 것을 보고, "CORS 로 API 를 보호한다" 는 말이 왜 틀렸는지 설명해 보세요. _____

⑥ 정적 파일을 API 라우트 위에 두면 무엇이 위험한가요? 실제로 public 안에 api 폴더를 만들어 확인해 보세요. _____

잘 안 될 때 보는 곳

화면은 나오는데 버튼이 안 됨 (같은 출처) → CORS 문제가 아닙니다. API 라우트나 주소를 확인하세요.

TypeError: Failed to fetch → F12 → Console 의 빨간 글씨를 끝까지 읽으세요. 진짜 이유는 거기 있습니다.

GET 은 되는데 POST 만 안 됨 → 프리플라이트입니다. cors 를 라우트보다 위에 붙였는지 확인하세요.

콘솔에 preflight 이라는 단어가 보임 → OPTIONS 응답 문제입니다. 진짜 요청이 아니라 그 앞 요청을 보세요.

200 인데 X-Total-Count 가 null → exposedHeaders 입니다. 에러가 안 나니 이걸 떠올려야 합니다.

OPTIONS 가 404 로 찍힘 → origin 을 함수로 썼고, 그 함수가 false 를 돌려준 것입니다. 정상입니다.

서버가 안 켜지고 포트 관련 에러 → Number(process.env.PORT) 를 안 썼습니다. "3000" + 100 은 "3000100" 입니다.

Postman 은 되는데 브라우저만 안 됨 → 정상입니다. CORS 는 브라우저만 따집니다. 그게 이 단원의 핵심입니다.

파일 업로드

OPEN 09_파일_업로드

```
const app = express();
app.use(express.json());
res.json({
  Base64문제: [
```

- 01 파일 업로드의 원리
- 02 multer 시작하기 (그리고 한글 함정 세 가지)
- 03 제한과 검증
- 04 올린 파일 보여 주고 내려받게 하기
- 05 파일과 기록을 함께 관리하기
- 연습문제
- 서버 연습문제

파일 업로드의 원리

RUN node 개념01_파일_업로드의_원리.js

그러면 Ctrl + C

지금까지 주고받은 것은 전부 글자였습니다. JSON 도 결국 글자입니다. 그런데 사진·PDF·엑셀은 글자가 아닙니다.

이번 단원에서 파일을 받는 법을 배웁니다. 그 전에 “파일이 어떻게 서버까지 오는가” 부터 봅시다.

```
const express = require("express");

const app = express();
const PORT = Number(process.env.PORT) || 3000;

app.use(express.json());
```

왜 JSON 으로 못 보내나

섹션 1

```
app.get("/why-not-json", (req, res) => {
  res.json({
    JSON은: "글자만 담을 수 있습니다. 사진의 바이트를 그대로 넣을 수 없습니다.",
    억지로하면: "Base64 로 글자로 바꿔서 담을 수는 있습니다.",
    Base64문제: [
      "크기가 약 33% 커집니다. 3MB 사진이 4MB 가 됩니다.",
      "서버가 통째로 메모리에 올려야 합니다. 큰 파일이면 서버가 멈춥니다.",
      "브라우저가 파일을 읽어 변환하는 시간이 따로 듭니다.",
      "진행률을 보여 주기 어렵습니다.",
    ],
    그래서: "파일은 multipart/form-data 라는 다른 형식으로 보냅니다.",
  });
});
```

GET /why-not-json

```
200 {"JSON은":"글자만 담을 수 있습니다. 사진의 바이트를 그대로 넣을 수 없습니
다.", "억지로하면":"Base64 로 글자로 바꿔서 담을 수는 있습니다.", "Base64문제":
["크기가 약 33% 커집니다. 3MB 사진이 4MB 가 됩니다.", "서버가 통째로 메모리에 올
려야 합니다. 큰 파일이면 서버가 멈춥니다.", "브라우저가 파일을 읽어 변환하는 시간
이 따로 듭니다.", "진행률을 보여 주기 어렵습니다."], "그래서":"파일은 multipart/fo
rm-data 라는 다른 형식으로 보냅니다."}
```

★ Base64 를 아예 안 쓰는 건 아닙니다. 아주 작은 이미지(아이콘 등)는 JSON 에 담아 보내기도 합니다. 하지만 사용자가 올리는 파일은 크기를 알 수 없으니 multipart 를 씁니다.

multipart/form-data 는 어떻게 생겼나

섹션 2

실제로 서버에 도착하는 본문을 그대로 찍어 봅니다. `express.json()` 이 못 읽는 형식이라, 조각을 직접 모읍니다. (03단원 개념04)

```
app.post("/raw", (req, res) => {
  let 쌓은것 = "";

  req.setEncoding("utf-8");
  req.on("data", (조각) => (쌓은것 += 조각));
  req.on("end", () => {
    res.json({
      "Content-Type 헤더": req.get("Content-Type"),
      본문줄수: 쌓은것.split("\n").length,
      본문에담긴것: 쌓은것
        .split("\n")
        .filter((줄) => 줄.includes("Content-Disposition") || 줄.includes("Content-Type:"))
        .map((줄) => 줄.trim()),
    });
  });
});
```

```
POST /raw multipart:@file=report.pdf&desc=hello
200
```

09

★ 터미널이 아니라 Postman 으로 직접 보내 보세요. 실제 본문은 이렇게 생겼습니다.

-----WebKitFormBoundaryABC123 Content-Disposition: form-data; name="desc"

설비 사진입니다 -----WebKitFormBoundaryABC123 Content-Disposition: form-data;
name="file"; filename="작업표준서.pdf" Content-Type: application/pdf

%PDF-1.4 ...(파일의 진짜 바이트)... -----WebKitFormBoundaryABC123--

하나씩 보면

-----WebKitFormBoundary... 조각을 나누는 구분선(boundary)입니다.

Content-Disposition	본문에 절대 안 나올 만한 글자를 브라우저가 만듭니다.
filename=	이 조각이 무슨 필드인지 알려 줍니다.
Content-Type	파일이면 원래 이름이 여기 붙습니다. 파일이면 형식이 붙습니다.

(빈 줄) 여기부터 내용입니다.

★ 글자 필드와 파일이 '한 요청 안에' 함께 들어 있습니다. 그래서 "사진 + 설명" 을 한 번에 보낼 수 있습니다. multi(여럿) + part(조각) — 이름 그대로입니다.

★ Content-Type 헤더를 직접 쓰지 마세요 multipart/form-data; boundary=----WebKitFormBoundaryABC123 이 boundary 는 요청마다 다릅니다. 브라우저가 만들어 붙입니다. 우리가 직접 쓰면 boundary 가 안 맞아서 서버가 못 읽습니다. 섹션 4 에서 다시 봅니다.

express.json() 은 이것 못 읽습니다

섹션 3

```
app.post("/try-json-parser", (req, res) => {
  res.json({
    "req.body": req.body,
    타입: typeof req.body,
    설명: "express.json() 은 application/json 만 읽습니다. multipart 는 그냥 지나칩니다.",
  });
});
```

POST /try-json-parser multipart:@file=report.pdf&desc=hello

```
200 {"타입":"undefined","설명":"express.json() 은 application/json 만 읽습니다. multipart 는 그냥 지나칩니다."}
```

★★ req.body 가 undefined 입니다. (JSON.stringify 는 undefined 인 속성을 아예 빼고 찍습니다)

04단원 개념03 에서 본 그대로입니다. express.json() 은 Content-Type 이 application/json 일 때만 일합니다. multipart 요청은 "내가 읽을 게 아니네" 하고 그냥 넘깁니다.

express.urlencoded() 도 못 읽습니다. 이름이 비슷하지만 완전히 다른 형식입니다.

그래서 multipart 를 읽어 주는 도구가 따로 필요합니다. 그게 multer 입니다.

프론트에서 보내는 법

섹션 4

```
app.get("/", (req, res) => {
  res.type("html").send(`<!DOCTYPE html>
<html lang="ko"><head><meta charset="UTF-8"><title>업로드 원리</title>
<style>body{font-family:"맑은 고딕",sans-serif;max-width:680px;margin:40px
auto;padding:0 20px;line-height:1.7}
h1{border-bottom:3px solid #333;padding-bottom:8px}
.안내{background:#fff8dc;border-left:4px solid #f0c040;padding:12px 16px}
pre{background:#f5f5f5;border:1px solid #ddd;padding:12px;white-space:pre-wrap;word-
break:break-all}
button{font-size:15px;padding:8px 16px;cursor:pointer}</style></head>
```

```

<body>
<h1>파일이 어떻게 가는지 보기</h1>
<p class="안내">파일을 하나 고르고 버튼을 누르세요.<br>
F12 → Network 에서 그 요청의 <b>Payload</b> 를 함께 보세요.</p>

<input type="file" id="파일" />
<input type="text" id="설명" placeholder="설명을 적으세요" value="설비 사진입니다" />
<button id="보내기">보내기</button>
<pre id="결과">아직 안 보냄</pre>

<script>
document.querySelector("#보내기").addEventListener("click", async () => {
  const 결과 = document.querySelector("#결과");
  const 파일 = document.querySelector("#파일").files[0];

  if (!파일) { 결과.textContent = "파일을 먼저 고르세요."; return; }

```

★ FormData 를 쓰면 브라우저가 multipart 형식을 알아서 만들어 줍니다.

```

const 폼 = new FormData();
폼.append("file", 파일);
폼.append("desc", document.querySelector("#설명").value);

결과.textContent = "보내는 중...";

```

★★ headers 를 직접 쓰지 마세요. Content-Type 을 우리가 쓰면 boundary 가 안 붙어서 서버가 못 읽습니다.

```

const 응답 = await fetch("/raw", { method: "POST", body: 폼 });

결과.textContent =
  "고른 파일: " + 파일.name + "\\n" +
  "크기: " + 파일.size + " 바이트\\n" +
  "형식: " + (파일.type || "(브라우저가 모름)") + "\\n\\n" +
  JSON.stringify(await 응답.json(), null, 2);
});
</script></body></html>`);
});

```

```

GET /
200

```

★ HTML 폼으로도 보낼 수 있습니다. 자바스크립트 없어요.

```

<form action="/upload" method="post" enctype="multipart/form-data">

```

```



```

```

</form>

```

enctype="multipart/form-data" 를 빼면 파일이 안 갑니다. 파일 '이름' 만 글자로 갑니다. 내용은 안 갑니다. 폼으로 업로드가 안 될 때 1번 원인입니다.

서버가 받는 순간 무슨 일이 일어나나

섹션 5

```

app.get("/what-happens", (req, res) => {
  res.json({
    순서: [
      "브라우저가 파일을 조각내어 보냅니다. 큰 파일은 여러 번에 나눠 옵니다.",
      "서버는 조각을 받는 대로 처리해야 합니다. 다 모을 때까지 기다리면 메모리가 찹니다.",
      "multer 가 조각을 받아 바로 디스크에 씁니다. (또는 메모리에 담습니다)",
      "다 받으면 req.file 에 정보를 담고 next() 를 부릅니다.",
      "우리 라우트가 실행됩니다. 이때 파일은 이미 저장이 끝난 상태입니다.",
    ],
    핵심: "multer 는 미들웨어입니다. 라우트가 실행될 때는 이미 저장이 끝나 있습니다.",
    주의: "그래서 '검사해서 거절' 을 하려면 저장 전에 해야 합니다. 개념03 에서 봅니다.",
  });
});

```

GET /what-happens

200

★ 03단원 개념04 의 스트림 이야기가 여기서 다시 나옵니다. "본문은 조각으로 온다" 를 배웠는데, 파일은 그게 훨씬 중요합니다. 500MB 동영상을 통째로 메모리에 올리면 서버가 죽습니다.

```

app.use((req, res) => {
  res.status(404).json({ error: { code: "NOT_FOUND", message: "찾을 수 없습니다" } });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  res.status(500).json({ error: { code: "INTERNAL_ERROR", message: "서버에서 문제가 생겼습니다" } });
});

```

```
app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}`);
  console.log("브라우저로 열어서 파일을 하나 보내 보세요.");
});
```

세 가지 본문 형식 정리

형식 무엇을 담나 읽는 도구

application/json	글자·숫자·구조	express.json()
application/x-www-form-urlencoded	글자만 (짧은 폼)	express.urlencoded()
multipart/form-data	글자 + 파일	multer

★ 셋 다 '본문에 담아 보내는 것' 입니다. 담는 방법이 다를 뿐입니다. 그래서 읽는 도구도 각각 다릅니다.

★ 하나의 요청에 하나의 형식만 쓸 수 있습니다. "JSON 도 보내고 파일도 보내고" 는 안 됩니다. 파일이 있으면 multipart 를 쓰고, 글자 값은 그 안에 함께 담습니다.

직접 해 볼 것

▫ 직접 해보기 1

브라우저로 열어 파일을 보내고, F12 → Network → 그 요청 → Payload 를 보세요. boundary 글자가 어떻게 생겼나요?

▫ 직접 해보기 2

같은 파일을 두 번 보내 보세요. boundary 가 같나요?

▫ 직접 해보기 3

화면의 fetch 에 headers 를 직접 넣어 보세요.

```
headers: { "Content-Type": "multipart/form-data" }
```

서버가 본문을 읽나요? 왜 안 될까요? (확인 후 되돌리세요)

▫ 직접 해보기 4

아주 큰 파일(10MB 이상)을 골라 보내 보세요. Network 탭에서 시간이 얼마나 걸리나요?

▫ 직접 해보기 5

확장자가 없는 파일을 골라 보세요. 화면에 찍히는 '형식' 이 무엇으로 나오나요? 브라우저는 형식을 어떻게 알아낼까요?

HTML 폼 방식으로도 만들어 보세요. `enctype` 을 빼면 서버에 무엇이 도착하나요?

자주 하는 실수

이런 실수를 합니다

JSON 에 파일을 담으려 함

JSON 은 글자만 담습니다. Base64 로 바꾸면 되지만 크기와 메모리 문제가 생깁니다.

이런 실수를 합니다

fetch 에 Content-Type 을 직접 씀

boundary 가 안 붙어서 서버가 조각을 못 나눕니다. FormData 를 body 에 넣으면 브라우저가 알아서 붙입니다. 건드리지 마세요.

이런 실수를 합니다

HTML 폼에 enctype 을 안 씀

파일 이름만 글자로 가고 내용은 안 갑니다.

이런 실수를 합니다

express.json() 으로 파일을 받으려 함

req.body 가 undefined 입니다. multer 가 필요합니다.

이런 실수를 합니다

파일을 통째로 메모리에 올림

큰 파일 하나로 서버가 멈출 수 있습니다. 디스크에 흘려보내야 합니다.

이런 실수를 합니다

라우트에서 파일을 '받기 전에' 검사하려 함

multer 가 미들웨어라, 라우트가 실행될 때는 이미 저장이 끝나 있습니다. 검사는 multer 에게 시켜야 합니다. (개념03)

정리

- 1 파일은 JSON 으로 못 보낸다. JSON 은 글자만 담는 형식이다.
- 2 그래서 multipart/form-data 라는 다른 형식을 쓴다. 구분선으로 조각을 나누고, 조각마다 이름과 내용을 담는다.
- 3 express.json() 은 이 형식을 못 읽는다. 읽는 도구가 따로 필요하다(multer).
- 4 fetch 로 보낼 때 Content-Type 을 직접 쓰지 않는다. 구분선 값이 들어가야 하는데 그건 브라우저만 안다. FormData 를 주면 알아서 붙인다.
- 5 HTML 폼으로 보낼 때는 enctype="multipart/form-data" 를 반드시 적는다.
- 6 파일을 통째로 메모리에 올리지 않는다. 큰 파일 몇 개면 서버가 넘어간다.
- 7 검사는 파일을 받은 뒤에 한다. 받기 전에는 무엇이 오는지 알 수 없다.

multer 시작하기 (그리고 한글 함정 세 가지)

RUN node 개념02_multer_시작하기.js

그러면 Ctrl + C

★★ 이 파일의 섹션 4·5·6 은 한국에서 개발할 때 반드시 만나는 문제입니다.

영어권 자료에는 아예 안 나옵니다. 꼭 읽으세요.

```
const express = require("express");
const multer = require("multer");
const path = require("path");
const fs = require("fs");

const app = express();
const PORT = Number(process.env.PORT) || 3000;

app.use(express.json());
```

가장 단순한 사용법

섹션 1

```
npm install multer
```

dest 만 주면 그 폴더에 저장해 줍니다.

```
const 업로드폴더 = path.join(__dirname, "uploads");
fs.mkdirSync(업로드폴더, { recursive: true });

const 간단업로드 = multer({ dest: 업로드폴더 });
```

★ upload.single("필드이름") "이 이름으로 파일 하나가 올 것이다" 라고 알려 주는 것입니다. 프론트의 formData.append("file", 파일) 의 "file" 과 같아야 합니다.

```
app.post("/upload", 간단업로드.single("file"), (req, res) => {
```

파일 정보는 req.file 에, 글자 필드는 req.body 에 담깁니다.

```
res.json({
  파일있나: !!req.file,
  fieldname: req.file?.fieldname,
  mimetype: req.file?.mimetype,
  저장된이름: req.file?.filename ? "무작위 글자" : null,
  본문: req.body,
});
});
```

POST /upload multipart:@file=report.pdf&desc=설비 사진입니다

```
200 {"파일있나":true,"fieldname":"file","mimetype":"application/octet-stream","저장
된이름":"무작위 글자","본문":{"desc":"설비 사진입니다"}}
```

★ 저장된 파일 이름이 무작위입니다. uploads 폴더를 열어 보면 105c3f104f8bd626... 같은 이름이 있습니다. 확장자도 없습니다.

왜 이렇게 하나

같은 이름의 파일을 두 사람이 올리면 덮어써지기 때문입니다.
그리고 파일 이름에 위험한 글자가 들어올 수 있습니다. (../../.env 같은)

그런데 이대로는 나중에 무슨 파일인지 알 수가 없습니다. 그래서 보통 이름을 직접 정합니다. 섹션 3 에서 합니다.

req.file 에는 무엇이 들어 있나

섹션 2

```
app.post("/file-info", 간단업로드.single("file"), (req, res) => {
  if (!req.file) {
    return res.status(400).json({ error: { code: "NO_FILE", message: "파일이
    없습니다" } });
  }

  const f = req.file;

  res.json({
    fieldname: f.fieldname, // 프론트가 쓴 필드 이름 originalname: f.originalname, //
    원래 파일 이름 * 섹션 5 를 꼭 보세요 mimetype: f.mimetype, // 브라우저가 알려 준
    형식 size: f.size, // 바이트 destination: "(저장 폴더)", // 컴퓨터마다 달라서
    가렸습니다 filename: "(저장된 이름)",
    path: "(폴더 + 이름)",
  });
});
```

```
POST /file-info multipart:@file=report.pdf
```

```
200 {"fieldname":"file","originalname":"report.pdf","mimetype":"application/octet-stream","size":20,"destination":"(저장 폴더)","filename":"(저장된 이름)","path":"(폴더 + 이름)"}
```

★ size 는 바이트입니다. 글자 수가 아닙니다. 여기서 보낸 내용은 “가짜 파일 내용” 인데 20 바이트입니다. 한글 한 글자가 UTF-8 에서 3바이트라서 그렇습니다. (02단원 개념02)

★★ mimetype 을 믿지 마세요. 이걸 ‘브라우저가 알려 준 것’ 입니다. 보내는 쪽이 마음대로 정할 수 있습니다. Postman 으로 exe 파일에 image/png 라고 붙여 보낼 수 있습니다. 진짜 검사는 개념03 에서 봅니다.

```
POST /file-info
```

```
400 {"error":{"code":"NO_FILE","message":"파일이 없습니다"}}
```

★★★ 파일을 안 보내도 에러가 안 납니다. multer 는 그냥 req.file 을 `undefined` 로 두고 넘어갑니다. “파일이 필수” 라면 우리가 직접 확인해서 400 을 내야 합니다. 이걸 빠뜨리면 req.file.originalname 에서 `TypeError` 로 서버가 500 을 냅니다.

저장 이름을 직접 정하기 (diskStorage)

섹션 3

```
const 저장방식 = multer.diskStorage({
```

어느 폴더에 저장할지

```
destination: (req, file, cb) => {  
  cb(null, 업로드폴더);  
},
```

어떤 이름으로 저장할지

```
filename: (req, file, cb) => {
```

★ 여기서도 originalname 은 깨져 있습니다. 섹션 5 를 보세요.

```
const 원래이름 = Buffer.from(file.originalname, "latin1").toString("utf8");  
const 확장자 = path.extname(원래이름).toLowerCase();
```

겹치지 않는 이름을 만듭니다.

```
const 고유값 = `${Date.now()}-${Math.round(Math.random() * 1e9)}`;
```

```

},
});

const 업로드 = multer({ storage: 저장방식 });

app.post("/upload-named", 업로드.single("file"), (req, res) => {
  const 원래이름 = Buffer.from(req.file.originalname, "latin1").toString("utf8");

  res.json({
    원래이름,
    확장자: path.extname(원래이름).toLowerCase(),
    저장된이름끝: path.extname(req.file.filename),
    이름에한글있나: /[가-힣]/.test(req.file.filename),
  });
});

```

POST /upload-named multipart:@file=작업표준서.PDF

```

200 {"원래이름":"작업표준서.PDF","확장자":".pdf","저장된이름끝":".pdf","이름에한글있나":false}

```

★ 저장된 이름에는 한글이 없습니다. 확장자만 남겼기 때문입니다.

왜 원래 이름을 그대로 안 쓰나

- ① 같은 이름이 겹칩니다
- ② 파일 이름에 위험한 글자가 들어올 수 있습니다 (../ 같은)
- ③ 서버 운영체제에 따라 한글 파일 이름이 깨지기도 합니다

원래 이름은 데이터베이스나 JSON 에 따로 적어 둡니다. 보여 줄 때는 원래 이름, 저장은 안전한 이름 — 이렇게 나눕니다. (개념05)

★ 확장자를 소문자로 바꾼 것에 주의하세요. .PDF 로 올려도 .pdf 로 저장됩니다. (02단원 연습문제 2·13)

★★★ 한글 함정 ① — 필드 이름

섹션 4

프론트에서 이렇게 보내면 어떻게 될까요?

```

formData.append("파일", 파일);

```

그리고 서버에서 upload.single("파일") 이라고 씁니다. 당연히 될 것 같습니다.

```

app.post("/ko-field", 간단업로드.single("파일"), (req, res) => {
  res.json({ 여기까지왔나: true });
});

```

POST /ko-field multipart:@파일=report.pdf

400

★★★ 400 이 납니다. MulterError: Unexpected field

왜 그럴까요? multipart 본문의 필드 이름은 latin1 로 해석됩니다. "파일" 이 "ᄁᆞᆫ" 처럼 바뀌어서 도착합니다. upload.single("파일") 은 "파일" 을 기다리는데 "ᄁᆞᆫ" 가 오니 못 알아봅니다.

에러 메시지가 "Unexpected field" 라서 원인을 짐작하기 어렵습니다. "필드 이름을 분명히 맞췄는데 왜 안 되지?" 로 한참 헤매게 됩니다.

★ 규칙: 필드 이름은 반드시 영어로.

```
formData.append("file", 파일)
upload.single("file")
```

04단원의 "라우트 주소는 영어로", 05단원의 "헤더는 영어로" 와 같은 이야기입니다. 한글은 '값' 에만 쓰세요.

★★★ 한글 함정 ㉔ — 파일 이름이 깨진다

섹션 5

```
app.post("/ko-filename", 간단업로드.single("file"), (req, res) => {
  const 그대로 = req.file.originalname;
  const 되돌림 = Buffer.from(그대로, "latin1").toString("utf8");
```

★ 깨진 글자를 응답에 그대로 담으면, 눈에 안 보이는 글자가 섞여 비교할 때마다 결과가 달라집니다. 그래서 길이만 함께 보여 줍니다. 깨진 모습은 터미널에서 직접 보세요.

```
console.log(`  깨진 그대로: ${그대로}`);

res.json({
  깨진글자수: 그대로.length,
  되돌린것: 되돌림,
  되돌린글자수: 되돌림.length,
  바뀌었나: 그대로 !== 되돌림,
});
});
```

POST /ko-filename multipart:@file=작업표준서.pdf

200 {"깨진글자수":19,"되돌린것":"작업표준서.pdf","되돌린글자수":9,"바뀌었나":true}

```
POST /ko-filename multipart:@file=report-2026.png
```

```
200 {"깨진글자수":15,"되돌린것":"report-2026.png","되돌린글자수":15,"바뀌었나":false}
```

★ 글자 수를 보세요. 한글 5글자 + “.pdf” = 9글자여야 하는데 19글자로 도착했습니다. 한글 한 글자가 UTF-8 에서 3바이트인데, 그 바이트 하나하나를 글자 하나로 잘못 읽어서 $5 \times 3 = 15$, 거기에 .pdf 4글자를 더해 19 입니다.

터미널에는 이렇게 찍힙니다.

```
깨진 그대로: ìììììì.pdf
```

★★★ 작업표준서.pdf 가 이렇게 도착합니다.

왜 그런가

```
multipart 규격이 아주 오래전에 만들어져서, 파일 이름을 latin1 로 읽습니다.
브라우저는 UTF-8 로 보내는데 받는 쪽이 latin1 로 해석하니 깨집니다.
```

어떻게 고치나

```
Buffer.from(이름, "latin1").toString("utf8")
```

```
"latin1 로 잘못 읽은 것을 바이트로 되돌린 뒤, utf8 로 다시 읽어라" 입니다.
```

★ 두 번째를 보세요. 영어 파일 이름은 안 바뀝니다. 그래서 이 되돌리기를 ‘항상’ 적용해도 안전합니다. 조건문 없이 그냥 감싸면 됩니다.

★★ 이 함정이 왜 무서운가 영어 파일로만 시험하면 아무 문제가 없습니다. 한글 이름 파일을 올린 사용자만 이름이 깨집니다. 그리고 그 파일은 이미 저장된 뒤라 되돌리기 어렵습니다.

개발할 때 반드시 한글 파일 이름으로 시험해 보세요.

```
app.post("/ko-body", 간단업로드.single("file"), (req, res) => {
  const 원본 = req.body.desc;
  const 괜히되돌림 = Buffer.from(원본, "latin1").toString("utf8");

  console.log(`   괜히 되돌리면: ${괜히되돌림}`);

  res.json({
    본문값그대로: 원본,
    되돌리면망가지나: 원본 !== 괜히되돌림,
    파일이름은: Buffer.from(req.file.originalname, "latin1").toString("utf8"),
  });
});
```

POST /ko-body multipart:@file=작업표준서.pdf&desc=설비 사진입니다

200 {"본문값그대로":"설비 사진입니다","되돌리면망가지나":true,"파일이름은":"작업표준서.pdf"}

★★★ 여기가 가장 헛갈리는 부분입니다.

req.body.desc 는 안 깨졌습니다. “설비 사진입니다” 그대로입니다. 그런데 여기에 되돌리기를 적용하면 오히려 망가집니다.

왜 다른가

파일 '이름' 은 헤더 줄(Content-Disposition)에 들어 있어서 latin1 로 읽힙니다.
글자 필드의 '값' 은 본문 내용이라 UTF-8 로 제대로 읽힙니다.

같은 요청 안에서 한쪽은 깨지고 한쪽은 안 깨집니다.

★ 규칙 한 줄

되돌리기는 file.originalname 에만 적용합니다.
req.body 의 값에는 절대 적용하지 마세요.

“한글이 깨지네? 전부 되돌리자” 하고 감싸면 멀쩡한 값까지 망가집니다.

메모리에 담기 (memoryStorage)

섹션 7

```
const 메모리업로드 = multer({ storage: multer.memoryStorage() });

app.post("/to-memory", 메모리업로드.single("file"), (req, res) => {
  res.json({
    버퍼인가: Buffer.isBuffer(req.file.buffer),
    크기: req.file.size,
    앞부분: req.file.buffer.toString("utf8").slice(0, 10),
    path있나: req.file.path !== undefined,
  });
});
```

```
POST /to-memory multipart:@file=report.txt
```

```
200 {"버퍼인가":true,"크기":20,"앞부분":"가짜 파일 내용","path있나":false}
```

★ 디스크에 안 쓰고 메모리에만 담습니다. req.file.buffer 로 내용을 바로 볼 수 있습니다. 대신 path 와 filename 은 없습니다. 저장한 게 아니니까요.

★ 언제 쓰나 · 받자마자 S3 같은 곳으로 넘길 때 (PART 4) · 엑셀·CSV 를 읽어서 데이터베이스에 넣고 파일은 버릴 때 · 이미지 크기를 줄여서 저장할 때

★★ 반드시 용량 제한을 함께 두세요. 제한이 없으면 500MB 파일 하나로 서버 메모리가 찹니다. 여러 명이 동시에 올리면 서버가 죽습니다. (개념03)

```
app.use((req, res) => {
  res.status(404).json({ error: { code: "NOT_FOUND", message: "찾을 수 없습니다" } });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.name}: ${err.message}`);

  if (err.name === "MulterError") {
    return res.status(400).json({
      error: { code: err.code, message: err.message, field: err.field },
    });
  }

  res.status(500).json({ error: { code: "INTERNAL_ERROR", message: "서버에서 문제가 생겼습니다" } });
});
```

```
app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}`);
  console.log(`업로드 폴더: ${업로드폴더}`);
});
```

한글 함정 세 가지 정리

무엇 한글이 되나 어떻게

필드 이름 × 영어로 쓰세요 (file, photo) 파일 이름 △ Buffer.from(x,"latin1").toString("utf8") 글자 필드의 값 ○ 그대로 쓰세요. 건드리면 망가집니다 응답 JSON ○ 그대로

△ 는 "받을 수는 있는데 되돌려야 한다" 는 뜻입니다.

★ 이 자료에서 지금까지 나온 '한글이 안 되는 곳' 을 모으면

라우트 주소 (04단원) 조용히 404 응답 헤더 값 (05단원) 500 으로 터짐 요청 헤더 값 (05단원) 요청이 안 나감 업로드 필드 이름 (09단원) Unexpected field 업로드 파일 이름 (09단원) 깨져서 도착

공통점: 전부 '헤더에 실려 가는 것' 입니다. 본문에 담기는 것(JSON, 폼 값, 응답 내용)은 한글이 잘 됩니다.

직접 해 볼 것

▫ 직접 해보기 1

Postman 으로 파일을 올려 보세요. Body → form-data → 왼쪽 칸에 file, 오른쪽에서 File 을 고릅니다. 한글 이름 파일과 영어 이름 파일을 각각 올려 비교하세요.

▫ 직접 해보기 2

uploads 폴더를 열어 보세요. /upload 로 올린 것과 /upload-named 로 올린 것의 이름이 어떻게 다른가요?

▫ 직접 해보기 3

/ko-filename 에서 되돌리기를 지우고 한글 파일을 올려 보세요. 깨진 이름이 그대로 나옵니다. 그 이름으로 저장하면 어떻게 될까요?

▫ 직접 해보기 4

/ko-body 의 되돌리기를 req.body 전체에 적용해 보세요. 설명이 어떻게 되나요? (확인 후 되돌리세요)

▫ 직접 해보기 5

diskStorage 의 filename 에서 확장자를 안 붙여 보세요. 저장된 파일을 더블클릭하면 열리나요?

▹ 직접 해보기 6

되돌리기를 함수로 빼서 `utils` 에 두세요.

```
function 파일이름되돌리기(이름) {
  return Buffer.from(이름, "latin1").toString("utf8");
}
```

앞으로 모든 업로드에서 쓰게 됩니다.

▹ 직접 해보기 7

`memoryStorage` 로 텍스트 파일을 올리고 `req.file.buffer.toString("utf8")` 로 내용을 통째로 찍어 보세요.

자주 하는 실수

이런 실수를 합니다

필드 이름을 한글로 씀

```
MulterError: Unexpected field.
```

원인을 짐작하기 어렵습니다.

이런 실수를 합니다

`originalname` 을 그대로 씀

한글 파일 이름이 깨진 채로 저장되거나 화면에 나옵니다.

이런 실수를 합니다

되돌리기를 `req.body` 에도 적용

멀쩡한 글자가 망가집니다. `originalname` 에만 쓰세요.

이런 실수를 합니다

`req.file` 이 있는지 확인 안 함

파일 없이 요청하면 `undefined` 라서 `TypeError` 로 500 이 납니다.

이런 실수를 합니다

`mimetype` 을 믿음

보내는 쪽이 마음대로 정합니다. 진짜 검사는 개념03 에서 합니다.

이런 실수를 합니다

원래 파일 이름 그대로 저장

같은 이름이 덮어써지고, ../ 같은 위험한 글자가 들어올 수 있습니다.

이런 실수를 합니다

memoryStorage 를 용량 제한 없이 씀

큰 파일 몇 개로 서버 메모리가 찹니다.

정리

- 1 multer 가 multipart 를 읽어서 req.file(또는 req.files)에 담아 준다.
- 2 필드 이름에 한글을 쓰면 안 된다. Unexpected field 로 거절당한다. 헤더에 실려 가는 것에는 한글을 못 쓰기 때문이다.
- 3 파일 이름의 한글은 깨져서 도착한다. Buffer.from(originalname, "latin1").toString("utf8") 로 되돌린다.
- 4 그 되돌리기를 req.body 값에는 하면 안 된다. 글자 필드는 안 깨져서 오므로 되돌리면 오히려 망가진다. 깨지는 것은 헤더에 실리는 것뿐이다.
- 5 originalname 을 그대로 저장 이름으로 쓰지 않는다. 같은 이름이 오면 덮어쓰고, 이상한 글자가 섞이면 경로가 망가진다.
- 6 저장 이름은 diskStorage 에서 직접 정한다. 겹치지 않게 만든다.
- 7 req.file 이 있는지 먼저 확인한다. 안 보냈으면 undefined 다.
- 8 mimetype 은 보내는 쪽이 정하는 값이라 믿으면 안 된다.
- 9 memoryStorage 는 용량 제한과 함께 쓴다. 안 그러면 메모리가 바로 찬다.

제한과 검증

```
RUN node 개념03_제한과_검증.js
```

그러면 Ctrl + C

업로드는 서버에서 가장 위험한 입구입니다.

· 500MB 파일 몇 개면 디스크가 찹니다 · 실행 파일을 올려 두고 그 주소를 열면 서버에서 실행될 수도 있습니다 · 파일 이름에 ../ 를 넣어 서버 파일을 덮어쓸 수 있습니다

그래서 “무엇을, 얼마나, 어떤 이름으로” 를 전부 정해 줘야 합니다.

```
const express = require("express");
const multer = require("multer");
const path = require("path");
const fs = require("fs");

const app = express();
const PORT = Number(process.env.PORT) || 3000;

app.use(express.json());

const 업로드폴더 = path.join(__dirname, "uploads");
fs.mkdirSync(업로드폴더, { recursive: true });
```

09단원 내내 쓸 도우미입니다. (개념02 섹션 5)

```
function 이름되돌리기(이름) {
  return Buffer.from(이름, "latin1").toString("utf8");
}
```

날짜별 폴더에 저장하기

섹션 1

한 폴더에 파일이 십만 개 쌓이면 열어 보기도 힘듭니다. 날짜로 나눠 두면 찾기도 쉽고, 오래된 것을 지우기도 쉽습니다.

```
function 오늘폴더() {
  const 지금 = new Date();
  const 년 = 지금.getFullYear();
  const 월 = String(지금.getMonth() + 1).padStart(2, "0");

  const 폴더 = path.join(업로드폴더, String(년), 월);
  fs.mkdirSync(폴더, { recursive: true }); // * 그 달의 첫 업로드에서 폴더를
  만듭니다 return 폴더;
}
```

★ 02단원 연습문제 13·15 에서 만든 그 함수입니다. “저장하기 직전에 항상 mkdirSync 를 부르라” 고 했던 이유가 여기 있습니다.

```
const 저장방식 = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 오늘폴더());
  },

  filename: (req, file, cb) => {
    const 원래이름 = 이름되돌리기(file.originalname);
    const 확장자 = path.extname(원래이름).toLowerCase();
    const 고유값 = `${Date.now()}-${Math.round(Math.random() * 1e9)}`;

    cb(null, `${고유값}${확장자}`);
  },
});
```

확장자 검사 (fileFilter)

섹션 2

```
const 허용확장자 = [".jpg", ".jpeg", ".png", ".gif", ".pdf"];

function 확장자검사(req, file, cb) {
  const 원래이름 = 이름되돌리기(file.originalname);
  const 확장자 = path.extname(원래이름).toLowerCase();

  if (!허용확장자.includes(확장자)) {
```

★ cb(에러) 를 부르면 그 파일은 저장되지 않습니다. 에러는 우리 에러 처리기로 갑니다.

```
const 에러 = new Error(`${확장자} || "확장자 없는 파일"은(는) 올릴 수 없습니다`);
에러.status = 400;
에러.code = "BAD_EXTENSION";
return cb(에러);
}

cb(null, true); // 통과
}
```

★★ fileFilter 안에서도 이름을 되돌려야 합니다. 안 하면 한글 이름 파일의 확장자를 못 읽어서 전부 거절됩니다. “영어 파일은 되는데 한글 파일만 거절돼요” 의 원인입니다.

★ 허용 목록 방식으로 쓰세요. “이건 안 됨” 목록(.exe, .bat ...)은 빠뜨리는 게 반드시 생깁니다. “이것만 됨” 목록은 빠뜨려도 안전한 쪽으로 틀립니다.

용량과 개수 제한

섹션 3

```
const 업로드 = multer({
  storage: 저장방식,
  fileFilter: 확장자검사,
  limits: {
    fileSize: 100, // 시험용으로 100 바이트. 실제로는 5 * 1024 * 1024 처럼
    씁니다 files: 2, // 한 번에 두 개까지 fields: 10, // 글자 필드 개수 fieldNameSize:
    100, // 필드 이름 길이
  },
});

app.post("/upload", 업로드.single("file"), (req, res) => {
  if (!req.file) {
    return res.status(400).json({ error: { code: "NO_FILE", message: "파일이
    없습니다" } });
  }

  res.status(201).json({
    data: {
      원래이름: 이름되돌리기(req.file.originalname),
      크기: req.file.size,
      저장위치: path.relative(업로드폴더, req.file.path).replace(/\\g, "/"),
    },
  });
});
```

POST /upload multipart:@file=작업표준서.pdf:50

201

POST /upload multipart:@file=큰파일.pdf:500

400 {"error":{"code":"LIMIT_FILE_SIZE","message":"파일이 너무 큼니다 (최대 100바이트)","field":"file"}}

POST /upload multipart:@file=악성.exe:10

400 {"error":{"code":"BAD_EXTENSION","message":".exe 은(는) 올릴 수 없습니다"}}

POST /upload multipart:@file=이름없음:10

400 {"error":{"code":"BAD_EXTENSION","message":"확장자 없는 파일 은(는) 올릴 수 없습니다"}}

POST /upload

400 {"error":{"code":"NO_FILE","message":"파일이 없습니다"}}

★ 실제 서비스에서 쓰는 값

fileSize: 5 * 1024 * 1024	사진 5MB
fileSize: 20 * 1024 * 1024	문서 20MB

숫자를 그대로 5242880 이라고 쓰지 마세요. 읽는 사람이 계산해야 합니다. 5 * 1024 * 1024 로 쓰면 한눈에 보입니다.

여러 개 올릴 때

섹션 4

```
app.post("/upload-many", 업로드.array("files", 2), (req, res) => {
  res.status(201).json({
    data: {
      개수: req.files.length,
      이름들: req.files.map((f) => 이름되돌리기(f.originalname)),
    },
  });
});
```

POST /upload-many multipart:@files=a.pdf:10&@files=b.png:10

201 {"data":{"개수":2,"이름들":["a.pdf","b.png"]}}

POST /upload-many multipart:@files=a.pdf:10&@files=b.png:10&@files=c.pdf:10

400 {"error":{"code":"LIMIT_FILE_COUNT","message":"파일이 너무 많습니다"}}

★ `array("필드", 최대개수)` 의 최대개수와 `limits.files` 는 둘 다 있어야 안전합니다. `array` 의 숫자는 그 필드만, `limits.files` 는 요청 전체를 봅니다.

★ `field` 가 없는 것에 주의하세요. `LIMIT_FILE_SIZE` 에는 `err.field` 가 있는데 `LIMIT_FILE_COUNT` 에는 없습니다. "몇 번째 파일이 문제인가" 를 셀 수 없는 상황이라 그렇습니다. 그래서 안내 문구를 만들 때 `err.field` 가 `undefined` 일 수 있다고 생각해야 합니다. (`JSON.stringify` 는 `undefined` 인 속성을 아예 빼고 찍습니다)

★ mimetype 을 믿으면 안 되는 이유

섹션 5

`req.file.mimetype` 은 '보내는 쪽이 알려 준 값' 입니다. Postman 이나 직접 만든 요청에서는 마음대로 정할 수 있습니다.

진짜로 확인하려면 파일 '내용의 첫 몇 바이트' 를 봐야 합니다. 파일 형식마다 정해진 시작 바이트가 있습니다. 이것 매직 넘버라고 합니다.

```
const 매직넘버 = {
  ".png": [0x89, 0x50, 0x4e, 0x47], // %PNG ".jpg": [0xff, 0xd8, 0xff],
  ".jpeg": [0xff, 0xd8, 0xff],
  ".gif": [0x47, 0x49, 0x46], // GIF ".pdf": [0x25, 0x50, 0x44, 0x46], // %PDF
};

function 진짜형식인가(버퍼, 확장자) {
  const 기대 = 매직넘버[확장자];
  if (!기대) return true; // 우리가 모르는 형식은 통과시킵니다 return
  기대.every((바이트, i) => 버퍼[i] === 바이트);
}
```

내용을 봐야 하니 메모리에 담습니다.

```
const 메모리업로드 = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: 1024 },
  fileFilter: 확장자검사,
});

app.post("/upload-checked", 메모리업로드.single("file"), (req, res) => {
  if (!req.file) {
    return res.status(400).json({ error: { code: "NO_FILE", message: "파일이 없습니다" } });
  }

  const 원래이름 = 이름되돌리기(req.file.originalname);
  const 확장자 = path.extname(원래이름).toLowerCase();

  if (!진짜형식인가(req.file.buffer, 확장자)) {
```

올린 파일 보여 주고 내려받게 하기

RUN node 개념04_올린_파일_보여주기.js

그러면 Ctrl + C

파일을 받아 저장까지 했습니다. 이제 다시 보여 줘야 합니다.

· 사진이면 화면에 로 · PDF 면 새 탭에서 열리게 · 엑셀이면 내려받아지게

셋이 조금씩 다릅니다. 그리고 여기에도 위험한 구멍이 있습니다.

```
const express = require("express");
const multer = require("multer");
const path = require("path");
const fs = require("fs");

const app = express();
const PORT = Number(process.env.PORT) || 3000;

app.use(express.json());

const 업로드폴더 = path.join(__dirname, "uploads");
fs.mkdirSync(업로드폴더, { recursive: true });

function 이름되돌리기(이름) {
  return Buffer.from(이름, "latin1").toString("utf8");
}
```

파일 기록을 따로 남깁니다

섹션 1

저장된 이름은 무작위입니다. 원래 이름은 어디에 둘까요? 파일과 짝을 이루는 '기록' 을 따로 만듭니다.

저장된 이름 1786683001682-123456789.pdf ← 겹치지 않고 안전한 이름 원래 이름 작업표준서.pdf

← 사람에게 보여 줄 이름

진짜 서버라면 데이터베이스에 넣습니다. 여기서는 배열에 둡니다. (07단원)

```
let 파일기록들 = [
  {
    id: 1,
    저장이름: "sample-report.pdf",
    원래이름: "작업표준서.pdf",
    크기: 12,
    형식: "application/pdf",
    올린날: "2026-08-01",
  },
];

let 다음번호 = 2;
```

시험용 파일을 하나 만들어 둡니다.

```
fs.writeFileSync(path.join(업로드폴더, "sample-report.pdf"), "가짜 PDF", "utf-8");

const 저장방식 = multer.diskStorage({
  destination: (req, file, cb) => cb(null, 업로드폴더),
  filename: (req, file, cb) => {
    const 확장자 = path.extname(이름되돌리기(file.originalname)).toLowerCase();
    cb(null, `${Date.now()}-${Math.round(Math.random() * 1e9)}${확장자}`);
  },
});

const 업로드 = multer({ storage: 저장방식, limits: { fileSize: 1024 } });

app.post("/api/v1/files", 업로드.single("file"), (req, res) => {
  if (!req.file) {
    return res.status(400).json({ error: { code: "NO_FILE", message: "파일이 없습니다" } });
  }

  const 기록 = {
    id: 다음번호,
    저장이름: req.file.filename,
    원래이름: 이름되돌리기(req.file.originalname),
    크기: req.file.size,
    형식: req.file.mimetype,
    올린날: "2026-08-14",
  };

  다음번호 += 1;
  파일기록들.push(기록);
```

★ 응답에 저장이름을 넣지 않았습니다. 밖에서 알 필요가 없습니다. id 로 주소를 만들면 되니까요.

```
res.status(201).json({
  data: { id: 기록.id, 원래이름: 기록.원래이름, 크기: 기록.크기 },
});
});
```

POST /api/v1/files multipart:@file=검사성적서.pdf:30

201 {"data":{"id":2,"원래이름":"검사성적서.pdf","크기":30}}

```
app.get("/api/v1/files", (req, res) => {
  res.json({
    data: 파일기록들.map((기록) => ({
      id: 기록.id,
      원래이름: 기록.원래이름,
      크기: 기록.크기,
      보기: `/api/v1/files/${기록.id}/view`,
      내려받기: `/api/v1/files/${기록.id}/download`,
    })),
  });
});
```

GET /api/v1/files

200 {"data":[{"id":1,"원래이름":"작업표준서.pdf","크기":12,"보기":"/api/v1/files/1/view","내려받기":"/api/v1/files/1/download"}, {"id":2,"원래이름":"검사성적서.pdf","크기":30,"보기":"/api/v1/files/2/view","내려받기":"/api/v1/files/2/download"}]}

★ 주소를 함께 알려 주는 것이 친절합니다. 프론트가 주소 규칙을 외우지 않아도 됩니다. 주소가 바뀌어도 프론트를 안 고쳐도 됩니다.

방법 ① — 정적 폴더로 통째로 열기

섹션 2

가장 쉬운 방법입니다. 한 줄이면 됩니다.

```
app.use("/uploads", express.static(업로드폴더));
```

GET /uploads/sample-report.pdf

200

★ 아주 편하지만 큰 문제가 있습니다.

저장 이름만 알면 누구나 받아 갑니다. “로그인한 사람만”이나 “자기가 올린 것만” 같은 규칙을 걸 수가 없습니다.

저장 이름이 무작위라 못 맞힌다고요? 한 사람이 자기 파일 주소를 알면, 그 주소를 남에게 그냥 넘길 수 있습니다. 그리고 무작위 이름은 '비밀번호'가 아닙니다.

★ 언제 써도 되나 · 프로필 사진, 공개 게시물 첨부처럼 누가 봐도 되는 것 · 어차피 공개인 자료

★ 쓰면 안 되는 것 · 계약서, 급여명세서, 신분증 사진 · 사내 문서

이런 건 다음 방법을 쓰세요.

방법 ② — 라우트로 하나씩 내주기

섹션 3

```
function 기록찾기(req, res, next) {
  const 번호 = Number(req.params.id);

  if (!Number.isInteger(번호)) {
    return res.status(400).json({ error: { code: "VALIDATION_FAILED", message:
"번호는 숫자여야 합니다" } });
  }

  const 기록 = 파일기록들.find((기록) => 기록.id === 번호);

  if (!기록) {
    return res.status(404).json({ error: { code: "NOT_FOUND", message: "파일을 찾을
수 없습니다" } });
  }

  req.기록 = 기록;
  next();
}
```

★ 여기에 05단원의 인증·권한 미들웨어를 엮을 수 있습니다. `app.get("/api/v1/files/:id/view", 인증, 기록찾기, 내것인지확인, ...)` 정적 폴더 방식으로는 이게 불가능합니다.

```
app.get("/api/v1/files/:id/view", 기록찾기, (req, res) => {
  const 진짜경로 = path.join(업로드폴더, req.기록.저장이름);
```

★ `res.sendFile` 은 절대 경로를 요구합니다. 그리고 `Content-Type` 을 확장자로 알아서 정해 줍니다.

```
res.sendFile(진짜경로);
});
```

```
GET /api/v1/files/1/view
```

```
200
```

GET /api/v1/files/99/view

404 {"error":{"code":"NOT_FOUND","message":"파일을 찾을 수 없습니다"}}

GET /api/v1/files/abc/view

400 {"error":{"code":"VALIDATION_FAILED","message":"번호는 숫자여야 합니다"}}

★★ 주소에 저장 이름이 안 나옵니다. id 만 있습니다. 그래서 규칙을 걸 수 있고, 파일을 다른 곳으로 옮겨도 주소가 안 바뀝니다.

보기와 내려받기의 차이

섹션 4

```
app.get("/api/v1/files/:id/download", 기록찾기, (req, res) => {  
  const 진짜경로 = path.join(업로드폴더, req.기록.저장이름);
```

★ res.download 는 Content-Disposition: attachment 를 붙입니다. 브라우저가 화면에 열지 않고 '저장' 창을 띄웁니다. 두 번째 인자로 '사용자에게 보일 이름' 을 줍니다.

```
res.download(진짜경로, req.기록.원래이름);  
});
```

GET /api/v1/files/1/download

200

content-disposition=attachment; filename="?????.pdf"; filename*=UTF-8'%EC%9E%91%EC%97%85%ED%91%9C%EC%A4%80%EC%84%9C.pdf

★★★ 헤더를 보세요. 한글 이름이 두 가지 방식으로 들어 있습니다.

filename="?????.pdf"	← 옛날 방식. 한글이 물음표가 됩니다
filename*=UTF-8'%EC%9E%91%EC%97%85...	← 새 방식. 퍼센트 인코딩

헤더에는 한글을 담을 수 없다고 했습니다. (05단원 개념02) 그래서 이런 방법이 생겼습니다.

요즘 브라우저는 filename* 을 보고 한글 이름으로 저장합니다. 아주 오래된 브라우저는 filename 을 보고 물음표로 저장합니다.

★ Express 가 이걸 알아서 해 줍니다. 직접 만들지 마세요.

```
res.set("Content-Disposition", `attachment; filename="${원래이름}"`)  
이렇게 쓰면 한글 이름에서 500 이 납니다. 헤더에 한글을 담은 셈이니까요.
```

```
app.get("/api/v1/files/:id/inline", 기록찾기, (req, res) => {
  const 진짜경로 = path.join(업로드폴더, req.기록.저장이름);
```

화면에 열되 이름은 알려 주고 싶을 때

```
res.set("Content-Disposition", `inline; filename*=UTF-8'`${encodeURIComponent(req.
기록.원래이름)}`);
res.sendFile(진짜경로);
});
```

GET /api/v1/files/1/inline

200

content-disposition=inline; filename*=UTF-8'%EC%9E%91%EC%97%85%ED%91%9C%EC%A4%80%EC%84%9C.pdf

★ 정리

attachment → 저장 창이 뜹니다
inline → 브라우저가 열 수 있으면 화면에 엽니다 (PDF, 이미지)

직접 쓸 때는 encodeURIComponent 로 감싸야 합니다. 한글을 그대로 넣으면 ERR_INVALID_CHAR 로 500 이 납니다.

★★ 경로 탈출을 막아야 합니다

섹션 5

이런 라우트를 만들면 안 됩니다. 아주 흔한 실수입니다.

```
app.get("/bad/:name", (req, res) => {

  res.sendFile(path.join(업로드폴더, req.params.name));

});
```

/bad/..%2F..%2Fpackage.json 을 부르면 서버 파일이 나갑니다. .env 파일이면 데이터베이스 비밀번호가 통째로 새어 나갑니다.

```
app.get("/api/v1/raw/:name", (req, res) => {
  const 요청이름 = req.params.name;
  const 노린경로 = path.join(업로드폴더, 요청이름);
```

★ 만들어진 경로가 정말 업로드 폴더 '안' 인지 확인합니다. path.resolve 로 ../ 를 전부 정리한 뒤 비교합니다.

```
const 정리된경로 = path.resolve(노린경로);
const 정리된폴더 = path.resolve(업로드폴더);

if (!정리된경로.startsWith(정리된폴더 + path.sep)) {
  return res.status(400).json({
    error: { code: "BAD_PATH", message: "폴더 밖으로 나갈 수 없습니다" },
  });
}

if (!fs.existsSync(정리된경로)) {
  return res.status(404).json({ error: { code: "NOT_FOUND", message: "파일을 찾을 수 없습니다" } });
}

res.sendFile(정리된경로);
});
```

```
GET /api/v1/raw/sample-report.pdf
200
```

```
GET /api/v1/raw/..%2F..%2Fpackage.json
400 {"error":{"code":"BAD_PATH","message":"폴더 밖으로 나갈 수 없습니다"}}
```

```
GET /api/v1/raw/없는파일.pdf
404 {"error":{"code":"NOT_FOUND","message":"파일을 찾을 수 없습니다"}}
```

★★ 두 번째를 보세요. ..%2F 는 ../ 를 인코딩한 것입니다. req.params 는 되돌려진 상태로 오기 때문에 ../ 가 그대로 들어옵니다. path.join 이 그걸 계산해서 업로드 폴더 밖을 가리키게 됩니다.

★ 막는 방법 두 가지 ① 아예 이름을 안 받는다 → 섹션 3 방식. id 로만 찾습니다. 가장 안전합니다 ② 받되 폴더 안인지 확인한다 → 이 섹션 방식

①이 훨씬 낫습니다. 사용자가 파일 이름을 정할 이유가 없습니다. 08단원의 express.static 도 이 확인을 안에서 해 줍니다.

★ path.sep 을 붙인 이유 startsWith(정리된폴더) 만 쓰면 "uploads-secret" 같은 폴더도 통과합니다. 구분자까지 붙여야 정확히 '그 폴더 안' 이 됩니다.

화면

섹션 6

```
app.get("/", (req, res) => {
  res.type("html").send(`<!DOCTYPE html>
<html lang="ko"><head><meta charset="UTF-8"><title>파일 목록</title>
```



```

<style>body{font-family:"맑은 고딕",sans-serif;max-width:680px;margin:40px
auto;padding:0 20px;line-height:1.7}
h1{border-bottom:3px solid #333;padding-bottom:8px}
li{margin:6px 0}a{margin-right:10px}</style></head>
<body>
<h1>올린 파일</h1>
<input type="file" id="파일" /><button id="올리기">올리기</button>
<p id="알림"></p>
<ul id="목록"></ul>
<script>
async function 그리기() {
  const 답 = await (await fetch("/api/v1/files")).json();

```

★ 실무에서는 사용자가 정한 값(파일 이름)을 innerHTML 에 그대로 넣으면 안 됩니다(XSS).
textContent 나 이스케이프를 쓰세요. 여기서는 학습용으로 단순화했습니다.

```

document.querySelector("#목록").innerHTML = 답.data
  .map((f) => "<li>" + f.원래이름 + " (" + f.크기 + "바이트) " +
    '<a href="' + f.보기 + '" target="_blank">보기</a>' +
    '<a href="' + f.내려받기 + '">내려받기</a></li>')
  .join("");
}
document.querySelector("#올리기").addEventListener("click", async () => {
  const 파일 = document.querySelector("#파일").files[0];
  if (!파일) { document.querySelector("#알림").textContent = "파일을 고르세요.";
return; }
  const 폼 = new FormData();
  폼.append("file", 파일);
  const 응답 = await fetch("/api/v1/files", { method: "POST", body: 폼 });
  const 답 = await 응답.json();
  document.querySelector("#알림").textContent =
    응답.ok ? "올렸습니다: " + 답.data.원래이름 : "실패: " + 답.error.message;
  그리기();
});
그리기();
</script></body></html>`);
});

```

GET /

200

```

app.use((req, res) => {
  res.status(404).json({ error: { code: "NOT_FOUND", message: "찾을 수 없습니다" }
});
});

```

```

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.name}: ${err.message}`);

  if (err.name === "MulterError") {
    return res.status(400).json({ error: { code: err.code, message: err.message } });
  }

  res.status(500).json({ error: { code: "INTERNAL_ERROR", message: "서버에서 문제가 생겼습니다" } });
});

app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}`);
});

```

세 가지 방법 비교

방법 규칙을 걸 수 있나 주소에 파일명 언제

express.static × 나눔 공개 자료 라우트 + sendFile(id) ○ 안 나눔 비공개 자료 ★ 라우트 + 이름 받기
△ 나눔 쓰지 마세요

★ 기본은 두 번째입니다. id 로만 찾으면 경로 탈출이 원천적으로 불가능하고, 인증도 걸 수 있습니다.

★ 실제 서비스에서는 파일을 서버에 두지 않고 S3 같은 곳에 올립니다. (PART 4) 그리고 '잠깐만 유효한 주소' 를 만들어 줍니다. 구조는 지금과 같습니다. sendFile 자리가 주소 만들기로 바뀔 뿐입니다.

직접 해 볼 것

▫ 직접 해보기 1

브라우저로 열어 한글 이름 파일을 올리고 '내려받기' 를 누르세요. 저장 창의 파일 이름이 한글로 나오나요?

▫ 직접 해보기 2

F12 → Network 에서 내려받기 요청의 Content-Disposition 헤더를 보세요. filename 과 filename* 이 있나요?

▫ 직접 해보기 3

'보기' 와 '내려받기' 를 각각 눌러 보세요. 무엇이 다른가요? PDF 와 이미지에서 각각 해 보세요.

▫ 직접 해보기 4

/api/v1/raw/..%2F..%2Fpackage.json 을 브라우저로 열어 보세요. 막히나요? 확인 코드를 지우고 다시 해 보세요. package.json 이 나오나요? (확인 후 반드시 되돌리세요)

▫ 직접 해보기 5

/api/v1/files/:id/inline 에서 encodeURIComponent 를 지우고 한글 이름 파일을 열어 보세요. 어떤 에러가 나오나요?

▫ 직접 해보기 6

05단원의 인증 미들웨어를 가져와 /api/v1/files/:id/download 에만 붙여 보세요. /uploads/ 정적 주소로는 여전히 받아지나요? 그게 왜 문제인가요?

▫ 직접 해보기 7

기록에 '올린사람' 을 추가하고, 자기가 올린 파일만 내려받을 수 있게 만들어 보세요.

자주 하는 실수

이런 실수를 합니다

업로드 폴더를 통째로 static 으로 엮

비공개 파일도 주소만 알면 누구나 받습니다.

이런 실수를 합니다

주소에 파일 이름을 받음

../ 로 서버 파일이 새어 나갑니다. id 로 찾으세요.

이런 실수를 합니다

Content-Disposition 에 한글을 직접 씀

ERR_INVALID_CHAR 로 500 이 납니다. res.download 를 쓰거나 encodeURIComponent 하세요.

이런 실수를 합니다

sendFile 에 상대 경로를 줌

절대 경로여야 합니다. path.join(__dirname, ...) 을 쓰세요.

이런 실수를 합니다

기록 없이 파일만 저장

무작위 이름만 남아서 무슨 파일인지 알 수 없게 됩니다.

이런 실수를 합니다

startsWith 에 구분자를 안 붙임

uploads-secret 같은 옆 폴더가 통과합니다.

정리

- 1 파일만 저장하지 않는다. 누가 언제 무엇을 올렸는지 기록을 따로 남긴다. 기록이 없으면 폴더에 파일만 쌓이고 아무도 손댈 수 없게 된다.
- 2 보여 주는 방법은 둘이다 — 정적 폴더로 통째로 열기, 라우트로 하나씩 내주기.
- 3 업로드 폴더를 통째로 static 으로 열면 안 된다. 주소만 알면 남의 파일까지 다 받아 갈 수 있다.
- 4 주소에는 파일 이름이 아니라 기록의 번호를 받는다. 이름을 받으면 ../ 를 섞어 넣어 서버 안을 돌아다닐 수 있다.
- 5 그래도 경로 탈출은 따로 막는다. 만든 경로가 업로드 폴더 안인지 확인한다. startsWith 로 볼 때는 폴더 구분자까지 붙여서 본다. 안 붙이면 uploads2 가 통과한다.
- 6 보기와 내려받기는 Content-Disposition 으로 갈린다. inline 이면 보기, attachment 면 저장.
- 7 Content-Disposition 에 한글을 직접 쓰지 않는다. 헤더라서 500 이 난다. filename* 형식으로 인코딩해서 넣는다.
- 8 sendFile 에는 절대 경로를 준다. 상대 경로는 받지 않는다.

파일과 기록을 함께 관리하기

RUN node 개념05_파일과_기록_함께_관리하기.js

그러면 Ctrl + C

업로드는 '두 곳' 을 동시에 바꿉니다.

① 디스크에 파일을 씁니다 ② 기록(데이터베이스 또는 JSON)에 한 줄을 넣습니다

둘 중 하나만 되면 어긋납니다. 그리고 07단원에서 본 것처럼, 파일과 데이터베이스는 트랜잭션을 함께 못 씁니다.

이 단원에서 그 문제를 어떻게 다루는지 봅니다.

```
const express = require("express");
const multer = require("multer");
const path = require("path");
const fs = require("fs");

const app = express();
const PORT = Number(process.env.PORT) || 3000;

app.use(express.json());

const 업로드폴더 = path.join(__dirname, "uploads", "관리");
fs.rmSync(업로드폴더, { recursive: true, force: true });
fs.mkdirSync(업로드폴더, { recursive: true });

function 이름되돌리기(이름) {
  return Buffer.from(이름, "latin1").toString("utf8");
}
```

저장소 (07단원 구조)

섹션 1

진짜라면 데이터베이스입니다. 여기서는 배열로 둡니다.

```
let 기록들 = [];
let 다음번호 = 1;

const 파일저장소 = {
  전부: () => 기록들.map((기록) => ({ ...기록 })),
```

```

하나: (id) => {
  const 찾은것 = 기록들.find((기록) => 기록.id === id);
  return 찾은것 ? { ...찾은것 } : null;
},

추가: (값) => {
  const 새것 = { id: 다음번호, ...값 };
  다음번호 += 1;
  기록들.push(새것);
  return { ...새것 };
},

삭제: (id) => {
  const 전 = 기록들.length;
  기록들 = 기록들.filter((기록) => 기록.id !== id);
  return 기록들.length !== 전;
},
};

```

디스크를 다루는 것도 한곳에 모읍니다.

```

const 디스크 = {
  경로: (저장이름) => path.join(업로드폴더, 저장이름),

  있나: (저장이름) => fs.existsSync(디스크.경로(저장이름)),

  지우기: async (저장이름) => {
    try {
      await fs.promises.unlink(디스크.경로(저장이름));
      return true;
    } catch (에러) {

```

★ 이미 없는 파일을 지우려 한 것은 실패가 아닙니다. “없어야 한다” 는 목표가 이미 이뤄진 상태니까요.

```

    if (에러.code === "ENOENT") return false;
    throw 에러;
  },

  전부목록: () => fs.readdirSync(업로드폴더),
};

```

올리기 — 순서가 중요합니다

섹션 2

```

const 저장방식 = multer.diskStorage({
  destination: (req, file, cb) => cb(null, 업로드폴더),

```

파일 업로드

RUN node 연습문제.js

서버 없이 푸는 문제들입니다. 여기서 만든 함수를 연습문제_서버.js 에서 그대로 씁니다.

```
const path = require("path");
```

문제 1 깨진 파일 이름 되돌리기

multer 가 주는 깨진 이름을 흉내 내는 함수와, 되돌리는 함수를 만드세요.

```
깨뜨리기(이름)      Buffer.from(이름, "utf8").toString("latin1")
이름되돌리기(이름)  Buffer.from(이름, "latin1").toString("utf8")
```

깨뜨리기 는 시험용입니다. 실제 서버에서는 multer 가 이미 깨진 것을 줍니다.

출력할 것 (이 순서로) 깨뜨리기("작업표준서.pdf") 의 글자 수 그것을 되돌린 결과 되돌린 것의 글자 수
이름되돌리기("report-2026.png") 이름되돌리기(깨뜨리기("검사성적서_2026.pdf"))

```
기대 출력    19
              작업표준서.pdf
              9
              report-2026.png
              검사성적서_2026.pdf
```

★ 19 와 9 의 차이를 설명할 수 있어야 합니다. (힌트: 한글 한 글자가 UTF-8 에서 몇 바이트인가요?)

★ 네 번째를 보세요. 영어 이름은 되돌려도 그대로입니다. 그래서 조건 없이 항상 감싸도 되는지 생각해 보세요.

★★ 이 되돌리기를 req.body 의 값에도 적용하면 어떻게 될까요?

여기에 코드를 쓰세요

문제 2 안전한 저장 이름 만들기

원본 이름을 받아 저장할 이름을 만드는 함수를 만드세요.

안전한저장이름(원본이름, 시각, 무작위) → "시각-무작위.확장자"

· 원본 이름을 먼저 되돌립니다 · 확장자만 뽑아 소문자로 바꿉니다 · 원래 이름은 저장 이름에 넣지 않습니다

★ 시각과 무작위를 인자로 받는 이유 함수 안에서 Date.now() 를 부르면 시험할 때마다 결과가 달라집니다. 실제 코드에서는 안에서 불러도 됩니다.

시험할 것 (전부 시각 1786683001682, 무작위 123456789) 깨뜨리기("작업표준서.PDF")

"photo.JPEG" "확장자없음" "../.env"

기대 출력	1786683001682-123456789.pdf
	1786683001682-123456789.jpeg
	1786683001682-123456789
	1786683001682-123456789

★★ 네 번째가 이 문제의 핵심입니다. ../.env 를 올려도 ../ 가 한 조각도 안 남습니다. 왜일까요?

★ 그리고 확장자가 ".env" 가 아니라 아예 없습니다. path.extname(".env") 를 직접 찍어 보세요. 무엇이 나오나요? 왜 그렇게 정해져 있을까요? (힌트: 리눅스의 .gitignore, .bashrc)

여기에 코드를 쓰세요

문제 3 확장자 검사

원본 이름을 받아 문제가 있으면 이유를, 없으면 null 을 돌려주는 함수를 만드세요.

허용: .jpg .jpeg .png .gif .pdf

확장자가 없으면 "확장자가 없습니다" 허용 목록에 없으면 "{확장자} 는 올릴 수 없습니다" 통과하면 null

★ 되돌리기를 먼저 하세요. 안 하면 한글 이름 파일에서 엉뚱하게 동작합니다. ★ 소문자로 바꾸세요. .PNG 도 통과해야 합니다. ★ '허용 목록' 방식입니다. '금지 목록' 이 왜 위험한지 생각해 보세요.

시험할 것 깨뜨리기("작업표준서.pdf") / "photo.PNG" / "악성.exe" / "이름만있음" / 깨뜨리기("사진.jpg")


```

기대 출력  null
           null
           .exe 는 올릴 수 없습니다
           확장자가 없습니다
           null

```

여기에 코드를 쓰세요

문제 4 내용이 진짜 그 형식인가 (매직 넘버)

파일의 첫 바이트를 보고 진짜 그 형식인지 판단하는 함수를 만드세요.

```

.png 0x89 0x50 0x4E 0x47
.jpg 0xFF 0xD8 0xFF
.jpeg 0xFF 0xD8 0xFF
.gif 0x47 0x49 0x46      ("GIF")
.pdf 0x25 0x50 0x44 0x46  ("%PDF")

```

진짜형식인가(버퍼, 확장자) → true / false 모르는 확장자는 true (확인할 방법이 없습니다)

힌트: 기대.every((바이트, i) ⇒ 버퍼[i] === 바이트)

시험할 값

```

진짜PNG = Buffer.from([0x89, 0x50, 0x4e, 0x47, 0x0d, 0x0a])
진짜PDF = Buffer.from("%PDF-1.4 어쨌고", "utf8")

```

그냥글자 = Buffer.from("이건 그냥 글자입니다", "utf8")

시험할 것 (이 순서로) (진짜PNG, ".png") / (그냥글자, ".png") / (진짜PDF, ".pdf") (진짜PNG, ".pdf") / (그냥글자, ".txt")

```

기대 출력  true
           false
           true
           false
           true

```

★★ 왜 확장자 검사만으로는 부족한가요? mimetype 은 왜 못 믿나요? .png 로 이름만 바꾼 실행 파일이 왜 위험한가요?

여기에 코드를 쓰세요

문제 5 어긋난 것 찾기

기록 목록과 디스크 파일 목록을 받아, 어긋난 것을 찾는 함수를 만드세요.

어긋난것찾기(기록들, 디스크파일들)

```
→ { 고아파일: [이름들], 깨진기록: [id들] }
```

고아 파일 기록은 없는데 파일만 있음 깨진 기록 기록은 있는데 파일이 없음

★ Set 을 쓰세요. 배열 includes 로 하면 개수가 늘 때 아주 느려집니다.

시험할 값 기록들 = [{id:1,저장이름:"a.pdf"}, {id:2,저장이름:"b.png"}, {id:3,저장이름:"없어진것.pdf"}]

디스크 = ["a.pdf", "b.png", "버려진것.pdf", "또버려진것.png"]

각각 join(", ") 으로 한 줄씩 출력하세요.

```
기대 출력    버려진것.pdf, 또버려진것.png
              3
```

★★ 둘 중 어느 쪽이 더 나쁜가요? 사용자 눈에 무엇이 보이나요? 그 답이 '지울 때 무엇을 먼저 지울까' 를 정합니다. (문제 7)

여기에 코드를 쓰세요

문제 6 MulterError 안내 문구

에러 객체를 받아 사용자에게 보여 줄 문구를 만드는 함수를 만드세요.

LIMIT_FILE_SIZE	파일이 너무 큼니다
LIMIT_FILE_COUNT	파일이 너무 많습니다
LIMIT_UNEXPECTED_FILE	예상하지 못한 필드 이름입니다. 필드 이름을 영어로 맞췄나요?
LIMIT_FIELD_COUNT	글자 필드가 너무 많습니다
LIMIT_FIELD_KEY	필드 이름이 너무 길니다
LIMIT_FIELD_VALUE	필드 값이 너무 길니다

그 밖 업로드에 실패했습니다

err.field 가 있으면 뒤에 " (필드이름)" 을 붙입니다.

★★ field 가 없는 경우가 있습니다. LIMIT_FILE_COUNT 에는 안 들어 있습니다. 직접 재 본 결과입니다. 없을 수 있다고 생각하고 만드세요.

시험할 것

```
{ code: "LIMIT_FILE_SIZE", field: "file" }
{ code: "LIMIT_FILE_COUNT" }
{ code: "LIMIT_UNEXPECTED_FILE", field: 깨뜨리기("파일") }
{ code: "MYSTERY" }
```

기대 출력 파일이 너무 큼니다 (file)
 파일이 너무 많습니다
 예상하지 못한 필드 이름입니다. 필드 이름을 영어로 맞췄나요? (îl%)
 업로드에 실패했습니다

★ 세 번째의 깨진 글자를 보세요. 필드 이름을 한글로 썼을 때 정확히 이렇게 나옵니다. ★ 이 에러들은 400 인가요 500 인가요? 왜죠?

여기에 코드를 쓰세요

문제 7 무엇을 먼저 지우나

파일과 기록을 지울 때 두 가지 순서가 있습니다.

① 파일 먼저 → 기록 나중. 중간에 실패하면: _____

사용자 눈에 무엇이 보이나

② 기록 먼저 → 파일 나중. 중간에 실패하면: _____

어느 쪽을 골라야 하나, 왜: _____

왜 완벽하게 못 하나: _____ (힌트: 07단원 개념05 의 '중간실패')

남은 쓰레기는 어떻게 하나: _____

청소할 때 주의할 점은: _____ (힌트: 방금 올라온 파일은 기록이 아직 없을 수 있습니다)

업로드 라우트에서 400-404 를 낼 때 반드시 해야 할 일: _____

문제 8 한글이 안 되는 곳 총정리

이 자료에서 지금까지 나온 '한글이 안 되는 곳' 을 전부 적어 보세요. 단원과 증상을 함께 적으세요.

① _____ (__ 단원) 증상

② _____ (__ 단원) 증상

③ _____ (__ 단원) 증상

④ _____ (__ 단원) 증상

⑤ _____ (__ 단원) 증상

⑥ _____ (__ 단원) 증상

반대로 한글이 잘 되는 곳: _____

공통점은 무엇인가: _____

규칙 한 줄로: _____

파일 이름은 왜 예외인가: _____

설비 사진 API

RUN `npx nodemon 연습문제_서버.js`

★ 이 문제의 목적 업로드에서 '쓰레기 파일이 안 생기게' 만드는 것입니다.

에러를 낼 때마다 이미 저장된 파일을 지워야 합니다. 빠뜨리기 아주 쉽고, 티도 안 나며, 디스크가 조용히 잡니다.

다 만든 뒤 `/api/v1/photos/orphans` 가 계속 0 이면 성공입니다.

만들 주소

GET	<code>/api/v1/equipments/:id/photos</code>	그 설비의 사진 목록
POST	<code>/api/v1/equipments/:id/photos</code>	사진 올리기
GET	<code>/api/v1/photos/orphans</code>	고아 파일 확인 (시험용)
GET	<code>/api/v1/photos/:photoId/view</code>	보기
GET	<code>/api/v1/photos/:photoId/download</code>	내려받기

DELETE `/api/v1/photos/:photoId` 지우기

```
const express = require("express");
const multer = require("multer");
const path = require("path");
const fs = require("fs");

const app = express();
const PORT = Number(process.env.PORT) || 3000;

app.use(express.json());

const 사진폴더 = path.join(__dirname, "uploads", "사진");
fs.rmSync(사진폴더, { recursive: true, force: true });
fs.mkdirSync(사진폴더, { recursive: true });
```

준비 (여기는 그대로 두세요) —————

```
const 설비들 = [
  { id: 1, name: "컨베이어 1호", line: "A" },
  { id: 2, name: "프레스 1호", line: "B" },
];

let 사진들 = [];
let 다음번호 = 1;

const 설비당최대 = 3;
const 허용확장자 = [".jpg", ".jpeg", ".png", ".gif"];
```

문제 1 도우미

연습문제.js 의 이름되돌리기 를 가져오세요. 그리고 상태코드가 붙은 에러를 만드는 `HttpError(status, code, message)` 도 만드세요.

파일을 지우는 도우미도 만드세요.

```
async function 파일지우기(저장이름) { ... }
```

★ ENOENT 는 에러가 아닙니다. 이미 없는 파일을 지우려 한 것은 '목표가 이미 이뤄진 것' 입니다.

여기에 코드를 쓰세요

문제 2 multer 설정

diskStorage 로 아래를 정하세요.

```
destination 사진폴더
filename "시각-무작위.확장자" (연습문제.js 문제 2)
```

limits

```
fileSize 200 (시험용. 실제로는 5 * 1024 * 1024)
files 1
```

fileFilter 허용확장자 가 아니면 `HttpError(400, "BAD_EXTENSION", ...)` 을 cb 로 넘기세요.

★ filename 과 fileFilter 안에서도 이름되돌리기 를 먼저 하세요. 안 하면 한글 이름 파일이 전부 거절됩니다.

여기에 코드를 쓰세요

문제 3 미들웨어

설비찾기(req, res, next) :id 로 설비를 찾아 req.설비 에 담기 (없으면 404, 숫자 아니면 400) 사진찾기 (req, res, next) :photoid 로 사진을 찾아 req.사진 에 담기 (없으면 404)

여기에 코드를 쓰세요

문제 4 사진 목록

GET /api/v1/equipments/:id/photos

기대 출력 /1/photos → 200 {"data":[],"meta":{"설비":"컨베이어
1호","개수":0,"최대":3}}
/99/photos → 404

★ 06단원에서 강조한 구별입니다. 설비는 있고 사진이 0장 → 200 과 빈 배열 설비 자체가 없음 → 404

여기에 코드를 쓰세요

문제 5 사진 올리기 ** 이 문제가 핵심입니다

POST /api/v1/equipments/:id/photos

규칙 · 설비가 없으면 404 · 파일이 없으면 400 · 설비당 3장을 넘으면 409 TOO_MANY_PHOTOS · 잘 왔으면 201 + Location

★★★ 여기가 함정입니다.

multer 는 라우트보다 '먼저' 둡니다. 그래서 이 함수가 실행될 때는 파일이 이미 저장돼 있습니다.

404 나 409 를 그냥 내면, 그 파일은 아무도 모르는 고아가 됩니다. 에러를 내기 전에 반드시 req.file 을 지우세요.

```
const 정리하고 = async (에러) => {  
  if (req.file) await 파일지우기(req.file.filename);  
  next(에러);  
};
```

★★ 그리고 설비찾기 를 미들웨어로 붙이면 안 됩니다. 왜일까요? 미들웨어에서 next(에러) 를 부르면 라우트 함수가 실행되지 않습니다. 그러면 파일을 지울 기회가 없습니다. 이 라우트에서는 설비 확인을 '직접' 하세요.

기대 출력	/1/photos	설비사진.png (50바이트)	→ 201, id 1
	/1/photos	악성.exe	→ 400 BAD_EXTENSION
	/1/photos	500바이트 파일	→ 400 LIMIT_FILE_SIZE
	/99/photos	설비사진.png	→ 404
	세 장을 채운 뒤 한 장 더		→ 409 TOO_MANY_PHOTOS

★ 409 인 이유를 설명할 수 있어야 합니다. 왜 400 이 아닌가요?

여기에 코드를 쓰세요

문제 6 고아 파일 확인

GET /api/v1/photos/orphans

디스크 파일 수, 기록 수, 고아 파일 수를 돌려주세요.

★★ 이 라우트를 /api/v1/photos/:photold 보다 '위' 에 두세요. 아래에 두면 :photold 에 "orphans" 가 들어갑니다. (04단원 개념02 섹션 7)

기대 결과 (사진 세 장을 올리고 에러를 여러 번 낸 뒤):

```
{"data":{"디스크파일수":3,"기록수":3,"고아파일수":0}}
```

★★ 고아파일수가 0 이 아니면 문제 5의 정리하고() 를 빠뜨린 것입니다.

여기에 코드를 쓰세요

문제 7 보기와 내려받기

GET /api/v1/photos/:photold/view res.sendFile GET /api/v1/photos/:photold/download
res.download (원래 이름으로)

★ res.download 의 두 번째 인자에 원래 이름을 주면 Express 가 한글을 알아서 인코딩해 줍니다. 직접 Content-Disposition 을 쓰면 한글에서 500 이 납니다.

여기에 코드를 쓰세요

문제 8 지우기

DELETE /api/v1/photos/:photold → 204

★ 기록을 먼저 지우고 파일을 나중에 지우세요. 왜인지 설명할 수 있어야 합니다.

```
기대 출력    /3 → 204
              /3 → 404 (이미 지웠으니)
              그 뒤 /orphans → 고아 0
```

여기에 코드를 쓰세요

문제 9 404 와 에러 처리기

MulterError 는 400 으로, 우리가 만든 에러는 err.status 로, 그 밖은 500 으로 내보내세요.

MulterError 안내 문구는 연습문제.js 문제 6 을 참고하세요.

여기에 코드를 쓰세요

```
app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다.
http://localhost:${PORT}/api/v1/equipments/1/photos`);
});
```

Postman 시나리오

Body → form-data → Key 에 photo, 오른쪽 드롭다운에서 File 선택

```
1 GET    /api/v1/equipments/1/photos      0장
2 GET    /api/v1/equipments/99/photos      404 3 POST
/api/v1/equipments/1/photos (사진.png) 201 4 POST /api/v1/equipments/1/photos
(악성.exe) 400 5 POST /api/v1/equipments/1/photos (큰 파일) 400 6 POST
/api/v1/equipments/99/photos (사진.png) 404 7 GET /api/v1/photos/orphans
고아 0 *
8 세 장을 채운 뒤 한 장 더      409 9 GET
/api/v1/photos/orphans          여전히 0 *
10 GET   /api/v1/photos/1/download        한글 이름으로 저장되나?
11 DELETE /api/v1/photos/1 204
12 GET   /api/v1/photos/orphans        여전히 0
```

★ 7·9·12 가 이 문제의 채점 기준입니다. 에러가 여섯 번 났는데도 쓰레기가 안 쌓였는지 봅니다.

★ 한글 이름 파일로 꼭 해 보세요. 영어 파일로만 하면 함정을 못 만납니다.

다 만든 뒤 답해 보세요

- ① 문제 5에서 설비찾기 를 미들웨어로 붙이면 무엇이 문제인가요? 실제로 붙여 보고 /99/photos 로 파일을 올린 뒤 /orphans 를 보세요. _____
- ② 409 와 400 의 차이를 이 상황에 맞춰 설명해 보세요. _____
- ③ /orphans 를 /:photold 아래에 두면 무슨 일이 생기나요? 실제로 옮겨서 확인해 보세요.

- ④ fileFilter 에서 이름되돌리기 를 빼면 어떤 파일이 거절되나요?

- ⑤ 문제 8에서 파일을 먼저 지우고 기록을 나중에 지우면 중간에 실패했을 때 사용자 눈에 무엇이 보이나요?

- ⑥ 이 서버에 05단원의 인증을 붙인다면 어디에 붙이나요? /view 와 /download 에도 붙여야 하나요?

잘 안 될 때 보는 곳

MulterError: Unexpected field → 필드 이름이 photo 가 맞나요? 한글로 쓰지 않았나요?

한글 파일만 거절됨 → fileFilter 에서 이름되돌리기 를 안 했습니다.

파일 이름이 깨져서 저장됨 → filename 에서 이름되돌리기 를 안 했습니다.

req.file 이 undefined → 파일을 안 보냈거나, 필드 이름이 다릅니다. → Postman 에서 File 이 아니라 Text 로 골랐을 수도 있습니다.

고아파일수가 계속 늘어남 → 에러를 내기 전에 req.file 을 안 지웁니다. 문제 5를 다시 보세요.

/api/v1/photos/orphans 가 404 → :photold 라우트가 먼저 걸렸습니다. 위로 올리세요.

내려받기에서 500 → Content-Disposition 에 한글을 직접 썼습니다. res.download 를 쓰세요.

500 인데 400 이어야 함 → 에러 처리기에서 MulterError 를 안 다뤘습니다.

프론트엔드 연동

OPEN 10_프론트엔드_연동

```
const path = require("path");
const app = express();
app.use(express.json());
app.get("/", (req, res) => {
```

- 01 fetch 로 API 부르기
- 02 로딩·에러·빈 결과 보여 주기
- 03 품과 검증 결과 보여 주기
- 04 목록 화면 만들기 (페이징 · 검색 · 정렬)
- 05 업로드 화면 만들기
- 연습문제
- 서버 연습문제

fetch 로 API 부르기

```
RUN node 개념01_fetch로_API_부르기.js
```

브라우저에서 `http://localhost:3000` 을 여세요.

09단원까지 서버를 만들었습니다. 이제 화면을 붙입니다.

★ 이 단원은 '프론트엔드 수업' 이 아닙니다. 화면을 예쁘게 만드는 법이 아니라, **서버가 준 응답을 화면이 어떻게 다뤄야 하는가** 를 배웁니다.

그래서 06단원에서 응답 형식을 통일해 둔 것이 여기서 값을 합니다.

이 파일은 화면과 API 를 같은 서버에서 줍니다. (08단원 ① 방식) 같은 출처라 CORS 를 신경 쓸 필요가 없습니다.

★★ 여기 화면은 **순수 HTML/JS** 입니다. React 가 아닙니다. 서버가 무엇을 주고받는지에 집중하려고 일부러 그렇게 했습니다.

★ React 쪽에서 같은 일을 하는 자료가 따로 있습니다.

```
`React자료/실습프로젝트/src/14_서버와_연동`  
거기서는 useForm 으로 폼을 만들고 FormData 로 파일을 올립니다.  
주소도 `/api/v1/equipments` 로 여기와 똑같이 맞춰 두었습니다.
```

★★ 두 자료가 **같은 이야기의 양쪽** 끝입니다.

PART 2 에서 붙는 쪽을 배웠고, 여기서 받는 쪽을 만듭니다.

```
const express = require("express");  
const path = require("path");  
  
const app = express();  
const PORT = Number(process.env.PORT) || 3000;  
  
app.use(express.json());
```

화면은 정적 파일로

섹션 1

```
app.use(express.static(path.join(__dirname, "public")));

app.get("/", (req, res) => {
  res.redirect("/01_기본.html");
});
```

```
GET /01_기본.html
200
```

```
GET /api.js
200
```

★ public/api.js 를 꼭 열어 보세요. 이 단원의 핵심이 그 파일에 있습니다. 화면 코드가 아니라고요.

화면 확인

아래 선언은 진짜 크롬으로 버튼을 눌러서 확인합니다.

```
node _검증도구/화면검증.js 10_프론트엔드_연동
```

화면 01_기본.html

```
button[data-할일="목록"]
```

#결과 컨베이어 1호

화면 01_기본.html

```
button[data-할일="하나"]
```

#결과 성공

화면 01_기본.html

```
button[data-할일="만들기"]
```

#결과 용접로봇 1호

화면 01_기본.html

```
button[data-할일="지우기"]
```

#결과 "data": null

화면 01_기본.html

button[data-할일="없음"]

#결과 NOT_FOUND

화면 01_기본.html

button[data-할일="검증"]

#결과 VALIDATION_FAILED

화면 01_기본.html

button[data-할일="터짐"]

#결과 INTERNAL_ERROR

화면 01_기본.html

button[data-할일="html"]

#결과 BAD_RESPONSE

화면 01_기본.html

button[data-할일="끊김"]

#결과 NETWORK

★★ 마지막 두 개가 이 파일의 핵심입니다. BAD_RESPONSE — JSON 이 아닌 응답이 와도 화면이 안 터집니다 NETWORK — 서버에 닿지도 못해도 화면이 안 터집니다 api.js 가 없었다면 둘 다 콘솔에 빨간 에러를 내고 화면은 멈췄을 것입니다.

화면이 부를 API

섹션 2

```
let 설비들 = [
  { id: 1, name: "컨베이어 1호", line: "A", status: "가동" },
  { id: 2, name: "프레스 1호", line: "B", status: "정지" },
];

app.get("/api/v1/equipments", (req, res) => {
  res.json({ data: 설비들, meta: { total: 설비들.length } });
});
```


GET /api/v1/equipments

```
200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"},{"id":2,"name":"프레스 1호","line":"B","status":"정지"}],"meta":{"total":2}}
```

```
app.get("/api/v1/equipments/:id", (req, res) => {
  const 설비 = 설비들.find((설비) => 설비.id === Number(req.params.id));

  if (!설비) {
    return res.status(404).json({
      error: { code: "NOT_FOUND", message: "설비를 찾을 수 없습니다" },
    });
  }

  res.json({ data: 설비 });
});
```

GET /api/v1/equipments/1

```
200 {"data":{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"}}
```

GET /api/v1/equipments/99

```
404 {"error":{"code":"NOT_FOUND","message":"설비를 찾을 수 없습니다"}}
```

```
app.post("/api/v1/equipments", (req, res) => {
  const { name, line } = req.body || {};
  const 항목 = [];

  if (!name) 항목.push({ 키: "name", 이유: "필수입니다" });
  else if (name.length < 2) 항목.push({ 키: "name", 이유: "2글자 이상이어야 합니다" });

  if (!line) 항목.push({ 키: "line", 이유: "필수입니다" });
  else if (!["A", "B", "C"].includes(line))
    항목.push({ 키: "line", 이유: "A, B, C 중 하나여야 합니다" });

  if (항목.length > 0) {
    return res.status(400).json({
      error: { code: "VALIDATION_FAILED", message: "입력값이 올바르지 않습니다",
      details: 항목 },
    });
  }

  const 새설비 = { id: 3, name, line, status: "정지" };
```

```
res.status(201).json({ data: 새설비 });
});
```

POST /api/v1/equipments

```
{ "name": "용접로봇 1호", "line": "C" }
```

201 { "data": { "id": 3, "name": "용접로봇 1호", "line": "C", "status": "정지" } }

POST /api/v1/equipments

```
{ "name": "용" }
```

400 { "error": { "code": "VALIDATION_FAILED", "message": "입력값이 올바르지 않습니다", "details": [{ "키": "name", "이유": "2글자 이상이어야 합니다" }, { "키": "line", "이유": "필수입니다" }] } }

```
app.delete("/api/v1/equipments/:id", (req, res) => {
  res.sendStatus(204);
});
```

DELETE /api/v1/equipments/1

204

★ 204 는 본문이 없습니다. 화면에서 res.json() 을 부르면 터집니다. api.js 가 204 를 먼저 걸러 내는 이유입니다.

일부러 실패하는 주소들

섹션 3

```
app.get("/api/v1/boom", (req, res) => {
  const 없는것 = undefined;
  res.json(없는것.속성); // TypeError
});
```

GET /api/v1/boom

500 { "error": { "code": "INTERNAL_ERROR", "message": "서버에서 문제가 생겼습니다" } }

```
app.get("/api/v1/html", (req, res) => {
```

JSON 을 기대했는데 HTML 이 오는 경우입니다.

```
res.status(500).type("html").send("<!DOCTYPE html><p>서버 점검 중입니다</p>");
});
```

GET /api/v1/html

500

★★ 이 경우가 실무에서 정말 자주 납니다. 앞단에 Nginx 같은 것이 있으면, 서버가 죽었을 때 HTML 에러 페이지를 대신 줍니다. 화면이 res.json() 을 부르면 "Unexpected token '<'" 로 터집니다.

api.js 는 json() 을 try/catch 로 감싸서 BAD_RESPONSE 로 바꿉니다. 덕분에 화면은 "서버 응답을 읽지 못했습니다" 를 보여 줄 수 있습니다.

404 와 에러 처리기

섹션 4

```
app.use((req, res) => {
  res.status(404).json({ error: { code: "ROUTE_NOT_FOUND", message: "그런 주소가 없습니다" } });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  res.status(500).json({ error: { code: "INTERNAL_ERROR", message: "서버에서 문제가 생겼습니다" } });
});

app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}`);
});
```

public/api.js 가 하는 일

화면이 부르는 것은 세 줄뿐입니다.

```
const 답 = await api.get("/api/v1/equipments");
화면그리기(답.data);
```

그런데 그 사이에 이런 일이 다 처리됩니다.

① fetch 가 던지는 경우 → NETWORK (서버에 닿지도 못함)

② 204 라 본문이 없는 경우 → { data: null }

③ JSON 이 아닌 응답 → BAD_RESPONSE ④ 몸통에 error 가 있는 경우 → API에러 로 던짐 ⑤ 그 밖 → 몸통을 그대로 돌려줌

화면은 try/catch 하나로 ①③④를 전부 받습니다.

★★ ①이 왜 중요한가 fetch 는 404·500 에 던지지 않습니다. 응답이 왔으니까요. 던지는 것은 '서버에 닿지도 못한' 경우뿐입니다.

서버가 꺼짐 / 인터넷 끊김 / CORS 로 막힘 (08단원)

그래서 fetch 만 쓰면 500 을 성공으로 착각합니다. 반드시 응답 내용을 확인해야 합니다.

res.ok 로 판단할까, error 필드로 판단할까

res.ok 상태코드가 200~299 면 true
몸통.error 06단원에서 정한 형식

둘 다 됩니다. 이 자료는 error 필드로 판단합니다.

· 06단원에서 "성공엔 error 없고 실패엔 data 없다" 고 정해 뒀습니다 · 상태코드를 안 봐도 되니 화면 코드가 단순해집니다 · code 로 갈래를 나눌 수 있습니다 (NOT_FOUND 면 이 화면, UNAUTHENTICATED 면 저 화면)

★ 다만 남이 만든 API 를 부를 때는 res.ok 를 봐야 합니다. 형식이 우리와 다를 테니까요. 그때도 api.js 같은 파일을 하나 만들어 그 안에서만 처리하세요.

직접 해 볼 것

▫ 직접 해보기 1

아홉 개 버튼을 전부 눌러 보세요. F12 → Network 에서 상태코드가 각각 무엇인가요?

▫ 직접 해보기 2

'서버에 못 닿음' 버튼을 누르고 Console 을 보세요. 빨간 에러가 하나 더 있습니다. 그건 왜 나올까요?
(힌트: 브라우저가 연결 실패를 알리는 것입니다. 우리 코드와 별개입니다)

▫ 직접 해보기 3

서버를 Ctrl+C 로 끄고 'GET 목록' 을 눌러 보세요. 화면에 무엇이 나오나요? code 가 무엇인가요?

▫ 직접 해보기 4

api.js 의 204 처리를 지우고 'DELETE' 를 눌러 보세요. 어떤 에러가 나나요? (확인 후 되돌리세요)

▫ 직접 해보기 5

api.js 에서 json() 의 try/catch 를 지우고 'JSON 이 아닌 응답' 을 눌러 보세요. Console 의 에러 메시지를 그대로 적어 보세요.

▫ 직접 해보기 6

api.js 의 판단을 res.ok 로 바꿔 보세요. 아홉 버튼이 전부 그대로 동작하나요? 무엇이 달라지나요?

☞ 직접 해보기 7

api.put 을 추가해 보세요. api.patch 를 복사하면 됩니다.

자주 하는 실수

이런 실수를 합니다

fetch 만 쓰고 응답을 확인 안 함

404·500 도 '성공' 으로 흘러갑니다. 화면에 `undefined` 가 나옵니다.

이런 실수를 합니다

204 에 res.json() 을 부름

본문이 없어서 터집니다. 상태코드를 먼저 보세요.

이런 실수를 합니다

json() 을 try/catch 없이 부름

HTML 에러 페이지가 오면 "Unexpected token '<'" 로 터집니다.

이런 실수를 합니다

화면마다 fetch 를 직접 씀

에러 처리를 화면 수만큼 반복하게 됩니다. 한 군데를 빠뜨리면 그 화면만 이상해집니다.

이런 실수를 합니다

에러 메시지 글자로 갈래를 나눔

서버에서 맞춤법만 고쳐도 화면이 깨집니다. code 로 판단하세요. (06단원 개념04)

이런 실수를 합니다

catch 에서 아무것도 안 함

사용자는 버튼을 눌렀는데 아무 일도 안 일어난 것처럼 보입니다. 반드시 화면에 알려야 합니다. 개념02 에서 자세히 합니다.

정리

- 1 fetch 는 응답이 오기만 하면 성공으로 친다. 404 도 500 도 catch 로 안 간다. res.ok 를 반드시 확인한다.
- 2 catch 로 오는 것은 아예 못 갔을 때다 — 서버가 꺼졌거나, 네트워크가 끊겼거나, CORS.
- 3 204 에는 본문이 없다. res.json() 을 부르면 그 자리에서 터진다.
- 4 서버가 JSON 이 아닌 것을 줄 수도 있다. json() 은 try/catch 로 감싼다.
- 5 화면마다 fetch 를 직접 쓰지 않는다. 한곳에 모아 두고 그것만 부른다. 안 그러면 규칙이 바뀔 때 모든 화면을 고쳐야 한다.
- 6 갈래는 에러 메시지 글자로 나누지 않는다. 06단원에서 정한 에러 코드로 나눈다. 메시지는 언제든지 바뀐다.
- 7 catch 에서 아무것도 안 하면 안 된다. 사용자는 화면이 멈춘 줄 안다.

로딩·에러·빈 결과 보여 주기

RUN node 개념02_로딩과_에러_화면.js

브라우저에서 `http://localhost:3000` 을 여세요.

화면에는 상태가 넷 있습니다.

① 로딩 중 ② 성공 ③ 실패 ④ 성공했는데 결과가 없음 ← 이걸 제일 많이 빠뜨립니다

④를 빠뜨리면 빈 화면이 나옵니다. 사용자는 “고장났나?” 하고 새로고침만 계속합니다.

```
const express = require("express");
const path = require("path");

const app = express();
const PORT = Number(process.env.PORT) || 3000;

app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));

app.get("/", (req, res) => res.redirect("/02_로딩과에러.html"));
```

GET /02_로딩과에러.html

200

```
const 설비들 = [
  { id: 1, name: "컨베이어 1호", line: "A", status: "가동" },
  { id: 2, name: "프레스 1호", line: "B", status: "정지" },
];

let 받은횟수 = 0;

const 잠깐 = (ms) => new Promise((r) => setTimeout(r, ms));
```

네 가지 상태를 만들어 주는 API

섹션 1

```
app.get("/api/v1/equipments", async (req, res) => {
  if (req.query.count) 받은횟수 += 1;
```

일부러 늦추기 (로딩 화면을 보려고)

```
if (req.query.delay) await 잠깐(Number(req.query.delay));
```

일부러 실패하기

```
if (req.query.fail) {
  return res.status(500).json({
    error: { code: "INTERNAL_ERROR", message: "서버에서 문제가 생겼습니다" },
  });
}

const 결과 = req.query.line ? 설비들.filter((설비) => 설비.line === req.query.line)
: 설비들;

res.json({ data: 결과, meta: { total: 결과.length } });
});
```

GET /api/v1/equipments

```
200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"},{"id":2,"name":"프레스 1호","line":"B","status":"정지"}],"meta":{"total":2}}
```

GET /api/v1/equipments?line=Z

```
200 {"data":[],"meta":{"total":0}}
```

GET /api/v1/equipments?fail=1

```
500 {"error":{"code":"INTERNAL_ERROR","message":"서버에서 문제가 생겼습니다"}}
```

```
app.post("/api/v1/count/reset", (req, res) => {
  받은횟수 = 0;
  res.json({ data: { 횟수: 0 } });
});
```

POST /api/v1/count/reset

```
200 {"data":{"횟수":0}}
```

★ 버튼을 누를 때마다 0 으로 되돌립니다. 그래야 “이번에 몇 번 나갔나” 를 그대로 볼 수 있습니다.


```
app.get("/api/v1/count", (req, res) => {
  res.json({ data: { 횟수: 받은횟수 } });
});

app.get("/api/v1/label", async (req, res) => {
  if (req.query.delay) await 잠깐(Number(req.query.delay));
  res.json({ data: { label: req.query.label ?? "-" } });
});
```

```
GET /api/v1/label?label=시험
200 {"data":{"label":"시험"}}
```

화면으로 확인

섹션 2

화면 02_로딩과에러.html

#정상

#목록칸 컨베이어 1호

화면 02_로딩과에러.html

#빈것

#목록칸 조건에 맞는 설비가 없습니다

★★ 두 번째가 이 파일의 핵심입니다. 서버는 200 과 빈 배열을 줍니다. 실패가 아닙니다. 그런데 화면이 아무것도 안 그리면 사용자는 실패로 받아들입니다.

06단원에서 “목록이 비었다고 404 를 주지 마라” 고 했는데, 화면 쪽에도 짝이 되는 일이 있는 것입니다.

화면 02_로딩과에러.html

#실패

#목록칸 서버에서 문제가 생겼습니다

#다시 다시 시도

★ 실패 화면에는 ‘다시 해 볼 방법’ 이 함께 있어야 합니다. 에러 글만 띄우면 사용자는 새로고침밖에 할 게 없습니다. 새로고침하면 지금까지 입력한 것이 다 날아갑니다.

화면 02_로딩과에러.html

#느림

#목록칸 컨베이어 1호

★ 2초 걸리는 요청입니다. 화면검증이 기다렸다가 확인합니다. 브라우저로 직접 눌러 보면 그동안 “불러오는 중...” 이 보입니다.

버튼 잠그기

섹션 3

화면 02_로딩과에러.html

#안잠금

#횟수 3

화면 02_로딩과에러.html

#잠금

#횟수 1

★★ 서버가 받은 횟수가 다릅니다. 3번과 1번입니다.

느린 요청 중에 버튼을 또 누를 수 있으면, 같은 요청이 여러 번 나갑니다. 조회면 서버만 힘들고 끝이지만, ‘등록’ 이면 데이터가 세 개 만들어집니다.

결제 버튼이라면? 세 번 결제됩니다. 실제로 나는 사고입니다.

★ 잠금 해제에는 반드시 finally 에서

```
try { await 보내기(); } finally { 버튼.disabled = false; }
```

catch 에만 쓰면 성공했을 때 안 풀리고, try 끝에만 쓰면 실패했을 때 안 풀립니다. 버튼이 영영 안 눌립니다.

★ 서버도 막아야 합니다 화면 잠금은 ‘실수’ 를 막을 뿐입니다. Postman 으로는 얼마든지 여러 번 보낼 수 있습니다. 중요한 요청은 서버에서도 중복을 막아야 합니다. (08단원 CORS 와 같은 이야기입니다)

★ 늦게 온 응답이 덮어쓰는 문제

섹션 4

화면 02_로딩과에러.html

#경쟁안막음

#경쟁칸 느린 응답

화면 02_로딩과에러.html

#경쟁막음

#경쟁칸 빠른 응답

★★★ 같은 순서로 두 요청을 보냈는데 결과가 다릅니다.

안 막으면

느린 요청(800ms) 보냄 → 빠른 요청(100ms) 보냄
 100ms 뒤 "빠른 응답" 이 화면에 나옴
 800ms 뒤 "느린 응답" 이 도착해서 덮어씀 ← 옛날 결과가 이김

막으면

요청마다 번호를 붙여 두고, 마지막 번호가 아니면 결과를 버립니다.
 "빠른 응답" 이 남습니다.

★ 이게 언제 진짜로 일어나나 검색창에 글자를 칠 때마다 요청을 보내는 화면을 떠올리세요.

"설" 을 치면 → 요청 A (결과가 많아서 느림)
 "설비" 를 치면 → 요청 B (결과가 적어서 빠름)

B 가 먼저 도착해 화면을 그렸는데, A 가 나중에 도착해 덮어씁니다. 검색창에는 "설비" 가 적혀 있는데 목록은 "설" 의 결과입니다.

사용자는 "검색이 이상해요" 라고 합니다. 개발자는 재현을 못 합니다. 네트워크가 빠른 사무실에서는 잘 안 나기 때문입니다.

★ 막는 법 세 가지 ① 요청 번호를 붙여 마지막 것만 반영 (지금 방식. 가장 간단합니다) ②

AbortController 로 앞 요청을 취소 ③ 입력이 멈춘 뒤에 보내기 (디바운스). 요청 수 자체를 줄입니다

①과 ③을 함께 쓰는 것이 흔합니다.

```
app.use((req, res) => {
  res.status(404).json({ error: { code: "ROUTE_NOT_FOUND", message: "그런 주소가 없습니다" } });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  res.status(500).json({ error: { code: "INTERNAL_ERROR", message: "서버에서 문제가 생겼습니다" } });
});
```

```
app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}`);
});
```

상태 네 가지, 화면에 무엇을 그리나

상태 보여 줄 것 빠뜨리면

로딩 중 "불러오는 중..." + 버튼 잠금 두 번 누름 / 멈춘 줄 앞

성공	결과	-
실패	무엇이 문제인지 + 다시 시도 버튼	새로고침밖에 할 게 없음

결과 없음 "조건에 맞는 것이 없습니다" 고장난 줄 앞

★ 로딩을 '언제나' 보여 줄 필요는 없습니다. 0.2초 안에 끝나면 깜빡이기만 해서 오히려 거슬립니다. 0.3초쯤 지나도 안 끝났을 때만 보여 주는 방법도 있습니다. 다만 처음 배울 때는 그냥 항상 보여 주는 편이 낫습니다.

직접 해 볼 것

▫ 직접 해보기 1

네 버튼을 눌러 보세요. '느림' 을 누르고 2초 동안 화면을 보세요.

▫ 직접 해보기 2

그리기 함수에서 값.length === 0 갈래를 지워 보세요. '목록 (0건)' 을 누르면 무엇이 나오나요? 사용자는 그걸 보고 뭐라고 생각할까요? (확인 후 되돌리세요)

▫ 직접 해보기 3

실패 화면에서 '다시 시도' 버튼을 지워 보세요. 사용자가 할 수 있는 게 무엇이 남나요?

▫ 직접 해보기 4

'잠그고 요청' 의 finally 를 try 안으로 옮기고, 서버가 500 을 내게 해 보세요 (?fail=1 추가). 버튼이 다시 눌리나요?

▫ 직접 해보기 5

F12 → Network 에서 속도를 Slow 3G 로 바꾸고 '안 막고 두 번 요청' 을 눌러 보세요. 더 잘 보이나요?

▹ 직접 해보기 6

검색창을 만들어 글자를 칠 때마다 요청을 보내 보세요. 그리고 늦게 온 응답 문제를 재현해 보세요. (힌트: Slow 3G 로 하면 잘 보입니다)

▹ 직접 해보기 7

마지막번호 방식을 AbortController 로 바꿔 보세요. 앞 요청이 Network 탭에서 취소되는 것을 확인하세요.

자주 하는 실수

이런 실수를 합니다

빈 결과를 안 다룸

화면이 텅 비니다. 사용자는 고장으로 받아들입니다. 제일 많이 빠뜨립니다.

이런 실수를 합니다

로딩을 안 보여 줌

느린 요청에서 화면이 멈춘 것처럼 보입니다. 사용자가 계속 누릅니다.

이런 실수를 합니다

에러를 콘솔에만 찍음

사용자는 아무 일도 안 일어난 줄 압니다. 화면에 보여 주세요.

이런 실수를 합니다

다시 시도할 방법을 안 줌

새로고침하면 입력한 것이 다 날아갑니다.

이런 실수를 합니다

잠금 해제를 finally 에 안 씀

실패했을 때 버튼이 영영 안 눌립니다.

이런 실수를 합니다

늦게 온 응답을 그대로 그림

검색 결과가 검색어와 안 맞습니다. 개발자는 재현을 못 합니다.

이런 실수를 합니다

화면에서 막았으니 서버는 안 막아도 된다고 생각

화면 잠금은 실수를 막을 뿐입니다. Postman 으로는 그냥 뚫립니다.

정리

- 1 화면에는 네 가지 상태가 있다 — 로딩 중 · 성공 · 빈 결과 · 에러. 넷 다 그려야 한다. 성공만 그리면 나머지 셋에서 화면이 멈춘 것처럼 보인다.
- 2 빈 결과는 에러가 아니다. “없습니다” 를 보여 준다.
- 3 로딩을 안 보여 주면 사용자는 눌렀는지 아닌지 모르고 또 누른다.
- 4 에러를 콘솔에만 찍으면 사용자는 아무것도 모른다. 화면에 띄운다.
- 5 에러를 띄웠으면 다시 시도할 방법도 준다. 새로고침하라고 하지 않는다.
- 6 버튼은 요청이 나가는 동안 잠근다. 푸는 것은 finally 에 둔다. 성공에만 두면 실패했을 때 영영 잠긴 채로 남는다.
- 7 늦게 온 응답이 최신 화면을 덮어쓸 수 있다. 요청마다 번호를 붙여 두고 마지막 것만 그린다.
- 8 화면에서 막았다고 서버 검증을 빼면 안 된다. 화면을 거치지 않는 요청이 있다.

폼과 검증 결과 보여 주기

RUN node 개념03_폼과_검증_보여주기.js

브라우저에서 `http://localhost:3000` 을 여세요.

06단원에서 검증 실패를 이렇게 돌려주기로 했습니다.

```
{ "error": { "code": "VALIDATION_FAILED",
```

```
  "message": "입력값이 올바르지 않습니다",
  "details": [ { "키": "name", "이유": "2글자 이상이어야 합니다" } ] } }
```

details 를 왜 넣었는지가 이 파일에서 드러납니다. message 만 있으면 “입력값이 올바르지 않습니다” 하나만 띄우게 됩니다. 사용자는 어느 칸이 틀렸는지 모릅니다.

```
const express = require("express");
const path = require("path");

const app = express();
const PORT = Number(process.env.PORT) || 3000;

app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));

app.get("/", (req, res) => res.redirect("/03_폼.html"));
```

GET /03_폼.html

200

```
let 설비들 = [];
let 다음번호 = 1;

const 라인값들 = ["A", "B", "C"];
```

서버는 06단원 형식 그대로

섹션 1

```
app.get("/api/v1/equipments", (req, res) => {
  res.json({ data: 설비들 });
});
```

GET /api/v1/equipments

200 {"data":[]}

```
app.post("/api/v1/equipments", (req, res) => {
  const { name, line } = req.body || {};
  const 항목 = [];

  if (!name) 항목.push({ 키: "name", 이유: "필수입니다" });
  else if (name.length < 2) 항목.push({ 키: "name", 이유: "2글자 이상이어야 합니다" });
  else if (name.length > 20) 항목.push({ 키: "name", 이유: "20글자 이하여야 합니다" });

  if (!line) 항목.push({ 키: "line", 이유: "필수입니다" });
  else if (!라인값들.includes(line))
    항목.push({ 키: "line", 이유: "A, B, C 중 하나여야 합니다" });

  if (항목.length > 0) {
    return res.status(400).json({
      error: { code: "VALIDATION_FAILED", message: "입력값이 올바르지 않습니다",
details: 항목 },
    });
  }

  if (설비들.some((설비) => 설비.name === name)) {
    return res.status(409).json({
      error: { code: "DUPLICATED", message: "이미 있는 설비 이름입니다" },
    });
  }

  const 새설비 = { id: 다음번호, name, line, status: "정지" };
  다음번호 += 1;
  설비들.push(새설비);

  res.status(201).json({ data: 새설비 });
});
```

POST /api/v1/equipments

{"name":"용","line":"Z"}

400 {"error":{"code":"VALIDATION_FAILED","message":"입력값이 올바르지 않습니다","details":[{"키":"name","이유":"2글자 이상이어야 합니다"}, {"키":"line","이유":"A, B, C 중 하나여야 합니다"}]}}

POST /api/v1/equipments

```
{"name": "컨베이어 1호", "line": "A"}
```

201 {"data": {"id": 1, "name": "컨베이어 1호", "line": "A", "status": "정지"}}

POST /api/v1/equipments

```
{"name": "컨베이어 1호", "line": "B"}
```

409 {"error": {"code": "DUPLICATED", "message": "이미 있는 설비 이름입니다"}}

화면으로 확인

섹션 2

비운 채로 보내면 두 칸 모두에 오류가 붙습니다.

화면 03_폼.html

#보내기

#오류-name 필수입니다

#오류-line 필수입니다

★★ 오류가 '칸 아래' 에 각각 붙었습니다. message 만 썼다면 "입력값이 올바르지 않습니다" 하나뿐이었을 것입니다. 사용자는 무엇을 고쳐야 할지 몰라 아무거나 바꿔 보게 됩니다.

짧은 이름 + 없는 라인

화면 03_폼.html #보내기

입력: #name=용 #line=Z 보임: #오류-name 2글자 이상이어야 합니다 보임: #오류-line A, B, C 중 하나여야 합니다

제대로 넣으면 성공하고 목록에 붙습니다.

화면 03_폼.html #보내기

입력: #name=컨베이어 1호 #line=A 보임: #알림 등록했습니다 보임: #목록 컨베이어 1호

★ 성공하면 폼을 비웁니다 (폼.reset). 안 비우면 사용자가 같은 것을 또 등록하기 쉽습니다.

같은 이름을 또 등록하면 409 가 name 칸에 붙습니다. 한 번 등록하고, 화면을 새로 열어 같은 이름으로 또 보냅니다. (서버는 켜져 있는 동안 앞의 등록을 기억하고 있습니다)

화면 03_폼.html #보내기

입력: #name=프레스 1호 #line=B 보임: #알림 등록했습니다

입력: #name=프레스 1호 #line=B 보임: #오류-name 이미 있는 설비 이름입니다

★★ 여기가 재미있는 부분입니다. 409 DUPLICATED 에는 details 가 없습니다. message 만 있습니다. 그런데 화면은 그걸 name 칸에 붙였습니다.

“이름 중복은 name 칸 문제다” 라는 것을 화면이 알기 때문입니다. 서버가 모든 것을 알려 줄 필요는 없습니다. code 로 갈래를 나누고, 화면이 어디에 보여 줄지 정하면 됩니다.

이게 06단원에서 code 를 따로 둔 이유입니다. message 글자로 판단했다면 서버가 맞춤법만 고쳐도 깨졌을 것입니다.

오류가 난 뒤 제대로 고쳐 보내면 지난 오류가 사라져야 합니다. 빈 채로 한 번 보내고(오류), 값을 채워 다시 보냅니다(성공).

화면 03_폼.html

#보내기 → 입력(#name=검사기 1호) → 입력(#line=C) → #보내기

#오류-name !필수입니다

#알림 등록했습니다

★ 첫 번째 줄의 ! 는 ‘없어야 한다’ 는 뜻입니다. 첫 번째 클릭에서 “필수입니다” 가 붙었고, 두 번째 클릭 전에 값을 넣었으니 그 오류는 사라져야 합니다.

오류지우기() 를 안 부르면 고친 칸에도 옛날 오류가 남아 있습니다. “다 고쳤는데 왜 빨간 글씨가 있지?” 하고 헤매게 됩니다.

```
app.use((req, res) => {
  res.status(404).json({ error: { code: "ROUTE_NOT_FOUND", message: "그런 주소가 없습니다" } });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  res.status(500).json({ error: { code: "INTERNAL_ERROR", message: "서버에서 문제가 생겼습니다" } });
});

app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}`);
});
```

폼을 다룰 때 꼭 해야 할 것

① event.preventDefault()

안 하면 폼이 페이지를 통째로 새로고침합니다.
화면이 깜빡이고 우리 코드는 실행되다 맙니다.
"버튼을 눌렀는데 아무 일도 안 일어나요" 의 1번 원인입니다.

② 보내기 전에 지난 오류 지우기

안 지우면 고친 칸에도 옛날 빨간 글씨가 남습니다.

③ 버튼 잠그기 + finally 로 풀기

느린 요청 중에 또 누르면 두 번 등록됩니다. (개념02)

④ 성공하면 폼 비우기

안 비우면 같은 것을 또 등록하기 쉽습니다.

⑤ 모르는 칸 이름이 와도 안 터지게

서버가 우리 화면에 없는 칸 이름을 줄 수도 있습니다.
`if (!오류칸) continue;` 한 줄이면 됩니다.

★ FormData 로 값 꺼내기

```
const 값 = Object.fromEntries(new FormData(폼));
```

칸마다 `querySelector` 를 쓰면 칸이 늘 때마다 코드가 늙습니다. `input` 에 `name` 속성만 잘 달아 두면 이 한 줄로 끝납니다.

★ 주의: 값은 전부 글자입니다.

숫자 칸이 있으면 `Number` 로 바꿔서 보내세요. (04단원 개념03 과 같은 이야기)

화면 검증도 하나요, 서버만 하면 되나요

둘 다 합니다. 막는 것이 다릅니다.

화면 검증 사용자를 편하게 합니다. 서버에 다녀오기 전에 알려 줍니다 서버 검증 진짜 방어선입니다.

Postman 으로는 화면을 건너뛴니다 (05단원 개념04)

★ 화면 검증만 믿으면 안 됩니다. 그런데 서버 검증만 있으면 사용자가 답답합니다. 글자를 다 치고 버튼을 눌러야 "2글자 이상" 을 알게 되니까요.

★ 규칙이 두 곳에 생기는 문제 화면에도 "2글자 이상" 을 적으면, 서버 규칙이 바뀔 때 화면도 고쳐야 합니다. 한쪽만 고치면 어긋납니다.

그래서 이 파일은 화면 검증을 일부러 안 넣었습니다. 서버가 준 `details` 를 보여 주기만 합니다. 규칙은 한 곳에만 있습니다.

실무에서는 required, minlength 같은 HTML 속성으로 가벼운 것만 막고, 진짜 규칙은 서버에 두는 방식을 많이 씁니다.

직접 해 볼 것

▫ 직접 해보기 1

빈 폼으로 등록을 눌러 보세요. 두 칸에 오류가 붙나요? F12 → Network 에서 응답의 details 를 확인하세요.

▫ 직접 해보기 2

event.preventDefault() 를 지우고 눌러 보세요. 주소창이 어떻게 바뀌나요? 화면이 어떻게 되나요? (확인 후 반드시 되돌리세요)

▫ 직접 해보기 3

오류지우기() 를 지우고, 오류를 낸 뒤 고쳐서 다시 보내 보세요. 빨간 글씨가 사라지나요?

▫ 직접 해보기 4

서버 검증에 “이름에 숫자가 들어가야 한다” 를 추가하고, 화면은 한 줄도 안 고쳐도 오류가 잘 나오는지 확인하세요. 이게 details 를 쓰는 이유입니다.

▫ 직접 해보기 5

서버가 details 의 키를 “설비이름” 처럼 화면에 없는 이름으로 보내게 해 보세요. 화면이 터지나요? if (!오류칸) continue; 를 지우면 어떻게 되나요?

▫ 직접 해보기 6

입력칸에 required 를 붙여 보세요. 브라우저가 먼저 막나요? 서버 요청이 나가나요?

▫ 직접 해보기 7

06단원 연습문제 1의 조사붙이기를 화면에 가져와 “컨베이어 1호를 등록했습니다” 처럼 조사를 맞춰 보세요.

자주 하는 실수

이런 실수를 합니다

preventDefault 를 안 함

페이지가 새로고침됩니다. “아무 일도 안 일어나요” 의 1번 원인입니다.

이런 실수를 합니다

지난 오류를 안 지움

고친 칸에도 빨간 글씨가 남습니다.

이런 실수를 합니다

details 를 안 쓰고 message 만 띄움

사용자는 어느 칸이 틀렸는지 모릅니다.

이런 실수를 합니다

서버가 모르는 키를 줄 때 터짐

querySelector 결과가 `null` 인데 그대로 씁니다. 확인 한 줄이면 됩니다.

이런 실수를 합니다

성공했는데 폼을 안 비움

같은 것을 또 등록하기 쉽습니다.

이런 실수를 합니다

화면 검증만 하고 서버 검증을 안 함

Postman 으로 그냥 뚫립니다. 진짜 방어선은 서버입니다.

이런 실수를 합니다

같은 규칙을 화면과 서버에 각각 적음

한쪽만 고치면 어긋납니다. 규칙은 한 곳에 두세요.

정리

- 1 폼 submit 에는 preventDefault 를 부른다. 안 부르면 페이지가 통째로 새로고침된다.
- 2 서버는 06단원에서 정한 형식 그대로 준다. 화면은 그 형식만 알면 된다.
- 3 검증 실패는 details 를 써서 필드마다 표시한다. message 하나만 띄우면 어디를 고쳐야 하는지 알 수 없다.
- 4 다시 보내기 전에 지난 오류 표시를 지운다. 안 지우면 고친 것까지 빨강게 남는다.
- 5 서버가 화면에 없는 키를 줄 수도 있다. 그때 터지지 않게 한다.
- 6 성공하면 폼을 비운다. 안 비우면 같은 것을 또 보내게 된다.
- 7 화면 검증만 하고 서버 검증을 빼지 않는다. 화면 검증은 편의고, 진짜 방어는 서버다.
- 8 같은 규칙을 화면과 서버에 따로 적으면 언젠가 어긋난다. 한쪽을 기준으로 삼는다.

목록 화면 만들기 (페이징 · 검색 · 정렬)

RUN node 개념04_목록_화면_만들기.js

브라우저에서 <http://localhost:3000> 을 여세요.

06단원에서 만든 목록 API 를 화면에 붙입니다.

?page= ?limit= ?sort= ?q= ?line=

서버는 이미 다 만들어져 있습니다. 화면이 할 일은 셋입니다.

① 지금 조건을 기억한다 ② 조건이 바뀌면 주소를 만들어 다시 부른다 ③ 받은 결과와 meta 로 표와 쪽 버튼을 그린다

```
const express = require("express");
const path = require("path");
const { 설비목록 } = require("../06_REST_API_설계/data/설비데이터");

const app = express();
const PORT = Number(process.env.PORT) || 3000;

app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));

app.get("/", (req, res) => res.redirect("/04_목록.html"));
```

```
GET /04_목록.html
200
```

★ 데이터를 06단원에서 가져다 씁니다. 같은 데이터를 두 번 적지 않으려는 것입니다. 설비 12개가 들어 있어서 페이징을 확인하기 좋습니다.

```

function 쪽정보(query) {
  let page = Number(query.page);
  let limit = Number(query.limit);

  if (!Number.isInteger(page) || page < 1) page = 1;
  if (!Number.isInteger(limit) || limit < 1) limit = 10;
  if (limit > 100) limit = 100;

  return { page, limit };
}

function 정렬조건(sort) {
  if (!sort) return [];

  return sort
    .split(",")
    .map((조각) => 조각.trim())
    .filter(Boolean)
    .map((조각) =>
      조각.startsWith("-") ? { 키: 조각.slice(1), 방향: -1 } : { 키: 조각, 방향: 1 }
    );
}

function 비교하기(a, b, 조건들) {
  for (const { 키, 방향 } of 조건들) {
    const 왼쪽 = a[키];
    const 오른쪽 = b[키];

    if (typeof 왼쪽 === "string" && typeof 오른쪽 === "string") {
      const 결과 = 왼쪽.localeCompare(오른쪽, "ko", { numeric: true });
      if (결과 !== 0) return 결과 * 방향;
      continue;
    }

    if (왼쪽 < 오른쪽) return -1 * 방향;
    if (왼쪽 > 오른쪽) return 1 * 방향;
  }
}

```



```

    }

    return 0;
}

app.get("/api/v1/equipments", (req, res) => {
    const q = (req.query.q ?? "").trim().toLowerCase();
    const { line } = req.query;

    let 결과 = 설비목록();

    if (q) {
        결과 = 결과.filter((설비) =>
            ["name", "line", "status"].some((키) => String(설비
[키]).toLowerCase().includes(q))
        );
    }

    if (line) 결과 = 결과.filter((설비) => 설비.line === line);

    const 조건들 = 정렬조건(req.query.sort);
    if (조건들.length > 0) 결과 = [...결과].sort((a, b) => 비교하기(a, b, 조건들));

    const total = 결과.length; // * 자르기 전에 셉니다 (06단원 개념03) const { page,
limit } = 쪽정보(req.query);
    const 시작 = (page - 1) * limit;

    res.json({
        data: 결과.slice(시작, 시작 + limit),
        meta: { page, limit, total, totalPages: Math.ceil(total / limit) },
    });
});

```

GET /api/v1/equipments?limit=5&page=1&sort=id

```

200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동","설치일":"2021-03-15"},
{"id":2,"name":"컨베이어 2호","line":"A","status":"정지","설치일":"2021-03-15"},
{"id":3,"name":"프레스 1호","line":"B","status":"가동","설치일":"2019-07-02"},
{"id":4,"name":"프레스 2호","line":"B","status":"점검중","설치일":"2019-07-02"},
{"id":5,"name":"용접로봇 1호","line":"C","status":"가동","설치일":"2023-11-20"}],
"meta":{"page":1,"limit":5,"total":12,"totalPages":3}}

```

GET /api/v1/equipments?q=로봇&limit=5&sort=id

```
200 {"data":[{"id":5,"name":"용접로봇 1호","line":"C","status":"가동","설치일":"2023-11-20"}, {"id":6,"name":"용접로봇 2호","line":"C","status":"가동","설치일":"2023-11-20"}, {"id":11,"name":"적재로봇 1호","line":"A","status":"가동","설치일":"2024-02-11"}, {"id":12,"name":"적재로봇 2호","line":"B","status":"가동","설치일":"2024-02-11"}], "meta":{"page":1,"limit":5,"total":4,"totalPages":1}}
```

화면으로 확인

섹션 2

처음 열면 5개씩 1쪽

화면 04_목록.html

#요약 전체 12건 중 5건

#쪽정보 1 / 3

#본문 컨베이어 1호

다음 쪽으로

화면 04_목록.html

#다음쪽

#쪽정보 2 / 3

#본문 용접로봇 2호

마지막 쪽에서는 '다음' 이 잠깁니다

화면 04_목록.html

#다음쪽 → #다음쪽

#쪽정보 3 / 3

#본문 적재로봇 2호

★ 갈 수 없는 쪽의 버튼을 잠그는 것이 중요합니다. 안 잠그면 사용자가 눌러 보고 "아무 일도 안 일어나네"를 겪습니다. 또는 4쪽으로 가서 빈 화면을 보게 됩니다.

검색

화면 04_목록.html

입력(#검색=로봇) → #라인

#요약 전체 4건 중 4건

#본문 용접로봇 1호

라인으로 거르기

화면 04_목록.html

입력(#라인=C) → #라인

#요약 전체 4건

#본문 포장기 1호

결과가 없을 때

화면 04_목록.html

입력(#검색=없는이름) → #라인

#요약 조건에 맞는 설비가 없습니다

#쪽정보 0 / 0

★★ 여기가 개념02 와 이어집니다. 서버는 200 과 빈 배열을 줍니다. 화면이 안내를 안 하면 텅 빈 표만 남습니다.

정렬

화면 04_목록.html

입력(#정렬=id) → #정렬

#본문 적재로봇 2호

#쪽정보 1 / 3

★ #라인 이나 #정렬 을 '누른' 이유 select 는 값을 넣기만 해서는 change 이벤트가 안 납니다. 화면검증 도구가 값을 넣으면서 change 를 함께 일으키는데, 그 뒤에 아무거나 눌러서 화면이 다시 그려질 시간을 주는 것입니다.

조건이 바뀌면 1쪽으로

섹션 3

3쪽을 보다가 검색하면 다시 1쪽이어야 합니다.

화면 04_목록.html

#다음쪽 → #다음쪽 → 입력(#검색=로봇) → #라인

#쪽정보 1 / 1

#요약 전체 4건

★★★ 이게 목록 화면에서 가장 많이 나는 버그입니다.

3쪽을 보다가 검색을 하면, 검색 결과는 1쪽뿐인데 여전히 page=3 으로 요청합니다. 서버는 200 과 빈 배열을 줍니다. (06단원에서 그렇게 만들었습니다) 화면에는 "조건에 맞는 설비가 없습니다" 가 나옵니다.

사용자는 "검색했는데 결과가 없다" 고 생각합니다. 사실은 4건이 있는데요.

막는 법은 한 줄입니다. 조건이 바뀌면 page 를 1 로 되돌립니다. 검색·필터·정렬·쪽크기 — 전부 마찬가지로입니다. 쪽 이동만 예외입니다.

```
app.use((req, res) => {
  res.status(404).json({ error: { code: "ROUTE_NOT_FOUND", message: "그런 주소가 없습니다" } });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  res.status(500).json({ error: { code: "INTERNAL_ERROR", message: "서버에서 문제가 생겼습니다" } });
});

app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}`);
});
```

목록 화면이 켜져야 할 것

① 상태를 한 곳에 모은다

{ page, limit, sort, q, line } 하나로 관리합니다.
흩어 두면 "지금 몇 쪽이지?" 를 알기 어렵고, 빠뜨리기 쉽습니다.

② 조건이 바뀌면 1쪽으로

쪽 이동만 빼고 전부 page = 1 입니다.

③ 빈 값은 주소에 안 넣는다

?q= 를 보내면 서버가 '빈 검색어' 를 받습니다. (06단원 개념03)
URLSearchParams 로 만들되, 값이 있을 때만 set 하세요.

④ 늦게 온 응답 버리기

개념02 섹션 4 의 그 문제입니다. 검색창에서 특히 자주 납니다.

⑤ 디바운스

글자를 칠 때마다 요청하면 "설비관리" 를 치는 동안 5번 나갑니다.
멈춘 뒤 0.3초쯤 기다렸다 보냅니다.

⑥ 갈 수 없는 쪽의 버튼 잠그기

meta.page 와 meta.totalPages 를 비교하면 됩니다.

⑦ 결과 없음 안내

빈 표만 남으면 고장으로 보입니다.

주소창에 상태를 담는 것에 대하여

지금 화면은 상태를 자바스크립트 변수에만 두고 있습니다. 그래서 이런 것들이 안 됩니다.

· 새로고침하면 처음으로 돌아갑니다 · “이 검색 결과 좀 봐” 하고 링크를 보낼 수 없습니다 · 뒤로 가기가
목록 안에서 안 먹습니다

상태를 주소창에도 담으면 셋 다 해결됩니다.

```
history.replaceState(null, "", "?" + 쿼리.toString());
```

그리고 화면이 처음 뜰 때 주소에서 읽어 옵니다.

```
const 쿼리 = new URLSearchParams(location.search);  
상태.page = Number(쿼리.get("page")) || 1;
```

★ 06단원에서 “하나를 콕 집는 것은 경로, 걸러 보는 것은 쿼리” 라고 했습니다. 화면 주소도 같은 규칙을
씁니다. 목록 조건은 쿼리에 담습니다.

연습문제에서 직접 붙여 봅니다.

직접 해 볼 것

➡ 직접 해보기 1

3쪽까지 가 보세요. '다음' 버튼이 잠기나요?

▫ 직접 해보기 2

조건바뀔() 에서 상태.page = 1 을 지우세요. 3쪽으로 간 뒤 '로봇' 으로 검색하면 어떻게 되나요? F12 → Network 에서 page 값이 무엇으로 나가나요? (확인 후 반드시 되돌리세요)

▫ 직접 해보기 3

주소만들기 에서 if (상태.q) 를 지우고 항상 넣게 해 보세요. 검색창을 비웠을 때 어떤 주소가 나가나요?

▫ 직접 해보기 4

디바운스(setTimeout)를 지우고 '컨베이어' 를 쳐 보세요. Network 탭에 요청이 몇 개 나가나요?

▫ 직접 해보기 5

마지막번호 확인을 지우고, Network 를 Slow 3G 로 바꾼 뒤 빠르게 검색어를 바꿔 보세요. 결과가 검색어와 어긋나나요?

▫ 직접 해보기 6

상태를 주소창에 담아 보세요. history.replaceState 로 쓰고, 처음 뜰 때 읽어 옵니다. 새로고침해도 그 자리에 있나요?

▫ 직접 해보기 7

표의 머리글을 눌러 정렬되게 해 보세요. 같은 것을 또 누르면 반대로 정렬되게요.

자주 하는 실수

이런 실수를 합니다

조건이 바뀌었는데 쪽 번호를 안 되돌림

검색 결과가 있는데도 빈 화면이 나옵니다. 목록 화면 1번 버그입니다.

이런 실수를 합니다

빈 값을 주소에 넣음

?q= 가 서버로 갑니다. 서버가 어떻게 다루느냐에 따라 결과가 달라집니다.

이런 실수를 합니다

total 을 화면에서 세려고 함

화면에는 지금 쪽 것만 있습니다. meta.total 을 쓰세요.

이런 실수를 합니다

갈 수 없는 쪽 버튼을 안 잠금

눌러도 아무 일이 없거나 빈 화면이 나옵니다.

이런 실수를 합니다

글자를 칠 때마다 요청

요청이 다섯 배로 늘고, 늦게 온 응답 문제까지 함께 옵니다.

이런 실수를 합니다

상태를 여기저기 변수로 흩어 놓음

조건이 하나 늘 때마다 고칠 곳이 늘어납니다.

이런 실수를 합니다

결과 없음을 안 다룸

빈 표만 남습니다. 개념02 에서 본 그대로입니다.

정리

- 1 목록 화면은 06단원의 목록 API 를 그대로 쓴다. 화면이 새로 만들 것은 없다.
- 2 검색어나 필터가 바뀌면 쪽 번호를 1로 되돌린다. 안 되돌리면 결과가 세 건인데 5쪽을 보여 달라고 해서 빈 화면이 나온다.
- 3 빈 값은 주소에 넣지 않는다. ?q=&type= 처럼 붙으면 서버가 빈 검색어를 받는다.
- 4 total 은 서버가 준 값을 쓴다. 화면에서 세면 지금 쪽의 개수만 나온다.
- 5 갈 수 없는 쪽 버튼은 잠근다. 첫 쪽에서 '이전', 끝 쪽에서 '다음'.
- 6 글자를 칠 때마다 요청하지 않는다. 잠깐 기다렸다 보내거나 버튼으로 보낸다.
- 7 조건(검색어·쪽·정렬)을 여기저기 변수로 흩어 놓지 않는다. 한곳에 모아 두고 그것만 본다.
- 8 결과 없음을 반드시 다룬다. 목록이 비면 빈 표가 아니라 "없습니다" 를 보여 준다.

업로드 화면 만들기

RUN node 개념05_업로드_화면_만들기.js

브라우저에서 `http://localhost:3000` 을 여세요.

09단원에서 만든 업로드 API 에 화면을 붙입니다.

화면 쪽에서 새로 배울 것은 셋입니다.

- ① 고른 파일을 보내기 전에 보여 주기 (미리보기) ② 진행률 보여 주기 ← fetch 로는 못 합니다 ③ 09단원의 에러(용량.확장자)를 화면에 알려 주기

```
const express = require("express");
const multer = require("multer");
const path = require("path");
const fs = require("fs");

const app = express();
const PORT = Number(process.env.PORT) || 3000;

app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));

app.get("/", (req, res) => res.redirect("/05_업로드.html"));
```

GET /05_업로드.html

200

09단원 업로드 서버를 그대로

섹션 1

```

const 업로드폴더 = path.join(__dirname, "uploads");
fs.rmSync(업로드폴더, { recursive: true, force: true });
fs.mkdirSync(업로드폴더, { recursive: true });

function 이름되돌리기(이름) {
  return Buffer.from(이름, "latin1").toString("utf8");
}

const 허용확장자 = [".png", ".jpg", ".jpeg", ".gif"];

const 업로드 = multer({
  storage: multer.diskStorage({
    destination: (req, file, cb) => cb(null, 업로드폴더),
    filename: (req, file, cb) => {
      const 확장자 = path.extname(이름되돌리기(file.originalname)).toLowerCase();
      cb(null, `${Date.now()}-${Math.round(Math.random() * 1e9)}${확장자}`);
    },
  }),
  limits: { fileSize: 2000 }, // 시험용. 실제로는 5 * 1024 * 1024
  fileFilter: (req, file, cb) => {
    const 확장자 = path.extname(이름되돌리기(file.originalname)).toLowerCase();

    if (!허용확장자.includes(확장자)) {
      const 에러 = new Error(`${확장자} || "확장자 없는 파일"은(는) 올릴 수 없습니다`);
      에러.status = 400;
      에러.code = "BAD_EXTENSION";
      return cb(에러);
    }

    cb(null, true);
  },
});

let 기록들 = [];
let 다음번호 = 1;

app.get("/api/v1/files", (req, res) => {
  res.json({ data: 기록들 });
});

```

GET /api/v1/files

200 {"data":[]}

```

app.post("/api/v1/files", 업로드.single("file"), (req, res) => {
  if (!req.file) {
    return res.status(400).json({ error: { code: "NO_FILE", message: "파일이 없습니다" } });
  }

  const 기록 = {
    id: 다음번호,
    원래이름: 이름되돌리기(req.file.originalname),
    크기: req.file.size,
  };

  다음번호 += 1;
  기록들.push(기록);

  res.status(201).json({ data: 기록 });
});

```

POST /api/v1/files multipart:@file=설비사진.png:30

201 {"data":{"id":1,"원래이름":"설비사진.png","크기":30}}

POST /api/v1/files multipart:@file=악성.exe:10

400 {"error":{"code":"BAD_EXTENSION","message":".exe 은(는) 올릴 수 없습니다"}}

POST /api/v1/files multipart:@file=큰파일.png:3000

400 {"error":{"code":"LIMIT_FILE_SIZE","message":"파일이 너무 큼니다 (최대 2000바이트)"} }

화면으로 확인

섹션 2

파일을 고르면 정보가 먼저 보입니다.

화면 05_업로드.html

파일(#파일=10_프론트엔드_연동/시험파일/설비사진.png)

#파일정보 설비사진.png

★ 서버에 보내기 전입니다. 브라우저가 아는 것만으로 보여 준 것입니다. 이름·크기·형식은 File 객체에 이미 들어 있습니다.

fetch 로 올리기

화면 05_업로드.html

파일(#파일=10_프론트엔드_연동/시험파일/설비사진.png) → #올리기

#알림 올렸습니다

#목록 설비사진.png

확장자가 안 되는 파일

화면 05_업로드.html

파일(#파일=10_프론트엔드_연동/시험파일/악성.exe) → #올리기

#알림 .exe 은(는) 올릴 수 없습니다

용량이 넘는 파일

화면 05_업로드.html

파일(#파일=10_프론트엔드_연동/시험파일/너무큰파일.png) → #올리기

#알림 파일이 너무 큼니다

★★ 09단원에서 만든 에러가 그대로 화면에 나왔습니다. 서버가 code 와 message 를 잘 만들어 뒀기 때문에 화면은 message 를 띄우기만 하면 됩니다.

09단원에서 “MulterError 를 400 으로 바꾸고 안내 문구를 만들라” 고 한 이유가 이것입니다. 그걸 안 했다면 화면에 “서버에서 문제가 생겼습니다” 만 나왔을 것입니다.

진행률로 올리기

화면 05_업로드.html

파일(#파일=10_프론트엔드_연동/시험파일/설비사진.png) → #진행률로

#진행칸 100%

#알림 올렸습니다

파일을 안 고르고 누르면

화면 05_업로드.html

#올리기

#알림 파일을 먼저 고르세요

```
app.get("/api/v1/why-xhr", (req, res) => {
  res.json({
    data: {
      fetch가못하는것: "올리는 도중의 진행률. upload progress 이벤트가 없습니다.",
      fetch가할수있는것: "내려받는 진행률은 됩니다. response.body 를 조금씩 읽으면 됩니다.",
      그래서: "업로드 진행률이 필요하면 아직도 XMLHttpRequest 를 씁니다.",
      언제필요한가: [
        "큰 파일을 올릴 때. 몇 초 이상 걸리면 사용자는 멈춘 줄 압니다.",
        "여러 파일을 올릴 때. 몇 번째인지 알려 줘야 합니다.",
      ],
      언제필요없나: "작은 파일이면 '올리는 중...' 만 보여 줘도 충분합니다.",
    },
  });
});
```

GET /api/v1/why-xhr

200

★★ 2026년인데도 XMLHttpRequest 를 쓰는 이유입니다. fetch 가 대부분을 대체했지만, 업로드 진행률만은 아직 안 됩니다.

axios 같은 라이브러리에 onUploadProgress 가 있는데, 그것도 안에서는 XMLHttpRequest 를 씁니다.

★ 진행률이 100% 가 됐는데도 안 끝나는 경우가 있습니다. 100% 는 '다 보냈다' 는 뜻이지 '서버가 다 처리했다' 는 뜻이 아닙니다. 서버가 파일을 저장하고 기록을 남기는 시간이 더 걸립니다. 그래서 100% 뒤에도 "처리 중..." 을 보여 주는 화면이 많습니다.

```
app.use((req, res) => {
  res.status(404).json({ error: { code: "ROUTE_NOT_FOUND", message: "그런 주소가 없습니다" } });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.name}: ${err.message}`);

  if (err.name === "MulterError") {
    const 안내 = {
      LIMIT_FILE_SIZE: "파일이 너무 큼니다 (최대 2000바이트)",
      LIMIT_FILE_COUNT: "파일이 너무 많습니다",
      LIMIT_UNEXPECTED_FILE: "필드 이름이 file 이 맞나요?",
    };

    return res.status(400).json({
```

화면 코드도 떼어 내면 시험할 수 있다

RUN node 연습문제.js

브라우저 없이 푸는 문제들입니다. 여기서 만든 함수를 연습문제_서버.js 의 화면에서 그대로 씁니다.

★ 문제 1·2·3·6 은 그냥 실행되고, 4·5 는 기다려야 해서 async 함수 안에 둡니다. 순서는 1 → 2 → 3 → 6 → (main 안에서) 4 → 5 입니다.

문제 1 목록 주소 만들기

상태 객체를 받아 목록 API 주소를 만드는 함수 주소만들기(기준, 상태) 를 만드세요.

page 와 limit 은 언제나 넣습니다. sort · q · line 은 값이 있을 때만 넣습니다.

★ URLSearchParams 를 쓰세요. 직접 이어 붙이면 한글 인코딩을 빠뜨립니다.

시험할 것 (이 순서로, 기준은 전부 "/api/v1/equipments")

```
{ page: 1, limit: 5, sort: "id", q: "", line: "" }
{ page: 2, limit: 10, sort: "-id", q: "로봇", line: "A" }
{ page: 1, limit: 5, sort: "", q: " ", line: "" }
```

```
기대 출력  /api/v1/equipments?page=1&limit=5&sort=id
           /api/v1/equipments?page=2&limit=10&sort=-id&q=%EB%A1%9C%EB%B4%87&line=A
           /api/v1/equipments?page=1&limit=5&q=++
```

★★ 세 번째가 함정입니다. q 가 공백 두 칸입니다. 빈 글자가 아니라서 조건을 통과해 버립니다. 어디서 막아야 할까요? (힌트: 상태에 담을 때)

여기에 코드를 쓰세요

문제 2 쪽 버튼 잠그기

meta 를 받아 이전·다음 버튼을 잠글지 판단하는 함수를 만드세요.

```
쪽버튼상태(meta) → { 이전잠김, 다음잠김, 표시 }
```

버튼찍기(meta) → "1 / 3 이전:잠김 다음:열림" 처럼 출력

시험할 것 (이 순서로)

```
{ page: 1, totalPages: 3 }
{ page: 2, totalPages: 3 }
{ page: 3, totalPages: 3 }
{ page: 1, totalPages: 0 }
{ page: 1, totalPages: 1 }
```

기대 출력	1 / 3	이전:잠김	다음:열림
	2 / 3	이전:열림	다음:열림
	3 / 3	이전:열림	다음:잠김
	1 / 0	이전:잠김	다음:잠김
	1 / 1	이전:잠김	다음:잠김

★ 네 번째는 결과가 0건일 때입니다. totalPages 가 0 입니다. (06단원 연습문제 2) 그래도 잘 잠기나요?

여기에 코드를 쓰세요

문제 3 에러를 어디에 보여 줄까

에러 code 를 받아 화면 어디에 보여 줄지 정하는 함수를 만드세요.

VALIDATION_FAILED	→ 각 칸 아래에 (details 를 씁니다)
DUPLICATED	→ 이름 칸 아래에
UNAUTHENTICATED	→ 로그인 화면으로 보냅니다
FORBIDDEN	→ 폼 전체에 '권한이 없습니다'
NETWORK	→ 폼 전체에 '연결 확인' + 다시 시도 버튼

그 밖 → 폼 전체에 message 그대로

기대 출력	VALIDATION_FAILED → 각 칸 아래에 (details 를 씁니다)
	DUPLICATED → 이름 칸 아래에
	UNAUTHENTICATED → 로그인 화면으로 보냅니다
	FORBIDDEN → 폼 전체에 '권한이 없습니다'
	NETWORK → 폼 전체에 '연결 확인' + 다시 시도 버튼
	INTERNAL_ERROR → 폼 전체에 message 그대로

★★ 왜 message 가 아니라 code 로 판단해야 하나요? ★ DUPLICATED 에는 details 가 없습니다.
그런데도 이름 칸에 붙일 수 있는 이유는?

여기에 코드를 쓰세요

문제 6 (4·5 보다 먼저 실행됩니다)

로딩 여부·에러·목록을 받아 화면 상태를 정하는 함수 화면상태 를 만드세요.

로딩 / 실패 / 결과없음 / 성공

★ 판단 순서가 중요합니다. 무엇을 먼저 봐야 할까요? 순서를 잘못 두면 화면이 뜨자마자 “설비가 없습니다”가 깜빡입니다.

시험할 것 (이 순서로) `(true, null, null)` `(false, { message: “서버 오류” }, null)` `(false, null, [])` `(false, null, [{ id: 1 }])` `(false, null, null)`

기대 출력	로딩
	실패
	결과없음
	성공
	결과없음

여기에 코드를 쓰세요

문제 4 디바운스

함수와 시간을 받아, 마지막 호출 뒤 그 시간이 지나야 실행되는 함수를 돌려주세요.

디바운스(함수, 시간) → 새 함수

힌트: `setTimeout` 을 걸어 두고, 또 불리면 `clearTimeout` 으로 지웁니다.

인자는 (...인자들) 로 받아 그대로 넘기세요.

시험 방법 부른횟수·마지막글자를 세는 함수를 디바운스(100) 로 감쌉니다. “설” “설비” “설비관” “설비관리” 를 20ms 간격으로 부릅니다. 200ms 기다린 뒤 결과를 찍습니다.

기대 출력 디바운스: 1번 실행, 마지막 글자 "설비관리"

★ 네 번 쳤는데 요청이 한 번인 이유를 설명할 수 있어야 합니다.

`main()` 안에 코드를 쓰세요

문제 5 늦게 온 응답 버리기

마지막 요청의 결과만 반영하는 도우미를 만드세요.

`const 반영하기 = 마지막만반영();`

`await 반영하기(일하는함수, 반영하는함수)` → 반영했으면 `true`, 버렸으면 `false`

힌트: 바깥에 마지막번호를 두고, 부를 때마다 1 늘리고 내 번호를 기억합니다.

일이 끝났을 때 내 번호가 마지막이 아니면 버립니다.

시험 방법 느린 것(80ms, "느린 응답")을 먼저 보내고 빠른 것(10ms, "빠른 응답")을 나중에 보냅니다.
`Promise.all` 로 둘 다 기다린 뒤 결과를 찍습니다.

기대 출력 느린 것 반영됨: `false`
 빠른 것 반영됨: `true`

화면 빠른 응답

★★ 막지 않았다면 화면에 무엇이 남았을까요? ★ 이게 검색창에서 어떻게 나타나는지 말로 설명해 보세요.

`main()` 안에 코드를 쓰세요

`async function main() {`

문제 4·5 를 여기에 쓰세요


```
}  
  
main();
```

문제 7 fetch 는 언제 던지나

fetch 가 '던지는' 경우 (catch 로 가는 경우) 를 전부 적어 보세요:

fetch 가 '안 던지는' 경우: _____

그래서 무엇이 문제인가: _____

어떻게 하나: _____

204 는 왜 따로 다루나: _____

JSON 이 아닌 응답은 왜 따로 다루나: _____ (힌트: 서버 앞에
Nginx 가 있고 서버가 죽으면 무엇이 오나요?)

문제 8 화면 검증과 서버 검증

화면 검증이 하는 일: _____

서버 검증이 하는 일: _____

화면 검증만 하면 무엇이 뚫리나: _____

서버 검증만 하면 무엇이 불편한가: _____

같은 규칙을 두 곳에 적으면 무엇이 문제인가: _____

이 자료(개념03)는 어떤 방법을 골랐나, 왜: _____

설비 관리 화면 하나 완성하기

RUN npx nodemon 연습문제_서버.js

브라우저에서 `http://localhost:3000` 을 여세요.

★ 이 문제의 목적 개념01~04 에서 배운 것을 ‘한 화면’ 에 전부 모으는 것입니다.

★ 서버는 미리 만들어 두었습니다. 아래 코드는 고치지 마세요.

여러분이 만들 것은 화면 하나입니다.

`public/연습_관리화면.html` ← 이 파일을 만드세요

★★ 그 자리에 완성본이 이미 들어 있습니다.

직접 만들려면 먼저 `public/연습_관리화면.html` 을 지우거나
다른 이름(예: `연습_관리화면_완성본.html`)으로 바꿔 두고 시작하세요.
그대로 두면 서버를 켜자마자 완성된 화면이 떴서 연습이 되지 않습니다.

등록 폼(이름 · 라인)과 목록(검색 · 라인 거르기 · 페이지 · 삭제)이 있는 화면입니다. 아래 일곱 문제가 그 화면을 나눠 놓은 것입니다. 순서대로 하면 완성됩니다.

확인은 브라우저로 합니다. `http://localhost:3000`

문제 1 화면 뼈대와 `api.js` 쓰기

등록 폼과 목록이 들어갈 자리를 만들고, 목록을 한 번 그려 보세요.

★ `fetch` 를 직접 쓰지 마세요. `public/api.js` 의 함수만 부릅니다. 화면마다 `fetch` 를 불러 놓으면 규칙이 바뀔 때 전부 고쳐야 합니다(개념01).

눈으로 확인: 화면을 열면 설비 네 건이 표로 보인다.

문제 2 네 가지 상태 전부 그리기

로딩 중 · 성공 · 빈 결과 · 실패 넷을 각각 다르게 보여 주세요(개념02).

빈 결과는 에러가 아닙니다. “없습니다” 를 보여 줍니다. 실패는 콘솔이 아니라 화면에 띄우고, 다시 시도할 버튼도 같이 줍니다.

눈으로 확인: 검색어에 “ㄹㅇㄹㅇ” 을 넣으면 빈 결과 화면이,

서버를 끄고 새로고침하면 실패 화면이 나온다.

문제 3 검색과 라인 거르기

검색어 칸과 라인 선택을 붙이세요.

☐ 빈 값은 주소에 넣지 않습니다. ?q=&line= 이 붙으면 안 됩니다. ☐ 조건이 바뀌면 쪽 번호를 1로 되돌립니다. ← 제일 많이 빠뜨립니다

눈으로 확인: 3쪽을 보다가 검색어를 넣으면 1쪽으로 돌아간다.

F12 → Network 에서 주소에 빈 값이 안 붙는다.

문제 4 페이징

이전·다음 버튼과 “N / M 쪽” 표시를 붙이세요.

☐ total 은 서버가 준 값을 씁니다. 화면에서 세면 지금 쪽 개수만 나옵니다. ☐ 갈 수 없는 쪽 버튼은 잠급니다. 첫 쪽에서 이전, 끝 쪽에서 다음.

눈으로 확인: 첫 쪽에서 ‘이전’ 이 눌리지 않는다.

문제 5 등록 폼

이름과 라인을 받아 등록하세요(개념03).

☐ preventDefault 를 부릅니다. 안 부르면 페이지가 새로고침됩니다. ☐ 보내기 전에 지난 오류 표시를 지웁니다. ☐ 서버가 준 details 를 칸마다 붙여 보여 줍니다. message 하나만 띄우지 않습니다. ☐ 보내는 동안 버튼을 잠그고, 푸는 것은 finally 에 둡니다. ☐ 성공하면 폼을 비우고 목록을 새로 그립니다.

눈으로 확인: 이름을 비우고 보내면 이름 칸 아래에만 빨간 글씨가 뜬다.

고쳐서 다시 보내면 그 글씨가 사라진다.

문제 6 삭제

줄마다 삭제 버튼을 붙이세요.

★ 버튼마다 addEventListener 를 걸지 마세요. 이벤트 위임을 쓰세요. 목록을 다시 그릴 때마다 거는 방식은 그릴 때마다 핸들러가 쌓입니다.

눈으로 확인: 목록을 여러 번 새로 그린 뒤 삭제해도 한 번만 지워진다.

문제 7 디바운스와 늦게 온 응답

마지막 두 가지를 챙기세요(개념02 섹션4 · 개념04).

□ 검색은 글자마다 보내지 않고 잠깐 기다렸다 보냅니다(디바운스). □ 늦게 온 응답이 최신 화면을 덮어쓰지 않게 합니다.

요청마다 번호를 붙여 두고 마지막 것만 그립니다.

눈으로 확인: 검색어를 빠르게 치면 요청이 글자 수만큼 나가지 않는다.

느린 응답이 뒤늦게 와도 화면이 예전 결과로 돌아가지 않는다.

```
const express = require("express");
const path = require("path");

const app = express();
const PORT = Number(process.env.PORT) || 3000;

app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));

app.get("/", (req, res) => res.redirect("/연습_관리화면.html"));
```

서버 (여기는 그대로 두세요) —————

```
let 설비들 = [
  { id: 1, name: "컨베이어 1호", line: "A", status: "가동" },
  { id: 2, name: "프레스 1호", line: "B", status: "정지" },
  { id: 3, name: "용접로봇 1호", line: "C", status: "가동" },
  { id: 4, name: "검사기 1호", line: "A", status: "점검중" },
];

let 다음번호 = 5;

app.get("/api/v1/equipments", (req, res) => {
  const q = (req.query.q ?? "").trim().toLowerCase();
  const { line } = req.query;

  let 결과 = 설비들;

  if (q) {
    결과 = 결과.filter((설비) =>
      ["name", "line", "status"].some((키) => String(설비
[키]).toLowerCase().includes(q))
    );
  }

  if (line) 결과 = 결과.filter((설비) => 설비.line === line);

  const total = 결과.length;

  let page = Number(req.query.page);
  let limit = Number(req.query.limit);
  if (!Number.isInteger(page) || page < 1) page = 1;
  if (!Number.isInteger(limit) || limit < 1) limit = 10;

  const 시작 = (page - 1) * limit;

  res.json({
    data: 결과.slice(시작, 시작 + limit),
    meta: { page, limit, total, totalPages: Math.ceil(total / limit) },
  });
});
```

```

});

app.post("/api/v1/equipments", (req, res) => {
  const { name, line } = req.body || {};
  const 항목 = [];

  if (!name) 항목.push({ 키: "name", 이유: "필수입니다" });
  else if (name.length < 2) 항목.push({ 키: "name", 이유: "2글자 이상이어야 합니다" });

  if (!line) 항목.push({ 키: "line", 이유: "필수입니다" });
  else if (!["A", "B", "C"].includes(line))
    항목.push({ 키: "line", 이유: "A, B, C 중 하나여야 합니다" });

  if (항목.length > 0) {
    return res.status(400).json({
      error: { code: "VALIDATION_FAILED", message: "입력값이 올바르지 않습니다",
details: 항목 },
    });
  }

  if (설비들.some((설비) => 설비.name === name)) {
    return res.status(409).json({
      error: { code: "DUPLICATED", message: "이미 있는 설비 이름입니다" },
    });
  }

  const 새설비 = { id: 다음번호, name, line, status: "정지" };
  다음번호 += 1;
  설비들.push(새설비);

  res.status(201).json({ data: 새설비 });
});

app.delete("/api/v1/equipments/:id", (req, res) => {
  const 번호 = Number(req.params.id);
  const 설비 = 설비들.find((설비) => 설비.id === 번호);

  if (!설비) {
    return res.status(404).json({ error: { code: "NOT_FOUND", message: "설비를 찾을 수 없습니다" } });
  }
}

```

★ 07단원의 업무 규칙입니다. 화면이 이걸 어떻게 보여 줄지 생각해 보세요.

```

if (설비.status === "가동") {
  return res.status(409).json({

```

```

});
}

설비들 = 설비들.filter((설비) => 설비.id !== 번호);
res.sendStatus(204);
});

app.use((req, res) => {
  res.status(404).json({ error: { code: "ROUTE_NOT_FOUND", message: "그런 주소가 없습니다" } });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  res.status(500).json({ error: { code: "INTERNAL_ERROR", message: "서버에서 문제가 생겼습니다" } });
});

app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}`);
});

```

화면에 꼭 넣어야 할 id (검증이 이 이름으로 찾습니다)

```

#품          등록 품
#name        이름 입력칸          (name 속성도 "name" 으로)
#line        라인 select          (name 속성도 "line" 으로)
#오류-name   이름 칸 오류 자리
#오류-line   라인 칸 오류 자리
#보내기      등록 버튼
#알림        성공·실패 알림 자리

#검색        검색 입력칸
#라인거르기  라인 select
#요약        "전체 N건 중 M건" / 로딩 / 빈 결과 안내
#본문        표의 tbody
#쪽정보      "1 / 2"

```

#이전쪽 #다음쪽 쪽 버튼 .지우기 각 줄의 삭제 버튼 (data-id 에 번호)

다 만들었으면 이 순서대로 눌러 보세요

1 처음 화면	3건 + "1 / 2" (limit 을 3 으로 두세요)
2 '다음' 누르기	2쪽, '다음' 이 잠김
3 2쪽에서 '로봇' 검색	1쪽으로 돌아가고 1건 **
4 '없는것' 검색	"조건에 맞는 설비가 없습니다" 5 빈 폼으로 등록
두 칸에 빨간 글씨	
6 라인을 2 로 등록	라인 칸에만 빨간 글씨
7 제대로 등록	목록이 새로 그려지고 전체가 늘어남
8 같은 이름으로 또 등록	이름 칸에 "이미 있는 설비 이름입니다" 9 가동
중인 설비 삭제	"가동 중인 설비는 삭제할 수 없습니다"
10 정지 중인 설비 삭제	지워지고 목록이 새로 그려짐

★★ 3번이 이 실습의 채점 기준입니다. 2쪽을 보다가 검색했을 때 1쪽으로 안 돌아가면, 결과가 있는데도 "조건에 맞는 설비가 없습니다" 가 나옵니다.

다 됐으면 기계로도 확인하세요.

```
node _검증도구/화면검증.js 10_프론트엔드_연동
```

다 만든 뒤 답해 보세요

① 3번에서 1쪽으로 안 돌리면 서버는 무엇을 돌려주나요? 화면에는요?

② 삭제 버튼에 이벤트 위임을 쓴 이유는? 표를 다시 그린 뒤에도 버튼이 동작하나요?

③ 삭제한 뒤 목록을 다시 부르는 것과, 화면에서만 그 줄을 지우는 것. 각각 무엇이 좋고 무엇이 나쁜가요?

④ 9번에서 409 를 어디에 보여 줬나요? 왜 거기인가요? _____

⑤ 검색창의 디바운스를 지우면 요청이 몇 배로 늘어나나요? F12 → Network 에서 세어 보세요.

⑥ 이 화면에 업로드를 붙인다면 무엇을 더 챙겨야 하나요? (개념05)

잘 안 될 때 보는 곳

등록 버튼을 눌렀는데 페이지가 새로고침됨 → event.preventDefault() 를 안 했습니다.

검색했는데 "설비가 없습니다" → 조건이 바뀌었는데 page 를 1 로 안 돌렸습니다.

고쳤는데 빨간 글씨가 그대로 → 보내기 전에 지난 오류를 안 지웠습니다.

표를 다시 그린 뒤 삭제 버튼이 안 먹음 → 버튼마다 addEventListener 를 달았습니다. tbody 에 위임하세요.

화면이 뜨자마자 "설비가 없습니다" 가 깜빡임 → 로딩 상태를 안 다뤘습니다. 판단 순서를 확인하세요.

빠르게 검색하면 결과가 검색어와 안 맞음 → 늦게 온 응답을 안 버렸습니다. (개념02 섹션 4)

요청이 글자마다 나감 → 디바운스를 안 걸었습니다.

서버와 이야기하는 부분을 한곳에 모은다

10단원의 모든 화면이 이 파일을 씁니다.

왜 따로 빼나 화면마다 fetch 를 직접 쓰면, 에러 처리를 화면 수만큼 반복하게 됩니다. 그러다 한 군데를 빠뜨리면 그 화면만 조용히 이상해집니다.

06단원에서 응답 형식을 통일해 둔 덕에, 처리도 한곳에 모을 수 있습니다.

```
성공 { "data": ... , "meta": {...} }
실패 { "error": { "code": "...", "message": "...", "details": [...] } }
```

서버가 준 에러를 그대로 담아 두는 에러입니다. 화면에서 code 로 판단하고 message 를 보여 줄 수 있습니다.

```
class API에러 extends Error {
  constructor(status, 봉투) {
    super(봉투?.message ?? "요청에 실패했습니다");

    this.name = "API에러";
    this.status = status;
    this.code = 봉투?.code ?? "UNKNOWN";
    this.details = 봉투?.details;
  }
}
```

모든 요청이 여기를 지나갑니다.

```
async function 호출(경로, 옵션 = {}) {
  let 응답;

  try {
    응답 = await fetch(경로, 옵션);
  } catch (에러) {
```

★ fetch 가 던지는 경우는 '서버에 닿지도 못한' 경우입니다. 서버가 꺼졌거나, 인터넷이 끊겼거나, CORS 로 막혔거나. (08단원) 404·500 은 여기로 안 옵니다. 응답이 왔으니까요.

```
throw new API에러(0, { code: "NETWORK", message: "서버에 연결하지 못했습니다" });
}
```

204 는 본문이 없습니다. json() 을 부르면 터집니다.

```

if (응답.status === 204) return { data: null };

let 몸통;

try {
  몸통 = await 응답.json();
} catch {

```

JSON 이 아닌 응답. HTML 에러 페이지가 온 경우가 대부분입니다.

```

throw new API에러(응답.status, { code: "BAD_RESPONSE", message: "서버 응답을 읽지
못했습니다" });
}

```

★ res.ok 가 아니라 몸통.error 로 판단합니다. 06단원에서 "성공에는 error 가 없고 실패에는 data 가 없다" 고 정했기 때문입니다. 둘 다 봐도 되지만, 하나로 정하는 편이 헛갈리지 않습니다.

```

if (몸통.error) {
  throw new API에러(응답.status, 몸통.error);
}

return 몸통;
}

const api = {
  get: (경로) => 호출(경로),

  post: (경로, 값) =>
    호출(경로, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(값),
    }),

  patch: (경로, 값) =>
    호출(경로, {
      method: "PATCH",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(값),
    }),

  del: (경로) => 호출(경로, { method: "DELETE" }),

```

파일 업로드는 Content-Type 을 직접 쓰면 안 됩니다. (09단원 개념01)

```
올리기: (경로, 폼데이터) => 호출(경로, { method: "POST", body: 폼데이터 }),  
};
```

★ delete 는 자바스크립트 예약어라 속성 이름으로는 되지만 헛갈립니다. 그래서 del 로 지었습니다.
api.delete 로 써도 동작은 합니다.

단 원 11

종합 실습

OPEN 11_종합_실습

요청 번호와 기록 (05단원)

```
let 요청번호 = 0;

function 번호붙이기(req, res, next) {
  요청번호 += 1;
  req.요청번호 = 요청번호;
  res.set("X-Request-Id", String(요청번호)); // * 값은 반드시 영어·숫자 next();
}

module.exports = { 번호붙이기 };
```

11단원 · repositories/기본저장소.js

JSON 파일 저장소를 만들어 주는 함수

07단원에서 저장소 두 개가 거의 같은 코드였습니다.

세 번째가 생기기 전에 공통 부분을 묶었습니다.

★ 07단원 연습문제 ⑤의 답입니다. “같은 걸 세 번 쓰면 그때 묶는다”가 흔한 기준입니다. 여기서는 설비·점검기록·사진 셋이라 묶을 때가 됐습니다.

```
const fs = require("fs");
const path = require("path");

function 저장소만들기(파일이름) {
  const 파일경로 = path.join(__dirname, "..", "data", 파일이름);
  fs.mkdirSync(path.dirname(파일경로), { recursive: true });
```

쓰기를 줄 세웁니다. 동시에 쓰면 사라집니다. (07단원 개념04)

```
let 대기줄 = Promise.resolve();

function 줄서서(작업) {
  const 다음 = 대기줄.then(작업, 작업);
  대기줄 = 다음.catch(() => {});
  return 다음;
}

async function 읽기() {
  try {
    return JSON.parse(await fs.promises.readFile(파일경로, "utf8"));
  } catch (에러) {
    if (에러.code === "ENOENT") return [];
    throw 에러; // * 없는 것과 못 읽는 것은 다릅니다
  }
}
```

임시 파일에 쓰고 이름을 바꿉니다. 쓰다가 죽어도 안 깨집니다.

```

async function 쓰기(목록) {
  const 임시 = `${파일경로}.tmp`;
  await fs.promises.writeFile(임시, JSON.stringify(목록, null, 2), "utf8");
  await fs.promises.rename(임시, 파일경로);
}

return {
  전부: () => 읽기(),

  찾기: async (조건) => (await 읽기()).filter(조건),

  하나: async (id) => (await 읽기()).find((항목) => 항목.id === id) ?? null,

  추가: (값) =>
    줄서서(async () => {
      const 목록 = await 읽기();
      const 다음번호 = 목록.length === 0 ? 1 : Math.max(...목록.map((항목) =>
항목.id)) + 1;
      const 새것 = { id: 다음번호, ...값 };

      목록.push(새것);
      await 쓰기(목록);

      return 새것;
    })),

  수정: (id, 바꿀것) =>
    줄서서(async () => {
      const 목록 = await 읽기();
      const 자리 = 목록.findIndex((항목) => 항목.id === id);

      if (자리 === -1) return null; // * findIndex 는 -1 입니다

      목록[자리] = { ...목록[자리], ...바꿀것, id }; // id 는 못 바꾸게 await 쓰기
(목록);

      return 목록[자리];
    })
};

```



```

    }),

    삭제: (id) =>
      줄서서(async () => {
        const 목록 = await 읽기();
        const 남길것 = 목록.filter((항목) => 항목.id !== id);

        if (남길것.length === 목록.length) return false;

        await 쓰기(남길것);
        return true;
      }),

    초기화: (목록) => 줄서서(() => 쓰기(목록)),
  };
}

module.exports = { 저장소만들기 };

```

★ 이 파일 하나만 고치면 저장 방법이 통째로 바뀝니다. PART 4 에서 데이터베이스로 옮길 때, 여기만 다시 쓰면 됩니다. services·controllers·routes 는 한 줄도 안 고칩니다.

사진라우트

```
const express = require("express");

const 사진컨트롤러 = require("../controllers/사진컨트롤러");
const { 인증, 역할확인 } = require("../middlewares/인증");
const { 에러 } = require("../utils/AppError");

const router = express.Router();
```

★★ /orphans 를 /:id 보다 먼저 등록합니다. 아래에 두면 :id 에 "orphans" 가 들어갑니다. (04단원 개념02 섹션 7)

```
router.get("/orphans", 사진컨트롤러.고아확인);

router.param("id", (req, res, next, 값) => {
  const 번호 = Number(값);
  if (!Number.isInteger(번호)) {
    return next(에러.검증실패([ { 키: "id", 이유: "숫자여야 합니다" } ]));
  }
  req.번호 = 번호;
  next();
});

router.get("/:id/view", 사진컨트롤러.보기);
router.get("/:id/download", 사진컨트롤러.내려받기);
router.delete("/:id", 인증, 역할확인("admin"), 사진컨트롤러.삭제);

module.exports = router;
```

설비 사진의 업무 규칙

```
const path = require("path");
const fs = require("fs");

const 사진저장소 = require("../repositories/사진저장소");
const 설비저장소 = require("../repositories/설비저장소");
const { 에러 } = require("../utils/AppError");
const { 되돌리기 } = require("../utils/파일이름");

const 사진폴더 = path.join(__dirname, "..", "uploads");
fs.mkdirSync(사진폴더, { recursive: true });

const 설비당최대 = 3;
```

이미 없는 파일을 지우려 한 것은 실패가 아닙니다.

```
async function 파일지우기(저장이름) {
  try {
    await fs.promises.unlink(path.join(사진폴더, 저장이름));
  } catch (에러) {
    if (에러.code !== "ENOENT") throw 에러;
  }
}

async function 설비확인(equipmentId) {
  const 설비 = await 설비저장소.하나(equipmentId);
  if (설비 === null) throw 에러.없음("설비");
  return 설비;
}

async function 목록(equipmentId) {
  await 설비확인(equipmentId);
  return 사진저장소.찾기((사진) => 사진.equipmentId === equipmentId);
}

async function 하나(id) {
  const 사진 = await 사진저장소.하나(id);
  if (사진 === null) throw 에러.없음("사진");
  return 사진;
}
```

★★ 이 함수는 '이미 저장된 파일'을 받습니다. multer 가 라우트보다 먼저 돌기 때문입니다. 그래서 규칙에 걸리면 그 파일을 우리가 지워야 합니다. (09단원 개념05)

```
async function 등록(equipmentId, 파일) {
  const 지우고던지기 = async (던질에러) => {
    await 파일지우기(파일.filename);
    throw 던질에러;
  };

  const 설비 = await 설비저장소.하나(equipmentId);
  if (설비 === null) await 지우고던지기(에러.없음("설비"));

  const 지금 = await 사진저장소.찾기((사진) => 사진.equipmentId === equipmentId);
  if (지금.length >= 설비당최대) {
    await 지우고던지기(에러.상태충돌(`설비당 사진은 ${설비당최대}장까지입니다`));
  }

  return 사진저장소.추가({
    equipmentId,
    저장이름: 파일.filename,
    원래이름: 되돌리기(파일.originalname),
    크기: 파일.size,
  });
}

async function 삭제(id) {
  const 사진 = await 하나(id);
```

★ 기록을 먼저, 파일을 나중에. (09단원 개념05) 보이지 않는 쓰레기가, 보이는 고장보다 낫습니다.

```
await 사진저장소.삭제(id);
await 파일지우기(사진.저장이름);
}
```

청소용. 기록 없는 파일을 셉니다.

```

async function 고아세기() {
  const 기록이름 = new Set((await 사진저장소.전부()).map((사진) => 사진.저장이름));
  const 디스크 = fs.readdirSync(사진폴더).filter((이름) => !이름.endsWith(".tmp"));

  return {
    디스크파일수: 디스크.length,
    기록수: 기록이름.size,
    고아파일수: 디스크.filter((이름) => !기록이름.has(이름)).length,
  };
}

module.exports = { 목록, 하나, 등록, 삭제, 고아세기, 사진폴더, 설비당최대 };

```

사진컨트롤러

```
const path = require("path");
const 사진서비스 = require("../services/사진서비스");
const { 에러 } = require("../utils/AppError");

async function 목록(req, res) {
  const 사진들 = await 사진서비스.목록(req.번호);

  res.json({
    data: 사진들.map((사진) => ({
      id: 사진.id,
      원래이름: 사진.원래이름,
      크기: 사진.크기,
      보기: `/api/v1/photos/${사진.id}/view`,
      내려받기: `/api/v1/photos/${사진.id}/download`,
    })),
  });
}

async function 등록(req, res) {
  if (!req.file) throw 에러.검증실패([ { 키: "photo", 이유: "사진이 없습니다" } ]);

  const 사진 = await 사진서비스.등록(req.번호, req.file);

  res.status(201).json({ data: { id: 사진.id, 원래이름: 사진.원래이름, 크기: 사진.크기 } });
}

async function 보기(req, res) {
  const 사진 = await 사진서비스.하나(req.번호);
  res.sendFile(path.join(사진서비스.사진폴더, 사진.저장이름));
}

async function 내려받기(req, res) {
  const 사진 = await 사진서비스.하나(req.번호);
```

★ res.download 가 한글 이름을 알아서 인코딩해 줍니다. Content-Disposition 을 직접 쓰면 한글에서 500 이 납니다. (09단원)

```
res.download(path.join(사진서비스.사진폴더, 사진.저장이름), 사진.원래이름);
}
```

```
async function 삭제(req, res) {  
  await 사진서비스.삭제(req.번호);  
  res.sendStatus(204);  
}  
  
async function 고아확인(req, res) {  
  res.json({ data: await 사진서비스.고아세기() });  
}  
  
module.exports = { 목록, 등록, 보기, 내려받기, 삭제, 고아확인 };
```

주소 ↔ 함수, 그리고 누가 할 수 있는가

```
const express = require("express");

const 설비컨트롤러 = require("../controllers/설비컨트롤러");
const 점검기록컨트롤러 = require("../controllers/점검기록컨트롤러");
const 사진컨트롤러 = require("../controllers/사진컨트롤러");
const { 인증, 역할확인 } = require("../middlewares/인증");
const { 사진업로드 } = require("../middlewares/업로드");
const { 에러 } = require("../utils/AppError");

const router = express.Router();
```

:id 를 숫자로 바꿔 req.번호 에 답습니다. 이 라우터의 모든 :id 에 걸립니다.

```
router.param("id", (req, res, next, 값) => {
  const 번호 = Number(값);

  if (!Number.isInteger(번호)) {
    return next(에러.검증실패([ { 키: "id", 이유: "숫자여야 합니다" } ]));
  }

  req.번호 = 번호;
  next();
});
```

설비

```
router.get("/", 설비컨트롤러.목록);
router.get("/:id", 설비컨트롤러.하나);
router.post("/", 인증, 역할확인("admin"), 설비컨트롤러.등록);
router.patch("/:id", 인증, 설비컨트롤러.상태바꾸기);
router.delete("/:id", 인증, 역할확인("admin"), 설비컨트롤러.삭제);
```

그 설비의 점검기록 (중첩 자원)

```
router.get("/:id/logs", 점검기록컨트롤러.설비별목록);
router.post("/:id/logs", 인증, 점검기록컨트롤러.등록);
```


그 설비의 사진

```
router.get("/:id/photos", 사진컨트롤러.목록);  
router.post("/:id/photos", 인증, 사진업로드.single("photo"), 사진컨트롤러.등록);  
  
module.exports = router;
```

★ 이 파일만 보면 “누가 무엇을 할 수 있는가” 가 한눈에 보입니다. 조회는 누구나, 등록은 로그인, 설비 등록·삭제는 admin. 컨트롤러를 열어 볼 필요가 없습니다.

★ 업로드에서 미들웨어 순서 인증 → 사진업로드 순입니다. 반대로 두면 로그인도 안 한 사람의 파일을 먼저 받아 저장하게 됩니다.

설비의 업무 규칙

```
const 설비저장소 = require("../repositories/설비저장소");  
const { 에러 } = require("../utils/AppError");  
  
const 라인값들 = ["A", "B", "C"];  
const 상태값들 = ["가동", "정지", "점검중"];
```

목록 (검색·필터·정렬·페이징)

```

async function 목록(조건) {
  const q = (조건.q ?? "").trim().toLowerCase();
  let 결과 = await 설비저장소.전부();

  if (q) {
    결과 = 결과.filter((설비) =>
      ["name", "line", "status"].some((키) => String(설비
[키]).toLowerCase().includes(q))
    );
  }

  if (조건.line) {
    const 라인들 = 조건.line.split(",").map((조각) => 조각.trim());
    결과 = 결과.filter((설비) => 라인들.includes(설비.line));
  }

  if (조건.status) 결과 = 결과.filter((설비) => 설비.status === 조건.status);

  if (조건.sort) {
    const 조건들 = 조건.sort
      .split(",")
      .map((조각) => 조각.trim())
      .filter(Boolean)
      .map((조각) =>
        조각.startsWith("-") ? { 키: 조각.slice(1), 방향: -1 } : { 키: 조각, 방향: 1
      }
    );

    결과 = [...결과].sort((a, b) => {
      for (const { 키, 방향 } of 조건들) {
        const 왼쪽 = a[키];
        const 오른쪽 = b[키];

        if (typeof 왼쪽 === "string" && typeof 오른쪽 === "string") {
          const 결 = 왼쪽.localeCompare(오른쪽, "ko", { numeric: true });
          if (결 !== 0) return 결 * 방향;
          continue;
        }
      }
    });
  }
}

```

```

if (왼쪽 < 오른쪽) return -1 * 방향;
    if (왼쪽 > 오른쪽) return 1 * 방향;
    }
    return 0;
});
}

```

★ total 은 자르기 전에 셉니다. (06단원)

```

const total = 결과.length;

let page = Number(조건.page);
let limit = Number(조건.limit);
if (!Number.isInteger(page) || page < 1) page = 1;
if (!Number.isInteger(limit) || limit < 1) limit = 10;
if (limit > 100) limit = 100;

const 시작 = (page - 1) * limit;

return {
  data: 결과.slice(시작, 시작 + limit),
  meta: { page, limit, total, totalPages: Math.ceil(total / limit) },
};
}

async function 하나(id) {
  const 설비 = await 설비저장소.하나(id);
  if (설비 === null) throw 에러.없음("설비");
  return 설비;
}

```

규칙 ① 모양이 맞아야 한다

```

function 검증(값) {
  const 항목 = [];

  if (!값.name) 항목.push({ 키: "name", 이유: "필수입니다" });
  else if (값.name.length < 2) 항목.push({ 키: "name", 이유: "2글자 이상이어야 합니다" });
  else if (값.name.length > 20) 항목.push({ 키: "name", 이유: "20글자 이하여야 합니다" });

  if (!값.line) 항목.push({ 키: "line", 이유: "필수입니다" });
  else if (!라인값들.includes(값.line))
    항목.push({ 키: "line", 이유: `${라인값들.join(", ")} 중 하나여야 합니다` });
}

```

꺼내고, 부르고, 응답한다

```
const 설비서비스 = require("../services/설비서비스");

async function 목록(req, res) {
  const 결과 = await 설비서비스.목록(req.query);
  res.json(결과); // 서비스가 { data, meta } 를 그대로 줍니다
}

async function 하나(req, res) {
  res.json({ data: await 설비서비스.하나(req.번호) });
}

async function 등록(req, res) {
  const { name, line } = req.body || {};
  const 새설비 = await 설비서비스.등록({ name, line });

  res.status(201).location(`/api/v1/equipments/${새설비.id}`).json({ data: 새설비 });
}

async function 상태바꾸기(req, res) {
  const { status } = req.body || {};
  res.json({ data: await 설비서비스.상태바꾸기(req.번호, status) });
}

async function 삭제(req, res) {
  await 설비서비스.삭제(req.번호);
  res.sendStatus(204);
}

module.exports = { 목록, 하나, 등록, 상태바꾸기, 삭제 };
```

★ 검증도 404 도 여기 없습니다. 전부 서비스가 던집니다. 컨트롤러는 '잘된 경우' 만 적으면 됩니다.

multer 설정 (09단원)

```
const multer = require("multer");
const path = require("path");

const { 저장이름, 되돌리기 } = require("../utils/파일이름");
const { 사진폴더 } = require("../services/사진서비스");

const 허용확장자 = [".jpg", ".jpeg", ".png", ".gif"];

const 사진업로드 = multer({
  storage: multer.diskStorage({
    destination: (req, file, cb) => cb(null, 사진폴더),
```

★ 원래 이름을 그대로 쓰지 않습니다. ../ 같은 것이 들어올 수 있습니다.

```
    filename: (req, file, cb) => cb(null, 저장이름(file.originalname)),
  }),

  limits: { fileSize: 2000, files: 1 }, // 시험용. 실제로는 5 * 1024 *
  1024
  fileFilter: (req, file, cb) => {
```

★ 여기서도 이름을 되돌려야 합니다. 안 하면 한글 이름이 전부 거절됩니다.

```
const 확장자 = path.extname(되돌리기(file.originalname)).toLowerCase();

if (!허용확장자.includes(확장자)) {
  const 에러 = new Error(`${확장자} || "확장자 없는 파일" 은(는) 올릴 수
  없습니다`);
  에러.status = 400;
  에러.code = "BAD_EXTENSION";
  return cb(에러);
}

cb(null, true);
});

module.exports = { 사진업로드, 허용확장자 };
```

★ 필드 이름은 "photo" 입니다. 한글로 쓰면 Unexpected field 가 납니다. (09단원)

404 와 에러 처리기

```
const { AppError, 에러 } = require("../utils/AppError");

function 없는주소(req, res, next) {
  next(에러.주소없음());
}

function 에러처리기(err, req, res, next) {
```

★ 터미널에는 전부 남깁니다. 요청 번호와 함께요.

```
console.error(`[에러] #${req.요청번호} ${req.method} ${req.path} - ${err.name}:
${err.message}`);

if (err.type === "entity.parse.failed") {
  return res.status(400).json({ error: { code: "INVALID_JSON", message: "JSON
형식이 아닙니다" } });
}

if (err.name === "MulterError") {
  const 안내 = {
    LIMIT_FILE_SIZE: "파일이 너무 큼니다 (최대 2000바이트)",
    LIMIT_FILE_COUNT: "사진은 한 번에 한 장만 올릴 수 있습니다",
    LIMIT_UNEXPECTED_FILE: "필드 이름이 photo 가 맞나요? (한글로 쓰면 안 됩니다)",
  };

  return res.status(400).json({ error: { code: err.code, message: 안내[err.code] ??
err.message } });
}

if (err instanceof AppError) {
  const 몸통 = { code: err.code, message: err.message };
  if (err.details !== undefined) 몸통.details = err.details;
  return res.status(err.status).json({ error: 몸통 });
}
```

fileFilter 처럼 우리가 status 만 붙여 던진 에러

```
if (err.status) {  
    return res.status(err.status).json({ error: { code: err.code ?? "BAD_REQUEST",  
message: err.message } });  
}
```

★ 예상 못 한 에러는 자세한 내용을 밖으로 안 보냅니다.

```
    res.status(500).json({ error: { code: "INTERNAL_ERROR", message: "서버에서 문제가  
생겼습니다" } });  
}  
  
module.exports = { 없는주소, 에러처리기 };
```


누구인지 확인하고 권한을 본다 (05단원)

```
const { 에러 } = require("../utils/AppError");
```

진짜 서버라면 데이터베이스에서 찾습니다. PART 4 에서 바꿉니다.

```
const 키별사용자 = {
  "key-user-1": { name: "김민준", role: "user" },
  "key-admin-1": { name: "이서연", role: "admin" },
};

function 인증(req, res, next) {
  const 헤더값 = req.get("Authorization");
  if (!헤더값) return next(에러.로그인필요());

  const [방식, 키] = 헤더값.split(" ");
  if (방식 !== "Bearer" || !키) return next(에러.로그인필요());

  const 사용자 = 키별사용자[키];
  if (!사용자) return next(에러.로그인필요());

  req.user = 사용자;
  next();
}

function 역할확인(...허용역할들) {
  return (req, res, next) => {
    if (!req.user) return next(에러.로그인필요());
    if (!허용역할들.includes(req.user.role)) return next(에러.권한없음("이 작업을 할")));
    next();
  };
}

module.exports = { 인증, 역할확인, 키별사용자 };
```

점검기록라우트

```
const express = require("express");

const 점검기록컨트롤러 = require("../controllers/점검기록컨트롤러");
const { 인증, 역할확인 } = require("../middlewares/인증");
const { 에러 } = require("../utils/AppError");

const router = express.Router();

router.param("id", (req, res, next, 값) => {
  const 번호 = Number(값);
  if (!Number.isInteger(번호)) {
    return next(에러.검증실패([ { 키: "id", 이유: "숫자여야 합니다" } ]));
  }
  req.번호 = 번호;
  next();
});

router.get("/", 점검기록컨트롤러.목록);
router.get("/:id", 점검기록컨트롤러.하나);
router.delete("/:id", 인증, 역할확인("admin"), 점검기록컨트롤러.삭제);

module.exports = router;
```

★ PATCH 가 없습니다. 점검기록은 '일어난 사실' 이라 고치는 게 이상합니다. 잘못 적었으면 지우고 다시 남깁니다.

11단원 · services/점검기록서비스.js

점검기록의 업무 규칙

```
const 점검기록저장소 = require("../repositories/점검기록저장소");
const 설비저장소 = require("../repositories/설비저장소");
const { 에러 } = require("../utils/AppError");

const 결과값들 = ["정상", "이상"];
```

설비가 실제로 있는지 확인합니다. 없으면 404. ★ 이게 없으면 “없는 설비의 기록은 0건입니다” 라는 엉뚱한 답이 나옵니다.

```
async function 설비확인(equipmentId) {
  const 설비 = await 설비저장소.하나(equipmentId);
  if (설비 === null) throw 에러.없음("설비");
  return 설비;
}

async function 목록(equipmentId) {
  if (equipmentId === undefined) return 점검기록저장소.전부();

  await 설비확인(equipmentId);
  return 점검기록저장소.찾기((기록) => 기록.equipmentId === equipmentId);
}

async function 하나(id) {
  const 기록 = await 점검기록저장소.하나(id);
  if (기록 === null) throw 에러.없음("점검기록");
  return 기록;
}

async function 등록(값, 남긴사람) {
  const 설비 = await 설비확인(값.equipmentId);

  const 항목 = [];
  if (!결과값들.includes(값.result))
    항목.push({ 키: "result", 이유: `${결과값들.join(", ")} 중 하나여야 합니다` });
  if (!값.내용) 항목.push({ 키: "내용", 이유: "필수입니다" });
  else if (값.내용.length > 200) 항목.push({ 키: "내용", 이유: "200글자 이하여야 합니다" });

  if (항목.length > 0) throw 에러.검증실패(항목);

  const 새기록 = await 점검기록저장소.추가({
```

점검기록컨트롤러

```
const 점검기록서비스 = require("../services/점검기록서비스");

async function 목록(req, res) {
  const { equipmentId } = req.query;
```

★ Number(undefined) 는 NaN 입니다. "전부 달라" 가 "NaN번 설비 달라" 가 되면 안 됩니다.

```
const 기록들 = await 점검기록서비스.목록(
  equipmentId === undefined ? undefined : Number(equipmentId)
);

res.json({ data: 기록들 });
}

async function 설비별목록(req, res) {
  res.json({ data: await 점검기록서비스.목록(req.번호) });
}

async function 하나(req, res) {
  res.json({ data: await 점검기록서비스.하나(req.번호) });
}

async function 등록(req, res) {
  const { result, 내용 } = req.body || {};
```

★ 담당자는 본문에서 안 받습니다. 로그인한 사람으로 정합니다.

```
const 새기록 = await 점검기록서비스.등록(
  { equipmentId: req.번호, result, 내용 },
  req.user.name
);

res.status(201).json({ data: 새기록 });
}

async function 삭제(req, res) {
  await 점검기록서비스.삭제(req.번호);
  res.sendStatus(204);
}
```

```
module.exports = { 목록, 설비별목록, 하나, 등록, 삭제 };
```

업로드 파일 이름 다루기 (09단원)

```
const path = require("path");
```

multer 가 주는 이름은 latin1 로 읽혀 깨져 있습니다. 되돌립니다. ★ file.originalName 에만 씁니다.
req.body 값에 쓰면 오히려 망가집니다.

```
function 되돌리기(이름) {
  return Buffer.from(이름, "latin1").toString("utf8");
}
```

겹치지 않고 안전한 저장 이름을 만듭니다. 원래 이름은 쓰지 않습니다. ../ 같은 것이 들어올 수 있습니다.

```
function 저장이름(원본이름) {
  const 확장자 = path.extname(되돌리기(원본이름)).toLowerCase();
  return `${Date.now()}-${Math.round(Math.random() * 1e9)}${확장자}`;
}
```

```
module.exports = { 되돌리기, 저장이름 };
```

조사 붙이기 (06단원 연습문제)

```
function 받침있나(글자) {  
  const 코드 = 글자.charCodeAt(0);  
  if (코드 < 44032 || 코드 > 55203) return null;  
  return (코드 - 44032) % 28 !== 0;  
}  
  
function 조사(단어, 받침있을때, 받침없을때) {  
  const 있나 = 받침있나(단어[단어.length - 1]);  
  if (있나 === null) return `${단어}(${받침있을때}/${받침없을때})`;   
  return 단어 + (있나 ? 받침있을때 : 받침없을때);  
}  
  
module.exports = { 받침있나, 조사 };
```

서버와 이야기하는 부분을 한곳에 모은다

10단원의 api.js 에 '증표 붙이기' 를 더했습니다.

이 프로젝트는 로그인이 있습니다. (05단원의 Bearer 방식) 모든 요청에 Authorization 헤더를 붙여야 하는데, 화면마다 붙이면 한 군데를 빠뜨리고 그 요청만 401 이 납니다.

그래서 여기서 한 번만 붙입니다.

지금 고른 증표를 알려 주는 함수. 화면이 정해 줍니다.

```
let 증표가져오기 = () => null;

function 증표설정(함수) {
  증표가져오기 = 함수;
}

class API에러 extends Error {
  constructor(status, 봉투) {
    super(봉투?.message ?? "요청에 실패했습니다");

    this.name = "API에러";
    this.status = status;
    this.code = 봉투?.code ?? "UNKNOWN";
    this.details = 봉투?.details;
  }
}

async function 호출(경로, 옵션 = {}) {
  const 헤더 = { ...(옵션.headers ?? {}) };
  const 증표 = 증표가져오기();
```

★ 증표가 있을 때만 붙입니다. 빈 값으로 붙이면 "Bearer " 가 되어 형식 오류로 401 이 납니다.

```
if (증표) 헤더.Authorization = `Bearer ${증표}`;

let 응답;

try {
  응답 = await fetch(경로, { ...옵션, headers: 헤더 });
} catch {
```

서버에 닿지도 못한 경우입니다. 404·500 은 여기로 안 옵니다.


```
throw new API에러(0, { code: "NETWORK", message: "서버에 연결하지 못했습니다" });
}

if (응답.status === 204) return { data: null }; // 본문이 없습니다 let 몸통;

try {
  몸통 = await 응답.json();
} catch {
```

HTML 에러 페이지가 온 경우가 대부분입니다.

```
throw new API에러(응답.status, { code: "BAD_RESPONSE", message: "서버 응답을 읽지
못했습니다" });
}

if (몸통.error) throw new API에러(응답.status, 몸통.error);

return 몸통;
}

const api = {
  get: (경로) => 호출(경로),

  post: (경로, 값) =>
    호출(경로, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(값),
    }),

  patch: (경로, 값) =>
    호출(경로, {
      method: "PATCH",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(값),
    }),

  del: (경로) => 호출(경로, { method: "DELETE" }),
```

★ 파일 업로드에는 Content-Type 을 직접 쓰지 않습니다. (09단원 개념01) FormData 를 넣으면 브라우저가 boundary 와 함께 붙여 줍니다.

```
올리기: (경로, 폼데이터) => 호출(경로, { method: "POST", body: 폼데이터 }),

증표설정,
};
```

조립만 하는 곳

listen 은 여기 없습니다. server.js 가 합니다.

★ app 과 server 를 나누는 이유 나중에 테스트를 할 때 app 만 가져다 쓸 수 있습니다. 포트를 열지 않고도 요청을 흉내 낼 수 있습니다. 지금 당장 필요하진 않지만, 실무 프로젝트가 거의 다 이 모양입니다.

```
const express = require("express");
const cors = require("cors");
const morgan = require("morgan");
const path = require("path");

const { 번호붙이기 } = require("./middlewares/기록");
const { 없는주소, 에러처리 } = require("./middlewares/에러처리");

const 설비라우트 = require("./routes/설비라우트");
const 점검기록라우트 = require("./routes/점검기록라우트");
const 사진라우트 = require("./routes/사진라우트");

const app = express();
```

① 미들웨어 (라우트보다 위)

```
morgan.token("user", (req) => (req.user ? req.user.name : "-"));
morgan.token("reqid", (req) => (req.요청번호 ? `${req.요청번호}` : "-"));

app.use(번호붙이기); // morgan 보다 먼저여야 reqid 가 채워집니다
app.use(morgan(":reqid :method :url :status :response-time ms - :user"));

app.use(express.json());
```

프론트를 따로 띄울 때를 대비해 CORS 를 열어 둡니다. 지금은 같은 서버가 화면도 주니 필요 없지만, 업데이트를 만들어 둡니다.

```
const 허용출처들 = process.env.CORS_ORIGINS
  ? process.env.CORS_ORIGINS.split(",")
  : ["http://localhost:5173", "http://localhost:5500"];

app.use(
  "/api",
  cors({
```

상태코드와 코드가 붙은 에러

```
const { 조사 } = require("./한국어");

class AppError extends Error {
  constructor(status, code, message, details) {
    super(message);
    this.name = "AppError";
    this.status = status;
    this.code = code;
    this.details = details;
  }
}

const 에러 = {
  없음: (무엇) => new AppError(404, "NOT_FOUND", `${조사(무엇, "을", "를")}` 찾을 수 없습니다`),
  검증실패: (항목들) => new AppError(400, "VALIDATION_FAILED", "입력값이 올바르지 않습니다", 항목들),
  중복: (무엇) => new AppError(409, "DUPLICATED", `이미 있는 ${무엇}입니다`),
  상태충돌: (메시지) => new AppError(409, "CONFLICT", 메시지),
  로그인필요: () => new AppError(401, "UNAUTHENTICATED", "로그인이 필요합니다"),
  권한없음: (무엇) => new AppError(403, "FORBIDDEN", `${무엇} 권한이 없습니다`),
  주소없음: () => new AppError(404, "ROUTE_NOT_FOUND", "그런 주소가 없습니다"),
};

module.exports = { AppError, 에러 };
```

준비하고 문을 연다

RUN node server.js

브라우저에서 http://localhost:3000 을 여세요.

```
const fs = require("fs");
const path = require("path");

const app = require("./app");
const 설비저장소 = require("./repositories/설비저장소");
const 점검기록저장소 = require("./repositories/점검기록저장소");
const 사진저장소 = require("./repositories/사진저장소");
const { 사진폴더 } = require("./services/사진서비스");
```

★ Number 를 꼭 씌우세요. 환경변수는 글자입니다. (08단원 개념05)

```
const PORT = Number(process.env.PORT) || 3000;
```

수업용 초기 데이터입니다. 진짜 서버에는 이런 게 있으면 안 됩니다.

```
const 처음설비 = [
  { id: 1, name: "컨베이어 1호", line: "A", status: "가동" },
  { id: 2, name: "프레스 1호", line: "B", status: "정지" },
  { id: 3, name: "용접로봇 1호", line: "C", status: "가동" },
  { id: 4, name: "검사기 1호", line: "A", status: "점검중" },
  { id: 5, name: "포장기 1호", line: "C", status: "정지" },
];

const 처음기록 = [
  { id: 1, equipmentId: 1, result: "정상", 내용: "벨트 장력 확인", 담당자: "김민준" },
  { id: 2, equipmentId: 4, result: "이상", 내용: "센서 오작동", 담당자: "이서연" },
];

async function 시작하기() {
```

① 준비

```
fs.rmSync(사진폴더, { recursive: true, force: true });
fs.mkdirSync(사진폴더, { recursive: true });

await 설비저장소.초기화(처음설비);
await 점검기록저장소.초기화(처음기록);
await 사진저장소.초기화([]);
```

② 준비가 끝난 뒤에야 문을 엽니다

```
app.listen(PORT, () => {
  console.log(`설비 점검 관리 시스템이 켜졌습니다.`);
  console.log(`화면: http://localhost:${PORT}`);
  console.log(`API : http://localhost:${PORT}/api/v1/equipments`);
  console.log("");
  console.log("증표: Bearer key-user-1 (김민준, user)");
  console.log("      Bearer key-admin-1 (이서연, admin)");
});
}

시작하기().catch((에러) => {
  console.error("서버를 켜지 못했습니다:", 에러);
  process.exit(1);
});
```

검증 선언 — 요구사항이 정말 지켜지는지

```
node _검증도구/서버검증.js 11_종합_실습
node _검증도구/화면검증.js 11_종합_실습
```

★ 위에서부터 순서대로 실행됩니다. 순서가 곧 시나리오입니다.

조회 (로그인 없이 됩니다)

```
GET /api/v1/equipments?limit=3&sort=id

200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"}, {"id":2,"name":"프레스 1호","line":"B","status":"정지"}, {"id":3,"name":"용접로봇 1호","line":"C","status":"가동"}], "meta":{"page":1,"limit":3,"total":5,"totalPages":2}}
```

```
GET /api/v1/equipments?q=로봇&sort=id

200 {"data":[{"id":3,"name":"용접로봇 1호","line":"C","status":"가동"}], "meta":{"page":1,"limit":10,"total":1,"totalPages":1}}
```

GET /api/v1/equipments/1

200 {"data":{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"}}

GET /api/v1/equipments/99

404 {"error":{"code":"NOT_FOUND","message":"설비를 찾을 수 없습니다"}}

GET /api/v1/equipments/abc

400 {"error":{"code":"VALIDATION_FAILED","message":"입력값이 올바르지 않습니다","details":[{"키":"id","이유":"숫자여야 합니다"}]}}

중첩 자원: 설비의 점검기록

GET /api/v1/equipments/4/logs

200 {"data":[{"id":2,"equipmentId":4,"result":"이상","내용":"센서 오작동","담당자":"이서연"}]}

GET /api/v1/equipments/2/logs

200 {"data":[]}

GET /api/v1/equipments/99/logs

404 {"error":{"code":"NOT_FOUND","message":"설비를 찾을 수 없습니다"}}

★★ 위 두 줄이 06단원의 그 대비입니다. 설비는 있고 기록이 0건 → 200 과 빈 배열 설비 자체가 없음 → 404

인증과 권한

POST /api/v1/equipments

{"name":"적재로봇 1호","line":"A"}

401 {"error":{"code":"UNAUTHENTICATED","message":"로그인이 필요합니다"}}

POST /api/v1/equipments

Authorization: Bearer key-user-1

{"name":"적재로봇 1호","line":"A"}

403 {"error":{"code":"FORBIDDEN","message":"이 작업을 할 권한이 없습니다"}}

POST /api/v1/equipments

Authorization: Bearer key-admin-1

{ "name": "용", "line": "Z" }

400 {"error":{"code":"VALIDATION_FAILED","message":"입력값이 올바르지 않습니다","details":[{"키":"name","이유":"2글자 이상이어야 합니다"},{"키":"line","이유":"A, B, C 중 하나여야 합니다"}]}}

POST /api/v1/equipments

Authorization: Bearer key-admin-1

{ "name": "적재로봇 1호", "line": "A", "status": "가동" }

201 {"data":{"id":6,"name":"적재로봇 1호","line":"A","status":"정지"}}

★ status: "가동" 을 보냈는데 "정지" 로 만들어졌습니다. 규칙은 서버가 짭니다.

POST /api/v1/equipments

Authorization: Bearer key-admin-1

{ "name": "적재로봇 1호", "line": "B" }

409 {"error":{"code":"DUPLICATED","message":"이미 있는 설비 이름입니다"}}

점검기록을 남기면 설비 상태가 바뀝니다

POST /api/v1/equipments/1/logs

{ "result": "이상", "내용": "베어링 소음" }

401 {"error":{"code":"UNAUTHENTICATED","message":"로그인이 필요합니다"}}

POST /api/v1/equipments/1/logs

Authorization: Bearer key-user-1

{ "result": "이상함", "내용": "베어링 소음" }

400 {"error":{"code":"VALIDATION_FAILED","message":"입력값이 올바르지 않습니다","details":[{"키":"result","이유":"정상, 이상 중 하나여야 합니다"}]}}

GET /api/v1/equipments/1

200 {"data":{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"}}

POST /api/v1/equipments/1/logs

Authorization: Bearer key-user-1

{ "result": "이상", "내용": "베어링 소음" }

201 { "data": { "id": 3, "equipmentId": 1, "result": "이상", "내용": "베어링 소음", "담당자": "김민준" } }

GET /api/v1/equipments/1

200 { "data": { "id": 1, "name": "컨베이어 1호", "line": "A", "status": "점검중" } }

★★★ 기록을 남겼을 뿐인데 설비가 '가동' 에서 '점검중' 으로 바뀌었습니다. 이 규칙은 services/점검기록서비스.js 한 곳에만 있습니다. 담당자도 본문이 아니라 로그인한 사람(김민준)으로 정해졌습니다.

상태 전이 규칙

PATCH /api/v1/equipments/1

Authorization: Bearer key-user-1

{ "status": "가동" }

409 { "error": { "code": "CONFLICT", "message": "점검중인 설비는 정지를 거쳐야 가동할 수 있습니다" } }

PATCH /api/v1/equipments/1

Authorization: Bearer key-user-1

{ "status": "정지" }

200 { "data": { "id": 1, "name": "컨베이어 1호", "line": "A", "status": "정지" } }

DELETE /api/v1/equipments/3

Authorization: Bearer key-admin-1

409 { "error": { "code": "CONFLICT", "message": "가동 중인 설비는 삭제할 수 없습니다. 먼저 정지시키세요" } }

DELETE /api/v1/equipments/1

Authorization: Bearer key-user-1

403 { "error": { "code": "FORBIDDEN", "message": "이 작업을 할 권한이 없습니다" } }

DELETE /api/v1/equipments/1

Authorization: Bearer key-admin-1

204

사진 업로드

POST /api/v1/equipments/2/photos multipart:@photo=설비사진.png:50

401 {"error":{"code":"UNAUTHENTICATED","message":"로그인이 필요합니다"}}

POST /api/v1/equipments/2/photos

Authorization: Bearer key-user-1

| multipart:@photo=악성.exe:20

400 {"error":{"code":"BAD_EXTENSION","message":".exe 은(는) 올릴 수 없습니다"}}

POST /api/v1/equipments/2/photos

Authorization: Bearer key-user-1

| multipart:@photo=큰사진.png:5000

400 {"error":{"code":"LIMIT_FILE_SIZE","message":"파일이 너무 큼니다 (최대 2000바이트)"}}

POST /api/v1/equipments/99/photos

Authorization: Bearer key-user-1

| multipart:@photo=설비사진.png:50

404 {"error":{"code":"NOT_FOUND","message":"설비를 찾을 수 없습니다"}}

POST /api/v1/equipments/2/photos

Authorization: Bearer key-user-1

| multipart:@photo=설비사진.png:50

201 {"data":{"id":1,"원래이름":"설비사진.png","크기":50}}

GET /api/v1/equipments/2/photos

200 {"data":[{"id":1,"원래이름":"설비사진.png","크기":50,"보기":"/api/v1/photos/1/view","내려받기":"/api/v1/photos/1/download"}]}

GET /api/v1/photos/orphans

200 {"data":{"디스크파일수":1,"기록수":1,"고아파일수":0}}

★★ 에러가 네 번 났는데도 쓰레기 파일이 없습니다. 사진서비스가 규칙에 걸릴 때마다 이미 저장된 파일을 지웠기 때문입니다. (09단원)

GET /api/v1/photos/1/download

200

content-disposition=attachment; filename="????.png"; filename*=UTF-8''%EC%84%A4%EB%B9%84%EC%82%AC%EC%A7%84.png

DELETE /api/v1/photos/1

Authorization: Bearer key-user-1

403 {"error":{"code":"FORBIDDEN","message":"이 작업을 할 권한이 없습니다"}}

DELETE /api/v1/photos/1

Authorization: Bearer key-admin-1

204

GET /api/v1/photos/orphans

200 {"data":{"디스크파일수":0,"기록수":0,"고아파일수":0}}

화면과 없는 주소

GET /

200

GET /없는주소

404 {"error":{"code":"ROUTE_NOT_FOUND","message":"그런 주소가 없습니다"}}

화면 확인

화면 index.html

#요약 전체 5건 중 3건

#쪽정보 1 / 2

#본문 컨베이어 1호

화면 index.html

#다음쪽

#쪽정보 2 / 2

#본문 포장기 1호

2쪽을 보다가 검색하면 1쪽으로 돌아갑니다

화면 index.html

#다음쪽 → 입력(#검색=로봇) → #라인거르기

#쪽정보 1 / 1

#본문 용접로봇 1호

결과 없음

화면 index.html

입력(#검색=없는것) → #라인거르기

#요약 조건에 맞는 설비가 없습니다

빈 폼으로 등록

화면 index.html

#보내기

#오류-name 필수입니다

#오류-line 필수입니다

user 로 등록하면 403

화면 index.html

입력(#증표=key-user-1) → 입력(#name=신규설비 1호) → 입력(#line=A) → #보내기

#알림 권한이 없습니다

admin 으로 등록하면 됩니다

화면 index.html

입력(#name=신규설비 1호) → 입력(#line=A) → #보내기

#알림 등록했습니다

#요약 전체 6건

상세를 열면 점검기록과 사진이 함께 나옵니다

화면 index.html

.보기

#상세이름	컨베이어 1호
-------	---------

#기록목록	벨트 장력 확인
-------	----------

#사진목록	사진이 없습니다
-------	----------

★ 요청 세 개(설비·기록·사진)를 Promise.all 로 한꺼번에 보냈습니다. 차례로 보내면 세 배 느립니다. 그리고 '기록은 있고 사진은 없는' 상태가 각각 제대로 그려집니다.

가동 중인 설비는 삭제가 막힙니다

화면 index.html

.지우기

#알림	가동 중인 설비는 삭제할 수 없습니다
-----	----------------------

★ 07단원의 업무 규칙이 화면까지 그대로 전해졌습니다. 화면에는 그 규칙이 한 줄도 없습니다. 서버가 준 message 를 보여 줄 뿐입니다.

부 록 가

연습문제·종합연습 정답

먼저 스스로 풀어 본 다음에 보세요. 정답과 다르게 썼더라도 기대 출력이 같으면 맞은 것입니다.
다만 “왜 그렇게 되는지” 설명은 꼭 읽으세요.

07단원 연습문제 정답 데이터 저장하기

RUN node 연습문제_정답.js

07단원 연습문제 정답 (서버 없이 푸는 것)

★ 이 파일은 Express 를 한 줄도 안 씁니다. 그런데도 저장소와 서비스를 전부 시험할 수 있습니다. 층을 나눠 둔 덕입니다.

```
const 저장소 = require("./repositories/설비저장소");
const 서비스 = require("./services/설비서비스");
```

문제 3 (먼저 나옵니다. 아래 main 에서 씁니다)

```
function 다음번호_틀린방법(목록) {
  return 목록.length + 1;
}

function 다음번호_맞는방법(목록) {
  return 목록.length === 0 ? 1 : Math.max(...목록.map((x) => x.id)) + 1;
}
```

문제 5 줄 세우기

```
let 대기줄 = Promise.resolve();

function 줄서서(작업) {
  const 다음 = 대기줄.then(작업, 작업);
  대기줄 = 다음.catch(() => {});
  return 다음;
}

async function main() {
```

문제 1 저장소를 서버 없이 시험하기

```
await 저장소.초기화([
  { id: 1, name: "컨베이어 1호", line: "A", status: "가동" },
  { id: 2, name: "프레스 1호", line: "B", status: "정지" },
]);

console.log((await 저장소.전부()).length);
```

출력 2

코드	▶ 출력
console.log((await 저장소.하나(1)).name);	▶ 컨베이어 1호
console.log(await 저장소.하나(99));	▶ null
console.log(await 저장소.삭제(99));	▶ false
console.log(await 저장소.삭제(2));	▶ true

```
console.log((await 저장소.전부()).map((설비) => 설비.id));
```

출력 [1]

```
const 추가된것 = await 저장소.추가({ name: "용접로봇 1호", line: "C", status: "정지"
});
console.log(추가된것.id);
```

출력 2

★★ 2번이 다시 나왔습니다. 2번(프레스)을 지운 뒤라 지금 최댓값이 1 이고, 거기에 1을 더했기 때문입니다.

지운 번호가 재활용되면 무슨 일이 생길까요? 프레스 1호의 점검기록에 equipmentId: 2 가 남아 있다면, 그 기록이 갑자기 용접로봇의 기록으로 보입니다.

그래서 진짜 서버는 '지금까지 쓴 가장 큰 번호'를 따로 기억합니다. 데이터베이스는 이걸 알아서 해 줍니다. (AUTO_INCREMENT) 파일로 하려면 { 다음번호: 4, 목록: [...] } 처럼 함께 저장해야 합니다.

문제 2 서비스의 업무 규칙 시험하기

```
await 저장소.초기화([
  { id: 1, name: "컨베이어 1호", line: "A", status: "가동" },
  { id: 2, name: "프레스 1호", line: "B", status: "정지" },
  { id: 3, name: "검사기 1호", line: "A", status: "점검중" },
]);
```

규칙 ㉓ 가동 중인 설비는 못 지운다

```
try {
  await 서비스.삭제(1);
  console.log("지워짐 (이게 나오면 규칙이 안 지켜진 것)");
} catch (에러) {
  console.log(`${에러.status} ${에러.code}`);
}
```

출력 409 CONFLICT

```
}
```

정지 상태면 지워집니다

```
await 서비스.삭제(2);
console.log((await 저장소.전부()).map((설비) => 설비.id));
```

출력 [1, 3]

규칙 ㉔ 점검중 → 가동 은 바로 못 간다

```
try {
  await 서비스.상태바꾸기(3, "가동");
} catch (에러) {
  console.log(`${에러.status} ${에러.code}`);
}
```

출력 409 CONFLICT

```
}
```

정지를 거치면 됩니다

```
await 서비스.상태바꾸기(3, "정지");
const 가동됨 = await 서비스.상태바꾸기(3, "가동");
console.log(가동됨.status);
```

출력 가동

없는 것을 찾으려면

```
try {
  await 서비스.하나(99);
} catch (에러) {
  console.log(`${에러.status} ${에러.code} ${에러.message}`);
}
```

출력 404 NOT_FOUND 설비를 찾을 수 없습니다

```
}
```

규칙 ② 이름이 겹치면 안 된다

```
try {
  await 서비스.등록({ name: "컨베이어 1호", line: "B" });
} catch (에러) {
  console.log(`${에러.status} ${에러.code}`);
}
```

출력 409 DUPLICATED

```
}
```

규칙 ③ 새 설비는 무조건 '정지'

```
const 새것 = await 서비스.등록({ name: "포장기 1호", line: "C", status: "가동" });
console.log(새것.status);
```

출력 정지

★★ Express 를 한 줄도 안 켜고 업무 규칙 여섯 가지를 전부 확인했습니다. Postman 을 열 필요도, 포트를 정할 필요도 없었습니다.

서비스가 req-res 를 안 쓰기 때문에 가능합니다. 서비스에 res.status(409) 를 한 줄만 넣어도 이게 불가능해집니다.

이런 걸 자동으로 돌리는 것이 '테스트' 입니다. 층을 나누면 테스트를 짤 수 있고, 안 나누면 못 짭니다.

문제 3 다음 번호 붙이기

```
const 목록 = [{ id: 1 }, { id: 2 }, { id: 3 }];
const 지운뒤 = [{ id: 1 }, { id: 3 }];

console.log(다음번호_틀린방법(목록), 다음번호_맞는방법(목록));
```

출력 4 4

```
console.log(다음번호_틀린방법(지운뒤), 다음번호_맞는방법(지운뒤));
```

출력 3 4

```
console.log(다음번호_틀린방법([]), 다음번호_맞는방법([]));
```

출력 1 1

★ 두 번째 줄이 문제입니다. 2번을 지운 뒤 `length + 1` 은 3 입니다. 그런데 3번이 이미 있습니다. 번호가 겹쳐서 데이터가 통째로 어긋납니다.

★ 빈 배열을 따로 처리한 이유 `Math.max()` 는 인자가 없으면 `-Infinity` 입니다. `-Infinity + 1` 도 `-Infinity` 라 `id` 가 `-Infinity` 가 됩니다.

문제 4 무엇을 어디에 두나

```
const 어디에 = {
  "주소를 /api/v1/equipments 로 정한다": "routes",
  "req.params.id 를 숫자로 바꾼다": "controllers",
  "본문이 안 왔을 때 터지지 않게 || {} 를 붙인다": "controllers",
  "이름은 2글자 이상이어야 한다": "services",
  "가동 중인 설비는 못 지운다": "services",
  "같은 이름의 설비는 못 만든다": "services",
  "id 는 지금 최댓값 + 1 로 붙인다": "repositories",
  "파일이 없으면 빈 배열을 돌려준다": "repositories",
  "쓰기를 줄 세운다": "repositories",
  "404 를 어떤 JSON 모양으로 내보낼지 정한다": "에러 처리기",
};

for (const 일 of Object.keys(어디에)) {
  console.log(`${일} → ${어디에[일]}`);
}
```

출력 주소를 /api/v1/equipments 로 정한다 → routes
req.params.id 를 숫자로 바꾼다 → controllers
본문이 안 왔을 때 터지지 않게 || {} 를 붙인다 → controllers
이름은 2글자 이상이어야 한다 → services
가동 중인 설비는 못 지운다 → services
같은 이름의 설비는 못 만든다 → services
id 는 지금 최댓값 + 1 로 붙인다 → repositories
파일이 없으면 빈 배열을 돌려준다 → repositories
쓰기를 줄 세운다 → repositories
404 를 어떤 JSON 모양으로 내보낼지 정한다 → 에러 처리기

★ 헛갈리기 쉬운 것 두 개

“|| 는 검증 아닌가?” — 아닙니다. “본문이 아예 안 왔다” 는 HTTP 사정입니다. 업무 규칙이 아닙니다. 그래서 컨트롤러입니다.

“id 붙이기는 업무 규칙 아닌가?” — 아닙니다. 데이터베이스로 바꾸면 데이터베이스가 알아서 붙여 줍니다. 저장 방법에 딸린 일이라 저장소입니다.

★ 이 블록을 `main()` 밖으로 빼면 출력 순서가 통째로 앞으로 갑니다. `main()` 은 `async` 라서 첫 `await` 를 만나는 순간 아래로 넘겨주기 때문입니다. (02단원 개념03 의 실행 순서 이야기입니다)

문제 5 줄 세우기가 순서를 어떻게 바꾸나

```
const 순서 = [];

async function 일하기(이름, 걸리는시간) {
  순서.push(`${이름} 시작`);
  await new Promise((resolve) => setTimeout(resolve, 걸리는시간));
  순서.push(`${이름} 끝`);
}
```

그냥 동시에

```
await Promise.all([일하기("A", 30), 일하기("B", 10)]);
console.log(순서.join(" / "));
```

출력 A 시작 / B 시작 / B 끝 / A 끝

줄 세워서

```
순서.length = 0;
await Promise.all([줄서서(() => 일하기("A", 30)), 줄서서(() => 일하기("B", 10))]);
console.log(순서.join(" / "));
```

출력 A 시작 / A 끝 / B 시작 / B 끝

★ 위: A 와 B 가 겹쳐 있습니다. A 가 시작만 해 놓고 기다리는 사이에 B 가 끼어들었습니다. ‘읽고-고치고-쓰기’ 가 이렇게 겹치면 데이터가 사라집니다.

★ 아래: A 가 완전히 끝난 뒤에 B 가 시작했습니다. 겹칠 틈이 없습니다.

★ 대신 느립니다. 위는 30ms 만에 끝나고, 아래는 40ms 가 걸립니다. 사람이 늘수록 차이가 커집니다. 그게 줄 세우기의 대가입니다.

문제 7 findIndex 함정

```
const 설비들 = [{ id: 10 }, { id: 20 }];

const 첫자리 = 설비들.findIndex((설비) => 설비.id === 10);
const 없는것 = 설비들.findIndex((설비) => 설비.id === 99);

console.log(첫자리, 없는것);
```

출력 0 -1

```
console.log(`잘못된 확인 → ${!첫자리 ? "못 찾음" : "찾음"}`);
```

출력 잘못된 확인 → 못 찾음

```
console.log(`올바른 확인 → ${첫자리 === -1 ? "못 찾음" : "찾음"}`);
```

출력 올바른 확인 → 찾음

★★ 0번 자리를 '못 찾음' 으로 봤습니다. 0 은 falsy 라서 !0 이 true 이기 때문입니다.

목록의 '첫 번째' 항목만 수정이 안 되는 버그가 됩니다. 두 번째, 세 번째는 잘 되니까 원인을 짐작하기 어렵습니다.

findIndex 와 indexOf 는 반드시 === -1 로 확인하세요. find 는 undefined 를 주니 !설비 로 확인해도 됩니다. 둘이 다릅니다.

문제 9 저장 경로 만들기

```
const path = require("node:path");

function 저장경로(파일이름) {
  return path.join(__dirname, "data", 파일이름);
}
```

사람마다 앞부분이 다르니 뒤 두 칸만 잘라서 찍습니다.

```
const 조각 = 저장경로("설비.json").split(path.sep).slice(-2).join("/");
console.log(조각);
```

출력 data/설비.json

```
console.log(path.isAbsolute(저장경로("설비.json")));
```

출력 true

★ `__dirname` 을 기준으로 잡았기 때문에 언제나 절대 경로가 나옵니다. `"/data/설비.json"` 처럼 적으면 'node 를 어디서 쳤느냐' 에 따라 엉뚱한 곳을 보게 됩니다. 에러도 안 나고 빈 목록으로 시작해 버립니다.

문제 10 없는 파일과 진짜 고장 가려내기

```
function 읽기결과(에러) {
  if (에러 === null) return "정상";
  if (에러.code === "ENOENT") return "처음 실행 - 빈 목록으로 시작";
  return `진짜 고장 - ${에러.code}`;
}

console.log(읽기결과(null));
```

출력 정상

```
console.log(읽기결과({ code: "ENOENT" }));
```

출력 처음 실행 - 빈 목록으로 시작

```
console.log(읽기결과({ code: "EACCES" }));
```

출력 진짜 고장 - EACCES

```
console.log(읽기결과({ code: "EISDIR" }));
```

출력 진짜 고장 - EISDIR

★ `catch` 로 전부 `[]` 를 돌려주면 `EACCES`(권한 없음)도 '데이터가 없다' 가 됩니다. 파일은 멀쩡히 있는데 목록이 빈 채로 서비스가 돌고, 그 위에 새로 저장하면 원본을 덮어씁니다. `code` 로 반드시 가려내세요.

문제 11 읽고 → 고치고 → 쓰기 한 바퀴

```
function 한바퀴(원본, 고치기) {
  const 읽은것 = JSON.parse(원본);      // 읽기
  const 고친것 = 고치기(읽은것);        // 고치기
  return JSON.stringify(고친것);        // 쓰기
}

const 처음 = JSON.stringify([
  { id: 1, name: "컨베이어 1호", status: "가동" }
]);

const 바뀐것 = 한바퀴(처음, (목록) =>
```

```
);

console.log(JSON.parse(바뀐것)[0].status);
```

출력 정지

```
console.log(typeof 바뀐것);
```

출력 string

★ 마지막이 string 인 것이 핵심입니다. `JSON.stringify` 를 빼고 객체를 그대로 `writeFile` 에 넘기면 파일에 `[object Object]` 가 적힙니다. 에러는 안 납니다. 다음에 읽을 때 `JSON.parse` 가 터지고 나서야 알게 됩니다.

```
}

main();
```

문제 6 원자적 쓰기가 필요한 이유

`writeFile` 만 쓰면 무슨 일이 생기나: 파일을 먼저 비우고 새 내용을 씁니다. 비운 직후에 서버가 죽거나 전원이 나가면 반쯤 쓰인 파일이 남습니다. JSON 이 깨져서 아예 못 읽습니다. 데이터 전체가 날아갑니다.

또 하나 — 다른 요청이 그 순간에 읽으면: 반쯤 쓰인 글자를 읽습니다. `JSON.parse` 가 `SyntaxError` 를 냅니다. 이걸 ‘찢긴 읽기’ 라고 합니다. 가끔만 일어나서 재현이 아주 어렵습니다.

어떻게 고치나: ① 임시 파일에 전부 쓴다 `writeFile(파일 + ".tmp", ...)` ② 원래 이름으로 바꾼다 `rename(파일 + ".tmp", 파일)`

왜 이게 안전한가: 이름 바꾸기는 한순간에 일어납니다. 반쯤 바뀌는 일이 없습니다. 그래서 파일은 언제나 ‘옛날 것 전체’ 이거나 ‘새것 전체’ 입니다. 중간 상태가 존재하지 않습니다.

주의할 점: 임시 파일은 같은 디스크(같은 폴더 옆)에 만드세요. `rename` 은 같은 디스크 안에서만 되고, 다른 디스크로는 `EXDEV` 에러로 실패합니다. (`mv` 명령은 그때 복사 + 삭제로 대신해 줍니다. `rename` 이 그러는 게 아닙니다)

문제 8 파일 저장의 한계

줄 세우기로 해결되는 것: 한 프로세스 안에서 ‘읽고-고치고-쓰기’ 가 겹치는 문제

줄 세우기로도 해결이 안 되는 것: 서버를 두 대 띄우거나 PM2 로 프로세스를 여러 개 돌리면 각자 자기 대기줄만 알고 있어서 다시 덮어쓰기가 일어납니다.

왜 이게 치명적인가: 서버를 여러 대 띄우는 것은 트래픽이 늘면 반드시 하게 됩니다. 즉 “서비스가 잘되면 반드시 깨지는” 구조입니다.

그 밖에 파일로 못 하는 것: · 10만 건이면 하나 고치려고 10만 건을 전부 읽고 씁니다 · 조건 검색을 하려면 전부 훑어야 합니다 · 여러 작업을 “둘 다 되거나 둘 다 안 되거나” 로 묶을 수 없습니다 · 누가 언제 무엇을 바꿨는지 알 수 없습니다

데이터베이스가 해 주는 것: 여러 서버가 붙어도 데이터는 한 곳입니다. 색인으로 조건 검색을 빠르게 합니다. 트랜잭션으로 여러 작업을 묶어 줍니다. 필요한 줄만 잠그기 때문에, 우리 줄 세우기처럼 전부 기다리지 않습니다.

그래도 07단원이 헛되지 않은 이유: 층을 나누는 법, 저장소가 HTTP 를 모르게 하는 법, 왜 트랜잭션이 필요한지 — 전부 그대로 씁니다. 그리고 저장 방법을 바꿀 때 repositories 한 폴더만 고치면 됩니다.

문제 12 서버에서 Sync 를 쓰면

readFileSync 가 도는 동안 다른 요청은 어떻게 되나: 아무것도 못 합니다. 줄을 서서 기다립니다. Node 는 요청을 받는 일꾼이 한 명이라, 그 한 명이 파일을 읽느라 붙잡히면 그동안 도착한 요청은 큐에 쌓이기만 합니다. 접속이 끊기는 것도 아니고 에러가 나는 것도 아니라서 ‘그냥 느린 서버’ 로 보입니다.

요청이 100건 몰리면 마지막 사람은 얼마나 기다리나: 한 번 읽는 데 10ms 라면 100번째 사람은 약 1초를 기다립니다. 요청마다 두 번 읽으면 2초입니다. 비동기로 하면 읽기들이 겹쳐서 도니까 마지막 사람도 수십 ms 면 끝납니다. ★ 느려지는 정도가 ‘요청 수에 비례해서’ 늘어나는 것이 무서운 점입니다.

혼자 시험할 때는 절대 안 드러납니다.

그런데 서버가 처음 켜질 때 설정 파일을 Sync 로 읽는 것은 왜 괜찮나: 아직 listen 을 안 했기 때문입니다. 기다릴 사람이 없습니다. 오히려 설정을 다 읽은 뒤에 문을 여는 편이 안전합니다. 준비가 덜 된 채로 요청을 받으면 첫 몇 건이 이상하게 동작합니다(개념02 섹션5).

“우리 파일은 작아서 금방 읽힌다” 는 왜 위험한 생각인가: · 파일은 자랍니다. 오늘 10KB 인 것이 반년 뒤에 10MB 가 됩니다. · 디스크가 항상 빠른 것도 아닙니다. 네트워크 드라이브나 클라우드 디스크는

평소엔 빠르다가 가끔 수백 ms 씩 걸립니다.

· 무엇보다 ‘금방’ 은 요청 수를 곱하면 금방이 아닙니다. 비동기로 쓰는 데 드는 비용이 await 한 글자뿐이라, 아낄 이유가 없습니다.

07단원 연습문제 서버 정답 데이터 저장하기

RUN node 연습문제_서버_정답.js

감사무시: 대조 — 이 정답은 번호 블록이 아니라 ‘완성된 서버 한 벌’ 입니다.

07단원은 저장소·서비스·컨트롤러·라우트를 층으로 나누는 것이 문제라
정답도 층별 파일 구조 그대로 두었습니다.

설비에 이어 ‘점검기록’ 이라는 두 번째 자원을 붙였습니다. 아래 파일들을 함께 보세요. 전부 설비 것과 같은
모양입니다.

repositories/점검기록저장소.js services/점검기록서비스.js controllers/점검기록컨트롤러.js routes/
점검기록라우트.js

★ 이 문제의 목적 “구조를 나눠 두면 두 번째 자원부터는 쉽다” 를 손으로 겪는 것입니다. 설비 파일을
복사해서 이름만 바꾸고, 규칙만 다르게 넣으면 됩니다.

```
const express = require("express");

const 설비저장소 = require("./repositories/설비저장소");
const 점검기록저장소 = require("./repositories/점검기록저장소");
const 설비라우트 = require("./routes/설비라우트");
const 점검기록라우트 = require("./routes/점검기록라우트");
const { AppError, 에러 } = require("./utils/AppError");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());

app.use("/api/v1/equipments", 설비라우트);
app.use("/api/v1/logs", 점검기록라우트);
```

설비는 그대로 동작합니다

GET /api/v1/equipments

```
200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"},{"id":2,"name":"프레스 1호","line":"B","status":"정지"}]}
```


점검기록 목록

GET /api/v1/logs

```
200 {"data":[{"id":1,"equipmentId":1,"result":"정상","담당자":"김민준"},{"id":2,"equipmentId":2,"result":"이상","담당자":"이서연"}]}
```

GET /api/v1/logs?equipmentId=1

```
200 {"data":[{"id":1,"equipmentId":1,"result":"정상","담당자":"김민준"}]}
```

GET /api/v1/logs?equipmentId=99

```
404 {"error":{"code":"NOT_FOUND","message":"설비를 찾을 수 없습니다"}}
```

★★ 여기가 이 문제의 핵심입니다. 99번 설비는 없습니다. 그러니 “기록이 0건” 이 아니라 “그런 설비가 없다” 입니다. 점검기록서비스가 설비저장소를 함께 보기 때문에 이 구별이 가능합니다.

서비스끼리 서로를 볼 수 있습니다. 같은 층이니까요. 저장소가 서비스를 보는 것만 안 됩니다. 아래층은 위층을 모릅니다.

점검기록 하나

GET /api/v1/logs/1

```
200 {"data":{"id":1,"equipmentId":1,"result":"정상","담당자":"김민준"}}
```

GET /api/v1/logs/99

```
404 {"error":{"code":"NOT_FOUND","message":"점검기록을 찾을 수 없습니다"}}
```

★ “점검기록을” 입니다. 받침이 있으니 ‘을’ 이 붙었습니다. utils/한국어.js 의 조사 함수가 붙여 준 것입니다.

점검기록 등록

POST /api/v1/logs

```
{"equipmentId":99,"result":"정상","담당자":"박지훈"}
```

```
404 {"error":{"code":"NOT_FOUND","message":"설비를 찾을 수 없습니다"}}
```

POST /api/v1/logs

{ "equipmentId":1, "result":"이상함", "담당자":"박지훈" }

400 { "error": { "code": "VALIDATION_FAILED", "message": "입력값이 올바르지 않습니다", "details": [{ "키": "result", "이유": "정상, 이상 중 하나여야 합니다" }] } }

POST /api/v1/logs

{ "equipmentId":1, "result":"정상" }

400 { "error": { "code": "VALIDATION_FAILED", "message": "입력값이 올바르지 않습니다", "details": [{ "키": "담당자", "이유": "필수입니다" }] } }

POST /api/v1/logs

{ "result":"정상", "담당자":"박지훈" }

400 { "error": { "code": "VALIDATION_FAILED", "message": "입력값이 올바르지 않습니다", "details": [{ "키": "equipmentId", "이유": "숫자여야 합니다" }] } }

★ equipmentId 를 아예 빠뜨리면 404 가 아니라 400 입니다. Number(undefined) 는 NaN — 이대로 설비를 찾으면 “없는 설비”(404) 로 잘못 보고됩니다. 빠뜨린 입력(400)과 없는 설비(404)는 다른 문제입니다.

POST /api/v1/logs

{ "equipmentId":1, "result":"정상", "담당자":"박지훈" }

201 { "data": { "id":3, "equipmentId":1, "result":"정상", "담당자":"박지훈" } }

★ 없는 설비에 기록을 남기려 하면 404 입니다. 저장소는 이걸 못 막습니다. equipmentId 가 숫자인지만 알 뿐입니다. “그 설비가 실제로 있어야 한다” 는 업무 규칙이라 서비스가 막습니다.

점검 결과가 ‘이상’ 이면 설비가 점검중이 됩니다

GET /api/v1/equipments/1

200 { "data": { "id":1, "name":"컨베이어 1호", "line":"A", "status":"가동" } }

POST /api/v1/logs

{ "equipmentId":1, "result":"이상", "담당자":"박지훈" }

201 { "data": { "id":4, "equipmentId":1, "result":"이상", "담당자":"박지훈" } }

GET /api/v1/equipments/1

200 { "data": { "id":1, "name":"컨베이어 1호", "line":"A", "status":"점검중" } }

★★★ 기록 하나를 남겼는데 설비의 상태가 바뀌었습니다.

이런 걸 '업무 규칙' 이라고 합니다. 화면에서 두 번 클릭하게 만드는 게 아니라, 서버가 알아서 함께 처리합니다.

이 규칙은 점검기록서비스 안에 있습니다. 컨트롤러에도, 저장소에도, 프론트엔드에도 없습니다.

프론트가 이걸 하면 어떻게 될까요? 관리자 도구에서 기록을 남기면 설비 상태가 안 바뀝니다. 같은 회사 데이터인데 화면마다 규칙이 달라집니다.

점검기록 삭제

DELETE	/api/v1/logs/4
204	
DELETE	/api/v1/logs/4
404	{"error":{"code":"NOT_FOUND","message":"점검기록을 찾을 수 없습니다"}}

★ 기록을 지워도 설비 상태는 안 되돌립니다. "점검 결과를 실수로 잘못 입력해서 지웠다" 와 "설비가 정상으로 돌아왔다" 는 다른 일이기 때문입니다. 되돌리려면 PATCH /api/v1/equipments/1 로 따로 바꿔야 합니다.

이런 판단도 업무 규칙입니다. 정답은 회사마다 다릅니다. 중요한 건 '정하고 한 곳에 적어 두는 것' 입니다.

404 와 에러 처리기

app.use((req, res, next) => { next(에러.주소없음()); });	
GET	/api/v1/없는주소
404	{"error":{"code":"ROUTE_NOT_FOUND","message":"그런 주소가 없습니다"}}
app.use((err, req, res, next) => { console.error(`[에러] \${req.method} \${req.path} - \${err.name}: \${err.message}`); if (err.type === "entity.parse.failed") { return res.status(400).json({ error: { code: "INVALID_JSON", message: "JSON 형식이 아닙니다" } }); } if (err instanceof AppError) { const 몸통 = { code: err.code, message: err.message }; } });	

08단원 연습문제 정답 정적파일과 CORS

RUN node 연습문제_정답.js

08단원 연습문제 정답 (서버 없이 푸는 것)

문제 1

```
function 출처만(주소) {
  const 분석 = new URL(주소);
  return `${분석.protocol}//${분석.host}`;
}

function 같은출처인가(a, b) {
  try {
    return 출처만(a) === 출처만(b);
  } catch {
    return false; // 주소 모양이 아니면 비교할 수 없습니다
  }
}

console.log(같은출처인가("http://localhost:3000/a", "http://localhost:3000/b?q=1"));
```

출력 true

```
console.log(같은출처인가("http://localhost:3000", "http://localhost:5500"));
```

출력 false

```
console.log(같은출처인가("http://localhost:3000", "https://localhost:3000"));
```

출력 false

```
console.log(같은출처인가("http://localhost:3000", "http://127.0.0.1:3000"));
```

출력 false

```
console.log(같은출처인가("https://a.com", "https://a.com:443"));
```

출력 true

```
console.log(같은출처인가("http://a.com", "http://a.com:80"));
```

```
출력      true
```

```
console.log(같은출처인가("http://a.com", "http://a.com:8080"));
```

```
출력      false
```

★ 다섯 번째와 여섯 번째가 재미있습니다. https 의 기본 포트는 443, http 는 80 입니다. 기본 포트는 주소에 안 씁니다. URL 이 알아서 떼어 줍니다.

```
new URL("https://a.com:443").host → "a.com"
```

그래서 같은 출처입니다.

★ 네 번째 — localhost 와 127.0.0.1 같은 컴퓨터를 가리키지만 글자가 다릅니다. 다른 출처입니다. “분명 같은 곳인데 왜 막히지?” 의 흔한 원인입니다.

★ protocol 에는 이미 콜론이 붙어 있습니다

```
new URL("http://a.com").protocol → "http:"  
그래서 `${분석.protocol}//` 라고 씁니다. 콜론을 또 붙이면 안 됩니다.
```

문제 2

```
const 단순메서드들 = ["GET", "HEAD", "POST"];  
const 단순형식들 = [  
  "text/plain",  
  "multipart/form-data",  
  "application/x-www-form-urlencoded",  
];  
const 그냥붙는헤더들 = ["accept", "accept-language", "content-language", "content-type"];  
  
function 프리플라이트필요한가(요청) {  
  if (!단순메서드들.includes(요청.method)) {  
    return true; // PATCH · DELETE · PUT  
  }  
  
  const 헤더들 = 요청.headers ?? {};  
  
  for (const 이름 of Object.keys(헤더들)) {  
    if (!그냥붙는헤더들.includes(이름.toLowerCase())) {  
      return true; // Authorization, X-무엇이든
```

```

    }
  }

  const 형식 = 헤더들["Content-Type"] ?? 헤더들["content-type"];

  if (형식 && !단순형식들.includes(형식.split(";")[0].trim())) {
    return true; // application/json
  }

  return false;
}

console.log(프리플라이트필요한가({ method: "GET" }));

```

출력 false

```

console.log(프리플라이트필요한가({ method: "POST", headers: { "Content-Type":
"application/x-www-form-urlencoded" } }));

```

출력 false

```

console.log(프리플라이트필요한가({ method: "POST", headers: { "Content-Type":
"application/json" } }));

```

출력 true

```

console.log(프리플라이트필요한가({ method: "DELETE" }));

```

출력 true

```

console.log(프리플라이트필요한가({ method: "GET", headers: { Authorization: "Bearer
key-user-1" } }));

```

출력 true

```

console.log(프리플라이트필요한가({ method: "POST", headers: { "Content-Type":
"text/plain; charset=utf-8" } }));

```

출력 false

★ 세 번째가 우리 API 의 거의 전부입니다. JSON 을 보내는 순간 프리플라이트가 생깁니다. 그래서 “GET 은 되는데 POST 만 안 돼요” 가 나오면 OPTIONS 를 의심합니다.

★ 두 번째와 여섯 번째 — 이걸 왜 단순 요청인가 HTML 폼(<form>)으로도 보낼 수 있는 요청이기 때문입니다. 폼은 CORS 가 생기기 전부터 있었고, 지금도 다른 출처로 보낼 수 있습니다. 그래서 “폼으로도 되는 것” 은 미리 물어볼 필요가 없습니다.

★ 여섯 번째의 charset=utf-8

`split(";")[0]` 로 앞부분만 잘라서 비교했습니다.

실제 헤더에는 charset 이 붙어 오는 경우가 많습니다. (04단원 연습문제 10)

문제 3

```
const 허용목록 = ["http://localhost:5500", "http://localhost:5173"];

function 허용인가(출처) {
```

Postman·curl·같은 출처 요청에는 Origin 이 없습니다. 보통 허용합니다.

```
if (출처 === undefined) {
    return true;
}

if (허용목록.includes(출처)) {
    return true;
}

try {
    const 호스트 = new URL(출처).hostname;
    return 호스트 === "example.com" || 호스트.endsWith(".example.com");
} catch {
    return false; // 주소 모양이 아니면 거절
}

console.log(허용인가(undefined));
```

출력 true

코드	▶ 출력
<code>console.log(허용인가("http://localhost:5500"));</code>	▶ true
<code>console.log(허용인가("https://example.com"));</code>	▶ true
<code>console.log(허용인가("https://sub.example.com"));</code>	▶ true

```
console.log(허용인가("https://example.com.evil.net"));
```

출력 false

코드	▶ 출력
<code>console.log(허용인가("https://evilexample.com"));</code>	▶ false
<code>console.log(허용인가("not-a-url"));</code>	▶ false

★★ 다섯 번째가 이 문제의 핵심입니다.

이렇게 썼다면 통과해 버립니다.

```
출처.includes("example.com")
```

공격자가 example.com.evil.net 이라는 도메인을 사면 끝입니다. 실제로 있는 공격 방법입니다.

★ 여섯 번째 — evilexample.com endsWith("example.com") 으로 썼다면 통과합니다. 앞에 점을 붙여 ".example.com" 으로 써야 막힙니다.

★ 왜 hostname 을 꺼내서 비교했나 글자 전체를 놓고 비교하면 이런 것들에 뚫립니다.

```
"https://evil.net/?x=https://example.com"
```

URL 로 분석해서 hostname 만 보면 안전합니다. "글자 비교보다 구조로 비교하라" 가 요령입니다.

문제 4

```
const 에러별처방 = {
  "No 'Access-Control-Allow-Origin' header is present": "Access-Control-Allow-Origin",
  "Response to preflight request doesn't pass access control check": "OPTIONS 응답에도 Access-Control-Allow-Origin",
  "has a value 'http://다른곳' that is not equal to the supplied origin": "허용 목록에 내 출처 추가",
  "Request header field authorization is not allowed": "Access-Control-Allow-Headers",
  "Method DELETE is not allowed": "Access-Control-Allow-Methods",
  "200 인데 headers.get 이 null": "Access-Control-Expose-Headers",
  "must not be the wildcard '*' when the request's credentials mode is 'include'": "Allow-Origin 을 정확한 출처로",
};

for (const 메시지 of Object.keys(에러별처방)) {
  console.log(`${메시지} → ${에러별처방[메시지]}`);
}
```


출력

```
No 'Access-Control-Allow-Origin' header is present → Access-Control-Allow-Origin
Response to preflight request doesn't pass access control check → OPTIONS
응답에도 Access-Control-Allow-Origin
has a value 'http://다른곳' that is not equal to the supplied origin →
허용 목록에 내 출처 추가
Request header field authorization is not allowed → Access-Control-Allow-Headers
Method DELETE is not allowed → Access-Control-Allow-Methods
200 인데 headers.get()이 null → Access-Control-Expose-Headers
must not be the wildcard '*' when the request's credentials mode is
'include' → Allow-Origin 을 정확한 출처로
```

★ 여섯 번째만 에러가 안 납니다. 그래서 제일 찾기 어렵습니다. “목록은 나오는데 전체 건수만 0 이예요” 라는 증상으로 나타납니다.

★ 두 번째에 preflight 이라는 단어가 있으면 OPTIONS 문제입니다. 같은 “No ‘Access-Control-Allow-Origin’” 인데 앞에 붙은 말이 다릅니다. 그 차이가 어디를 고칠지를 알려 줍니다.

문제 5

```
const 상황별헤더 = {
  "공개 자료를 아무나 읽게": "Access-Control-Allow-Origin: *",
  "우리 프론트만 읽게": "Access-Control-Allow-Origin: (그 출처) + Vary: Origin",
  "JSON 을 POST 로 받게": "OPTIONS 에 Allow-Methods, Allow-Headers",
  "Authorization 헤더를 받게": "Access-Control-Allow-Headers: Authorization",
  "X-Total-Count 를 읽게": "Access-Control-Expose-Headers: X-Total-Count",
  "OPTIONS 를 10분간 기억하게": "Access-Control-Max-Age: 600",
  "쿠키를 함께 받게": "Access-Control-Allow-Credentials: true (* 금지)",
};

for (const 상황 of Object.keys(상황별헤더)) {
  console.log(`${상황} → ${상황별헤더[상황]}`);
}
```

출력

```
공개 자료를 아무나 읽게 → Access-Control-Allow-Origin: *
우리 프론트만 읽게 → Access-Control-Allow-Origin: (그 출처) + Vary: Origin
JSON 을 POST 로 받게 → OPTIONS 에 Allow-Methods, Allow-Headers
Authorization 헤더를 받게 → Access-Control-Allow-Headers: Authorization
X-Total-Count 를 읽게 → Access-Control-Expose-Headers: X-Total-Count
OPTIONS 를 10분간 기억하게 → Access-Control-Max-Age: 600
쿠키를 함께 받게 → Access-Control-Allow-Credentials: true (* 금지)
```

★ 두 번째의 Vary: Origin 출처마다 응답이 달라지므로, 중간 캐시가 섞여 쓰면 안 된다고 알리는 것입니다. 개발 중에는 티가 안 나고 실제 서비스에서 터집니다. cors 패키지는 알아서 붙여 줍니다.

문제 6

CORS 가 보안이 아닌 이유

막는 것은 누구인가: 브라우저입니다. 서버가 아닙니다. 서버는 200 과 데이터를 그대로 보냅니다. Allow-Origin 헤더만 안 붙일 뿐입니다. 브라우저가 그 헤더를 보고 "스크립트에게 안 넘겨준다" 고 판단합니다.

그래서 무엇이 뚫리나: Postman, curl, 다른 서버에서 부르는 요청은 전부 통과합니다. CORS 헤더를 아예 안 붙입니다. 즉 데이터를 가져가려는 사람은 브라우저를 안 쓰면 그만입니다.

그럼 CORS 는 무엇을 막나: "다른 사이트가 사용자의 로그인 상태를 이용해서 우리 API 를 부르는 것" 입니다. 사용자가 우리 서비스에 로그인해 둔 상태로 나쁜 사이트에 들어갔을 때, 그 사이트의 스크립트가 우리 데이터를 읽어 가는 것을 막습니다.

진짜 보안은 무엇인가: 인증입니다. (05단원) 증표가 없으면 데이터를 아예 안 주는 것. Postman 으로 불러도 401 이 나와야 진짜 막힌 것입니다.

한 줄로: CORS 는 '막는 장치' 가 아니라 '허락하는 방법' 입니다.

문제 7

프록시가 CORS 를 없애는 이유

프록시가 없을 때: 브라우저 → API 서버 (출처가 다름 → CORS 검사)

프록시가 있을 때: 브라우저 → 프론트 서버 (같은 출처 → 검사 없음) 프론트 서버 → API 서버 (서버끼리 → 브라우저가 아님 → 검사 없음)

핵심: 브라우저는 같은 출처와만 이야기합니다. CORS 를 따질 일이 없습니다. API 서버에 요청을 보내는 것은 Node 이지 브라우저가 아닙니다. Node 는 CORS 를 안 따집니다.

그래서 API 서버는: CORS 를 한 줄도 안 열어도 됩니다.

주의할 점: 프론트에서 주소를 "/api/..." 로만 써야 합니다. "http://localhost:3000/api/..." 라고 전체 주소를 쓰면 프록시를 건너뛰니다.

배포하면: 개발 서버의 프록시는 사라집니다. Nginx 로 같은 구조를 만들거나 CORS 를 엽니다.

"개발에서는 됐는데 배포하니 CORS 에러" 의 원인이 이것입니다.

문제 8

Expose-Headers 함정

무슨 일이 일어나나: 요청은 성공합니다. 상태코드 200 입니다. 에러도 안 납니다. 그런데 `response.headers.get("X-Total-Count")` 가 `null` 입니다.

왜 그런가: 브라우저가 기본으로 읽게 해 주는 응답 헤더는 일곱 개뿐입니다.

`Content-Type`, `Content-Length`, `Content-Language`,
`Cache-Control`, `Expires`, `Last-Modified`, `Pragma`

나머지는 서버가 `Expose-Headers` 로 "이건 읽어도 된다" 고 해야 보입니다.

왜 특히 위험한가: 에러가 안 나기 때문입니다. 콘솔에도 아무것도 안 나옵니다. "목록은 잘 나오는데 전체 건수만 0 이에요" 라는 증상으로 나타납니다. CORS 를 원인으로 의심하는 사람이 거의 없습니다.

어떻게 고치나: `Access-Control-Expose-Headers: X-Total-Count` cors 패키지라면
`exposedHeaders: ["X-Total-Count"]`

06단원과의 연결: 06단원에서 "전체 개수를 헤더로 주는 방법도 있다" 고 하면서 "CORS 를 쓰면 헤더를 따로 열어 줘야 한다" 고 적었습니다. 이 이야기입니다. 그래서 그 단원에서는 본문의 meta 에 담는 쪽을 골랐습니다.

문제 9

```
const path = require("path");

const 공개폴더 = path.join(__dirname, "public");

function 파일경로(주소) {
  const 끝난것 = 주소.endsWith("/") ? `${주소}index.html` : 주소;
  return path.join(공개폴더, 끝난것);
}
```

사람마다 절대 경로가 다르니 기준 폴더에서 본 상대 경로로 찍습니다.

```
function 보기(주소) {
  return path.relative(공개폴더, 파일경로(주소)).split(path.sep).join("/");
}

console.log(보기("/"));
```

출력 `index.html`

코드	출력
<code>console.log(보기("/index.html"));</code>	▶ index.html
<code>console.log(보기("/css/공통.css"));</code>	▶ css/공통.css
<code>console.log(보기("/img/"));</code>	▶ img/index.html

해설 ① path.join 은 구분자를 알아서 붙이고 ../ 도 정리해 줍니다.

직접 문자열을 이어 붙이면 // 나 \ 가 섞여서 윈도우에서 깨집니다.

해설 ② __dirname 을 기준으로 잡는 것이 핵심입니다.

안 쓰면 'node 를 어디서 쳤느냐' 에 따라 public 을 못 찾습니다.

해설 ③ express.static 도 속으로 이 일을 합니다. 다만 없는 파일이면

에러를 내지 않고 다음 미들웨어로 넘깁니다(개념01 섹션3).

문제 10

```
function 안전한가(주소) {
  const 경로 = 파일경로(주소);
```

★ 구분자까지 붙여서 봅시다. 이게 없으면 public2 같은 이름이 통과합니다.

```
  return 경로.startsWith(공개폴더 + path.sep);
}
```

```
console.log(안전한가("/index.html"));
```

출력 true

코드	출력
<code>console.log(안전한가("/css/../index.html"));</code>	▶ true
<code>console.log(안전한가("/../비밀.txt"));</code>	▶ false
<code>console.log(안전한가("/../etc/passwd"));</code>	▶ false

해설 ① path.join 이 ../ 를 미리 정리하기 때문에

`"/css/../index.html"` 은 `public/index.html` 이 되어 통과합니다.
정리 전 글자로 판단하면 이 정상적인 주소까지 막게 됩니다.

해설 ② 반대로 “../비밀.txt” 는 public 밖으로 나가므로 걸립니다. 해설 ③ path.sep 를 안 붙이면 C:\...\public2\비밀.txt 도 startsWith 를 통과합니다.

09단원 개념04에서 업로드 파일을 내줄 때 같은 검사를 씁니다.

해설 ④ express.static 은 이 검사를 이미 하고 있습니다(개념01 섹션5).

직접 sendFile 로 만들 때만 내가 해야 합니다.

문제 11

```
function 먼저걸리는것(등록들, 주소) {
  const 걸린것 = 등록들.find((하나) => 주소.startsWith(하나.접두));
  return 걸린것 ? 걸린것.이름 : "없음";
}

const 등록들 = [
  { 이름: "정적파일", 접두: "/" },
  { 이름: "API", 접두: "/api" },
];

console.log(먼저걸리는것(등록들, "/api/v1/equipments"));
```

출력 정적파일

```
console.log(먼저걸리는것(등록들.slice().reverse(), "/api/v1/equipments"));
```

출력 API

해설 ① 정적 파일을 위에 두면 “/” 로 시작하는 모든 주소가 거기 먼저 걸립니다.

public 안에 api 폴더가 없으면 파일을 못 찾아 다음으로 넘어가므로 평소에는 아무 문제가 없어 보입니다.

해설 ② 그런데 public 안에 api 라는 폴더가 생기는 순간

API 요청이 그 폴더의 파일로 응답됩니다. 에러는 안 납니다.
“어제까지 되던 API 가 갑자기 HTML 을 돌려준다” 가 이 사고의 증상입니다.

해설 ③ 그래서 API 라우트를 정적 파일보다 위에 둡니다.

아니면 정적 파일에 접두를 줍니다 – app.use("/static", express.static(...)).

문제 12

고른 것: ② .env, ④ 회원 명단 엑셀, ⑥ 백업.sql ③ 관리자 화면 html 은 조건부입니다(아래).

왜 안 되나: public 안의 파일은 주소만 알면 누구나 받아 갑니다. 서버가 아무것도 안 물어봅니다.

③ 관리자 화면은 왜 애매한가: 화면 파일 자체는 껌데기라 공개돼도 큰일은 아닙니다. 진짜 방어는 그 화면이 부르는 API 에 인증을 거는 것입니다(05단원). 화면을 숨겨서 지키려는 것을 '숨김으로 지키기' 라고 하는데, 지키는 것이 아닙니다. 다만 관리자용 주소 구조가 통째로 드러나므로 공개 폴더 밖에 두는 편이 낫습니다.

“주소를 아무도 모르니 괜찮다” 가 왜 안 통하나: · 브라우저 주소창 자동완성·확장 프로그램·회사 프록시에 기록이 남습니다. · 화면 코드에 그 주소가 적혀 있으면 F12 로 그냥 보입니다. · 검색 로봇이 링크를 타고 들어와 색인해 버리는 일이 실제로 있습니다. 막고 싶으면 '못 찾게' 가 아니라 '물어보게' 만들어야 합니다. 그게 인증입니다.

08단원 연습문제 서버 정답 정적파일과 CORS

RUN node 연습문제_서버_정답.js

```
const express = require("express");
const cors = require("cors");
const path = require("path");

const app = express();
const PORT = Number(process.env.PORT) || 3000;

app.use(express.json());
```

문제 1 출처 판단 함수

```
const 허용목록 = ["http://localhost:5500", "http://localhost:5173"];

function 허용인가(출처) {
```

Postman·curl·같은 출처 요청에는 Origin 이 없습니다.

```
if (출처 === undefined) return true;

if (허용목록.includes(출처)) return true;

try {
```

★ 글자로 비교하지 않고 hostname 을 꺼내서 비교합니다. includes("example.com") 로 하면 example.com.evil.net 이 통과합니다.

```
const 호스트 = new URL(출처).hostname;
return 호스트 === "example.com" || 호스트.endsWith(".example.com");
} catch {
return false;
}
}

function 출처판단(출처, 알려주기) {
```

★ 거절할 때 에러를 던지지 마세요. 500 이 됩니다.

```
    알려주기(null, 허용인가(출처));
  }
}
```

문제 2 공개 API 는 아무나

```
app.use("/public-api", cors());

app.get("/public-api/notice", (req, res) => {
  res.json({ data: { 제목: "8월 정기 점검 안내", 작성일: "2026-08-01" } });
});
```

```
GET /public-api/notice
Origin: http://anywhere.example.net

200 {"data":{"제목":"8월 정기 점검 안내","작성일":"2026-08-01"}}
access-control-allow-origin=*
```

★ 로그인 이 필요 없고, 누가 봐도 상관없는 자료에만 * 를 씁니다.

★ 시험용 출처를 영어로 쓴 이유 헤더 값에는 한글을 담을 수 없습니다. (05단원 개념02 섹션 4-2) Origin 에 한글 도메인을 넣으면 요청 자체가 안 나갑니다.

문제 3 내부 API 는 허용 목록만

```
app.use(
  "/api",
  cors({
    origin: 출처판단,
    methods: ["GET", "POST", "PATCH", "DELETE"],
    allowedHeaders: ["Content-Type", "Authorization"],
    exposedHeaders: ["X-Total-Count"],
    maxAge: 600,
  })
);

const 설비들 = [
  { id: 1, name: "컨베이어 1호", line: "A", status: "가동" },
  { id: 2, name: "프레스 1호", line: "B", status: "정지" },
];

app.get("/api/v1/equipments", (req, res) => {
```



```
res.set("X-Total-Count", String(설비들.length));
res.json({ data: 설비들 });
});
```

GET /api/v1/equipments

Origin: http://localhost:5500

200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"}, {"id":2,"name":"프레스 1호","line":"B","status":"정지"}]}

access-control-allow-origin=http://localhost:5500 | access-control-expose-headers=X-Total-Count | vary=Origin | x-total-count=2

GET /api/v1/equipments

Origin: https://sub.example.com

200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"}, {"id":2,"name":"프레스 1호","line":"B","status":"정지"}]}

access-control-allow-origin=https://sub.example.com

GET /api/v1/equipments

Origin: https://example.com.evil.net

200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"}, {"id":2,"name":"프레스 1호","line":"B","status":"정지"}]}

access-control-allow-origin=(없음)

★★ 세 번째를 보세요. 비슷하게 생긴 도메인을 막았습니다. hostname 을 꺼내 비교했기 때문입니다. 그리고 데이터는 그대로 나갔다는 것도 보세요. 서버는 안 막습니다.

front/다른출처.html 의 ☹ 버튼이 부르는 주소입니다. 08단원의 모든 서버가 같은 페이지로 확인할 수 있게 맞춰 두었습니다.

```
app.get("/api/v1/equipments-with-count", (req, res) => {
  res.set("X-Total-Count", String(설비들.length));
  res.json({ data: 설비들 });
});
```

GET /api/v1/equipments-with-count

Origin: http://localhost:5500

200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"}, {"id":2,"name":"프레스 1호","line":"B","status":"정지"}]}

access-control-allow-origin=http://localhost:5500 | x-total-count=2 | access-control-expose-headers=X-Total-Count

문제 4 프리플라이트

```
app.post("/api/v1/equipments", (req, res) => {
  const { name, line } = req.body || {};

  if (!name || !line) {
    return res.status(400).json({
      error: { code: "VALIDATION_FAILED", message: "name 과 line 이 필요합니다" },
    });
  }

  res.status(201).json({ data: { id: 3, name, line, status: "정지" } });
});

app.delete("/api/v1/equipments/:id", (req, res) => {
  console.log(` * DELETE 가 서버까지 도착했습니다. id=${req.params.id}`);
  res.sendStatus(204);
});
```

OPTIONS /api/v1/equipments

Origin: http://localhost:5500; Access-Control-Request-Method: POST; Access-Control-Request-Headers: content-type

204

access-control-allow-origin=http://localhost:5500 | access-control-allow-methods=GET,POST,PATCH,DELETE | access-control-allow-headers=Content-Type,Authorization | access-control-max-age=600

POST /api/v1/equipments

Origin: http://localhost:5500

```
{ "name": "용접로봇 1호", "line": "C" }
```

201 { "data": { "id": 3, "name": "용접로봇 1호", "line": "C", "status": "정지" } }

access-control-allow-origin=http://localhost:5500

DELETE /api/v1/equipments/1

Origin: http://localhost:5500

204

access-control-allow-origin=http://localhost:5500

OPTIONS /api/v1/equipments

Origin: https://example.com.evill.net; Access-Control-Request-Method: POST

404

access-control-allow-origin=(없음)

★ 마지막 — 허락 안 한 출처에는 프리플라이트 응답에도 헤더를 안 줍니다. 브라우저는 진짜 요청을 아예 안 보냅니다. 터미널의 ★ 도 안 찍힙니다.

★★ 그런데 상태코드가 204 가 아니라 404 입니다. 왜일까요?

origin 을 '함수' 로 준 경우, 그 함수가 `false` 를 돌려주면 cors 패키지는 응답을 만들지 않고 `next()` 를 부릅니다. 그래서 요청이 아래로 흘러가 404 처리기까지 내려갑니다.

origin 을 '배열' 로 주면 다릅니다. 직접 재 본 결과입니다.

origin 설정	허용 출처	거절 출처
배열	204 + Allow-Origin 204, 헤더 없음	
함수(false)	204 + Allow-Origin 404, 헤더 없음	

브라우저 입장에서는 둘 다 똑같이 막힙니다. Allow-Origin 헤더가 없으면 상태코드가 무엇이든 거절이니까요.

다만 서버 기록을 볼 때 헷갈릴 수 있습니다. "OPTIONS 가 404 로 잔뜩 찍히는데요?" → 허락 안 한 출처가 두드리고 있는 것입니다.

문제 5 정적 파일

★ API 라우트보다 아래에 두었습니다. `public` 안에 `api` 라는 폴더가 생겨도 API 가 먼저 걸리게 하려는 것입니다.

```
app.use(express.static(path.join(__dirname, "public")));
```

```
GET /
200
```

```
GET /css/공통.css
200
```

★ 같은 출처라 CORS 가 필요 없습니다. `http://localhost:3000` 으로 열면 화면과 API 가 같은 곳에서 옵니다.

문제 6 404 와 에러 처리기

```
app.use((req, res) => {
  res.status(404).json({ error: { code: "NOT_FOUND", message: "찾을 수 없습니다" } });
});
```

GET /없는주소

```
404 {"error":{"code":"NOT_FOUND","message":"찾을 수 없습니다"}}
```

```
app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  res.status(500).json({
    error: { code: "INTERNAL_ERROR", message: "서버에서 문제가 생겼습니다" },
  });
});

app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}`);
  console.log(`허용 출처: ${허용목록.join(", ")} + *.example.com`);
});
```

세 가지 방법으로 확인하기

① 같은 출처

http://localhost:3000 을 브라우저로 엽니다.
버튼을 누르면 목록이 나옵니다. CORS 가 전혀 안 걸립니다.

② 다른 출처 (CORS)

front/다른출처.html 을 Live Server(5500)로 엽니다.
다섯 버튼이 전부 됩니다. ⑤에서 X-Total-Count 도 읽힙니다.

③ 브라우저가 아닌 곳

Postman 으로 부릅니다. 잘 됩니다.
Headers 에 Origin: https://example.com.evil.net 을 넣어 보세요.
데이터가 그대로 나옵니다. Allow-Origin 헤더만 없습니다.

★ ③이 중요합니다. CORS 로는 아무것도 못 막습니다. 진짜로 막으려면 05단원의 인증을 붙여야 합니다.

이 서버의 구조

app.use(express.json()) 본문 읽기 app.use("/public-api", cors()) 공개 API 는 누구나

app.use("/api", cors({ 목록 })))	내부 API 는 허용 목록만
--------------------------------	-----------------

app.get("/api/...")	API 라우트들
app.use(express.static("public"))	화면 (API 아래에)
app.use(404)	못 찾음

app.use(에러 처리기) 인자 네 개

★ cors 를 라우트별로 나눠 붙였습니다. app.use(cors()) 한 줄로 끝냈다면 내부 API 도 아무나 부를 수 있게 됩니다.

★ static 을 API 아래에 두었습니다. public 에 api 라는 폴더가 실수로 생겨도 API 가 먼저 걸립니다. 반대로 두면 조용히 API 가 안 걸립니다. 에러도 안 납니다.

09단원 연습문제 정답 파일 업로드

RUN node 연습문제_정답.js

09단원 연습문제 정답 (서버 없이 푸는 것)

```
const path = require("path");
```

문제 1

multer 가 주는 깨진 이름을 흉내 냅니다. 브라우저는 UTF-8 로 보내는데 받는 쪽이 latin1 로 읽어서 이렇게 됩니다.

```
function 깨뜨리기(이름) {
  return Buffer.from(이름, "utf8").toString("latin1");
}

function 이름되돌리기(이름) {
  return Buffer.from(이름, "latin1").toString("utf8");
}

const 깨진것 = 깨뜨리기("작업표준서.pdf");

console.log(깨진것.length);
```

출력 19

코드

```
console.log(이름되돌리기(깨진것));
```

```
console.log(이름되돌리기(깨진것).length);
```

▶ 출력

▶ 작업표준서.pdf

▶ 9

```
console.log(이름되돌리기("report-2026.png"));
```

출력 report-2026.png

```
console.log(이름되돌리기(깨뜨리기("검사성적서_2026.pdf")));
```

출력 검사성적서_2026.pdf

★ 19 와 9 한글 5글자가 UTF-8 에서 15바이트입니다. 그 바이트 하나하나를 글자 하나로 잘못 읽어서 15글자가 되고, ".pdf" 4글자를 더해 19 입니다.

★ 네 번째를 보세요. 영어 이름은 되돌려도 그대로입니다. 그래서 조건 없이 항상 감싸도 안전합니다.

★★ 어디에 적용하나 file.originalname 에만 적용합니다. req.body 의 값에는 절대 적용하지 마세요. 멀쩡한 글자가 망가집니다. (파일 이름은 헤더 줄에, 글자 값은 본문에 담겨 와서 해석 방식이 다릅니다)

문제 2

```
function 안전한저장이름(원본이름, 시각, 무작위) {
  const 되돌린것 = 이름되돌리기(원본이름);
  const 확장자 = path.extname(되돌린것).toLowerCase();

  return `${시각}-${무작위}${확장자}`;
}

console.log(안전한저장이름(깨뜨리기("작업표준서.PDF"), 1786683001682, 123456789));
```

출력 1786683001682-123456789.pdf

```
console.log(안전한저장이름("photo.JPEG", 1786683001682, 123456789));
```

출력 1786683001682-123456789.jpeg

```
console.log(안전한저장이름("확장자없음", 1786683001682, 123456789));
```

출력 1786683001682-123456789

```
console.log(안전한저장이름("../..../.env", 1786683001682, 123456789));
```

출력 1786683001682-123456789

★★ 네 번째가 중요합니다. ../..../.env 를 올려도 저장 이름에는 ../ 가 한 조각도 안 남습니다. 확장자만 뽑아 쓰기 때문입니다. 경로 탈출이 원천적으로 막힙니다.

원래 이름을 그대로 저장 이름으로 쓰면 서버 파일을 덮어쓸 수 있습니다.

★ 그런데 확장자가 ".env" 가 아니라 아예 없습니다. 왜일까요?

```
path.extname("../..../.env") → ""
path.extname(".env") → ""
path.extname("a.env") → ".env"
```

점으로 '시작하는' 이름은 확장자가 아니라 '숨김 파일 이름' 으로 봅니다. 리눅스에서 .env, .gitignore 처럼
점으로 시작하는 파일이 숨김 파일입니다.

그래서 .env 를 올리면 확장자 없는 파일로 저장됩니다. 문제 3의 확장자검사가 "확장자가 없습니다" 로
거절할 것입니다.

★ 시각과 무작위를 인자로 받은 이유 함수 안에서 Date.now() 를 부르면 시험할 때마다 결과가
달라집니다. 밖에서 넣어 주면 결과가 정해져서 확인할 수 있습니다. 실제 코드에서는 함수 안에서 부르면
됩니다.

★ 세 번째 — 확장자가 없으면 그냥 안 붙습니다. 확장자 없는 파일을 받을지 말지는 fileFilter 에서
정합니다. (문제 3)

문제 3

```
const 허용확장자 = [".jpg", ".jpeg", ".png", ".gif", ".pdf"];

function 확장자검사(원본이름) {
  const 되돌린것 = 이름되돌리기(원본이름);
  const 확장자 = path.extname(되돌린것).toLowerCase();

  if (!확장자) return "확장자가 없습니다";
  if (!허용확장자.includes(확장자)) return `${확장자} 는 올릴 수 없습니다`;

  return null; // 통과
}

console.log(확장자검사(깨뜨리기("작업표준서.pdf")));
```

출력 null

```
console.log(확장자검사("photo.PNG"));
```

출력 null

```
console.log(확장자검사("악성.exe"));
```

출력 .exe 는 올릴 수 없습니다

```
console.log(확장자검사("이름만있음"));
```

출력 확장자가 없습니다

```
console.log(확장자검사(깨뜨리기("사진.jpg")));
```


출력 null

★ 첫 번째와 다섯 번째 — 되돌리기를 안 하면 어떻게 될까요? 깨진 이름에서 extname 을 뽑아도 확장자는 살아 있습니다. “.pdf” 가 나옵니다. 그래서 이 경우엔 우연히 통과합니다.

그런데 파일 이름이 “표준.pdf” 가 아니라 “표준서” 처럼 확장자가 없으면? 깨진 글자 안에 점이 들어 있으면 엉뚱한 것을 확장자로 봅니다. 되돌리고 나서 판단하는 것이 안전합니다.

★ 허용 목록 방식인 것에 주의하세요. 금지 목록(.exe, .bat ...)은 반드시 빠뜨리는 게 생깁니다. .cmd, .ps1, .jar, .html, .svg ... 끝이 없습니다.

문제 4

```
const 매직넘버 = {
  ".png": [0x89, 0x50, 0x4e, 0x47],
  ".jpg": [0xff, 0xd8, 0xff],
  ".jpeg": [0xff, 0xd8, 0xff],
  ".gif": [0x47, 0x49, 0x46],
  ".pdf": [0x25, 0x50, 0x44, 0x46],
};

function 진짜형식인가(버퍼, 확장자) {
  const 기대 = 매직넘버[확장자];
  if (!기대) return true; // 모르는 형식은 통과 return 기대.every((바이트, i) => 버퍼[i] === 바이트);
}

const 진짜PNG = Buffer.from([0x89, 0x50, 0x4e, 0x47, 0x0d, 0x0a]);
const 진짜PDF = Buffer.from("%PDF-1.4 어찌고", "utf8");
const 그냥글자 = Buffer.from("이건 그냥 글자입니다", "utf8");

console.log(진짜형식인가(진짜PNG, ".png"));
```

출력 true

코드	출력
console.log(진짜형식인가(그냥글자, ".png"));	false
console.log(진짜형식인가(진짜PDF, ".pdf"));	true
console.log(진짜형식인가(진짜PNG, ".pdf"));	false
console.log(진짜형식인가(그냥글자, ".txt"));	true

★ 네 번째 — 진짜 PNG 인데 이름이 .pdf 입니다. 확장자와 내용이 안 맞으니 거절합니다.

★ 다섯 번째 — .txt 는 매직 넘버가 없습니다. 글자 파일은 정해진 시작 바이트가 없어서 확인할 방법이 없습니다. 모르는 형식은 통과시키되, 애초에 허용 목록에 넣을지를 고민하세요.

★★ 왜 이렇게까지 하나 mimetype 은 보내는 쪽이 마음대로 정합니다. 확장자도 이름만 바꾸면 그만입니다. 내용의 첫 바이트만이 거짓말을 하기 어렵습니다.

.png 로 이름만 바꾼 실행 파일을 올려 두고, 나중에 남에게 내려받게 하는 공격이 있습니다.

문제 5

```
function 어긋난것찾기(기록들, 디스크파일들) {
  const 기록이름 = new Set(기록들.map((기록) => 기록.저장이름));
  const 디스크이름 = new Set(디스크파일들);

  return {
    고아파일: 디스크파일들.filter((이름) => !기록이름.has(이름)),
    깨진기록: 기록들.filter((기록) => !디스크이름.has(기록.저장이름)).map((기록) =>
기록.id),
  };
}

const 기록들 = [
  { id: 1, 저장이름: "a.pdf" },
  { id: 2, 저장이름: "b.png" },
  { id: 3, 저장이름: "없어진것.pdf" },
];
const 디스크파일들 = ["a.pdf", "b.png", "버려진것.pdf", "또버려진것.png"];

const 결과 = 어긋난것찾기(기록들, 디스크파일들);

console.log(결과.고아파일.join(", "));
```

출력 버려진것.pdf, 또버려진것.png

```
console.log(결과.깨진기록.join(", "));
```

출력 3

★ 두 가지가 다른 문제입니다.

고아 파일 기록은 없는데 파일만 있음 → 디스크만 차지. 사용자는 모름 깨진 기록 기록은 있는데 파일이 없음
→ 목록에 보이는데 누르면 깨짐 ★

깨진 기록이 훨씬 나쁩니다. 사용자에게 바로 보이니까요. 그래서 지울 때 '기록을 먼저' 지웁니다. (문제 7)

★ Set 을 쓴 이유 배열의 includes 는 하나 찾을 때마다 처음부터 훑습니다. 기록 만 개 × 파일 만 개면 일억 번 비교합니다. Set 의 has 는 개수와 상관없이 빠릅니다.

문제 6

```
const 에러안내 = {
  LIMIT_FILE_SIZE: "파일이 너무 큼니다",
  LIMIT_FILE_COUNT: "파일이 너무 많습니다",
  LIMIT_UNEXPECTED_FILE: "예상하지 못한 필드 이름입니다. 필드 이름을 영어로  
맞췄나요?",
  LIMIT_FIELD_COUNT: "글자 필드가 너무 많습니다",
  LIMIT_FIELD_KEY: "필드 이름이 너무 길니다",
  LIMIT_FIELD_VALUE: "필드 값이 너무 길니다",
};

function 안내문구(에러) {
  const 기본 = 에러안내[에러.code] ?? "업로드에 실패했습니다";
```

★ field 는 없을 수도 있습니다. LIMIT_FILE_COUNT 에는 안 들어 있습니다.

```
return 에러.field ? `${기본} (${에러.field})` : 기본;
}

console.log(안내문구({ code: "LIMIT_FILE_SIZE", field: "file" }));
```

출력 파일이 너무 큼니다 (file)

```
console.log(안내문구({ code: "LIMIT_FILE_COUNT" }));
```

출력 파일이 너무 많습니다

```
console.log(안내문구({ code: "LIMIT_UNEXPECTED_FILE", field: "ìì%" }));
```

출력 예상하지 못한 필드 이름입니다. 필드 이름을 영어로 맞췄나요? (ìì%)

```
console.log(안내문구({ code: "MYSTERY" }));
```

출력 업로드에 실패했습니다

★ 세 번째를 보세요. field 에 깨진 글자가 들어 있습니다. 필드 이름을 한글로 썼을 때 정확히 이렇게 나옵니다. 이 글자를 보면 “아, 필드 이름을 한글로 썼구나” 를 바로 알 수 있습니다.

★ 이 에러들은 전부 400 입니다. 500 이 아닙니다. 보낸 쪽이 고칠 수 있는 문제니까요. 500 을 주면 사용자가 “서버 고장” 이라고 생각하고 계속 다시 시도합니다.

문제 7

파일과 기록을 지울 때 무엇을 먼저 지우나

두 가지 순서와 각각의 실패 결과:

① 파일 먼저 → 기록 나중

중간에 실패하면 '기록은 있는데 파일이 없는' 상태가 됩니다.
목록에 파일이 보이는데 누르면 깨집니다.
사용자가 바로 알아채고 문의합니다.

② 기록 먼저 → 파일 나중 ★ 이쪽을 고릅니다

중간에 실패하면 '파일은 있는데 기록이 없는' 상태가 됩니다.
목록에 안 보입니다. 아무도 모릅니다.
디스크만 조금 차지하고, 나중에 청소하면 됩니다.

왜 완벽하게 못 하나: 파일 시스템과 데이터베이스는 트랜잭션을 함께 쓸 수 없습니다. “둘 다 되거나 둘 다 안 되거나”를 보장할 방법이 없습니다. (07단원 개념05의 ‘중간실패’와 같은 이야기입니다)

그래서 어떻게 하나: ‘어느 쪽으로 실패할지’를 정합니다. 보이지 않는 쓰레기가, 보이는 고장보다 낫습니다. 남은 쓰레기는 청소 작업으로 치웁니다.

청소할 때 주의할 점: 방금 올라온 파일은 아직 기록이 안 남았을 수 있습니다. “만들어진 지 한 시간이 넘은 것만” 같은 조건을 걸어야 합니다.

업로드 라우트에서 에러를 낼 때: req.file이 있는 상태로 400·404를 내면 그 파일이 고아가 됩니다. 에러를 내기 전에 반드시 지우세요. 아주 빠뜨리기 쉽습니다.

문제 8

이 자료에서 지금까지 나온 ‘한글이 안 되는 곳’을 전부 모으면

라우트 주소 (04단원) 조용히 404가 납니다 응답 헤더 값 (05단원) ERR_INVALID_CHAR로 500이 납니다 요청 헤더 값 (05단원) 요청 자체가 안 나갑니다 업로드 필드 이름 (09단원) MulterError: Unexpected field 업로드 파일 이름 (09단원) 깨져서 도착합니다 (되돌릴 수는 있음) Content-Disposition (09단원) encodeURIComponent로 감싸야 합니다

반대로 한글이 잘 되는 곳: 쿼리 값, 경로 파라미터 값, JSON 본문, 폼 필드 값, 응답 본문, 파일 내용

공통점은 무엇인가: 안 되는 것은 전부 ‘헤더에 실려 가는 것’입니다. HTTP 헤더는 아주 오래전 약속이라 영어와 숫자만 담을 수 있습니다. 본문은 아무 바이트나 담을 수 있어서 한글이 잘 됩니다.

규칙 한 줄로: 이름은 영어로, 값은 한글로. 주소·필드 이름·헤더 이름은 우리가 정하는 것이니 영어로 지으면 됩니다. 사용자가 넣는 값에만 한글이 들어오고, 그건 대부분 본문에 담깁니다.

파일 이름은 왜 예외인가: 파일 이름은 '값' 인데도 헤더 줄(Content-Disposition)에 실려 옵니다. 그래서 깨집니다. 우리가 정하는 게 아니라 사용자가 정하는 값이라 영어로 하라고 할 수도 없습니다. 그래서 되돌리기가 필요합니다.

09단원 연습문제 서버 정답 파일 업로드

RUN node 연습문제_서버_정답.js

06단원(중첩 자원) · 07단원(업무 규칙) · 09단원(업로드)을 합칩니다.

GET	/api/v1/equipments/:id/photos	그 설비의 사진 목록
POST	/api/v1/equipments/:id/photos	사진 올리기
GET	/api/v1/photos/:photoId/view	보기
GET	/api/v1/photos/:photoId/download	내려받기

DELETE /api/v1/photos/:photoId 지우기

```
const express = require("express");
const multer = require("multer");
const path = require("path");
const fs = require("fs");

const app = express();
const PORT = Number(process.env.PORT) || 3000;

app.use(express.json());

const 사진폴더 = path.join(__dirname, "uploads", "사진");
fs.rmSync(사진폴더, { recursive: true, force: true });
fs.mkdirSync(사진폴더, { recursive: true });
```

문제 1 도우미

```
function 이름되돌리기(이름) {
  return Buffer.from(이름, "latin1").toString("utf8");
}

function HttpError(status, code, message) {
  const 에러 = new Error(message);
  에러.status = status;
  에러.code = code;
  return 에러;
}
```

준비

```
const 설비들 = [
  { id: 1, name: "컨베이어 1호", line: "A" },
  { id: 2, name: "프레스 1호", line: "B" },
];

let 사진들 = [];
let 다음번호 = 1;

const 설비당최대 = 3;
const 허용확장자 = [".jpg", ".jpeg", ".png", ".gif"];
```

문제 2 multer 설정

```
const 저장방식 = multer.diskStorage({
  destination: (req, file, cb) => cb(null, 사진폴더),
  filename: (req, file, cb) => {
    const 확장자 = path.extname(이름되돌리기(file.originalname)).toLowerCase();
    cb(null, `${Date.now()}-${Math.round(Math.random() * 1e9)}${확장자}`);
  },
});

const 업로드 = multer({
  storage: 저장방식,
  limits: { fileSize: 200, files: 1 },
  fileFilter: (req, file, cb) => {
    const 확장자 = path.extname(이름되돌리기(file.originalname)).toLowerCase();

    if (!허용확장자.includes(확장자)) {
      return cb(
        HttpError(400, "BAD_EXTENSION", `${확장자} || "확장자 없는 파일" 은(는) 올릴 수 없습니다`)
      );
    }

    cb(null, true);
  },
});
```

문제 3 미들웨어

```
function 설비찾기(req, res, next) {
  const 번호 = Number(req.params.id);

  if (!Number.isInteger(번호)) {
    return next(HttpError(400, "VALIDATION_FAILED", "번호는 숫자여야 합니다"));
  }

  const 설비 = 설비들.find((설비) => 설비.id === 번호);

  if (!설비) {
    return next(HttpError(404, "NOT_FOUND", "설비를 찾을 수 없습니다"));
  }

  req.설비 = 설비;
  next();
}

function 사진찾기(req, res, next) {
  const 번호 = Number(req.params.photoId);
  const 사진 = 사진들.find((사진) => 사진.id === 번호);

  if (!사진) {
    return next(HttpError(404, "NOT_FOUND", "사진을 찾을 수 없습니다"));
  }

  req.사진 = 사진;
  next();
}
```

★ 파일을 지우는 도우미. 이미 없어도 에러가 아닙니다.

```
async function 파일지우기(저장이름) {
  try {
    await fs.promises.unlink(path.join(사진폴더, 저장이름));
  } catch (에러) {
    if (에러.code !== "ENOENT") throw 에러;
  }
}
```


문제 4 목록

```
app.get("/api/v1/equipments/:id/photos", 설비찾기, (req, res) => {
  const 결과 = 사진들.filter((사진) => 사진.equipmentId === req.설비.id);

  res.json({
    data: 결과.map((사진) => ({
      id: 사진.id,
      원래이름: 사진.원래이름,
      크기: 사진.크기,
      보기: `/api/v1/photos/${사진.id}/view`,
    })),
    meta: { 설비: req.설비.name, 개수: 결과.length, 최대: 설비당최대 },
  });
});
```

GET /api/v1/equipments/1/photos

200 {"data":[],"meta":{"설비":"컨베이어 1호","개수":0,"최대":3}}

GET /api/v1/equipments/99/photos

404 {"error":{"code":"NOT_FOUND","message":"설비를 찾을 수 없습니다"}}

★ 06단원에서 강조한 그 구별입니다. 설비는 있고 사진이 0장 → 200 과 빈 배열 설비 자체가 없음 → 404

문제 5 올리기

```
app.post(
  "/api/v1/equipments/:id/photos",
  업로드.single("photo"),
  async (req, res, next) => {
```

★★ multer 가 먼저 돌기 때문에, 여기 왔을 때는 파일이 이미 저장돼 있습니다. 그래서 에러를 내기 전에 반드시 지워야 합니다.

```
const 정리하고 = async (에러) => {
  if (req.file) await 파일지우기(req.file.filename);
  next(에러);
};
```

설비 확인을 여기서 직접 합니다. 미들웨어로 하면 파일 정리를 못 합니다.

```

const 번호 = Number(req.params.id);

if (!Number.isInteger(번호)) {
  return 정리하고(400, "VALIDATION_FAILED", "번호는 숫자여야 합니다");
}

const 설비 = 설비들.find((설비) => 설비.id === 번호);

if (!설비) {
  return 정리하고(404, "NOT_FOUND", "설비를 찾을 수 없습니다");
}

if (!req.file) {
  return next(400, "NO_FILE", "사진이 없습니다");
}

```

업무 규칙: 설비당 3장까지

```

const 지금개수 = 사진들.filter((사진) => 사진.equipmentId === 번호).length;

if (지금개수 >= 설비당최대) {
  return 정리하고(
    409, "TOO_MANY_PHOTOS", `설비당 사진은 ${설비당최대}장까지입니다`
  );
}

const 사진 = {
  id: 다음번호,
  equipmentId: 번호,
  저장이름: req.file.filename,
  원레이름: 이름되돌리기(req.file.originalname),
  크기: req.file.size,
};

다음번호 += 1;
사진들.push(사진);

res
  .status(201)
  .location(`/api/v1/photos/${사진.id}/view`)
  .json({ data: { id: 사진.id, 원레이름: 사진.원레이름, 크기: 사진.크기 } });
}
);

```

POST /api/v1/equipments/1/photos multipart:@photo=설비사진.png:50

201 {"data":{"id":1,"원래이름":"설비사진.png","크기":50}}

POST /api/v1/equipments/1/photos multipart:@photo=악성.exe:10

400 {"error":{"code":"BAD_EXTENSION","message":".exe 은(는) 올릴 수 없습니다"}}

POST /api/v1/equipments/1/photos multipart:@photo=큰사진.png:500

400 {"error":{"code":"LIMIT_FILE_SIZE","message":"파일이 너무 큽니다 (최대 200바이트)"}}

POST /api/v1/equipments/99/photos multipart:@photo=설비사진.png:50

404 {"error":{"code":"NOT_FOUND","message":"설비를 찾을 수 없습니다"}}

POST /api/v1/equipments/1/photos multipart:@photo=사진2.png:50

201 {"data":{"id":2,"원래이름":"사진2.png","크기":50}}

POST /api/v1/equipments/1/photos multipart:@photo=사진3.png:50

201 {"data":{"id":3,"원래이름":"사진3.png","크기":50}}

POST /api/v1/equipments/1/photos multipart:@photo=사진4.png:50

409 {"error":{"code":"T00_MANY_PHOTOS","message":"설비당 사진은 3장까지입니다"}}

★★★ 네 번째와 마지막을 보세요. 404 와 409 를 낼 때 이미 올라온 파일을 지웠습니다. 안 지웠다면 지금쯤 고아 파일이 두 개 쌓였을 것입니다. 다음 문제에서 확인합니다.

문제 6 고아 파일 확인

```
app.get("/api/v1/photos/orphans", (req, res) => {
  const 기록이름 = new Set(사진들.map((사진) => 사진.저장이름));
  const 디스크 = fs.readdirSync(사진폴더);

  res.json({
    data: {
      디스크파일수: 디스크.length,
      기록수: 기록이름.size,
      고아파일수: 디스크.filter((이름) => !기록이름.has(이름)).length,
    },
  });
});
```

GET /api/v1/photos/orphans

200 {"data":{"디스크파일수":3,"기록수":3,"고아파일수":0}}

★★ 0 입니다. 에러를 낸 요청이 네 번 있었는데도 쓰레기가 안 남았습니다. 문제 5의 정리하고() 덕입니다.

★ 이 라우트는 반드시 /api/v1/photos/:photoId 보다 '위'에 있어야 합니다.

아래에 두면 :photoId 에 "orphans" 가 들어가 버립니다. (04단원 개념02 섹션 7)

문제 7 보기와 내려받기

```
app.get("/api/v1/photos/:photoId/view", 사진찾기, (req, res) => {
  res.sendFile(path.join(사진폴더, req.사진.저장이름));
});
```

GET /api/v1/photos/1/view

200

GET /api/v1/photos/99/view

404 {"error":{"code":"NOT_FOUND","message":"사진을 찾을 수 없습니다"}}

```
app.get("/api/v1/photos/:photoId/download", 사진찾기, (req, res) => {
```

★ res.download 가 한글 이름을 알아서 인코딩해 줍니다.

```
res.download(path.join(사진폴더, req.사진.저장이름), req.사진.원래이름);
});
```

GET /api/v1/photos/1/download

200

content-disposition=attachment; filename="????.png"; filename*=UTF-8'%EC%84%A4%EB%B9%84%EC%82%AC%EC%A7%84.png

★ filename 과 filename* 두 개가 들어 있습니다. 헤더에 한글을 못 담기 때문에 생긴 방법입니다.
(09단원 개념04 섹션 4)

★ 물음표가 네 개인 것에 주의하세요. "설비사진" 이 네 글자라서입니다. 한글 한 글자가 물음표 하나가 됩니다. 확장자 .png 는 영어라 그대로 남습니다. 요즘 브라우저는 filename* 쪽을 보고 한글 이름으로 저장합니다.

문제 8 지우기

```
app.delete("/api/v1/photos/:photoId", 사진찾기, async (req, res, next) => {
  try {
```

★ 기록을 먼저, 파일을 나중에. (개념05 섹션 3)

```
    사진들 = 사진들.filter((사진) => 사진.id !== req.사진.id);
    await 파일지우기(req.사진.저장이름);

    res.sendStatus(204);
  } catch (에러) {
    next(에러);
  }
});
```

DELETE /api/v1/photos/3

204

DELETE /api/v1/photos/3

404 {"error":{"code":"NOT_FOUND","message":"사진을 찾을 수 없습니다"}}

GET /api/v1/photos/orphans

200 {"data":{"디스크파일수":2,"기록수":2,"고아파일수":0}}

GET /api/v1/equipments/1/photos

```
200 {"data":[{"id":1,"원래이름":"설비사진.png","크기":50,"보기":"/api/v1/photos/1/view"}, {"id":2,"원래이름":"사진2.png","크기":50,"보기":"/api/v1/photos/2/view"}], "meta":{"설비":"컨베이어 1호","개수":2,"최대":3}}
```

★ 한 장을 지웠으니 다시 올릴 수 있습니다.

POST /api/v1/equipments/1/photos multipart:photo=새사진.png:50

```
201 {"data":{"id":4,"원래이름":"새사진.png","크기":50}}
```

문제 9 404 와 에러 처리기

```
app.use((req, res) => {
  res.status(404).json({ error: { code: "ROUTE_NOT_FOUND", message: "그런 주소가 없습니다" } });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.name}: ${err.message}`);

  if (err.name === "MulterError") {
    const 안내 = {
      LIMIT_FILE_SIZE: "파일이 너무 큼니다 (최대 200바이트)",
      LIMIT_FILE_COUNT: "사진은 한 번에 한 장만 올릴 수 있습니다",
      LIMIT_UNEXPECTED_FILE: "필드 이름이 photo 가 맞나요? (한글로 쓰면 안 됩니다)",
    };

    return res.status(400).json({
      error: { code: err.code, message: 안내[err.code] ?? err.message },
    });
  }

  if (err.status) {
    return res.status(err.status).json({ error: { code: err.code, message: err.message } });
  }

  res.status(500).json({ error: { code: "INTERNAL_ERROR", message: "서버에서 문제가 생겼습니다" } });
});
```

```
app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다.
  http://localhost:${PORT}/api/v1/equipments/1/photos`);
});
```

이 파일에서 확인할 것

- ① 에러를 낼 때마다 파일을 지웠습니다 404 도, 409 도 마찬가지입니다. 안 그러면 고아가 쌓입니다. /api/v1/photos/orphans 가 계속 0 인 것이 그 증거입니다.
- ② 설비 확인을 미들웨어로 안 했습니다 설비찾기 를 미들웨어로 붙이면 파일 정리를 할 수가 없습니다. 미들웨어에서 next(에러) 를 부르면 라우트 함수가 실행되지 않으니까요. 그래서 업로드 라우트에서만 직접 확인했습니다.
- ③ /orphans 를 /:photoId 보다 위에 두었습니다 아래에 두면 :photoId 에 "orphans" 가 들어갑니다.
- ④ 지울 때 기록을 먼저 지웠습니다 보이지 않는 쓰레기가, 보이는 고장보다 낫습니다.
- ⑤ 업무 규칙(3장 제한)이 409 입니다 400 이 아닙니다. 값이 잘못된 게 아니라 '지금 상태와 안 맞는' 것입니다.

Postman 시나리오

```
1 GET    /api/v1/equipments/1/photos    0장
2 GET    /api/v1/equipments/99/photos    404 3 POST
/api/v1/equipments/1/photos (사진.png) 201 4 POST /api/v1/equipments/1/photos
(악성.exe) 400 5 POST /api/v1/equipments/1/photos (큰 파일) 400 6 POST
/api/v1/equipments/99/photos (사진.png) 404 7 GET /api/v1/photos/orphans
고아 0 *
8 세 장을 채운 뒤 한 장 더 409 9 GET
/api/v1/photos/orphans 여전히 0 *
10 GET   /api/v1/photos/1/download    한글 이름으로 저장됨
11 DELETE /api/v1/photos/1 204
12 GET   /api/v1/photos/orphans    여전히 0
```

★ 7·9·12 가 이 문제의 핵심입니다. 에러가 여러 번 났는데도 쓰레기가 안 쌓였는지 확인하는 것입니다.

★ Postman 에서 파일 올리기 Body → form-data → Key 에 photo, 오른쪽 드롭다운에서 File 선택 한글 이름 파일로도 꼭 해 보세요.

10단원 연습문제 정답 프론트엔드 연동

RUN node 연습문제_정답.js

10단원 연습문제 정답 (서버 없이 푸는 것)

화면 코드도 '순수한 함수' 로 떼어 내면 서버·브라우저 없이 시험할 수 있습니다. 07단원에서 서비스를 떼어 낸 것과 같은 이야기입니다.

문제 1

```
function 주소만들기(기준, 상태) {
  const 쿼리 = new URLSearchParams();

  쿼리.set("page", 상태.page);
  쿼리.set("limit", 상태.limit);
```

★ 빈 값은 아예 안 넣습니다. ?q= 를 보내면 서버가 '빈 검색어' 를 받습니다.

```
if (상태.sort) 쿼리.set("sort", 상태.sort);
if (상태.q) 쿼리.set("q", 상태.q);
if (상태.line) 쿼리.set("line", 상태.line);

return `${기준}?${쿼리.toString()}`;
}

console.log(주소만들기("/api/v1/equipments", { page: 1, limit: 5, sort: "id", q: "",
line: "" }));
```

출력 /api/v1/equipments?page=1&limit=5&sort=id

```
console.log(주소만들기("/api/v1/equipments", { page: 2, limit: 10, sort: "-id", q:
"로봇", line: "A" }));
```

출력 /api/v1/equipments?page=2&limit=10&sort=-id&q=%EB%A1%9C%EB%B4%87&line=A

```
console.log(주소만들기("/api/v1/equipments", { page: 1, limit: 5, sort: "", q: " ",
line: "" }));
```

출력 /api/v1/equipments?page=1&limit=5&q=++

★ 두 번째 — 한글이 퍼센트 인코딩됐습니다. URLSearchParams 가 알아서 해 줍니다. 직접 이어 붙이면 이걸 빠뜨립니다.

```
"?q=" + 상태.q    ← 한글이 그대로 들어가 주소가 깨질 수 있습니다
```

★★ 세 번째가 함정입니다. q 가 공백 두 칸입니다. " " 는 빈 글자가 아니라서 if (상태.q) 를 통과합니다. 그래서 ?q=++ 가 붙어 버립니다. (+ 는 공백이 인코딩된 것입니다)

상태에 담을 때 trim 을 해야 합니다.

```
상태.q = 입력값.trim();
```

화면에서 trim 을 안 하면 사용자가 실수로 넣은 공백이 그대로 서버에 갑니다. 06단원에서 서버도 trim 하도록 만들어 뒀지만, 안 보내는 편이 낫습니다.

문제 2

```
function 쪽버튼상태(meta) {
  return {
    이전잠김: meta.page <= 1,
    다음잠김: meta.page >= meta.totalPages,
    표시: `${meta.page} / ${meta.totalPages}`,
  };
}

function 버튼찍기(meta) {
  const 상태 = 쪽버튼상태(meta);
  console.log(`${상태.표시}  이전:${상태.이전잠김 ? "잠김" : "열림"}  다음:${상태.
  다음잠김 ? "잠김" : "열림"}`);
}

버튼찍기({ page: 1, totalPages: 3 });
```

```
출력      1 / 3  이전:잠김  다음:열림
```

```
버튼찍기({ page: 2, totalPages: 3 });
```

```
출력      2 / 3  이전:열림  다음:열림
```

```
버튼찍기({ page: 3, totalPages: 3 });
```

```
출력      3 / 3  이전:열림  다음:잠김
```

```
버튼찍기({ page: 1, totalPages: 0 });
```

출력 1 / 0 이전:잠김 다음:잠김

```
버튼찍기({ page: 1, totalPages: 1 });
```

출력 1 / 1 이전:잠김 다음:잠김

★ 네 번째 — 결과가 0건이면 totalPages 가 0 입니다. (06단원 연습문제 2) page(1) >= totalPages(0) 이라 다음도 잠깁니다. 잘 맞습니다.

★ 다섯 번째 — 딱 한 쪽뿐이면 둘 다 잠깁니다. 이것도 맞습니다.

문제 3

```
function 어디에보일까(에러) {  
  if (에러.code === "VALIDATION_FAILED") return "각 칸 아래에 (details 를 씁니다)";  
  if (에러.code === "DUPLICATED") return "이름 칸 아래에";  
  if (에러.code === "UNAUTHENTICATED") return "로그인 화면으로 보냅니다";  
  if (에러.code === "FORBIDDEN") return "폼 전체에 '권한이 없습니다'";  
  if (에러.code === "NETWORK") return "폼 전체에 '연결 확인' + 다시 시도 버튼";  
  return "폼 전체에 message 그대로";  
}  
  
for (const code of [  
  "VALIDATION_FAILED",  
  "DUPLICATED",  
  "UNAUTHENTICATED",  
  "FORBIDDEN",  
  "NETWORK",  
  "INTERNAL_ERROR",  
) {  
  console.log(`${code} → ${어디에보일까({ code })}`);  
}
```

출력 VALIDATION_FAILED → 각 칸 아래에 (details 를 씁니다)
 DUPLICATED → 이름 칸 아래에
 UNAUTHENTICATED → 로그인 화면으로 보냅니다
 FORBIDDEN → 폼 전체에 '권한이 없습니다'
 NETWORK → 폼 전체에 '연결 확인' + 다시 시도 버튼
 INTERNAL_ERROR → 폼 전체에 message 그대로

★★ code 로 판단하는 것이 핵심입니다. message 글자로 판단하면 서버가 맞춤법만 고쳐도 화면이 깨집니다. (06단원 개념04)

★ DUPLICATED 를 보세요. 서버는 details 를 안 줍니다. message 만 줍니다. 그런데 화면은 “이름 중복은 name 칸 문제” 라는 것을 압니다. 서버가 모든 것을 알려 줄 필요는 없습니다. 화면이 정할 수 있습니다.

문제 6 (async 가 아니라 먼저 나옵니다)

```
function 화면상태(로딩중, 에러, 목록) {
  if (로딩중) return "로딩";
  if (에러) return "실패";
  if (!목록 || 목록.length === 0) return "결과없음";
  return "성공";
}
```

```
console.log(화면상태(true, null, null));
```

출력 로딩

```
console.log(화면상태(false, { message: "서버 오류" }, null));
```

출력 실패

코드	▶ 출력
console.log(화면상태(false, null, []));	▶ 결과없음
console.log(화면상태(false, null, [{ id: 1 }]));	▶ 성공
console.log(화면상태(false, null, null));	▶ 결과없음

★ 순서가 중요합니다. 로딩 → 실패 → 결과없음 → 성공. 로딩을 맨 앞에 두지 않으면, 아직 안 불러온 상태에서 “결과없음” 이 나옵니다. 화면이 뜨자마자 “설비가 없습니다” 가 깜빡이게 됩니다.

★ 마지막 — 목록이 아직 null 인 경우도 ‘결과없음’ 으로 봤습니다. 여기서는 로딩이 false 니까요. 실무에서는 로딩 초기값을 true 로 두어 이 경우가 아예 안 생기게 하는 편이 낫습니다.

문제 4.5 (기다려야 하므로 async 안에서)

```
function 디바운스(함수, 시간) {
  let 타이머 = null;

  return (...인자들) => {
    clearTimeout(타이머);
    타이머 = setTimeout(() => 함수(...인자들), 시간);
  };
}
```

```

}

function 마지막만반영() {
  let 마지막번호 = 0;

  return async (일, 반영) => {
    마지막번호 += 1;
    const 내번호 = 마지막번호;

    const 결과 = await 일();
  }
}

```

내가 마지막 요청이 아니면 버립니다.

```

if (내번호 !== 마지막번호) return false;

  반영(결과);
  return true;
};
}

const 잠깐 = (ms) => new Promise((r) => setTimeout(r, ms));

async function main() {

```

문제 4

```

let 부른횟수 = 0;
let 마지막글자 = "";

const 검색보내기 = 디바운스((글자) => {
  부른횟수 += 1;
  마지막글자 = 글자;
}, 100);

```

“설비관리” 를 한 글자씩 치는 것을 흉내 냅니다.

```

for (const 글자 of ["설", "설비", "설비관", "설비관리"]) {
  검색보내기(글자);
  await 잠깐(20);
}

await 잠깐(200); // 타이머가 돌 시간 console.log(`디바운스: ${부른횟수}번 실행,
마지막 글자 "${마지막글자}"`);

```

출력 디바운스: 1번 실행, 마지막 글자 "설비관리"

★ 네 번 쳤는데 요청은 한 번입니다. 글자를 칠 때마다 보내면 요청이 네 배가 되고, 늦게 온 응답 문제(문제 5)까지 함께 옵니다.

문제 5

```
const 반영하기 = 마지막만반영();
let 화면 = "";

const 느린것 = 반영하기(
  async () => {
    await 잠깐(80);
    return "느린 응답";
  },
  (값) => (화면 = 값)
);

const 빠른것 = 반영하기(
  async () => {
    await 잠깐(10);
    return "빠른 응답";
  },
  (값) => (화면 = 값)
);

const [느린결과, 빠른결과] = await Promise.all([느린것, 빠른것]);

console.log(`느린 것 반영됨: ${느린결과}`);
```

출력 느린 것 반영됨: false

```
console.log(`빠른 것 반영됨: ${빠른결과}`);
```

출력 빠른 것 반영됨: true

```
console.log(`화면: ${화면}`);
```

출력 화면: 빠른 응답

★★ 느린 것을 먼저 보냈는데도 화면에는 빠른 것이 남았습니다.

막지 않았다면 순서가 이렇습니다.

```
10ms 뒤  "빠른 응답" 이 화면에 나옴
80ms 뒤  "느린 응답" 이 도착해 덮어씀   ← 옛날 결과가 이김
```

요청마다 번호를 붙여 두고 마지막 번호가 아니면 버리면 됩니다.

★ 이게 검색창에서 진짜로 일어납니다. "설"의 결과(느림)가 "설비관리"의 결과(빠름)를 덮어씁니다. 검색창에는 "설비관리"가 적혀 있는데 목록은 "설"의 결과입니다. 네트워크가 빠른 사무실에서는 재현이 잘 안 돼서 더 어렵습니다.

```
}  
  
main();
```

문제 7

fetch가 던지는 경우와 안 던지는 경우

fetch가 '던지는' 경우 (catch로 잡니다): 서버에 닿지도 못한 경우입니다.

- 서버가 꺼져 있음
- 인터넷이 끊김
- 주소를 잘못 씀 (없는 호스트)
- CORS로 막힘 (08단원)

fetch가 '안 던지는' 경우: 응답이 오기만 하면 안 던집니다. 상태코드가 무엇이든요.

404도, 500도, 403도 전부 '성공'으로 흘러갑니다.

그래서 무엇이 문제인가: fetch만 쓰고 응답 내용을 확인 안 하면, 500이 왔는데도 성공 갈래로 가서 화면에 `undefined`가 나옵니다. 에러도 안 나고 콘솔도 조용합니다. 원인을 찾기 어렵습니다.

어떻게 하나: `res.ok`를 보거나, 응답 몸통의 `error` 필드를 봅니다. 그리고 그 판단을 `api.js` 같은 파일 한 곳에 모읍니다. 화면마다 하면 한 군데를 빠뜨리고, 그 화면만 조용히 이상해집니다.

204는 왜 따로 다루냐: 본문이 없습니다. `res.json()`을 부르면 터집니다. 상태코드를 먼저 보고 걸러 내야 합니다.

JSON이 아닌 응답은 왜 따로 다루냐: 앞단에 Nginx 같은 것이 있으면 서버가 죽었을 때 HTML 에러 페이지를 줍니다. `res.json()`이 "Unexpected token '<'"로 터집니다. try/catch로 감싸서 "서버 응답을 읽지 못했습니다"로 바꿔야 합니다.

문제 8

화면 검증과 서버 검증

화면 검증이 하는 일: 사용자를 편하게 합니다. 서버에 다녀오기 전에 알려 주니 빠릅니다. `required`, `minlength` 같은 HTML 속성으로도 됩니다.

서버 검증이 하는 일: 진짜 방어선입니다. Postman·curl 로 보내면 화면을 통째로 건너웁니다. (05단원 개념04)

화면 검증만 하면: Postman 한 번이면 뚫립니다. 이름이 없는 설비, 라인이 Z 인 설비가 만들어집니다.

서버 검증만 하면: 동작은 안전합니다. 다만 사용자가 답답합니다. 글자를 다 치고 버튼을 눌러야 "2글자 이상" 을 알게 됩니다.

규칙이 두 곳에 생기는 문제: 화면에도 "2글자 이상" 을 적으면, 서버 규칙이 바뀔 때 화면도 고쳐야 합니다. 한쪽만 고치면 어긋납니다. "화면에서는 되는데 저장이 안 돼요" 또는 그 반대가 됩니다.

이 자료가 고른 방법: 규칙은 서버에만 둡니다. 화면은 서버가 준 details 를 칸마다 붙여 주기만 합니다. 규칙이 한 곳에 있으니 어긋날 일이 없습니다.

실무에서는 required 같은 가벼운 것만 화면에 두고, 진짜 규칙은 서버에 두는 방식을 많이 씁니다. 또는 규칙을 한 곳에 적어 두고 화면과 서버가 함께 쓰기도 합니다. (TS + zod)

10단원 연습문제 서버 정답 프론트엔드 연동

RUN node 연습문제_서버_정답.js

브라우저에서 `http://localhost:3000` 을 여세요.

감사무시: 대조 — 이 단원의 답은 서버가 아니라 화면(public/연습_관리화면.html)입니다.

이 파일은 문제에서 '고치지 마세요' 라고 준 서버 그대로라 번호 블록이 없습니다.

개념01~05 에서 배운 것을 한 화면에 모읍니다.

목록 + 검색 + 페이징 + 등록 폼 + 삭제 그리고 로딩·에러·빈 결과·검증 오류·중복 클릭 막기까지.

```
const express = require("express");
const path = require("path");

const app = express();
const PORT = Number(process.env.PORT) || 3000;

app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));

app.get("/", (req, res) => res.redirect("/연습_관리화면.html"));
```

GET /연습_관리화면.html

200

서버 (06단원 형식 그대로)

```

let 설비들 = [
  { id: 1, name: "컨베이어 1호", line: "A", status: "가동" },
  { id: 2, name: "프레스 1호", line: "B", status: "정지" },
  { id: 3, name: "용접로봇 1호", line: "C", status: "가동" },
  { id: 4, name: "검사기 1호", line: "A", status: "점검중" },
];

let 다음번호 = 5;

app.get("/api/v1/equipments", (req, res) => {
  const q = (req.query.q ?? "").trim().toLowerCase();
  const { line } = req.query;

  let 결과 = 설비들;

  if (q) {
    결과 = 결과.filter((설비) =>
      ["name", "line", "status"].some((키) => String(설비
[키]).toLowerCase().includes(q))
    );
  }

  if (line) 결과 = 결과.filter((설비) => 설비.line === line);

  const total = 결과.length;

  let page = Number(req.query.page);
  let limit = Number(req.query.limit);
  if (!Number.isInteger(page) || page < 1) page = 1;
  if (!Number.isInteger(limit) || limit < 1) limit = 10;

  const 시작 = (page - 1) * limit;

  res.json({
    data: 결과.slice(시작, 시작 + limit),
    meta: { page, limit, total, totalPages: Math.ceil(total / limit) },
  });
});

```

GET /api/v1/equipments?page=1&limit=3

```
200 {"data":[{"id":1,"name":"컨베이어 1호","line":"A","status":"가동"}, {"id":2,"name":"프레스 1호","line":"B","status":"정지"}, {"id":3,"name":"용접로봇 1호","line":"C","status":"가동"}], "meta":{"page":1,"limit":3,"total":4,"totalPages":2}}
```

```
app.post("/api/v1/equipments", (req, res) => {
  const { name, line } = req.body || {};
  const 항목 = [];

  if (!name) 항목.push({ 키: "name", 이유: "필수입니다" });
  else if (name.length < 2) 항목.push({ 키: "name", 이유: "2글자 이상이어야 합니다" });

  if (!line) 항목.push({ 키: "line", 이유: "필수입니다" });
  else if (!["A", "B", "C"].includes(line))
    항목.push({ 키: "line", 이유: "A, B, C 중 하나여야 합니다" });

  if (항목.length > 0) {
    return res.status(400).json({
      error: { code: "VALIDATION_FAILED", message: "입력값이 올바르지 않습니다",
details: 항목 },
    });
  }

  if (설비들.some((설비) => 설비.name === name)) {
    return res.status(409).json({
      error: { code: "DUPLICATED", message: "이미 있는 설비 이름입니다" },
    });
  }

  const 새설비 = { id: 다음번호, name, line, status: "정지" };
  다음번호 += 1;
  설비들.push(새설비);

  res.status(201).json({ data: 새설비 });
});
```

POST /api/v1/equipments

```
{"name":"포장기 1호","line":"C"}
```

```
201 {"data":{"id":5,"name":"포장기 1호","line":"C","status":"정지"}}
```

```
app.delete("/api/v1/equipments/:id", (req, res) => {
  const 번호 = Number(req.params.id);
  const 설비 = 설비들.find((설비) => 설비.id === 번호);

  if (!설비) {
    return res.status(404).json({ error: { code: "NOT_FOUND", message: "설비를 찾을 수 없습니다" } });
  }
});
```

업무 규칙: 가동 중인 설비는 못 지웁니다. (07단원)

```
if (설비.status === "가동") {
  return res.status(409).json({
    error: { code: "CONFLICT", message: "가동 중인 설비는 삭제할 수 없습니다" },
  });
}

설비들 = 설비들.filter((설비) => 설비.id !== 번호);
res.sendStatus(204);
});
```

```
DELETE /api/v1/equipments/2
204
```

```
DELETE /api/v1/equipments/1
409 {"error":{"code":"CONFLICT","message":"가동 중인 설비는 삭제할 수 없습니다"}}
```

화면 확인

처음 열면 3개씩 1쪽

화면 연습_관리화면.html

#요약 전체 4건 중 3건

#쪽정보 1 / 2

#본문 컨베이어 1호

다음 쪽

화면 연습_관리화면.html

#다음쪽

#쪽정보 2 / 2

#본문 검사기 1호

검색하면 1쪽으로 돌아갑니다

화면 연습_관리화면.html

#다음쪽 → 입력(#검색=로봇) → #라인거르기

#쪽정보 1 / 1

#본문 용접로봇 1호

★★ 2쪽을 보다가 검색했는데 1쪽으로 돌아왔습니다. 안 돌리면 page=2 로 요청해서 빈 화면이 나옵니다.
(개념04 섹션 3)

결과 없음

화면 연습_관리화면.html

입력(#검색=없는것) → #라인거르기

#요약 조건에 맞는 설비가 없습니다

빈 폼으로 등록

화면 연습_관리화면.html

#보내기

#오류-name 필수입니다

#오류-line 필수입니다

라인이 Z

화면 연습_관리화면.html #보내기

입력: #name=포장기 2호 #line=Z 보임: #오류-line A, B, C 중 하나여야 합니다

제대로 등록하면 목록이 새로 그려집니다

화면 연습_관리화면.html #보내기

입력: #name=적재로봇 1호 #line=A 보임: #알림 등록했습니다 보임: #요약 전체 5건

이름이 겹치면 409 가 이름 칸에

화면 연습_관리화면.html #보내기

입력: #name=컨베이어 1호 #line=A 보임: #오류=name 이미 있는 설비 이름입니다

가동 중인 설비를 지우려 하면 409

화면 연습_관리화면.html

.지우기

#알림 가동 중인 설비는 삭제할 수 없습니다

★ 첫 줄의 설비가 '컨베이어 1호(가동)' 라 삭제가 막힙니다. 07단원의 업무 규칙이 화면까지 그대로 전해졌습니다.

```
app.use((req, res) => {
  res.status(404).json({ error: { code: "ROUTE_NOT_FOUND", message: "그런 주소가 없습니다" } });
});

app.use((err, req, res, next) => {
  console.error(`[에러] ${req.method} ${req.path} - ${err.message}`);
  res.status(500).json({ error: { code: "INTERNAL_ERROR", message: "서버에서 문제가 생겼습니다" } });
});

app.listen(PORT, () => {
  console.log(`서버가 켜졌습니다. http://localhost:${PORT}`);
});
```

이 화면이 생긴 것

개념01 api.js 로 에러를 한곳에서 처리
 개념02 로딩 · 실패 · 빈 결과 · 버튼 잠금 · 늦게 온 응답 버리기
 개념03 preventDefault · 오류 지우기 · details 를 칸마다 · 성공하면 폼 비우기
 개념04 상태 한 곳에 · 조건 바뀌면 1쪽 · 빈 값은 주소에서 빼기 · 디바운스
 개념05 (이 화면에는 업로드가 없습니다. 11단원에서 붙입니다)

★ 삭제 버튼에 이벤트 위임을 쓴 이유 표는 다시 그려질 때마다 버튼이 새로 만들어집니다. 버튼마다 addEventListener 를 달면, 다시 그린 뒤에 또 달아야 합니다. tbody 에 한 번만 달아 두면 그럴 일이 없습니다. (JS자료 11단원)

★ 삭제 뒤에 목록을 다시 부르는 이유 화면에서만 그 줄을 지우면 meta.total 이 안 맞습니다. 쪽 수도 달라질 수 있습니다. 서버에 다시 물어보는 편이 정확합니다.

★ 다만 요청이 한 번 더 갑니다.

목록이 아주 크면 화면에서 지우고 total 만 줄이는 방법도 씁니다.
둘 다 맞습니다. 무엇을 더 중요하게 볼지 정하는 문제입니다.

직접 눌러 볼 것

1 처음 화면	3건 + "1 / 2" 2 '다음' 누르기
2쪽, '다음' 이 잠김	
3 2쪽에서 '로봇' 검색	1쪽으로 돌아가고 1건 ★
4 '없는것' 검색	"조건에 맞는 설비가 없습니다" 5 빈 폼으로 등록
두 칸에 빨간 글씨	
6 라인을 Z 로 등록	라인 칸에만 빨간 글씨
7 제대로 등록	목록이 새로 그려지고 전체가 늘어남
8 같은 이름으로 또 등록	이름 칸에 "이미 있는 설비 이름입니다" 9 가동
중인 설비 삭제	"가동 중인 설비는 삭제할 수 없습니다"
10 정지 중인 설비 삭제	지워지고 목록이 새로 그려짐

3번과 9번이 이 실습의 핵심입니다. 3번은 화면이 켜져야 할 것, 9번은 서버 규칙이 화면까지 그대로 전해지는 것입니다.

부 록 나

심화문제 정답

심화문제는 선택 과제입니다. 풀지 않고 넘어가도 다음 단원에 지장이 없습니다.